Topics

Sunday, July 20, 2025 10:08 AM

1. **Introduction to DSA and Python Basics**
2. **Arrays and Lists**
3. **Strings**
4. **Linked Lists**
5. **Stacks**
6. **Queues**
7. **Searching**
8. **Sorting**
9. **Recursion**
10. **Hashing**
11. **Patterns & Problem Solving**
12. **Trees**
13. **Graphs**

1 - Introduction

Saturday, July 26, 2025 9:29 AM

- **Importance of DSA**
- **Time & Space Complexity**
- **Python Refresher**

1.1 - Importance of DSA

Saturday, July 26, 2025 11:11 AM

What is DSA?

- DSA stands for **Data Structures and Algorithms**—two fundamental pillars of computer science and software development
- DSA helps manage and preprocess large datasets, optimize pipelines, and write fast algorithms for feature engineering or mode l evaluation

Data Structures: *particular way of organizing and storing data so it can be accessed and modified efficiently*

- **Linked Lists** → Streaming Data (can't load full data into memory)
- **Hash Tables** → One-hot Encoding, Label Encoding
- **Stacks/Queues** → Data Pipelines, Rolling Windows
- **Trees** → Decision Trees, Random Forests, Boosting methods
- **Graphs** → Social Networks, Recommendation Systems

DS → way of holding data/info

Algorithms: *step-by-step procedure to solve a specific problem or perform a task*

- **Searching** (Linear Search, Binary Search)
- **Sorting** (Bubble, Merge, Quick Sort)
- **Recursion**
- **Dynamic Programming**
- **Graph algorithms** (BFS, DFS)

Alg → way of processing data

Why Learn DSA?

- **Build Strong Problem-Solving Skills**
- **Crack Coding Interviews**
- **Better System Design Thinking**
- **Improve Debugging and Optimization**
- **Write Optimized Code**
  - Reduce time and space complexity
  - Handle large datasets efficiently
  - Write scalable programs
- **Core for Advanced Topics**
  - Competitive Programming
  - Machine Learning (matrix operations, search algorithms)
  - Web development (caching, queues)
  - Blockchain, Compilers, and Operating Systems

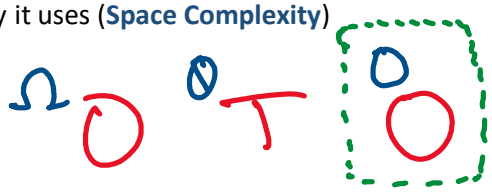
1.2 - Time & Space Complexity

Saturday, July 26, 2025 9:49 AM

Why Analyze Complexity?

When working with large datasets or building scalable ML/DS systems, it's crucial to:

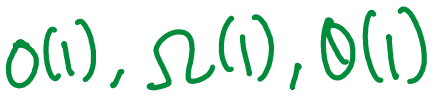
- Estimate how fast an algorithm runs (**Time Complexity**)
- Know how much memory it uses (**Space Complexity**)



Complexity Notations:

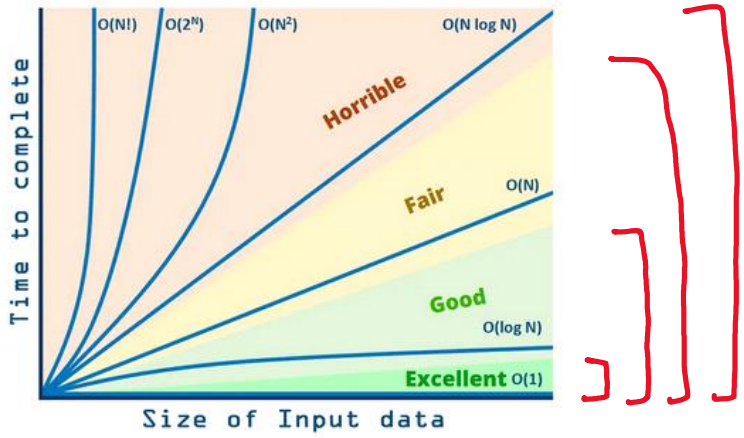
NOTATION	MEANING	REPRESENTS
Ω(f(n))	Big Omega	Best-case performance
Θ(f(n))	Big Theta	Average-case / tight bound
O(f(n))	Big O	Worst-case performance

Most Common Complexities:



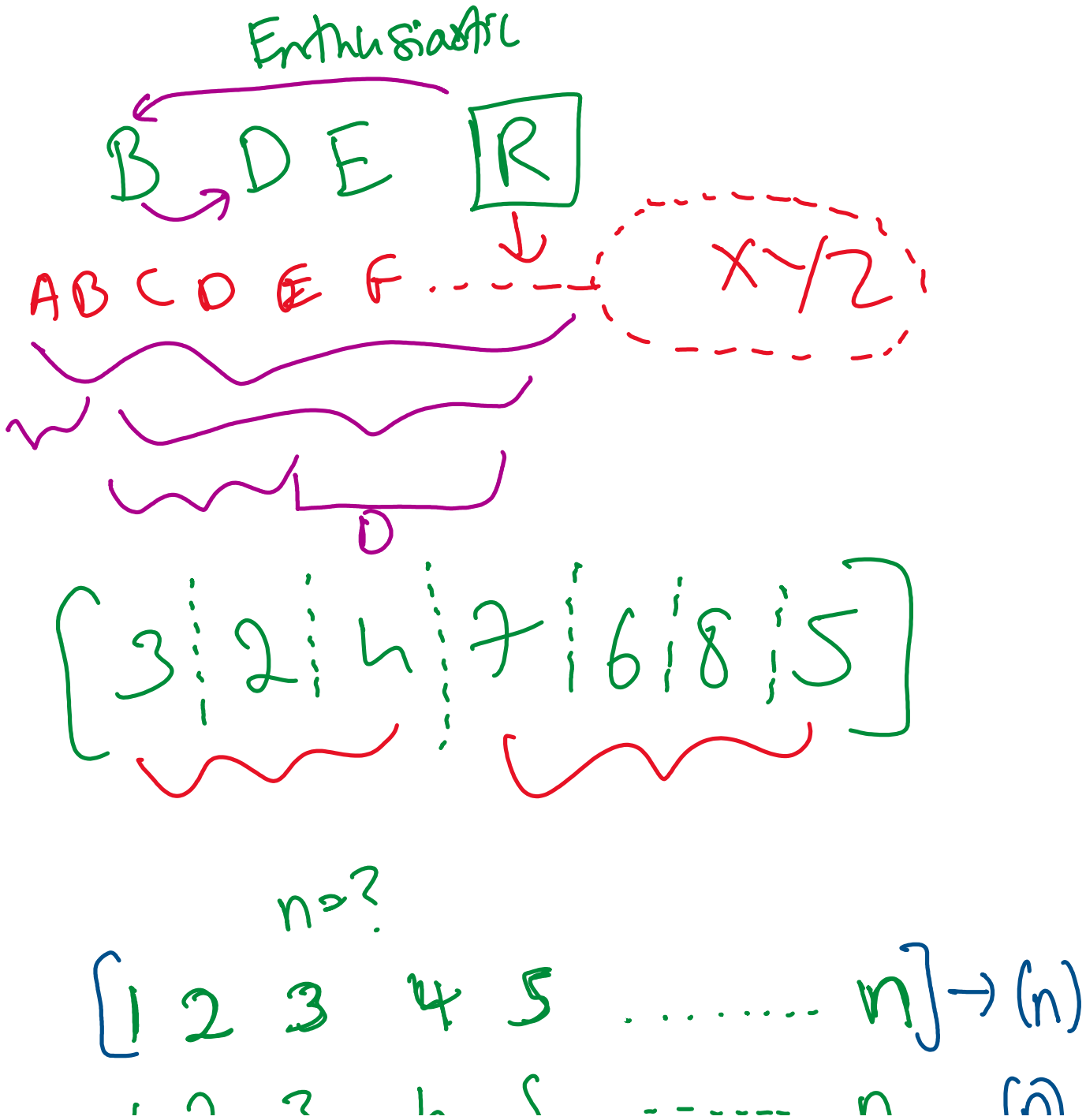
COMPLEXITY	NAME	ANALOGY
O(1)	Constant	Flipping a light switch
O(log n)	Logarithmic	Finding a word in a dictionary (Binary Search)
O(n)	Linear	Scanning every row in a dataset
O(n log n)	Linearithmic	Efficient sorting (Merge Sort)
O(n²)	Quadratic	Comparing each row with every other row
O(2ⁿ), O(n!)	Exponential/Factorial	Trying every possible combination (Brute-force models)

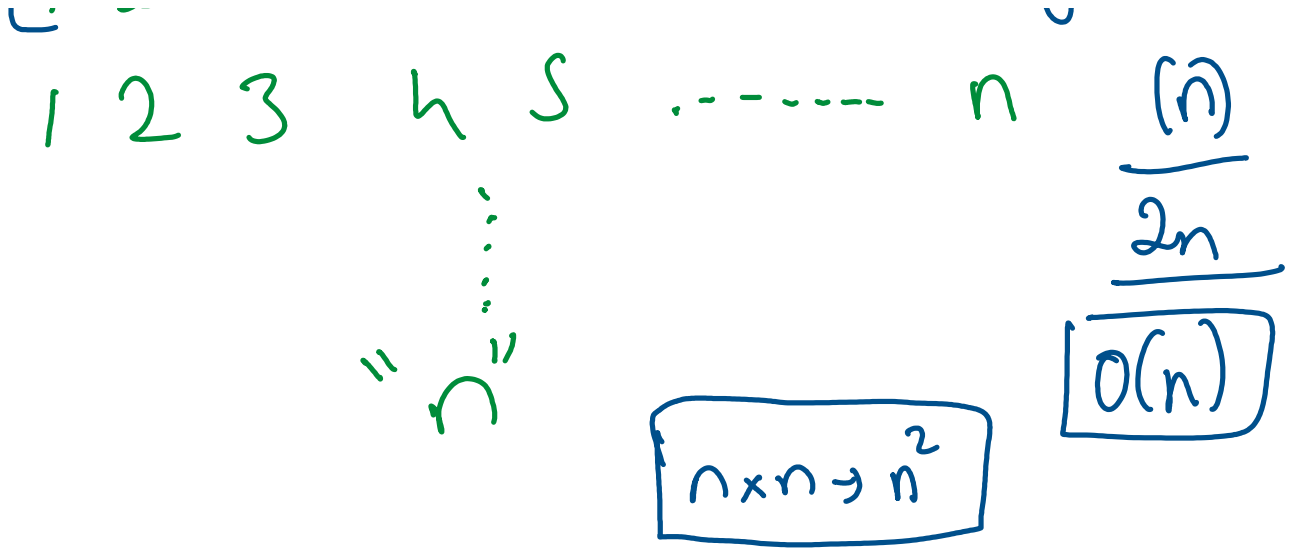
Complexities Analysis:



Summary:

OPERATION	COMPLEXITY	NOTE
Indexing	O(1)	Constant access
Loop over list	O(n)	Linear scan
Nested loop	$O(n^2)$	All pairs
Binary Search	O(log n)	Halves search space each step
Merge Sort	O(n log n)	Divide and conquer
Fibonacci (naive)	$O(2^n)$	Recursive tree explodes




1 2 3 4 5 ..... n
(n)
2n
O(n)
"n"
n x n → n²

1.2.1 - Practice

Saturday, July 26, 2025 10:57 AM

Space Time

Q1.

```
def print_first_element(arr):
    print(arr[0])
```

O(n) O(1)

Q2.

```
def print_all_elements(arr):
    for item in arr:
        print(item)
```

O(n) O(n)

Q3.

```
def print_all_pairs(arr):
    for i in arr:
        for j in arr:
            print(i, j)
```

n n

O(n) $O(n^2)$

Q4.

```
def log_loop(n):
    while n > 1:
        print(n)
        n = n // 2
```

O(1) O(log n)

n=10 5 2 1

10 5 2

Q5.

```
def sort_list(arr):
    arr.sort()
```

Quick Sort

O(n) O(n · log n)

Q6.

```
def fib(n):
```

O(1) $O(2^n)$

Q6.

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

$O(1)$ $O(2^n)$

Q7.

```python
def create_zero_list(n):
    return [0] * n
```

$O(n)$ $O(1)$

$[0] * n \rightarrow [0,0,0,0,\ldots.0]$ (n)

Q8.

```python
def sum_n(n):
    total = 0          # 1
    for i in range(n):  # n
        total += i      # 1
    return total        # 1
```

$O(1)$ $O(n)$

Q9.

```python
def print_elements_twice(arr):
    for item in arr:    # n
        print(item)     # 1
    for item in arr:    # n
        print(item)     # 1
```

$O(n)$ $O(n)$

$[n+1+n+1 \Rightarrow 2n+2 \Rightarrow 2n \Rightarrow n]$

Q10.

```python
def fixed_inner_loop(n):
    for i in range(n):          # n
        for j in range(100):    # 100
            print(i + j)        # 1
```

$O(1)$ $O(n)$

```
        print(i + j)
```

$n \times 100 \times 1 \Rightarrow 100n \Rightarrow n$

1.3 - Python Refresher

Saturday, July 26, 2025 11:17 AM

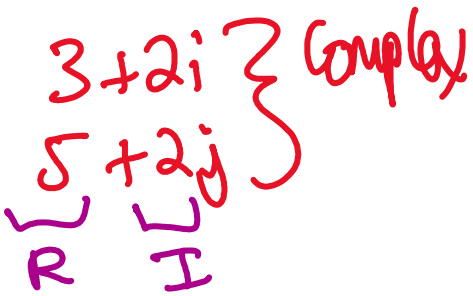
- **Built-in Data Types & Data Structures**
- **Control Structures**
- **Loops**
- **Functions**
- **Built-in Functions**
- **List Comprehensions**
- **Classes**
- **Reading Inputs**

1.3.1 - Built-in Data Types & Data Structures

Saturday, July 26, 2025 11:25 AM

1. Numeric Types:

TYPE	DESCRIPTION
int	Integer values
float	Floating-point numbers
complex	Complex numbers (real + imaginary part)



2. Sequence Types:

TYPE	DESCRIPTION
str	Text (string of characters)
list	Mutable ordered collection
tuple	Immutable ordered collection
range	Sequence of numbers

3. Set Types:

TYPE	DESCRIPTION
set	Mutable, unordered
frozenset	Immutable version of set

4. Mapping Types:

TYPE	DESCRIPTION
dict	Unordered key-value store

5. Boolean Type:

TYPE	DESCRIPTION
bool	True or False

6. None Type:

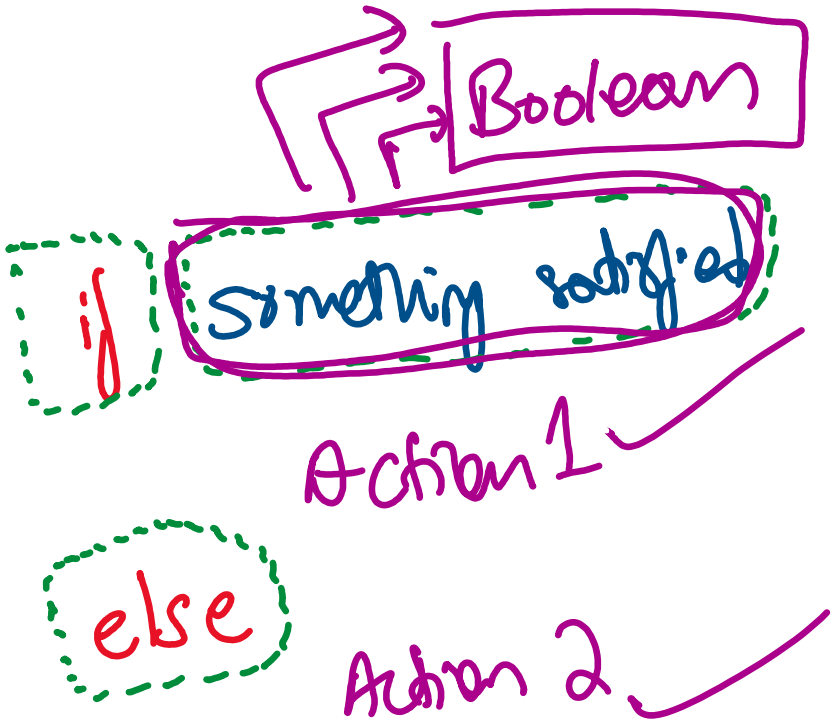
TYPE	DESCRIPTION
NoneType	Represents None

1.3.2 - Control Structures

Saturday, July 26, 2025 11:59 AM

if - else ✓

if - elif - else ✓



1.3.3 - Loops

Saturday, July 26, 2025 12:01 PM

1. **For loop** → if u know how many iterations
2. **While loop** → if u don't know; conditional

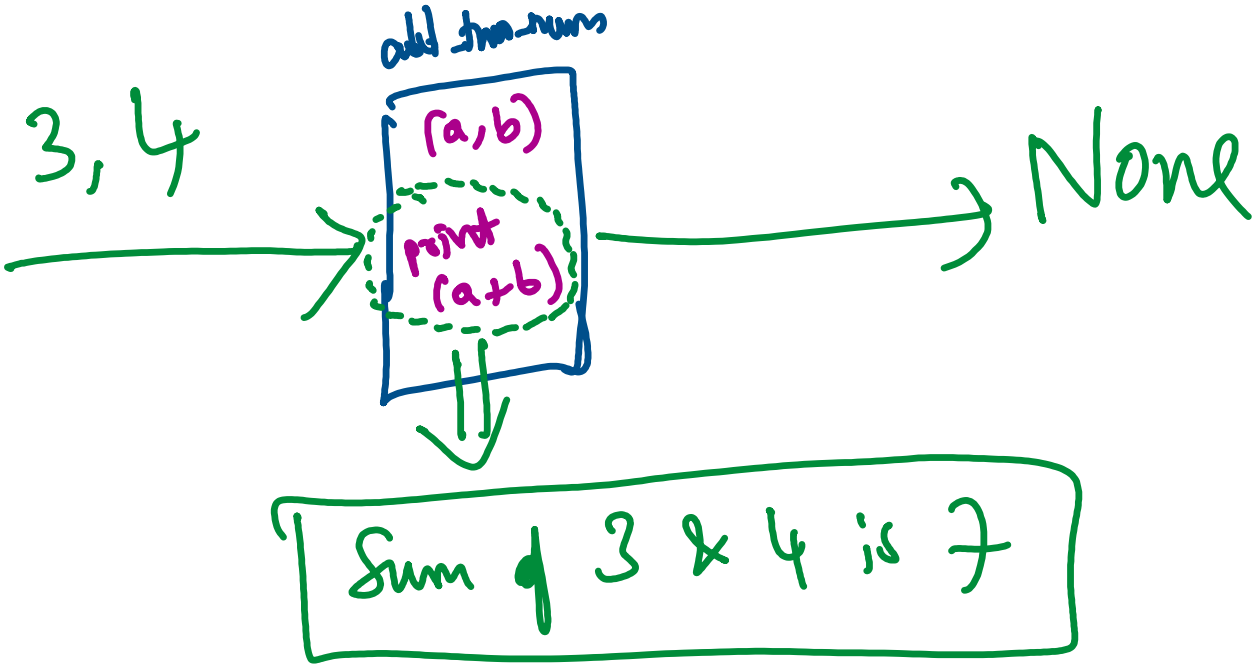
1.3.4 - Functions

Saturday, July 26, 2025 12:05 PM

- **Define/Call functions**
- **Input to functions** → parameters → Arguments
- **Output from functions**

Define → Storage

Call → Usage



1.3.5 - Built-in Functions

Saturday, July 26, 2025 12:07 PM

- **min**
- **max**
- **sum**
- **len**
- **sorted**
- **enumerate**
- **zip**

1.3.7 - Classes

Saturday, July 26, 2025 12:12 PM

- **Naming convention: *camel-case***
- **Define constructor: *__init__***
- **Attributes**
- **Methods**
- **Class Objects**

1.3.8 - Reading Inputs

Saturday, July 26, 2025 12:13 PM

- **input**
- **split**
- **map**

2. Arrays & Lists

Saturday, July 26, 2025 5:10 PM

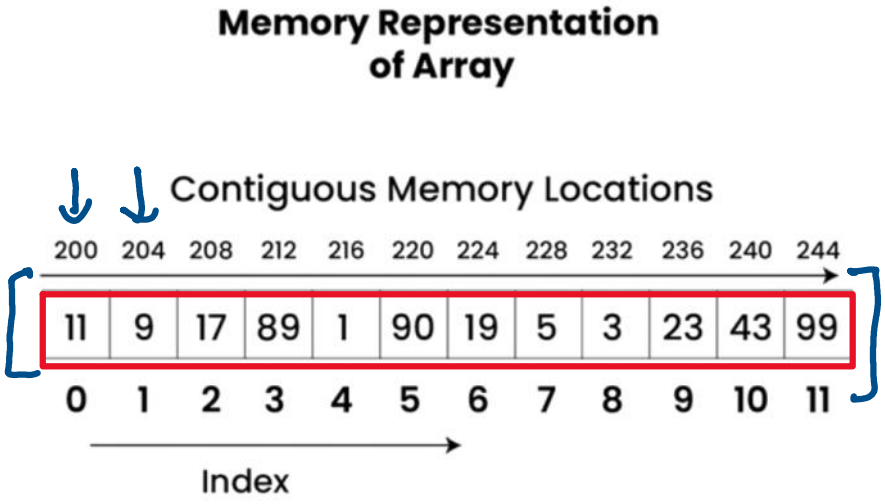
- **Arrays**
- **Lists**
- **Arrays vs Lists**
- **Complexity Analysis**
- **Coding Problems**

2.1 - Arrays

Saturday, August 9, 2025 9:04 AM

What is an Array?

An array is a data structure that stores a collection of elements of the same data type in ***contiguous memory locations***



Properties of an Array:

1. **Homogeneous Elements:** All elements are of the same data type (***model weights of type float32***)
2. **Contiguous Memory Allocation:** Elements are stored one after another in RAM
3. **Fixed Index Access:** Can instantly fetch any element with its index
4. **Efficient Iteration:** Traversing all elements is sequential and cache-friendly

Fixed-Size Arrays:

- Size is decided at creation
- Stored contiguously in memory
- More memory-efficient, but less flexible
- **Implementation:** ***array, numpy***
- **Usage:** Best for predictable, stable data (offline learning)

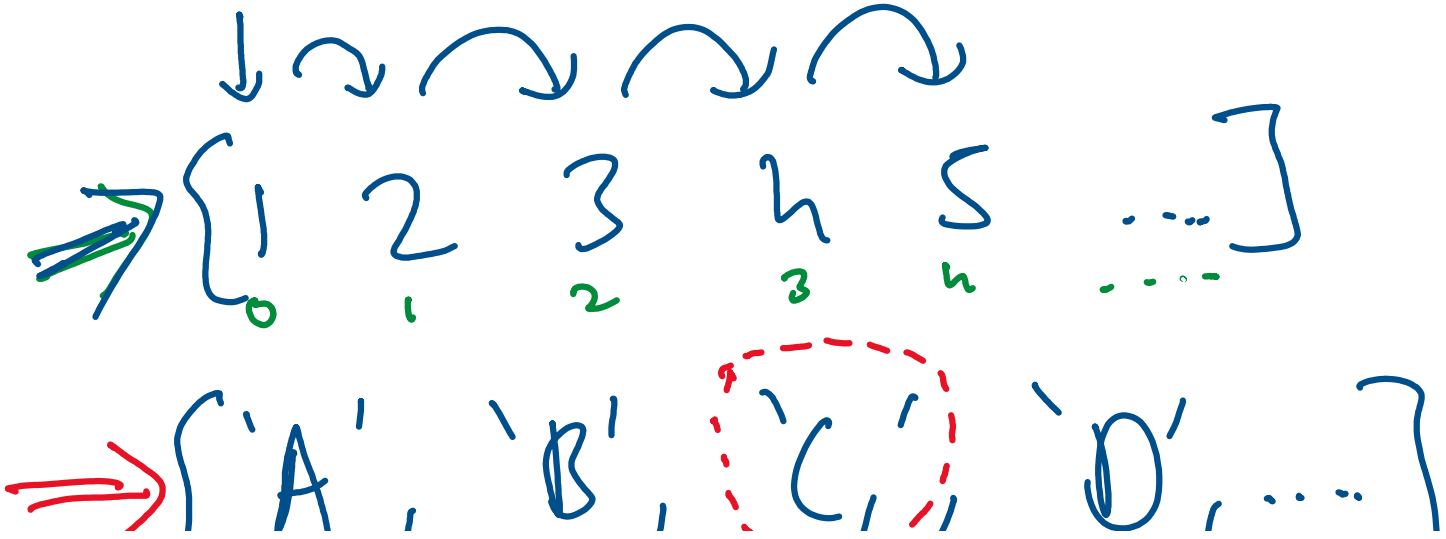
- **Usage:** Best for predictable, stable data (offline learning)

Dynamic-Size Arrays:

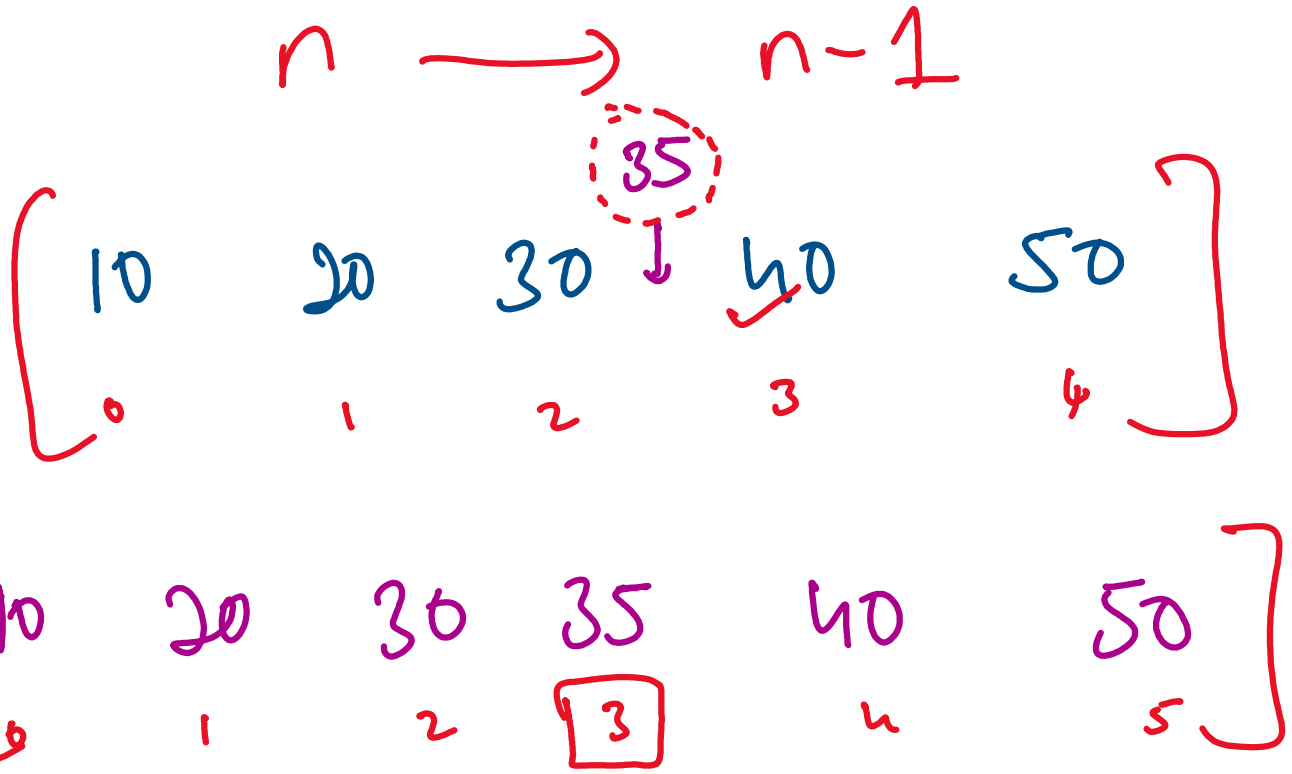
- Can grow or shrink during runtime
- Allocates extra "spare" memory to allow future growth
- **Implementation:** *lists*
- **Usage:** Best for unpredictable, growing datasets (online learning)

Basic Array Operations:

OPERATION	DESCRIPTION
Traversal	Visiting each element sequentially
Insertion	Adding a new element at a given position
Deletion	Removing an element from a given position
Access (Indexing)	Getting an element at a specific index
Searching	Finding the index of an element
Updating	Changing the value of an element at a specific index
Sorting	Rearranging elements in a certain order
Merging	Combining two arrays into one
Splitting	Dividing one array into multiple arrays



A
B
C
D
0
1
2
3


n
n-1
35
10
20
30
40
50
0
1
2
3
4

10
20
30
35
40
50
0
1
2
3
4
5

2.2 - Lists

Saturday, August 9, 2025 9:29 AM

What is a List?

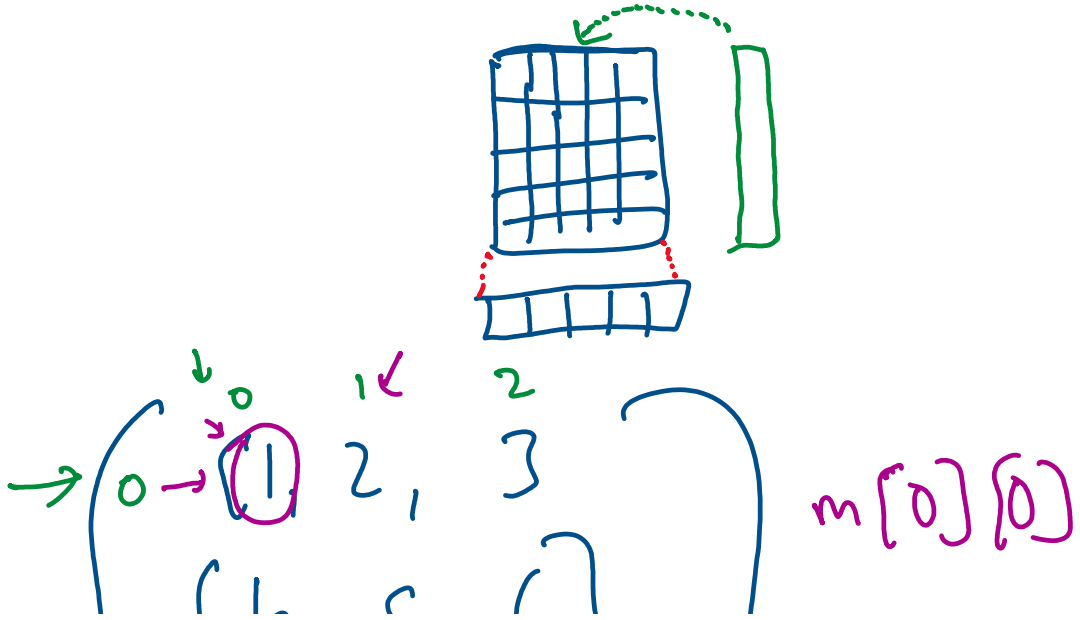
- A list is a built-in and versatile data structure that can hold elements of ***different data types***
- A list is a ***dynamic, ordered and mutable*** sequence in Python

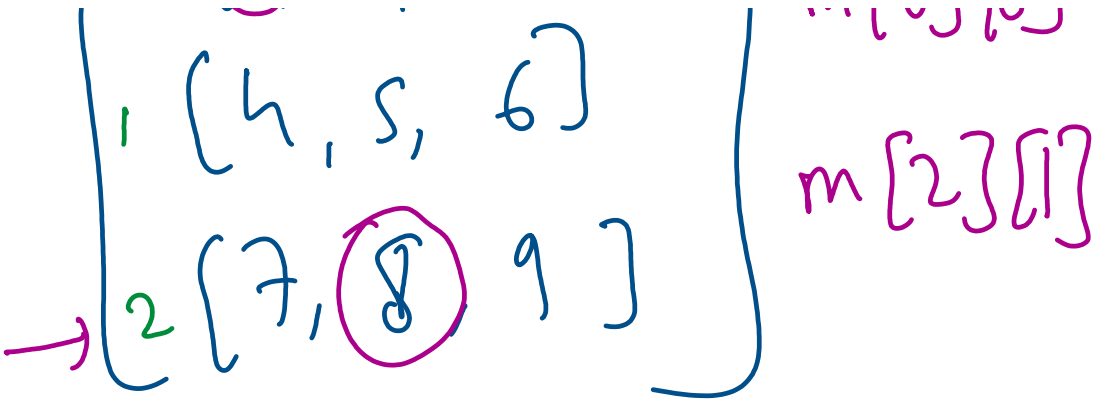
Properties of Lists:

1. **Ordered:** Items in a list maintain a specific order based on their insertion sequence
2. **Mutable:** Lists are changeable, i.e., elements can be added, removed, or modified after the list has been created
3. **Allows Duplicates:** Multiple identical elements allowed
4. **Indexing:** Supports direct element access using zero-based indexing
5. **Heterogeneous:** Can store elements of mixed data types (e.g., integers, floats, strings, objects)
6. **Dynamic Size:** Can grow or shrink without defining size in advance

Most Common List Methods:

METHOD	DESCRIPTION	ML ANALOGY
append(x)	Add element x to the end of the list	Add a new training sample to the dataset buffer
insert(i, x)	Insert element x at position i	Insert a missing feature into the correct index of a feature vector
extend(iterable)	Add all elements from another iterable	Add a mini-batch of new samples to an existing dataset
remove(x)	Remove first occurrence of x	Remove a specific label or value from dataset
pop([i])	Remove and return element at index i (default last)	Retrieve and remove the last prediction from stored results
clear()	Remove all elements from the list	Reset the temporary dataset storage
index(x)	Return index of first occurrence of x	Find where a specific label appears in dataset
count(x)	Return number of times x appears	Count samples of a specific class in the dataset
sort()	Sort the list in ascending order (in-place)	Sort prediction scores for ranking tasks
reverse()	Reverse the list in-place	Reverse a time-series sequence for backward RNN processing
copy()	Return a shallow copy of the list	Duplicate dataset before augmentation

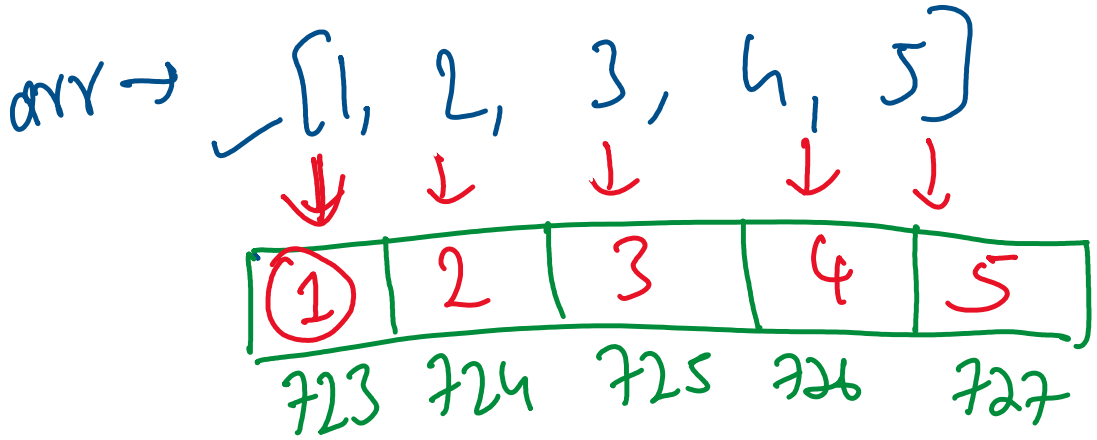


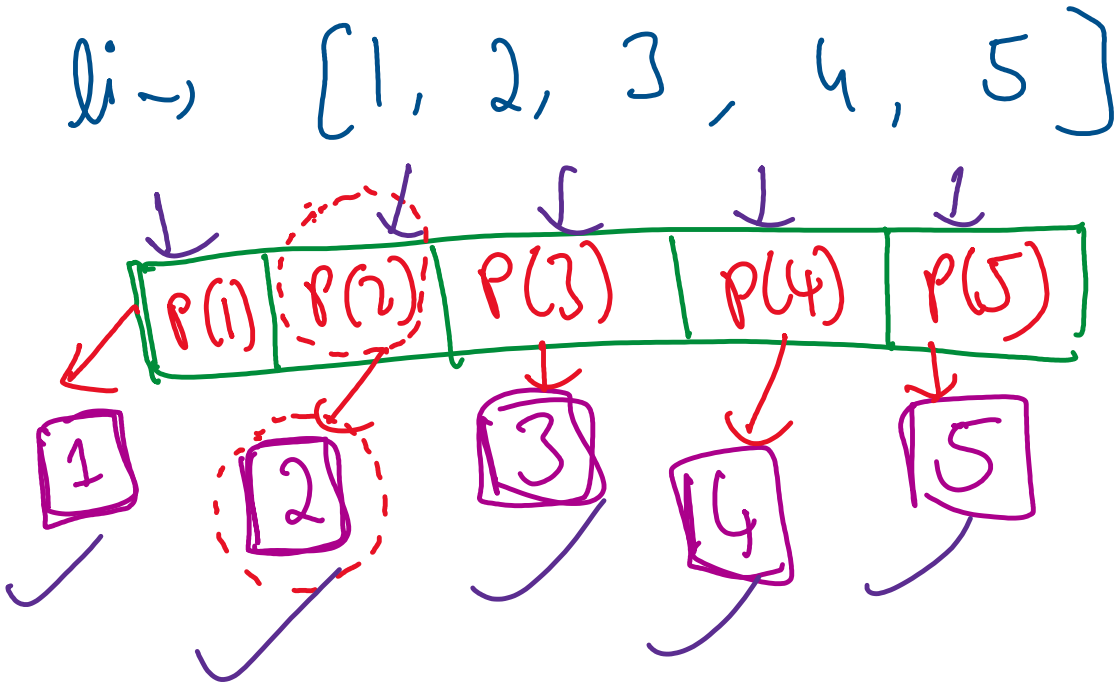

1 [4, 5, 6]
2 [7, 8, 9]
m[2][1]

2.3 - Arrays vs Lists

Saturday, August 9, 2025 9:29 AM

PROPERTY	ARRAY	LIST
Data Type	- collection of homogeneous (same type) elements	- heterogeneous collection of elements that can store any data types
Memory	- stored in contiguous memory - memory efficient - faster retrieval	- stores references to objects - less memory-efficient
Performance	- faster for numerical operations - supports vectorized processing (especially with NumPy)	- slower for element-wise math - requires explicit loops
Flexibility	- generally fixed in size	- can grow/shrink dynamically
Functionality	- supports mathematical operations directly (NumPy)	- built-in methods for insertion, deletion, reordering - needs manual loops for math
Use Cases	Scientific computing, ML datasets, image data, numerical simulations	Data collection, mixed-type storage, intermediate data handling

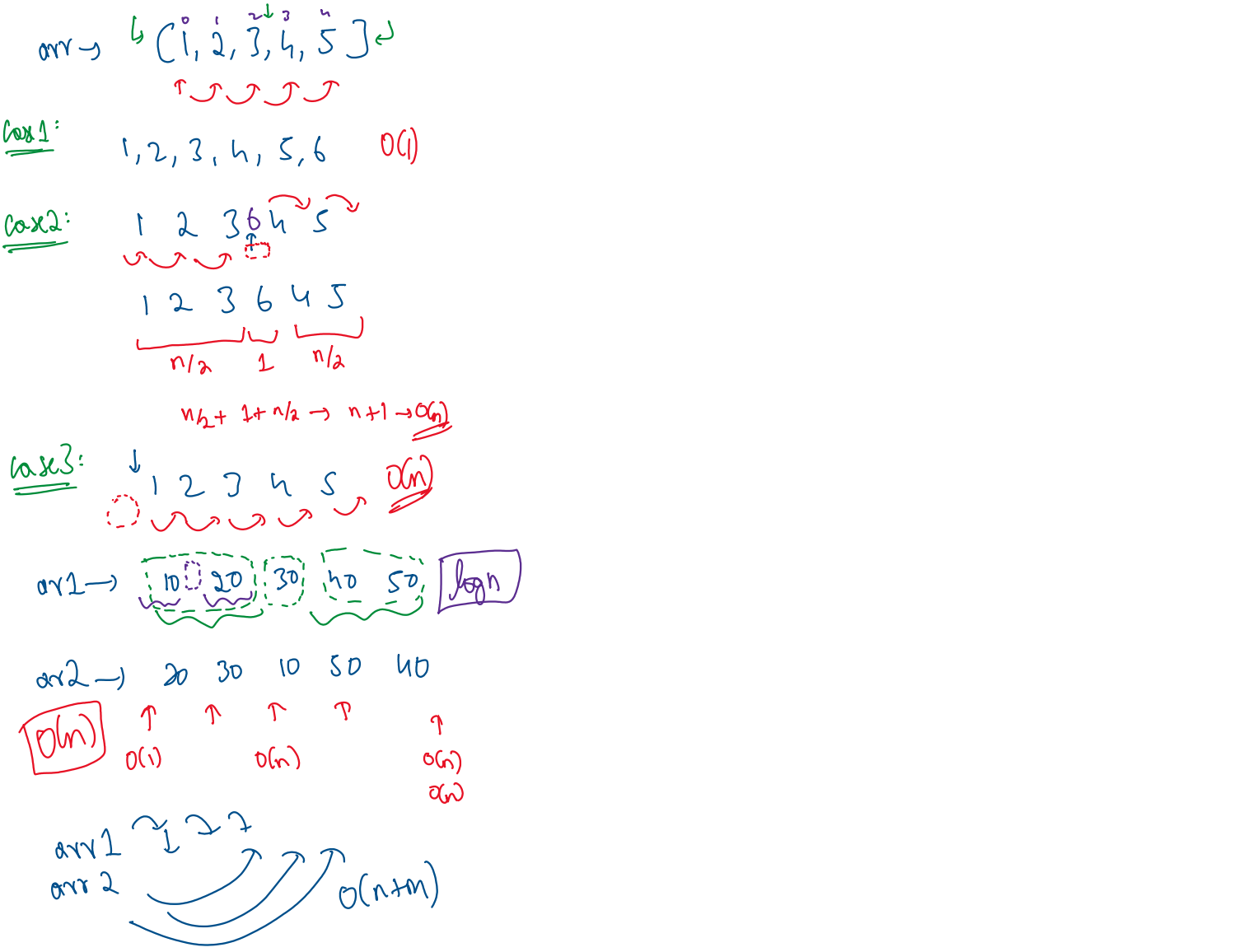




2.4 - Complexity Analysis

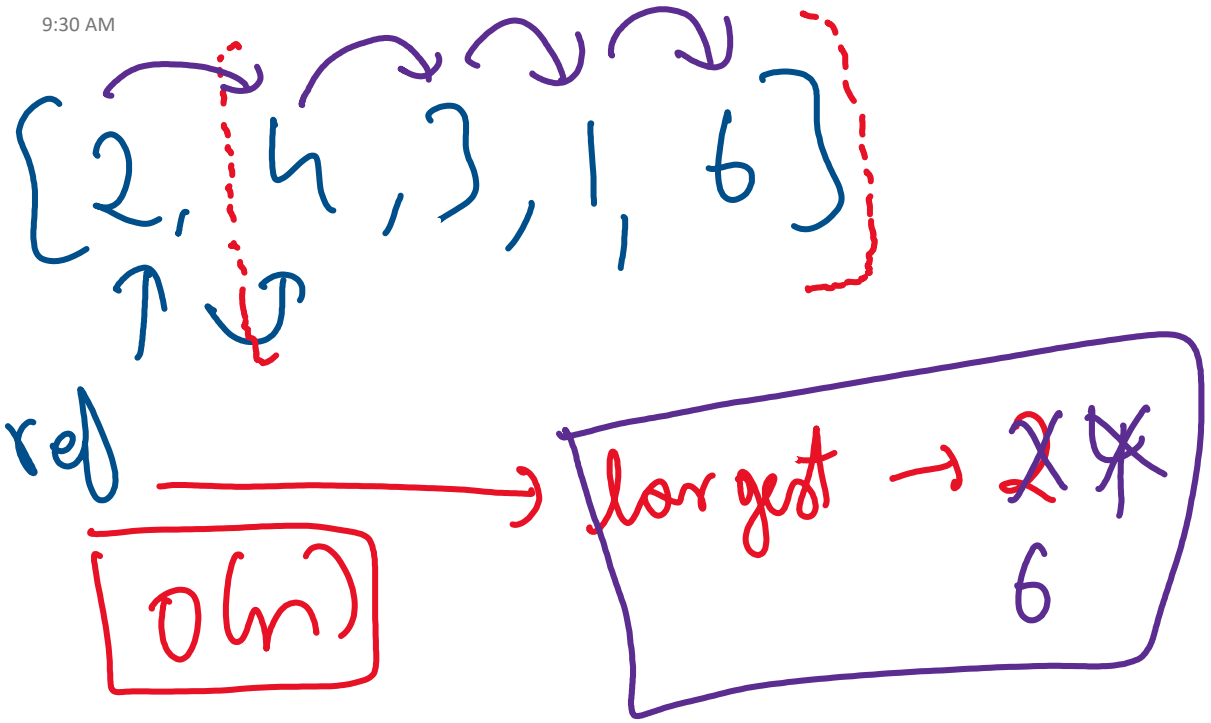
Saturday, August 9, 2025 9:29 AM

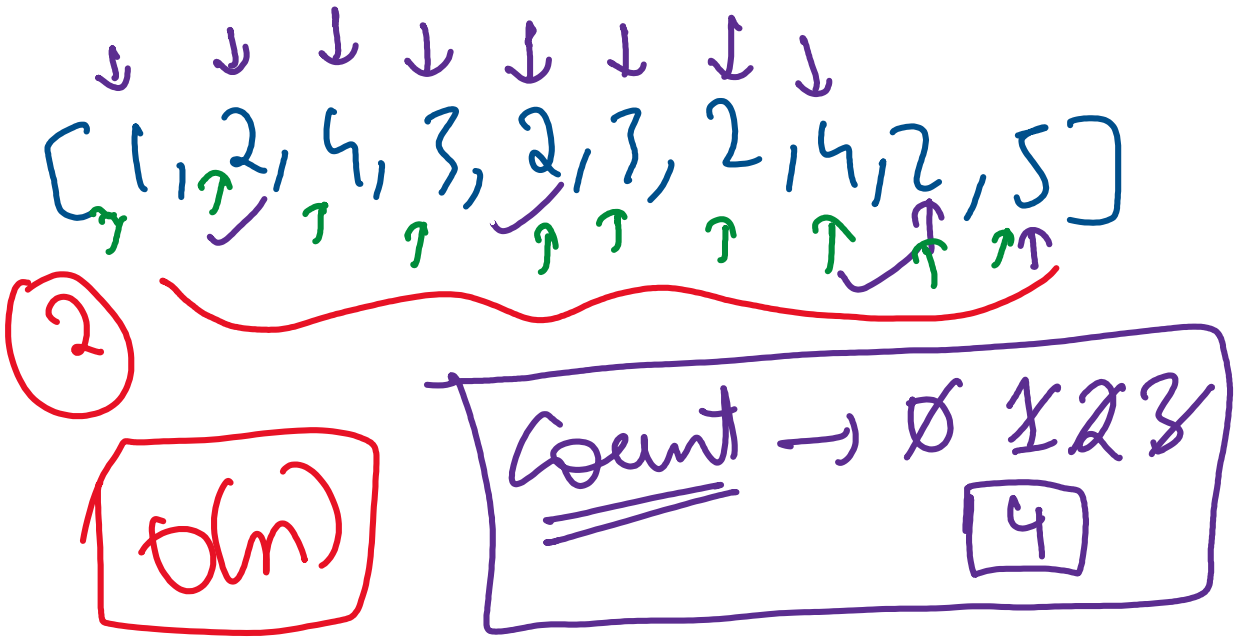
OPERATION	DESCRIPTION	TIME COMPLEXITY	ML ANALOGY
Traversal	Visiting each element sequentially	O(n)	Iterating over all feature values in a dataset to normalize them
Insertion	Adding a new element at a given position	O(n) (worst case, due to shifting elements)	Adding a new sample into a sorted list of training data
Deletion	Removing an element from a given position	O(n) (due to shifting elements)	Removing an outlier from a dataset stored in an array
Access (Indexing)	Getting an element at a specific index	O(1)	Accessing the pixel value at (x, y) in an image tensor
Searching	Finding the index of an element	O(n) (linear search) or O(log n) if sorted (binary search)	Looking up a feature value in a sorted array of unique IDs
Updating	Changing the value of an element at a specific index	O(1)	Updating a model weight in a flattened parameter array
Sorting	Rearranging elements in a certain order	O(n log n) (typical)	Sorting dataset samples by label before stratified batching
Merging	Combining two arrays into one	O(n + m)	Merging two mini-batches into a single batch before training
Splitting	Dividing one array into multiple arrays	O(n)	Splitting dataset into training and validation sets

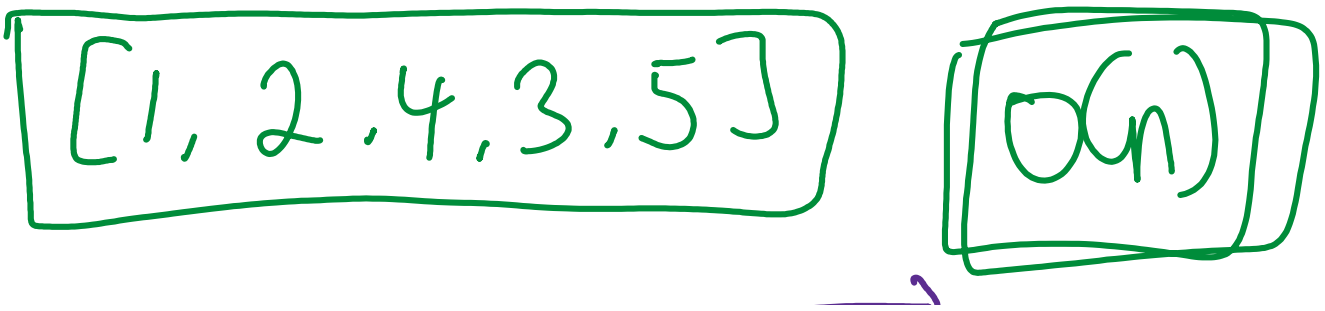


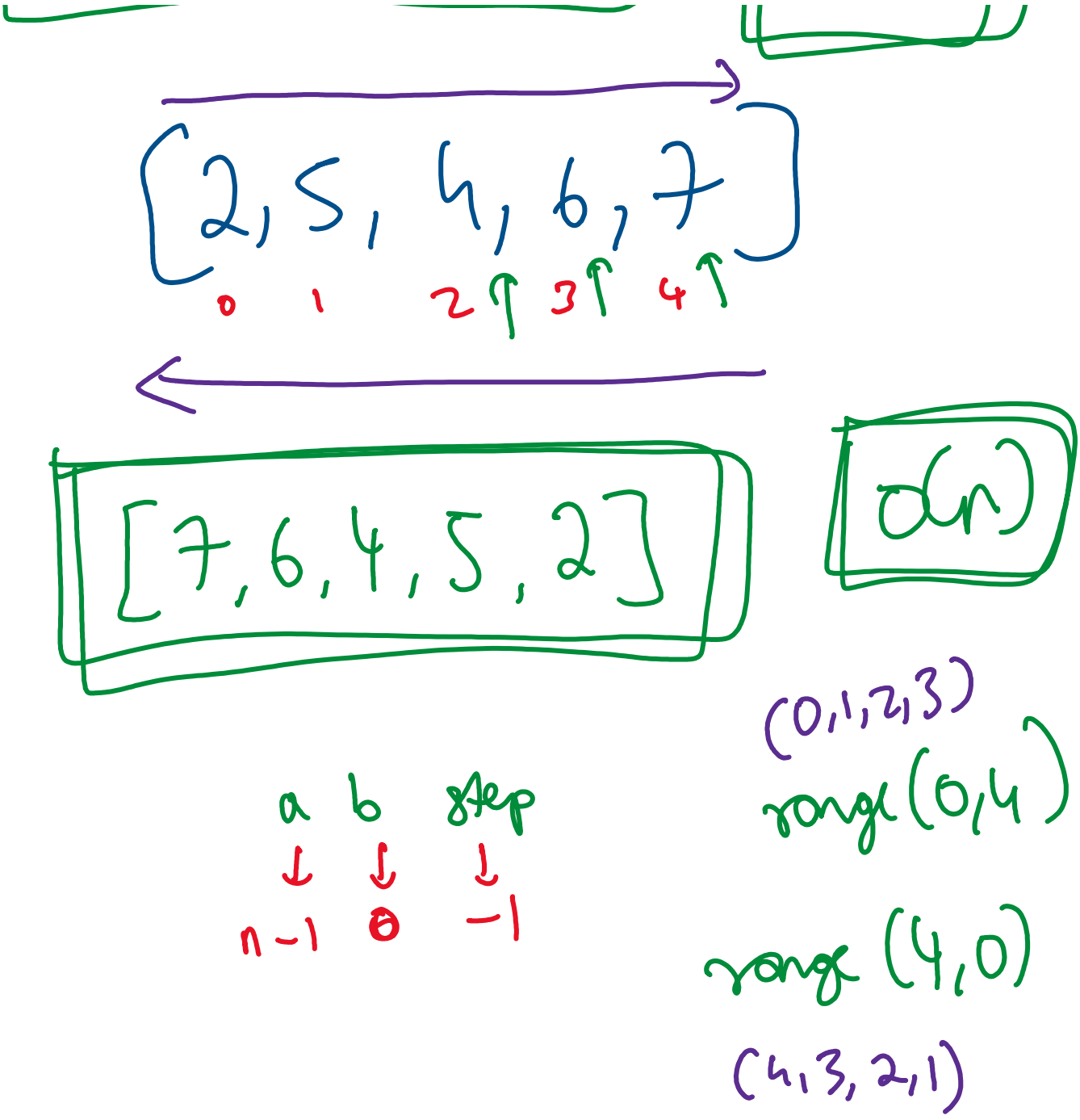
2.5 - Coding Problems

Saturday, August 9, 2025 9:30 AM








[2, 5, 4, 6, 7]
0 1 2 3 4
[7, 6, 4, 5, 2]
O(n)
a b step
n-1 0 -1
(0,1,2,3)
range(0,4)
range (4,0)
(4,3,2,1)

2.6 - DS Context

Tuesday, December 16, 2025 12:34 AM

Data Science / ML Correlation:

- Feature vectors, datasets, tensors
- Model inputs & outputs are arrays

Libraries & Usage:

- **NumPy:** ***ndarray***
- **Pandas:** ***Series & DataFrame*** columns
- **TensorFlow / PyTorch:** ***tensors***

3. Strings

Saturday, August 16, 2025 4:13 PM

- **Introduction**
- **Properties**
- **Usecases**
- **Common Functions**
- **String Manipulation Techniques**
- **String Comparison**
- **Coding Problems**

3.1 - Introduction to Strings

Monday, August 18, 2025 10:23 PM

What is a String?

- A string is a sequence of characters enclosed within **single (')**, **double (")**, or **triple quotes (''' / """)**
- Strings are ordered sequences, meaning characters have a defined index starting from 0

3.2 - Properties of Strings

Monday, August 18, 2025 10:36 PM

- **Ordered sequences:** Characters are stored in order & each character has an index
- **Immutable:** Once created, a string cannot be modified; Any operation creates a new string
- **Iterable:** Characters of a string can be looped over
- **Slicing:** Can extract substrings using [start:end:step]
- **Unicode-based:** Each character is stored as a Unicode code point; Python uses UTF-8 internally
- **Concatenation**
- **Repetition**

PROPERTY	DESCRIPTION
Sequence	Ordered collection of characters
Immutable	Cannot be changed
Iterable	Can be looped
Indexed	Positive & negative indexing
Slicing	Extract substrings
Concatenation/Repeat	"+" and "*" are supported
Unicode	Supports all characters & emojis
Homogeneous	Only characters
Built-in Methods	Rich library of functions
Hashable	Usable as dict keys

3.3 - Usecases of Strings

Monday, August 18, 2025 10:42 PM

1. **Handling Textual Data**
   - Names, emails, file paths, log entries
   - Almost every real-world program deals with strings

2. **Foundation for Algorithms:**
   - **Pattern Matching:** searching substrings (KMP, Rabin-Karp)
   - **Hashing:** rolling hash in string problems (finding anagrams)
   - **Parsing & Processing:** extracting useful information from text

3. **Basis for Natural Language Processing (NLP):**
   - **Tokenization, sentiment analysis, machine translation** – all work with strings

3.4 - Common Functions

Monday, August 18, 2025 10:49 PM

FUNCTION	DESCRIPTION
s.lower()	Converts all characters to lowercase
s.upper()	Converts all characters to uppercase
s.strip()	Removes leading and trailing whitespaces
s.replace(old, new)	Replaces all occurrences of a substring with another
s.split()	Splits string into a list of substrings (default separator = space)
"sep".join(list)	Joins list elements into a single string with a separator

3.5 - String Manipulation

Monday, August 18, 2025 10:55 PM

1. Case Conversion:

- **lower()**: Converts entire string to lowercase
- **upper()**: Converts entire string to uppercase
- **title():** Capitalizes the first letter of each word
- **capitalize():** Capitalizes only the first letter of the string
- **swapcase():** Swaps uppercase to lowercase and vice versa

2. Whitespace Handling:

- **strip():** Removes spaces (or specific characters) from both ends
- **lstrip() / rstrip():** Removes spaces from left/right only

3. Searching & Finding:

- **find(sub):** Returns first index of substring, or -1 if not found
- **index(sub):** Same as find() but raises ValueError if substring not found
- **startswith() / endswith():** Boolean check if string starts/ends with given text

4. Validation Checks:

- **isdigit():** Returns True if all characters are digits
- **isalpha():** Returns True if all are alphabets
- **isalnum():** Returns True if alphanumeric (letters + digits)
- **islower() / isupper():** Check case
- **isspace():** Checks if only whitespace

5. Splitting & Joining:

- **split(sep):** Splits string into list (default sep = space)
- **join(list):** Joins list elements with a separator into one string

6. Replacing Substrings:

- **replace(old, new):** Returns a new string where all occurrences of old are replaced with new

7. Counting & Measuring:

- **len(s):** Returns length of string
- **count(sub):** Counts how many times a substring appears

8. Formatting Strings:

- **f-strings** (Python 3.6+)

3.6 - String Comparison

Monday, August 18, 2025 11:14 PM

In Python, strings are compared **lexicographically** (like words in a dictionary):

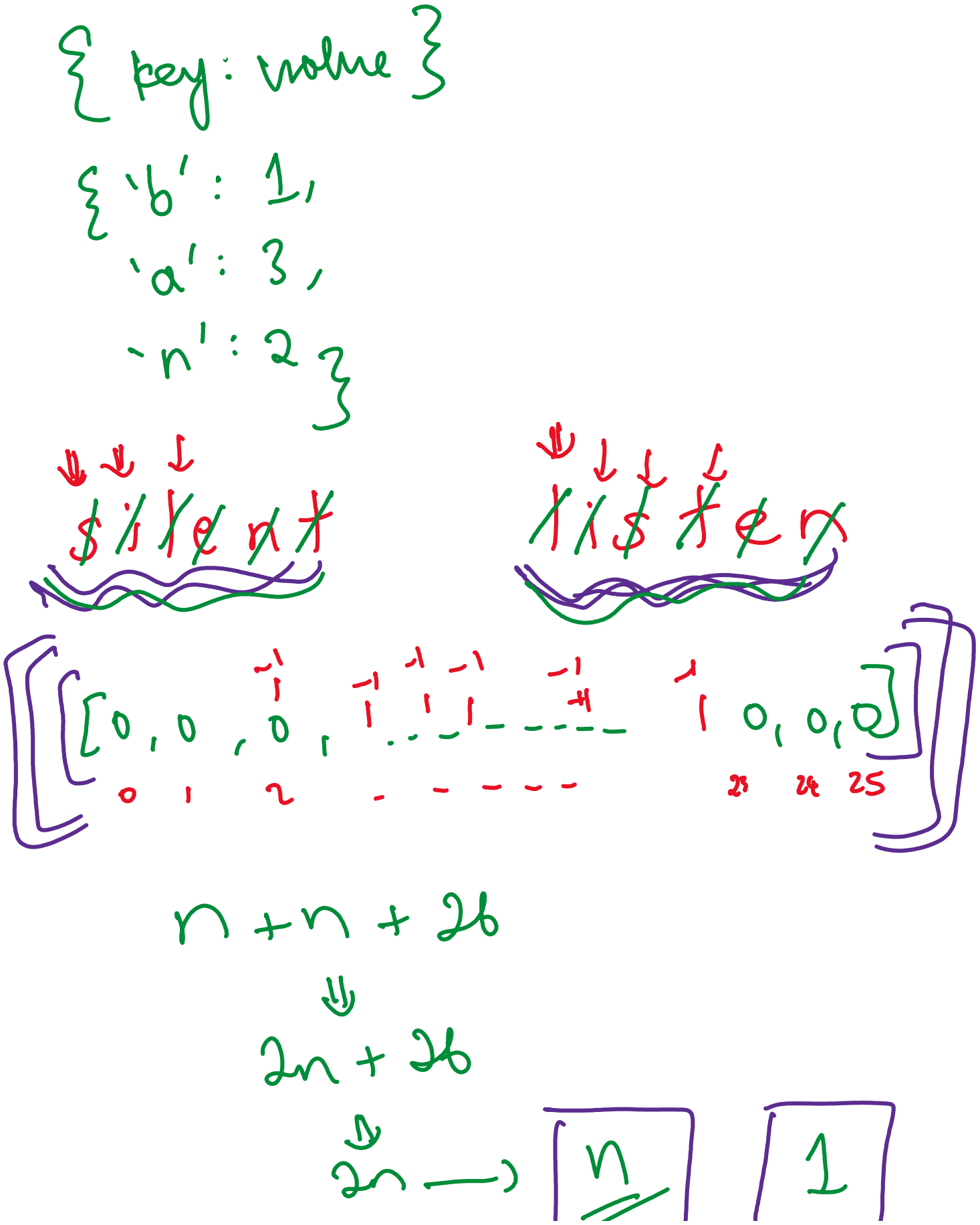
- Python checks character by character from left to right
- Comparison is based on **Unicode code points** (the number assigned to each character)
- If all characters match but lengths differ, the shorter string is smaller

Common Operators:

OPERATOR	DESCRIPTION	EXAMPLE
==	Equality	"hello" == "hello" → True
!=	Inequality	"hi" != "hello" → True
<	Less than (lexicographic)	"apple" < "banana" → True
>	Greater than	"dog" > "cat" → True
<=	Less than or equal	"app" <= "apple" → True
>=	Greater than or equal	"zoo" >= "zebra" → True

3.7 - Coding Problems

Monday, August 18, 2025 11:43 PM



3.8 - DS Context

Tuesday, December 16, 2025 12:44 AM

Data Science / ML Correlation:

- Text data processing
- NLP pipelines

Libraries & Usage:

- **re:** text cleaning
- **NLTK, spaCy:** tokenization
- **transformers:** BERT, GPT tokenizers

4. Linked Lists

Wednesday, August 27, 2025 1:55 PM

- **Introduction**
- **Linked Lists in Memory**
- **Linked Lists vs Arrays/Lists**
- **Types of Linked Lists**
- **Singly Linked List**
- **Circular Singly Linked List**
- **Doubly Linked List**
- **Circular Doubly Linked List**
- **Coding Problems**

4.1 - Introduction

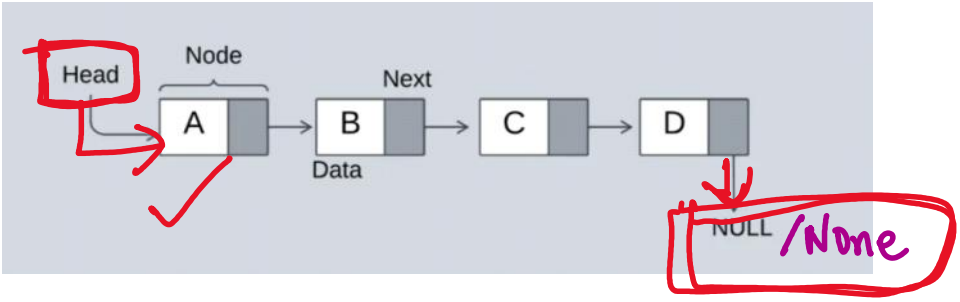
Saturday, August 30, 2025 10:59 AM

What is a Linked List?

- A linked list is a linear data structure composed of nodes connected by pointers (or references)
- Each node contains a data element and a pointer to the next node in the sequence
- Linked lists do not require contiguous memory and can dynamically change size, making them flexible for operations like insertions and deletions without rearranging the entire structure

Structure of a Linked List:

- Each element/node of a linked list typically consists of two parts:
  - **Data** - The value stored (e.g., integer, string, object, etc.)
  - **Pointer** - Address/reference to the next node in the sequence



- The first node is called the **head**
- The last node points to **None** (in singly linked list) or back to the **head** (in circular linked list)

Key Features:

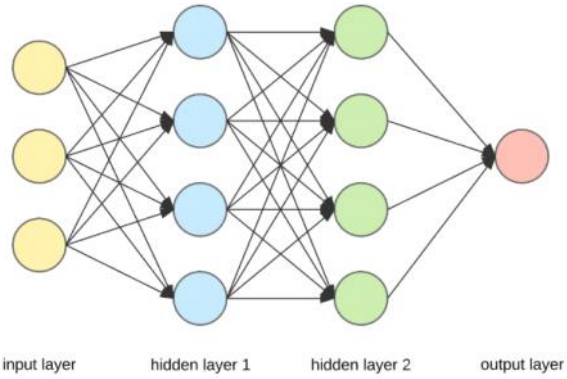
1. **Dynamic Size:**
   - Do not require a fixed size at the time of creation
   - Nodes can be added or removed at runtime as needed
2. **Non-Contiguous Memory Allocation:**
   - Nodes can be stored anywhere in memory
   - Each node has a pointer/reference that connects it to the next node
3. **Efficient Insertions and Deletions:**

- Insertion/Deletion at the beginning, middle, or end is efficient if the pointer to the node is known
- No shifting of elements is required

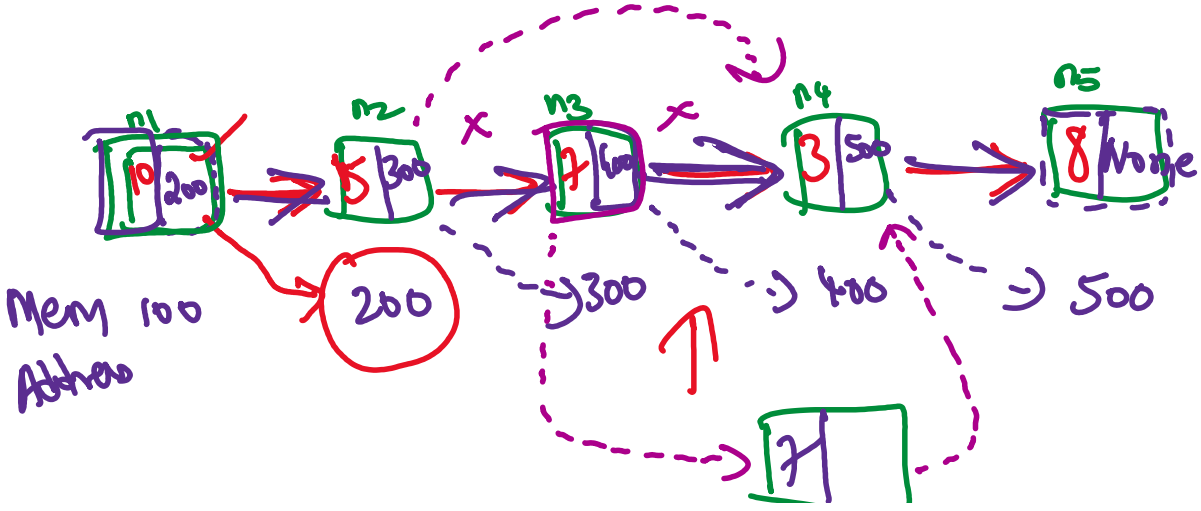
4. **Sequential Access Only:**
   - Elements cannot be accessed directly by index
   - To reach a particular node, traversal from the **head** is required

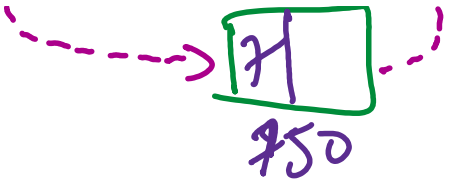
ML Analogy:

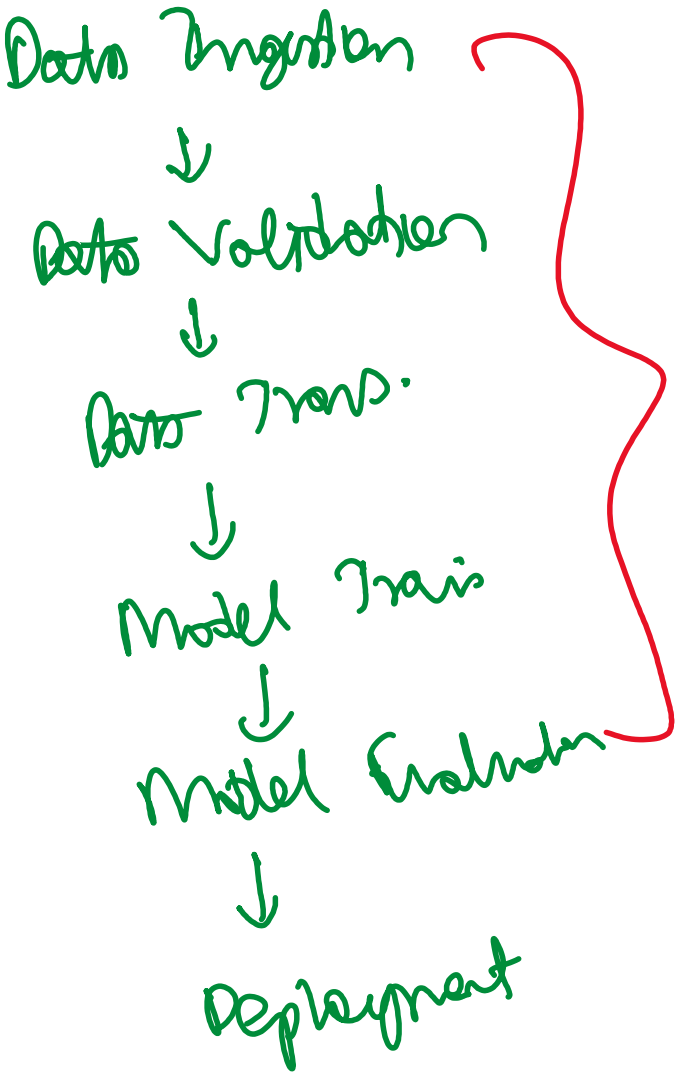
- **ML Pipelines**
- **Neural Network Architectures:**
  - Layers are stacked and connected sequentially

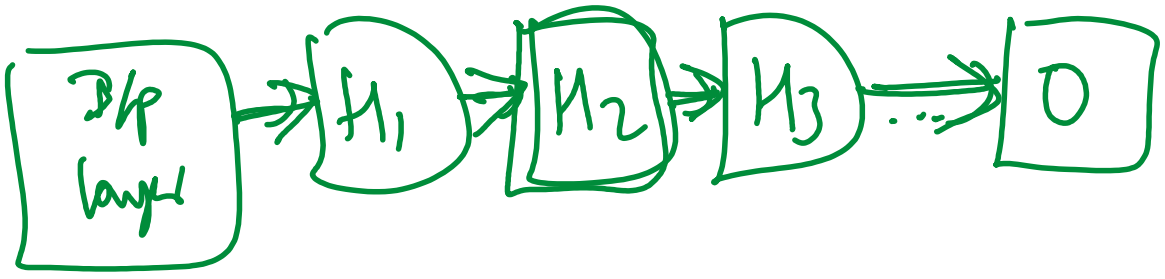


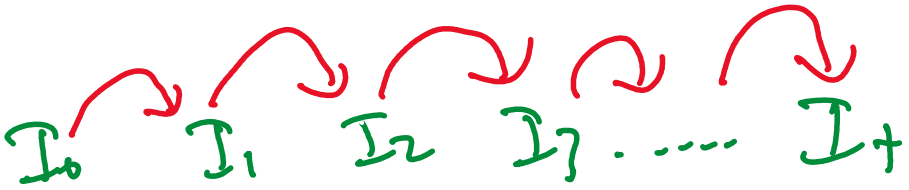
- **Gradient Descent Steps:**
  - Each update depends on previous state parameters
- **Graph Representation in ML:**
  - ML problems (social networks, knowledge graphs, recommendation systems) are naturally represented using **nodes** and **pointers**



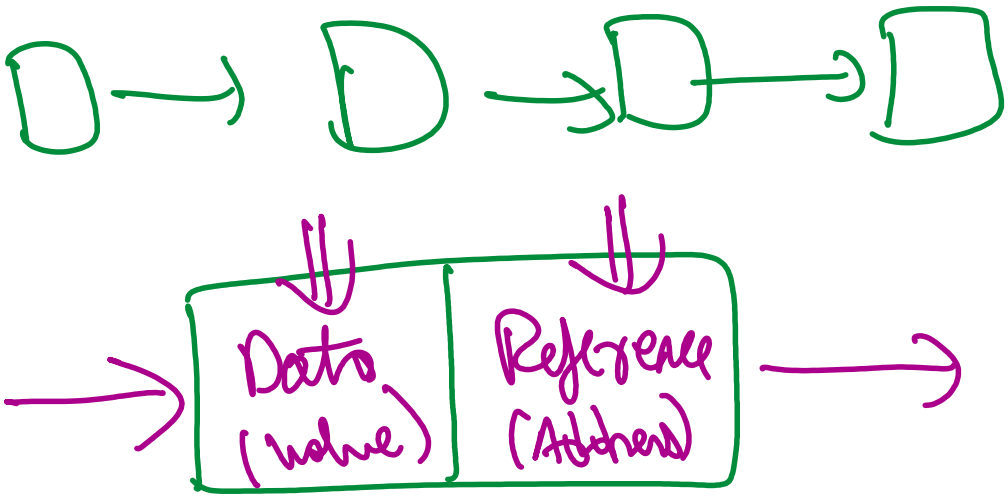

H
750


Data Ingestion
Data Validation
Data Trans.
Model Train
Model Evaluation
Deployment


I/p layer
H1
H2
H3
O


I0
I1
I2
I3 .....
It

I_0 I_1 I_2 I_3 ..... I_t



node

4.2 - Linked Lists in Memory

Saturday, August 30, 2025 11:12 AM

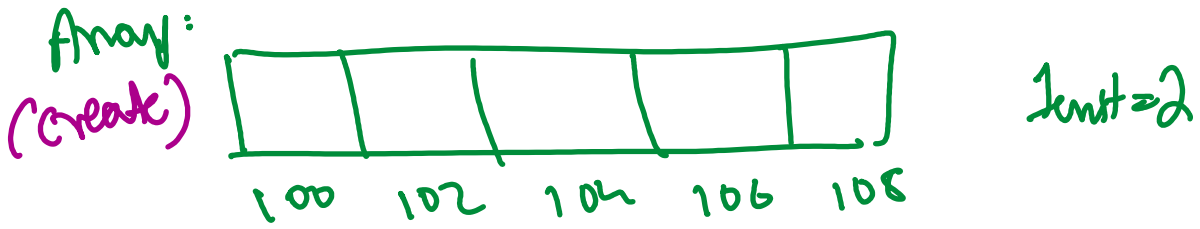
- Linked list nodes are not stored together in a continuous block
- Each node is dynamically allocated whenever a new element is added
- The memory location of the next node is not predictable, so we must store a pointer/reference inside each node

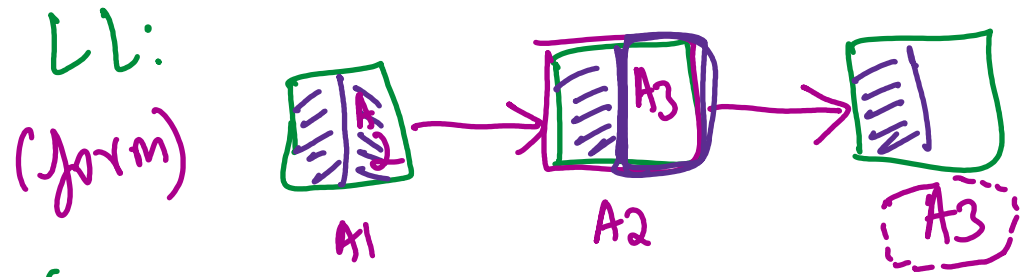
Pros:

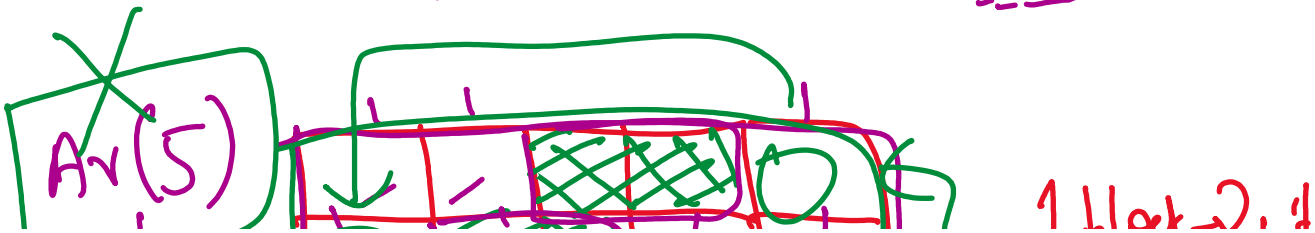
- No need for contiguous free memory
- Prevents **memory fragmentation** (scattered free memory that cannot always be used for large contiguous allocations)
- Flexible and grows/shrinks at runtime

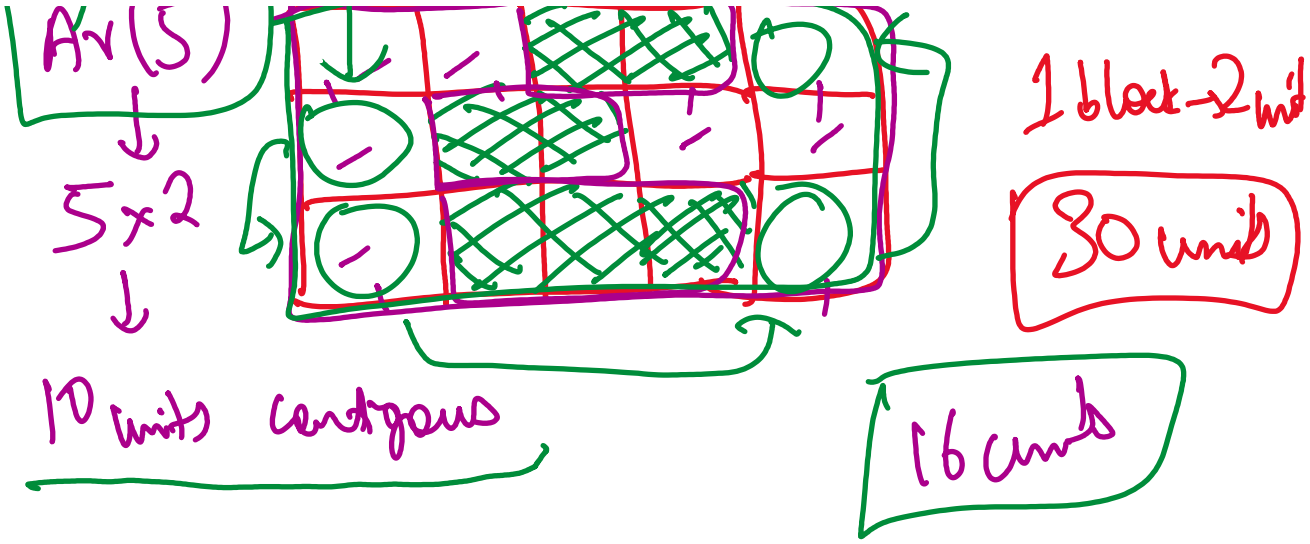
Cons:

- Extra memory required for pointers
- Since nodes are scattered, CPU caching is less effective
- Access is sequential since addresses are not predictable





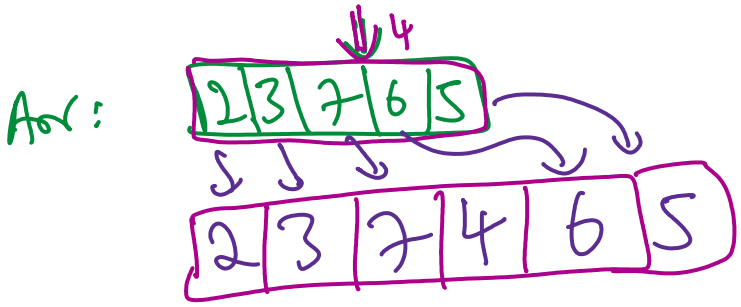



Ar(S)
5×2
10 units contiguous
1 block → 2 unit
30 units
16 units

4.3 - Linked Lists vs Arrays/Lists

Saturday, August 30, 2025 11:21 AM

FEATURE	ARRAYS / LISTS	LINKED LISTS
Memory storage	Contiguous memory block	Non-contiguous memory (nodes scattered)
Indexing support	Supported (direct access by index)	Not supported (no built-in indexing)
Memory usage	Fixed (or resized in chunks when using Python list)	Flexible (each node allocated separately)
Pointer overhead	None since only data is stored	Extra memory for storing pointers/references in each node
Traversal direction	Only forward using index	Forward in singly, forward/backward in doubly linked lists
Cache friendliness	High (elements stored together, better CPU caching)	Low (nodes scattered across memory, poor cache performance)
Resizing	Needs reallocation when limit is reached	Naturally grows/shrinks with dynamic allocation
Implementation complexity	Simple (built-in in most languages like Python list)	More complex (requires manual implementation of nodes & pointers)
Fragmentation issues	Needs large contiguous block of memory	Works well even with fragmented memory
Example in Python	arr = [10, 20, 30]	Custom **Node** and **LinkedList** class
Use cases	Best when frequent random access and fewer insertions/deletions	Best when frequent insertions/deletions and size is dynamic

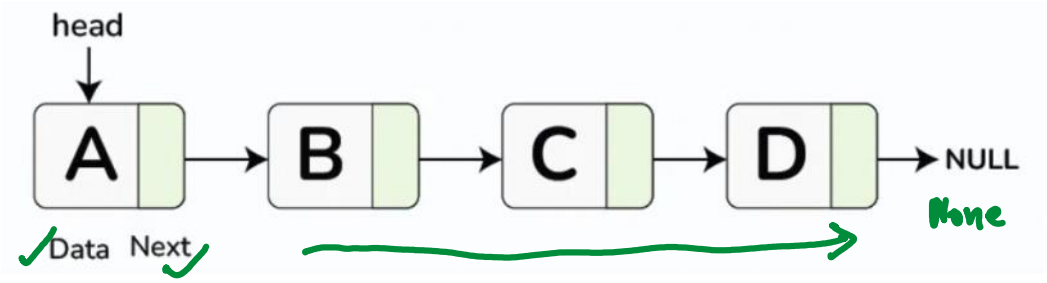


4.4 - Types of Linked Lists

Saturday, August 30, 2025 11:45 AM

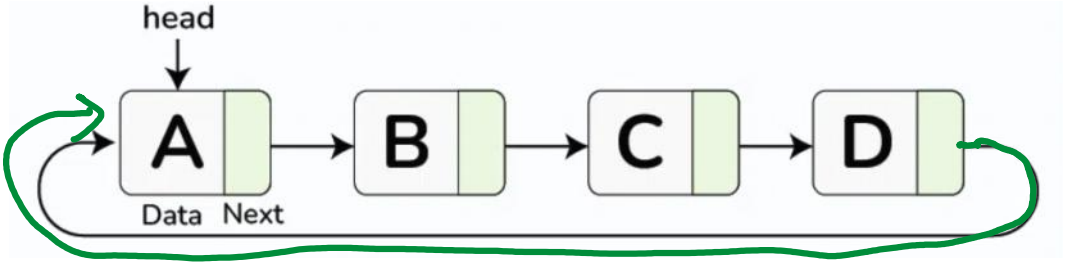
1. Singly Linked List:

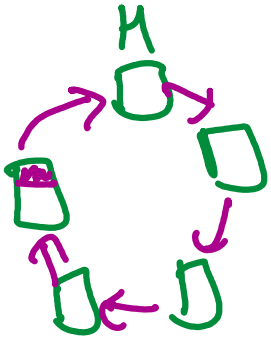
- Each node has two parts:
  - **Data:** stores the value
  - **Next:** pointer/reference to the next node
- The last node points to **None**
- Traversal is only in one direction (forward)



2. Circular Singly Linked List:

- Similar to singly linked list, but last node points back to **head** instead of **None**
- Forms a circular chain
- Traversal can continue indefinitely only in forward direction



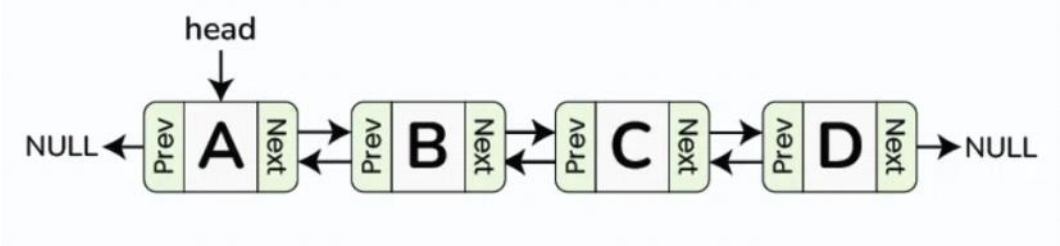


3. Doubly Linked List:

- Each node has three parts:
  - **Prev:** pointer to previous node

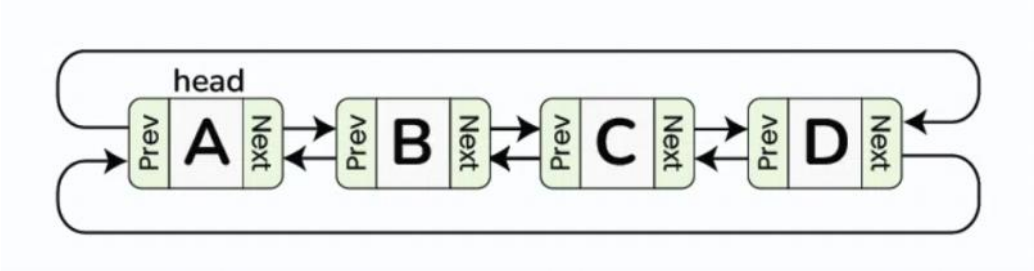
- **Data:** stores the value
- **Next:** pointer to next node

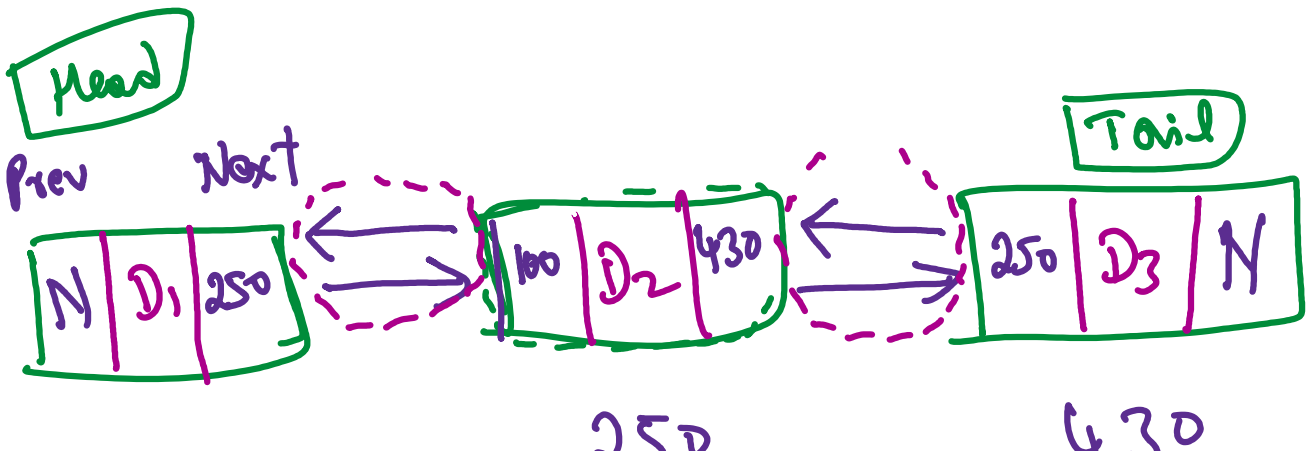
- Allows bidirectional traversal (forward & backward)

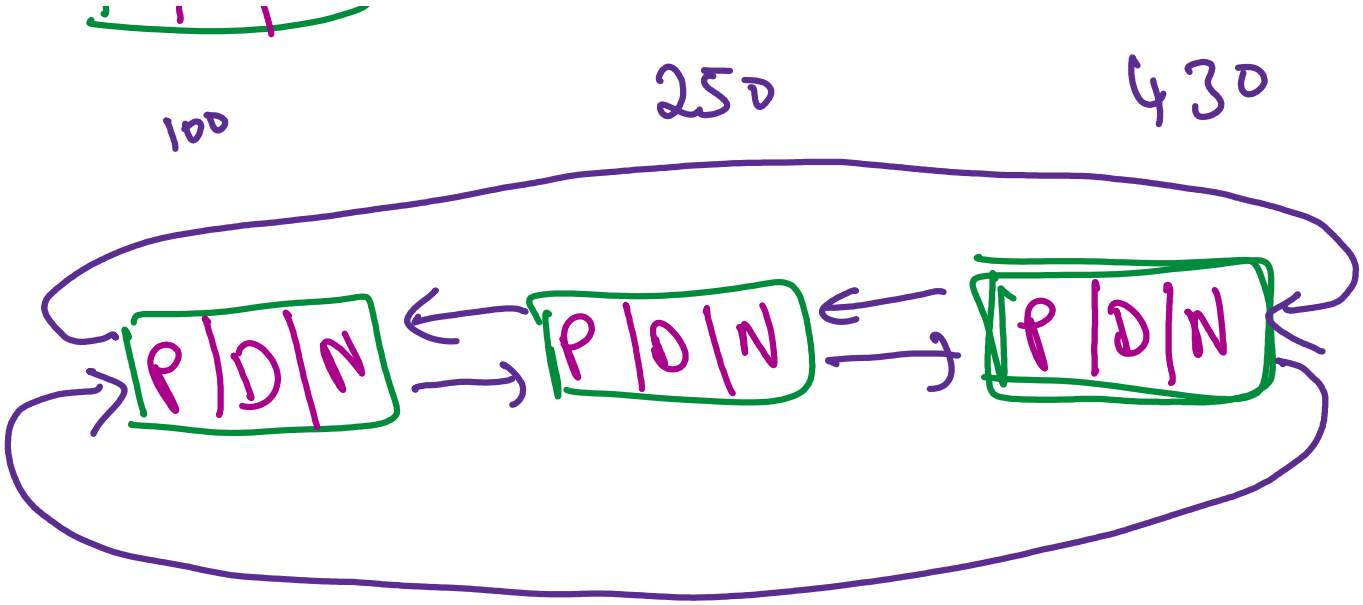


4. Circular Doubly Linked List:

- Combines features of circular + doubly linked lists
- Last node points back to **head**, and **head's Prev** points to last node
- Flexible traversal in both directions
- This is the most complex implementation



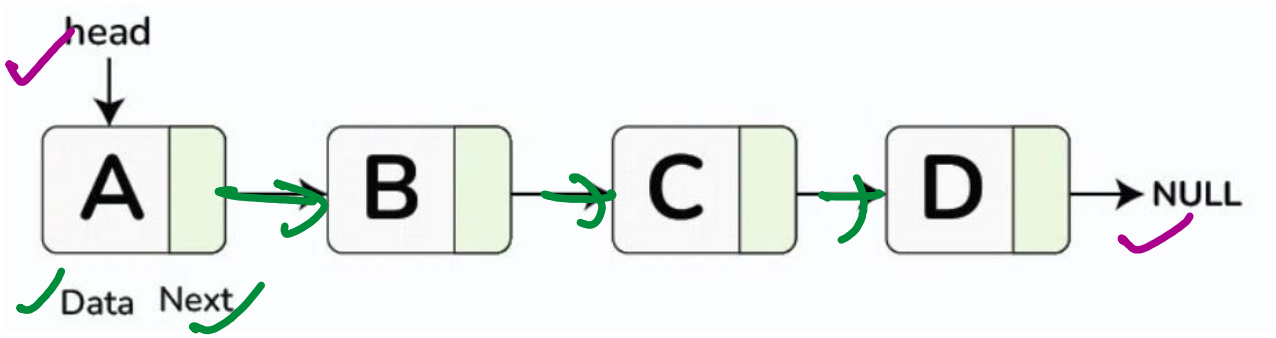



100
250
430
P D N
P D N
P D N

4.5 - Singly Linked List

Saturday, August 30, 2025 12:06 PM

- A **Singly Linked List** is the simplest form of a linked list
- Each node has two fields:
  - **Data:** stores the element
  - **Next:** pointer/reference to the next node
- The first node is called the **head**
- The last node's next points to **None**, marking the end of the list
- Traversal is only in one direction (forward)



Pros:

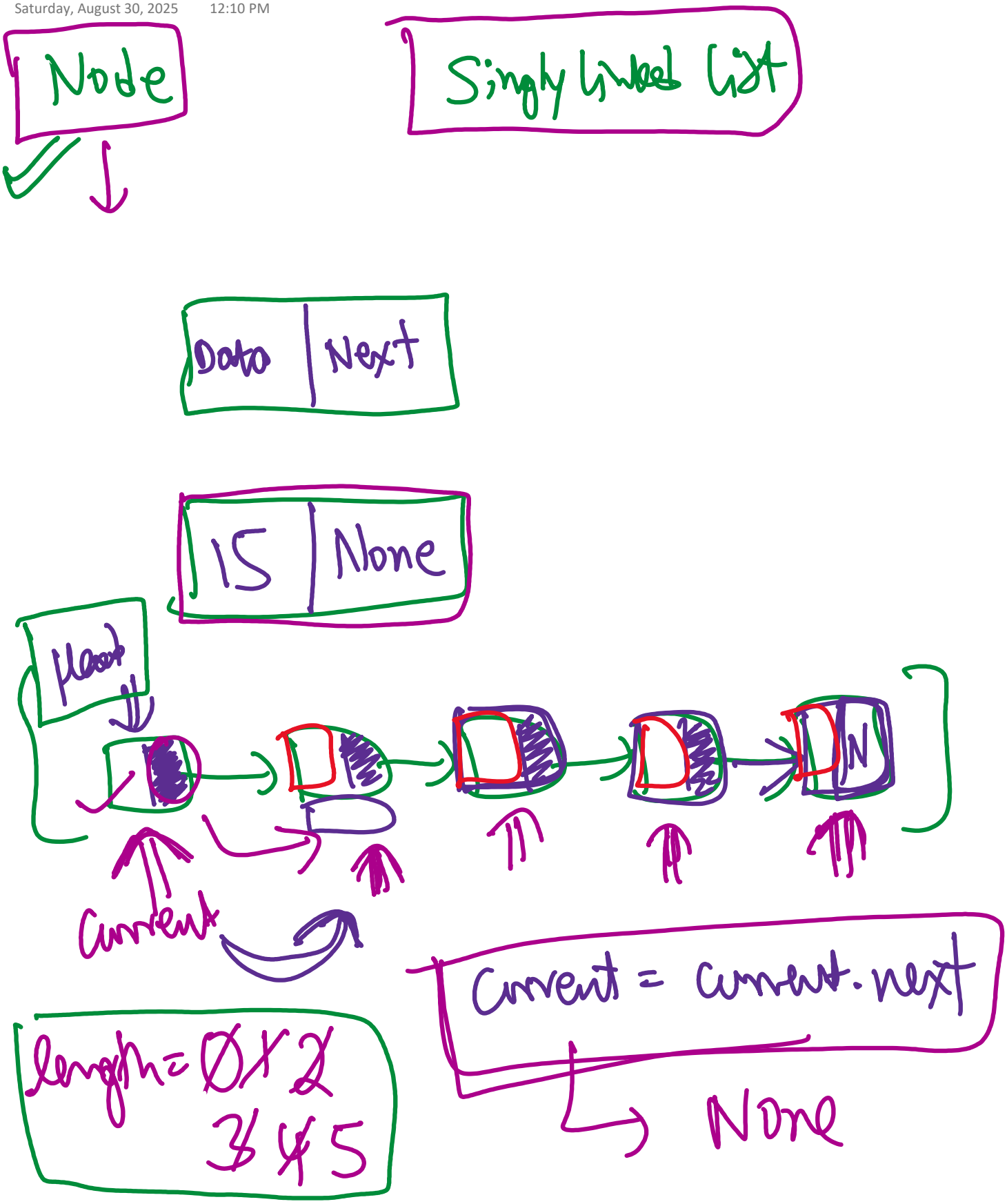
- Dynamic size (can grow/shrink easily)
- Efficient insertion/deletion at the beginning or middle (if pointer is known)

Cons:

- Cannot traverse backward
- Sequential access only
- Extra memory for pointer

4.5.1 - SLL Initial Setup

Saturday, August 30, 2025 12:10 PM



length -> 0

head -> None

current

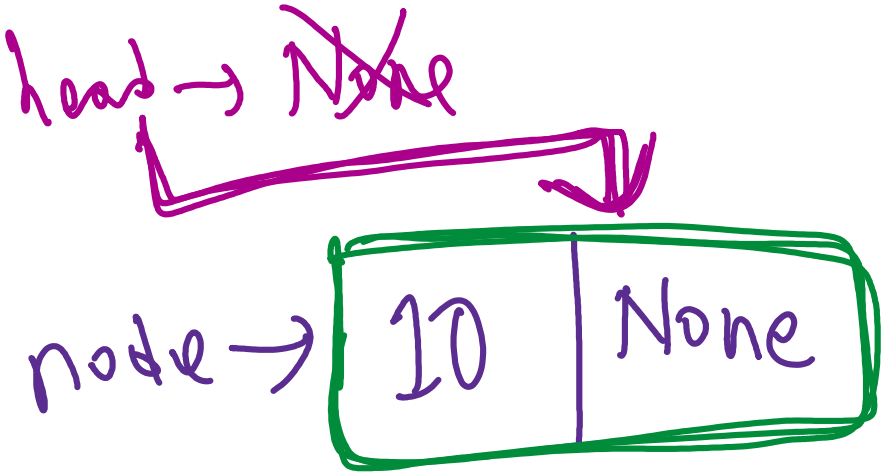
4.5.2 - SLL Insertion

Saturday, August 30, 2025 12:31 PM

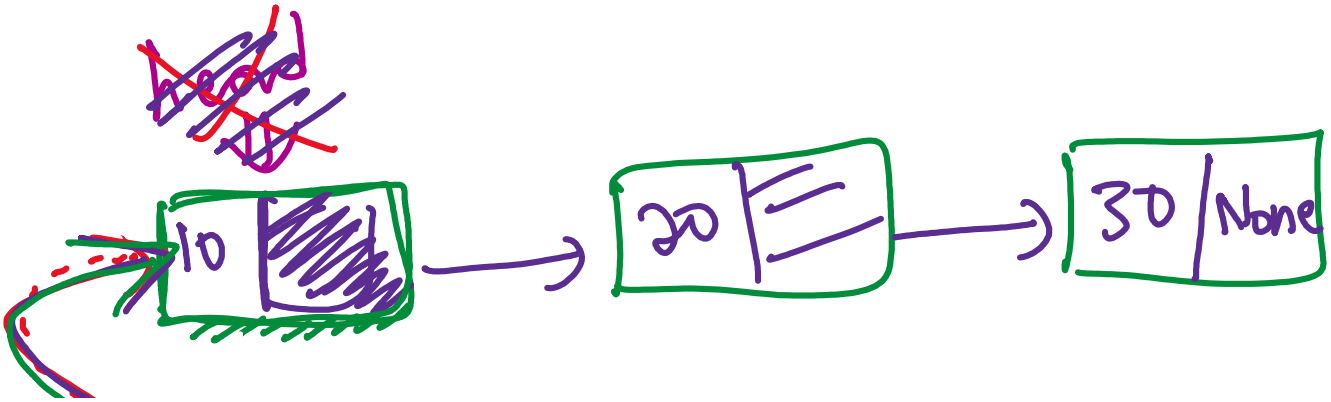
1. Insertion in SLL at beginning

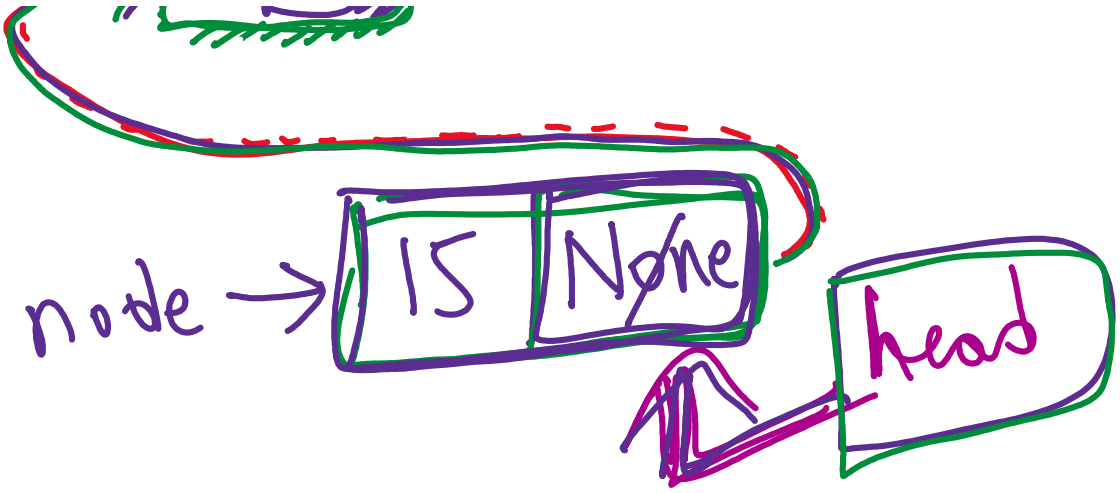
↳ list is empty

↳ list is not empty



head → node





node.next → head

head → node

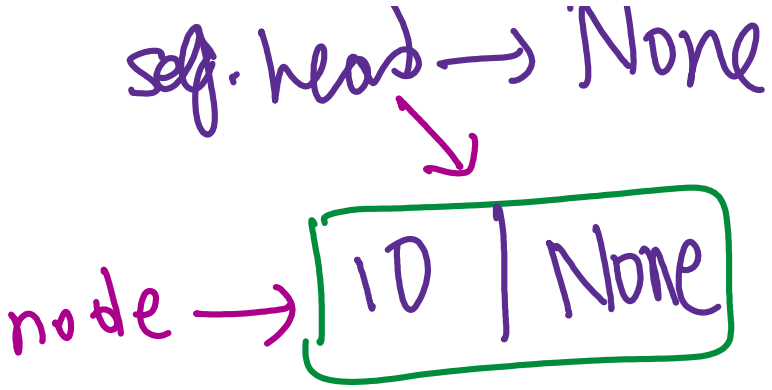
---

2. Insertion at End

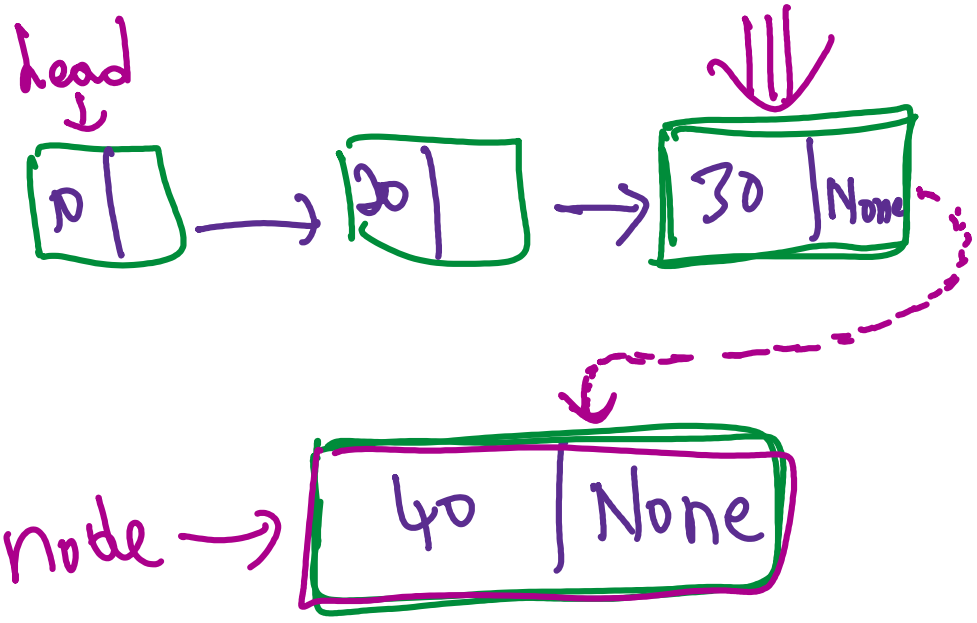
↳ if list empty

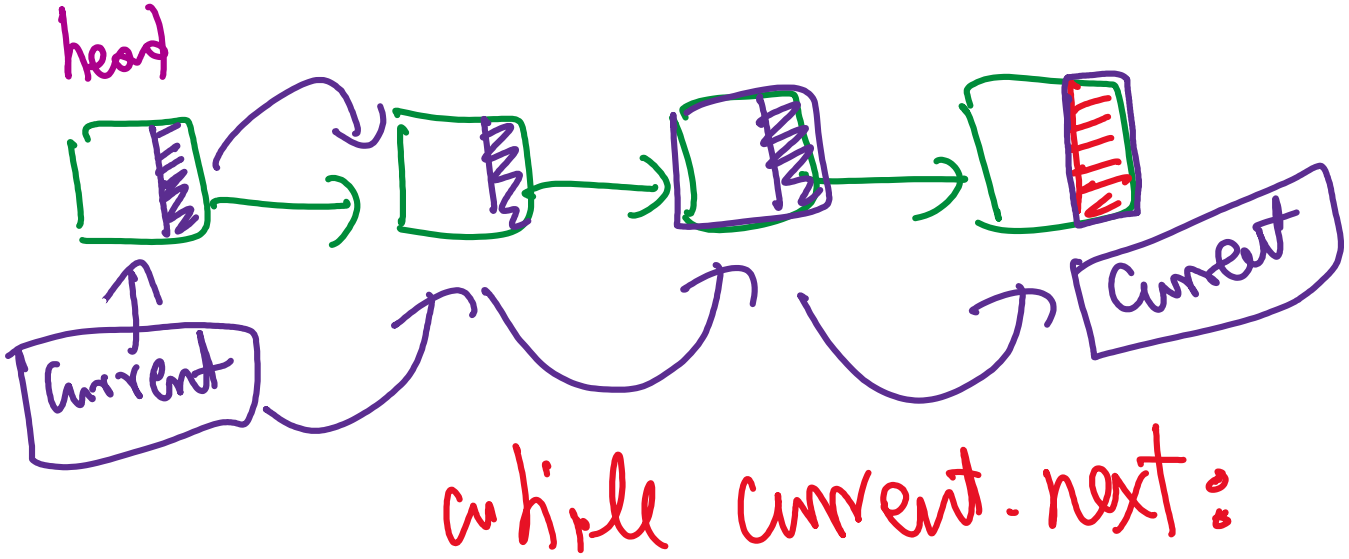
↳ if list not empty

self.head → None



head → node





while current.next:

while current.next:

current = current.next

current.next = node

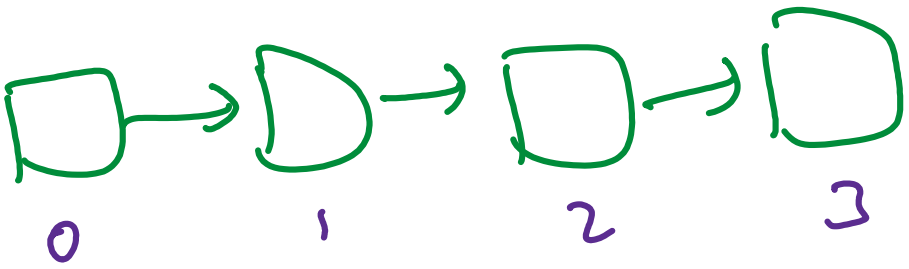
---

3. Insertion in the Middle

value

position (0-indexed)

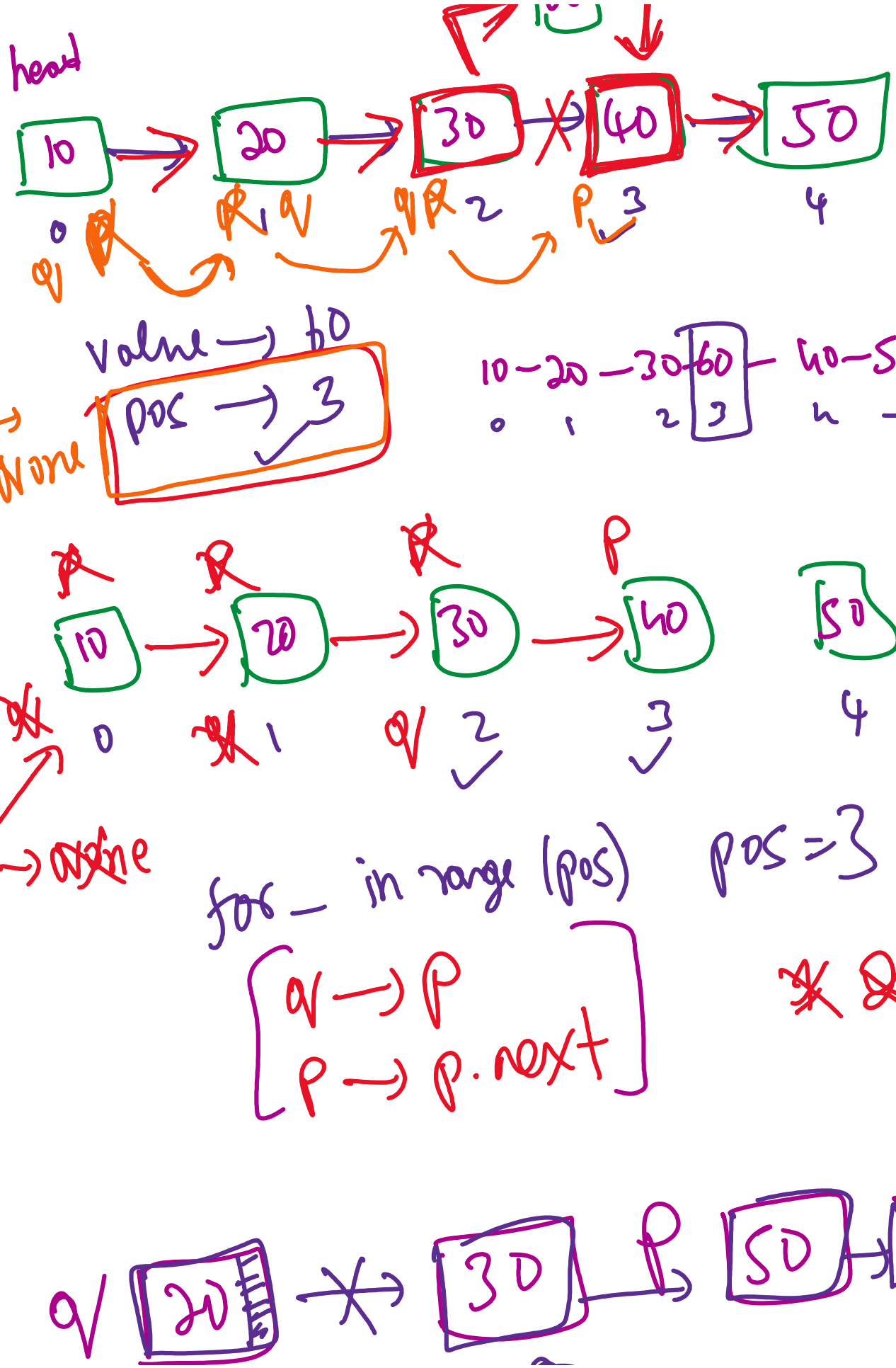
n = 4

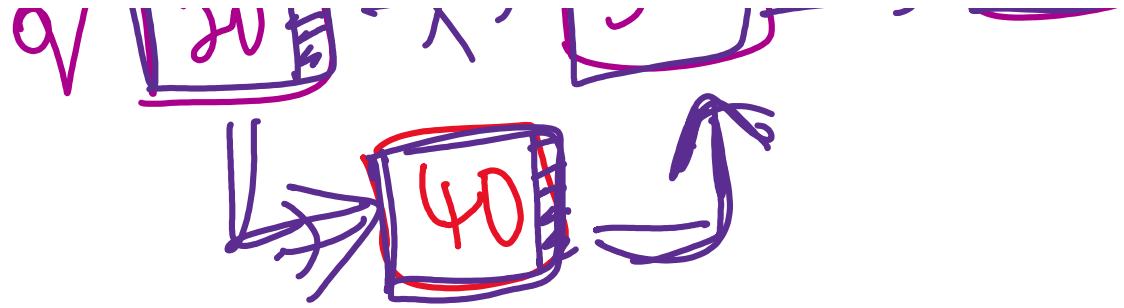


pos = 0, insertion at beginning

pos = n, insertion at end

head


head
10
20
30
40
50
0
1
2
3
4
p
q
value → 60
pos → 3
None
10-20-30-60-40-50
0 1 2 3 4 5
p
10
20
30
40
50
0
1
2
3
4
q → None
for _ in range (pos)
pos = 3
q → p
p → p.next
q
20
30
p
50
60

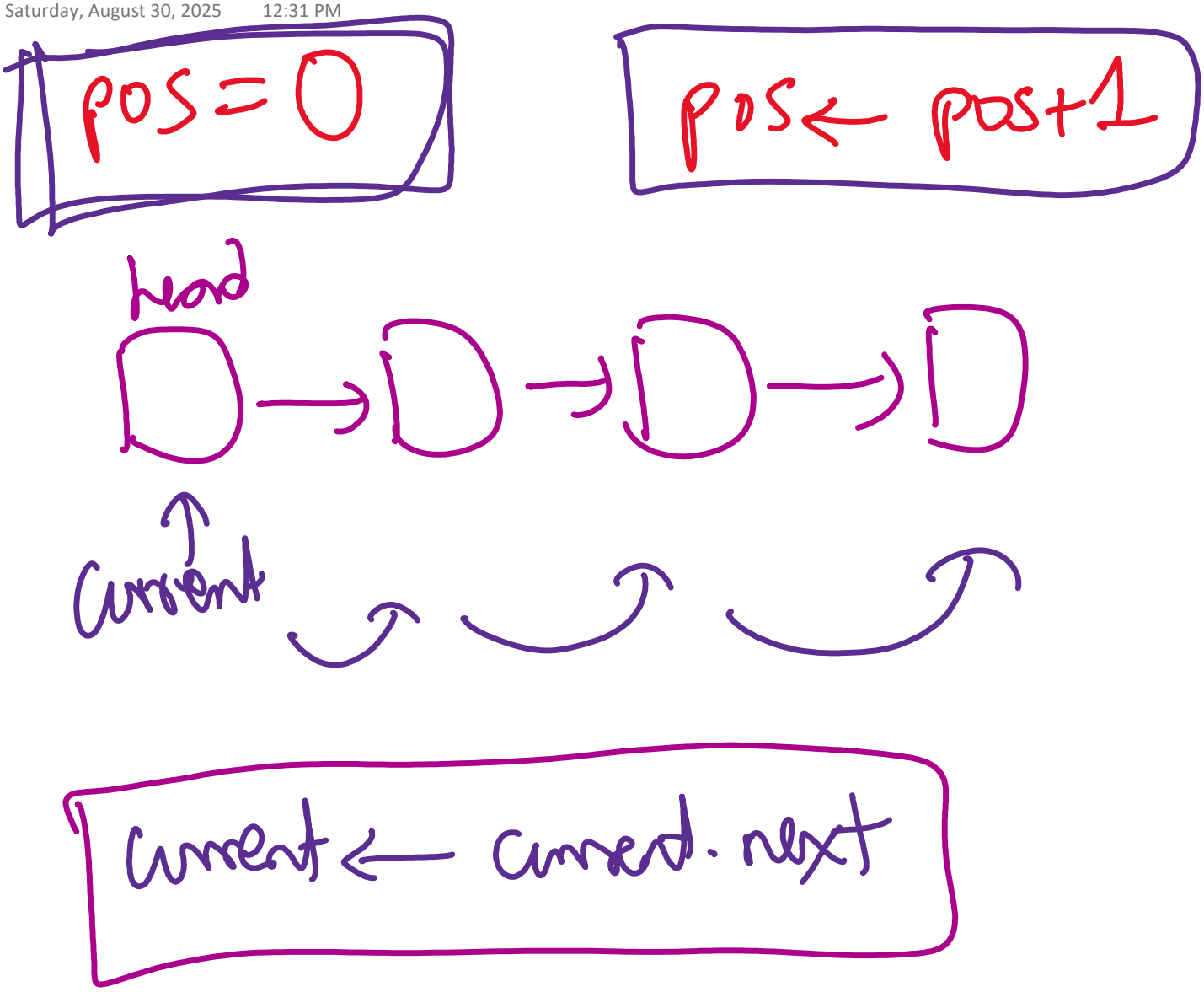


q.next → node

node.next → p

4.5.3 - SLL Search

Saturday, August 30, 2025 12:31 PM



4.5.4 - SLL Deletion

Saturday, August 30, 2025 12:31 PM

1. At begin
2. At end
3. At middle

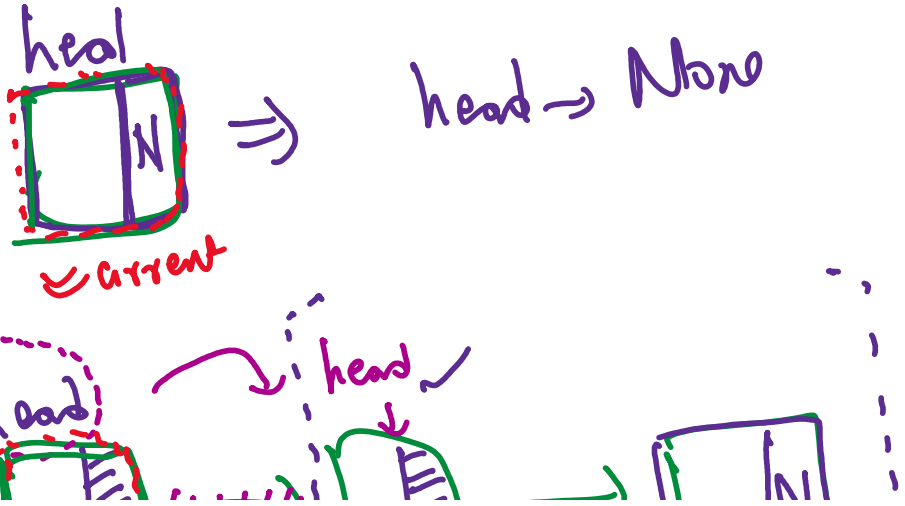
---

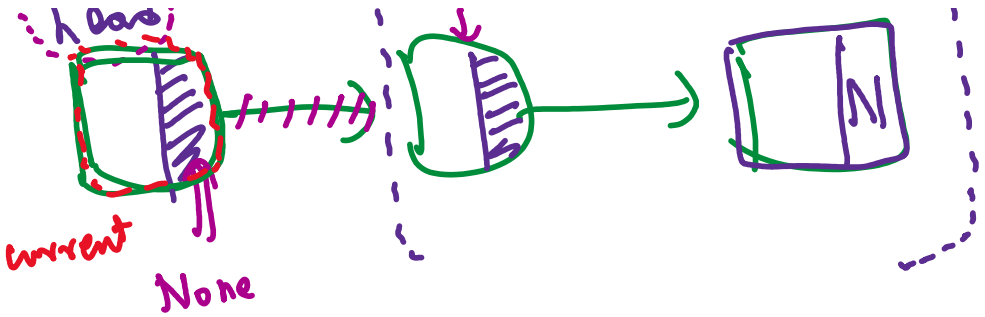
1. Beginning

i) List is empty

ii) Not empty

head → None





current ← head

head ← current.next

current.next ← None

---

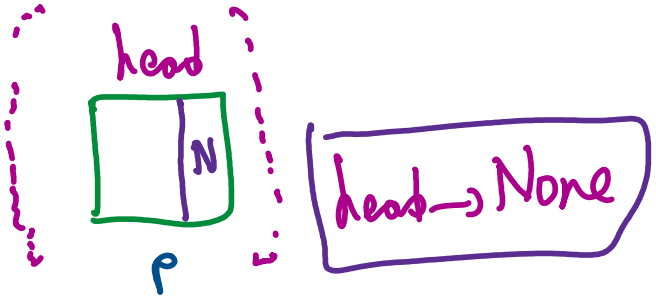
2. At the end

i) Empty ⟶ print()

ii) Not Empty

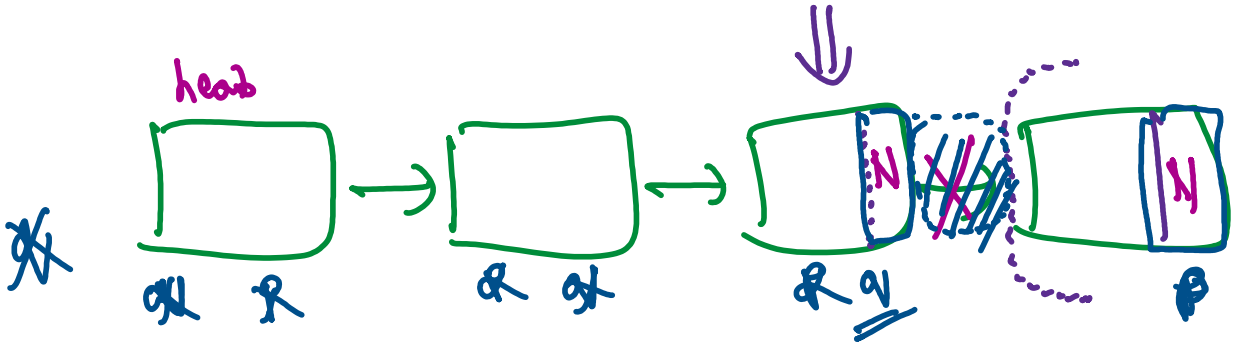
↓

Single Element



Multiple Elements

(next → None)



"next" attribute of 2nd last element

p → head

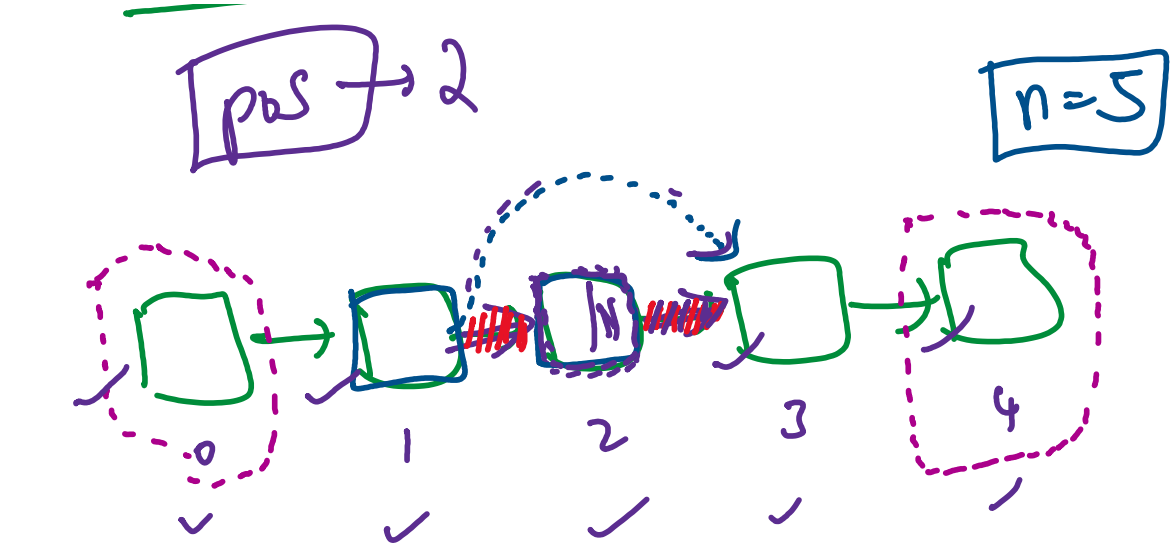
q → None

```
q → p
p → p.next
```
while p.next :

```
q.next = None
```

---

3. In the middle :



(pos < 0) or (pos ≥ n)

↳ Invalid Position

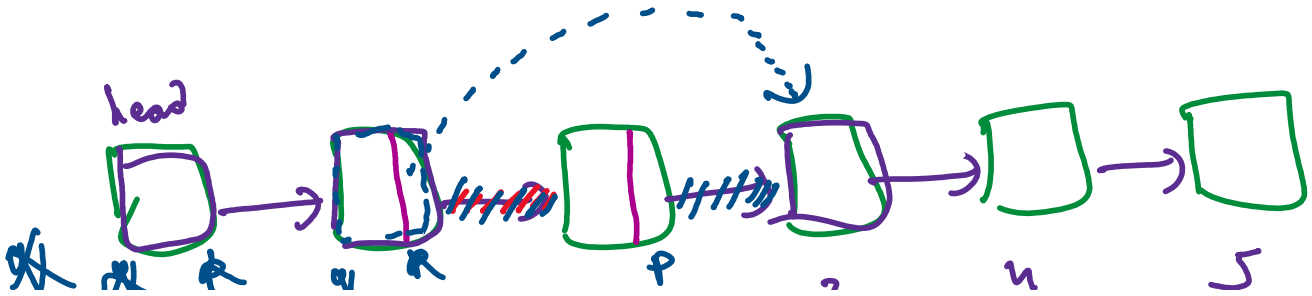
pos → 0

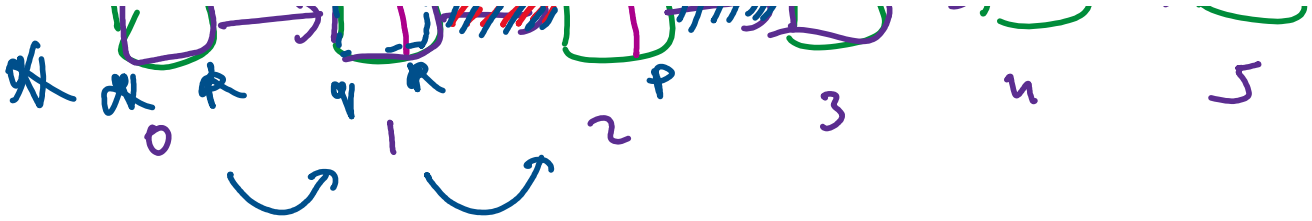
↳ delete_at_begin ()

pos → n-1

↳ delete_at_end ()

else:





pos ← 2

p → head, q = None

for range(pos):

[q → p
  p → p.next]

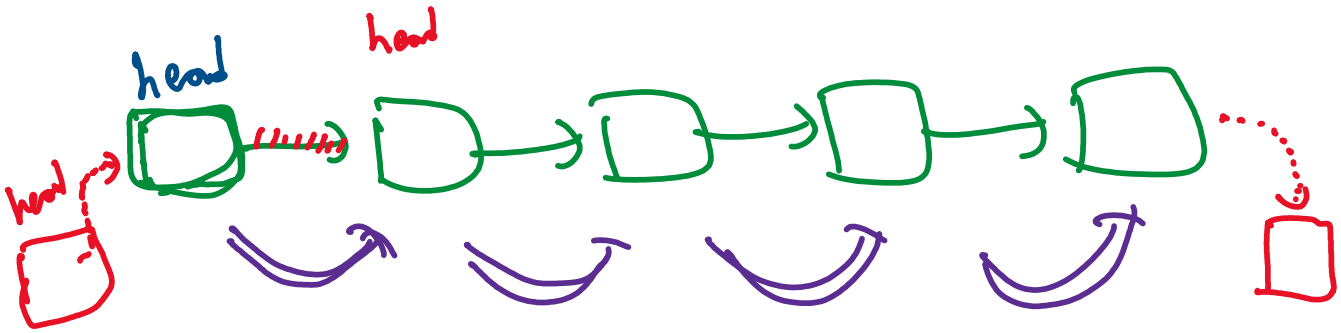
range(2)
↳ 0, 1

[q.next → p.next
  p.next → None]

4.5.5 - SLL Complexity Analysis

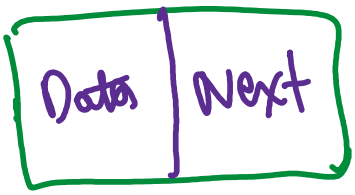
Saturday, August 30, 2025 12:32 PM

OPERATION	WORST CASE	REASONING
Traversal	O(n)	Must visit each node once
Search	O(n)	May need to scan the entire list
Insert at Beginning	O(1)	Just adjust **head** pointer.
Insert at End	O(n)	Need to traverse till last node
Insert at Middle	O(n)	Need to traverse to position
Delete at Beginning	O(1)	Just move **head** to next node
Delete at End	O(n)	Need to traverse till last node
Delete at Middle	O(n)	Traverse till position, then unlink node
Space per Node	O(1)	Each node stores data + one pointer (**next**)
Overall Space Usage	O(n)	Needs memory for all nodes



5 → 4

n → n-1 ⇒ O(n)



Node → 2 units

List → "n" nodes

→ 2n

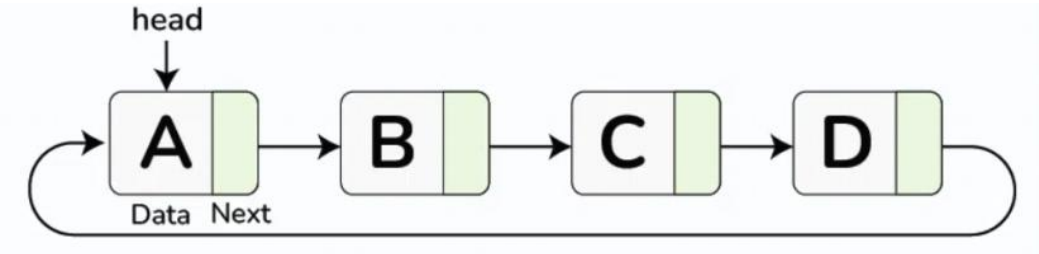
→ n

→ O(n)

4.6 - Circular Singly Linked List

Tuesday, September 9, 2025 9:27 AM

- A **Circular Singly Linked List** is a variation of a singly linked list
- Each node has two fields:
  - **Data:** stores the element
  - **Next:** pointer/reference to the next node
- The first node is called the **head**
- Unlike a normal singly linked list, the last node's **next** points back to the **head** node, **forming a circular structure**
- Traversal is only in one direction (forward), but since it is circular, starting from any node allows you to reach all other nodes

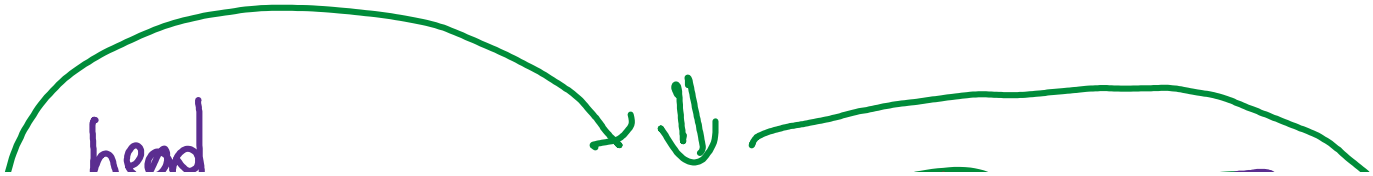


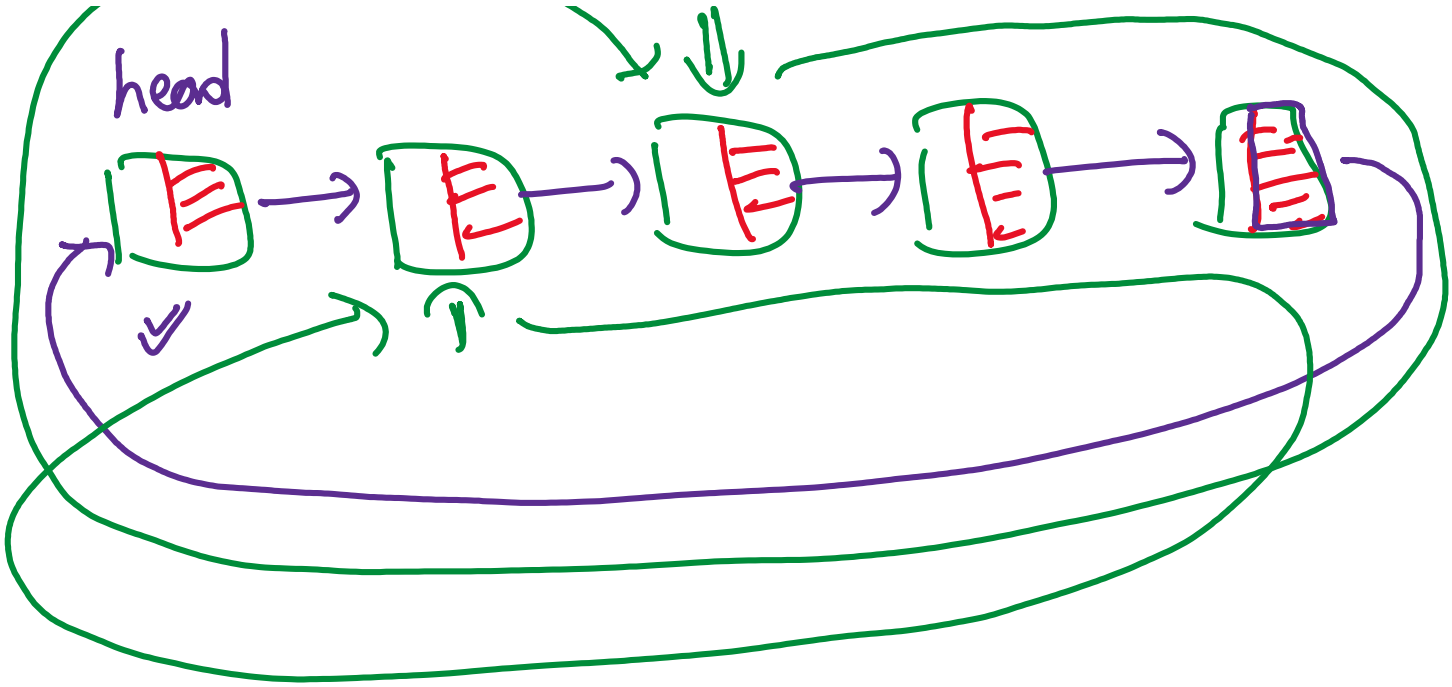
Pros:

- Efficient for circular traversal (e.g., implementing round-robin scheduling)
- Can start traversal from any node and still access the entire list
- No **None** at the end, so memory usage is more consistent

Cons:

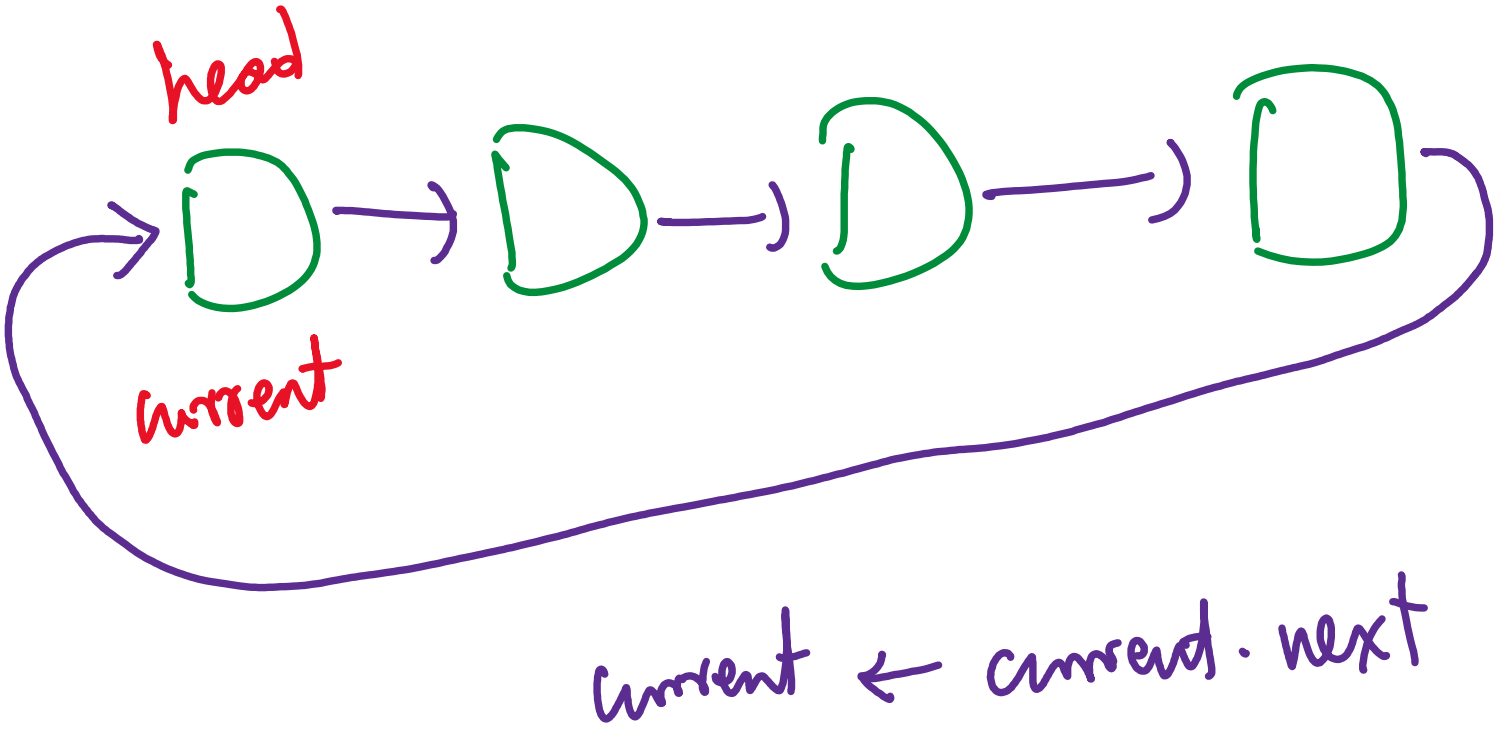
- More complex insertion and deletion logic compared to a simple singly linked list
- Risk of infinite loops if traversal stopping condition isn't handled properly
- Slightly harder to debug and implement than a standard singly linked list




head

4.6.1 - CSLL Initial Setup

Tuesday, September 9, 2025 9:34 AM

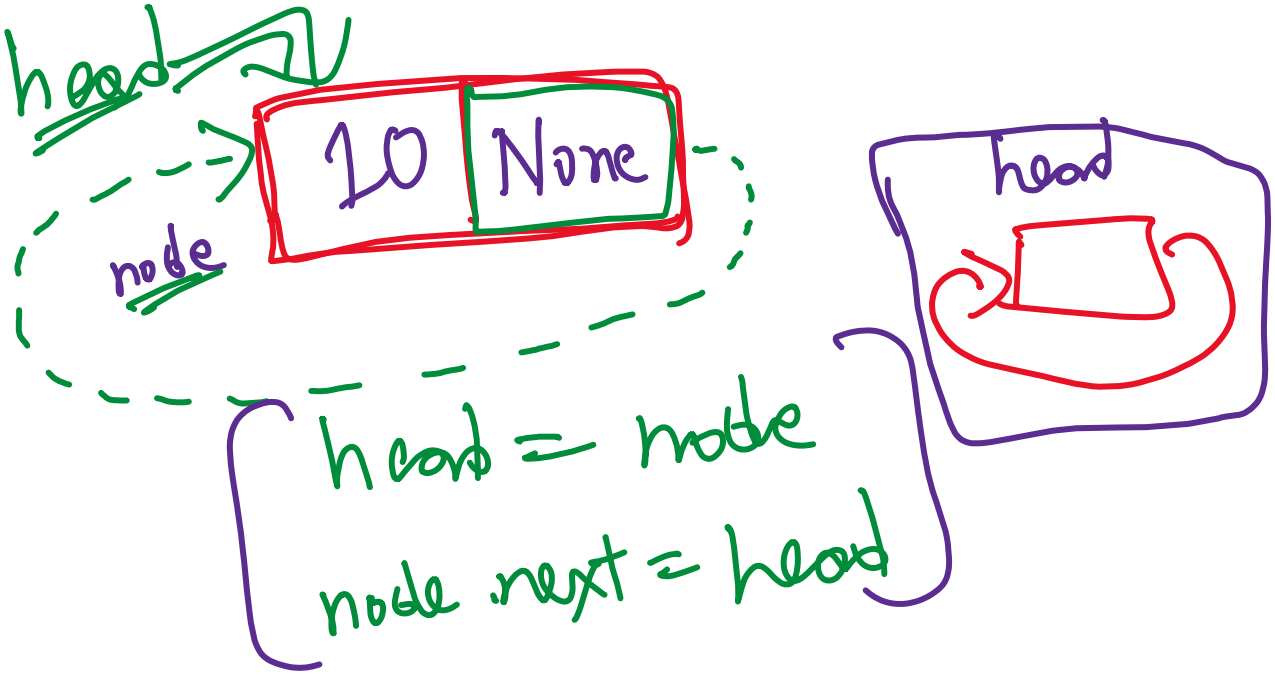


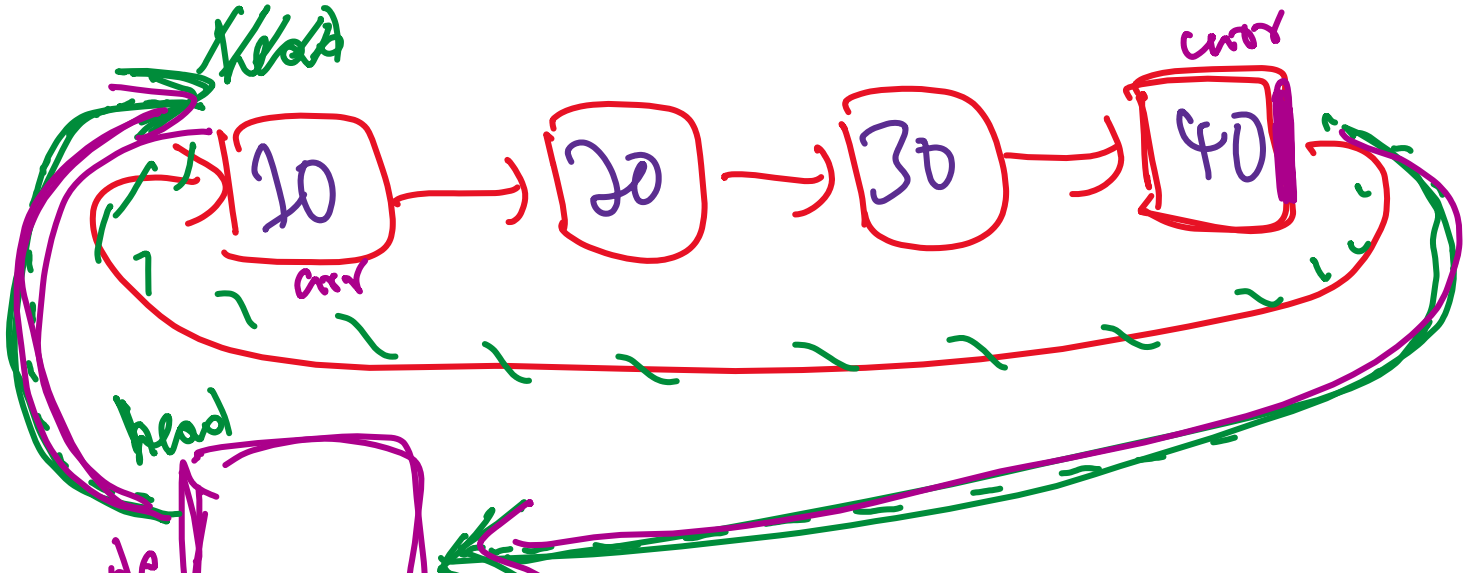
4.6.2 - CSLL Insertion

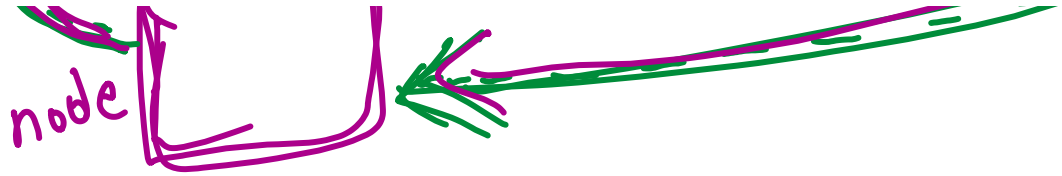
Tuesday, September 9, 2025 9:34 AM

1. Insertion at Beginning

head → None



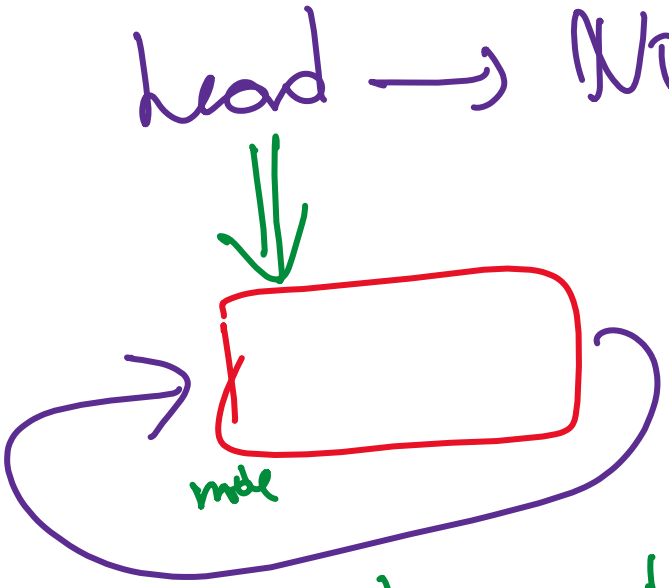





```
current.next = node
node.next = head
head = node
```

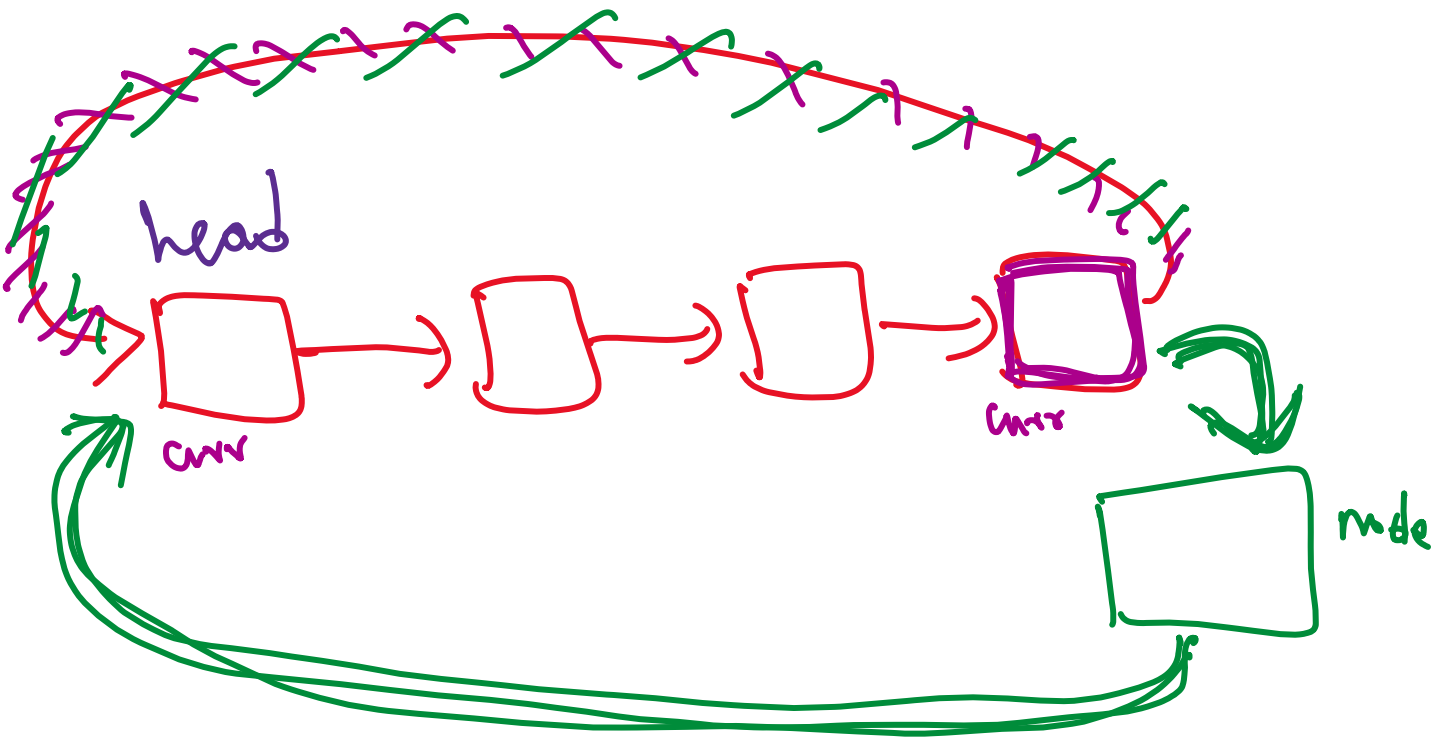
---

2. Insertion at End




```
head → node
node.next → head
```

node.next → head

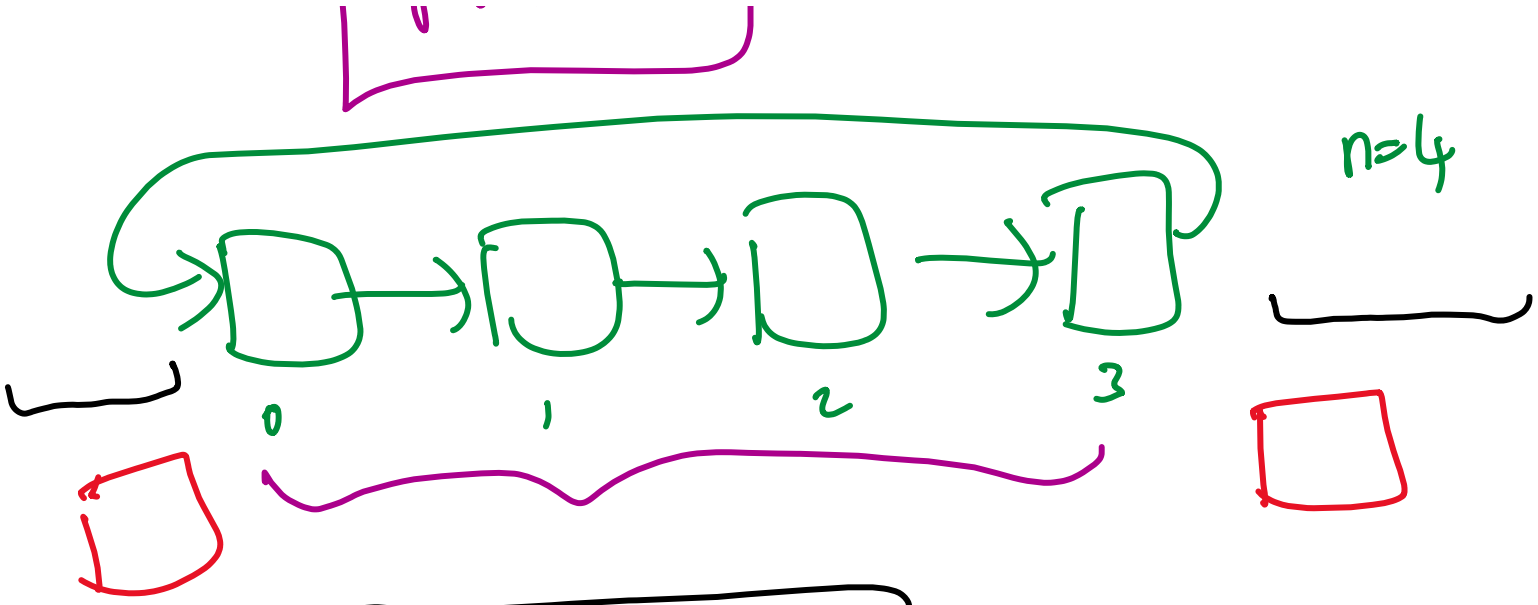


current.next = node
node.next = head

---

3. Insertion in Middle

value
position



$0 \leq pos \leq n$

1. (pos < 0) or (pos > n)
   ↳ Invalid index

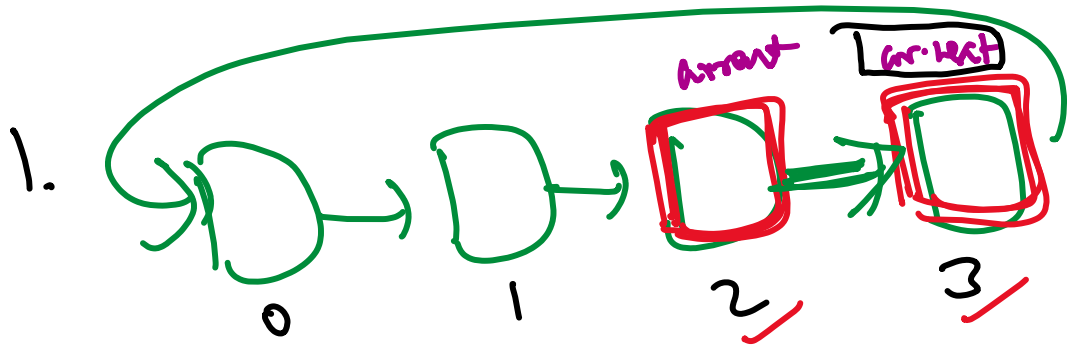
2. pos = 0
   ↳ insert_beg

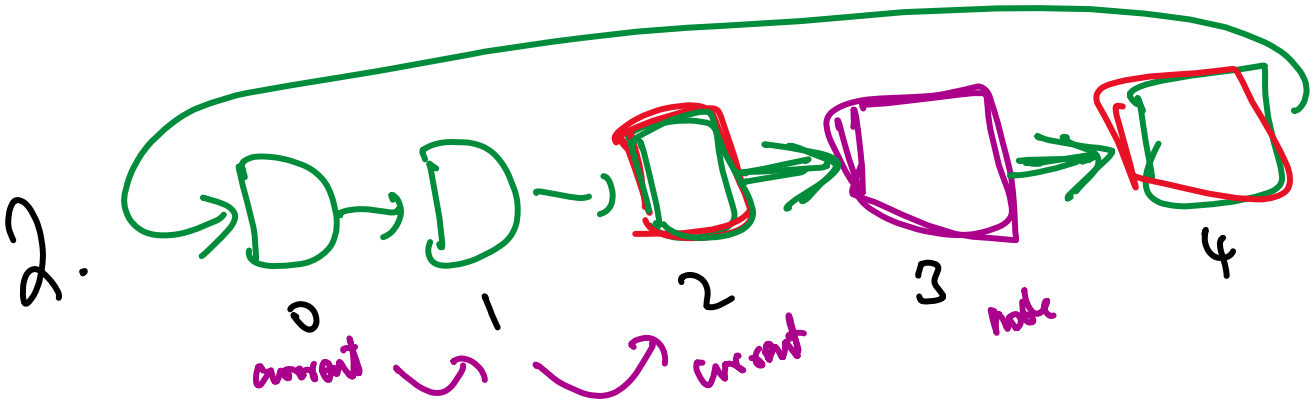
3. pos = n
   ↳ insert_end

4. else:

4. else:

pos=3





pos, pos-1

current → pos-1

current.next → pos

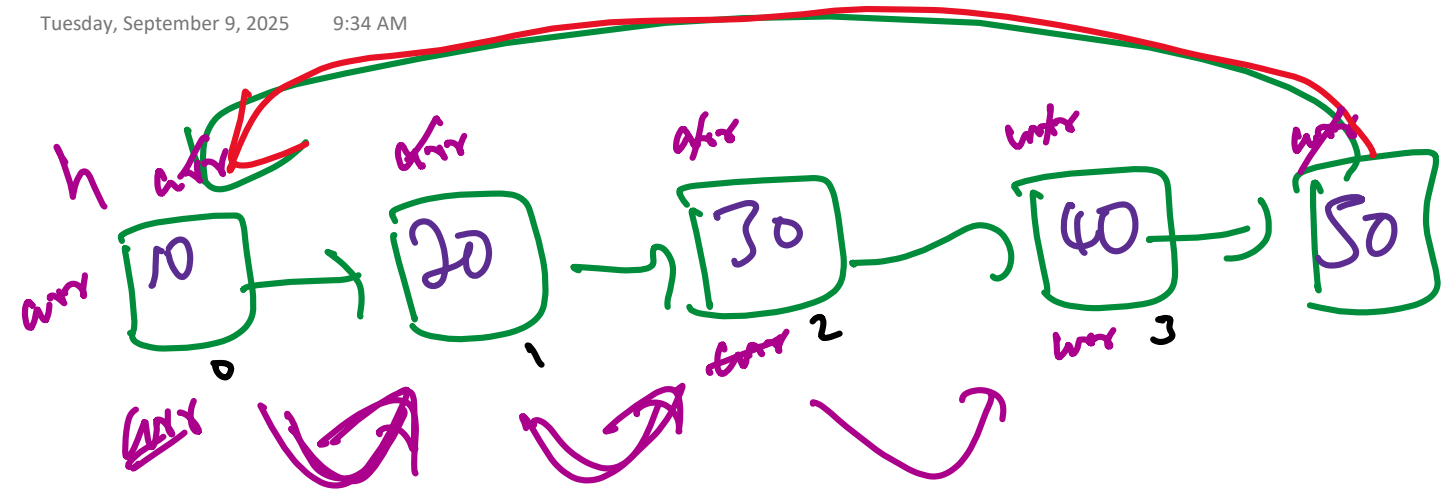
range(pos-1)

node.next = current.next

current.next = node

4.6.3 - CSLL Search

Tuesday, September 9, 2025 9:34 AM



pos = 0
+1

pos → ~~0~~ ~~1~~ ~~2~~ 3

key → 40

pos → ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ 5

key → 80

4.6.4 - CSLL Deletion

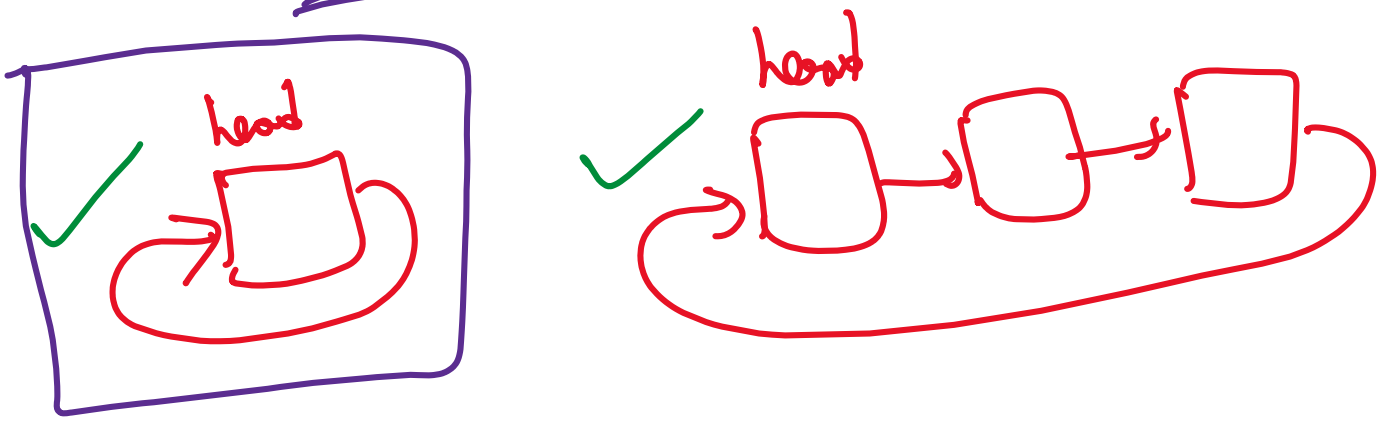
Tuesday, September 9, 2025 9:34 AM

1. Beginning

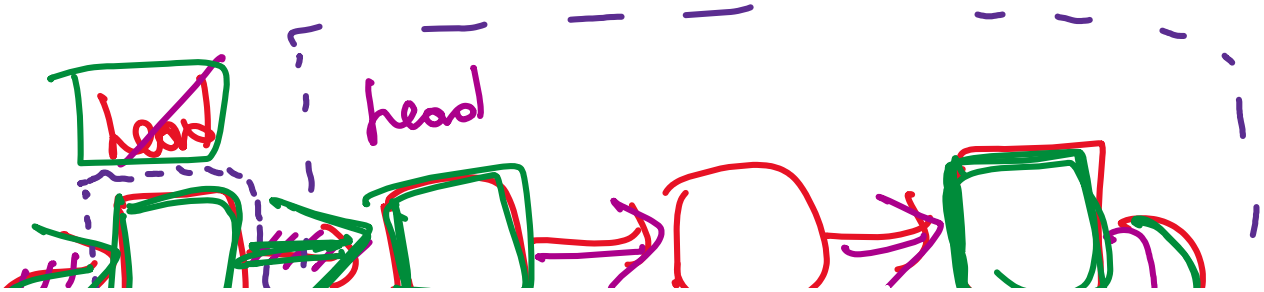
i) Empty

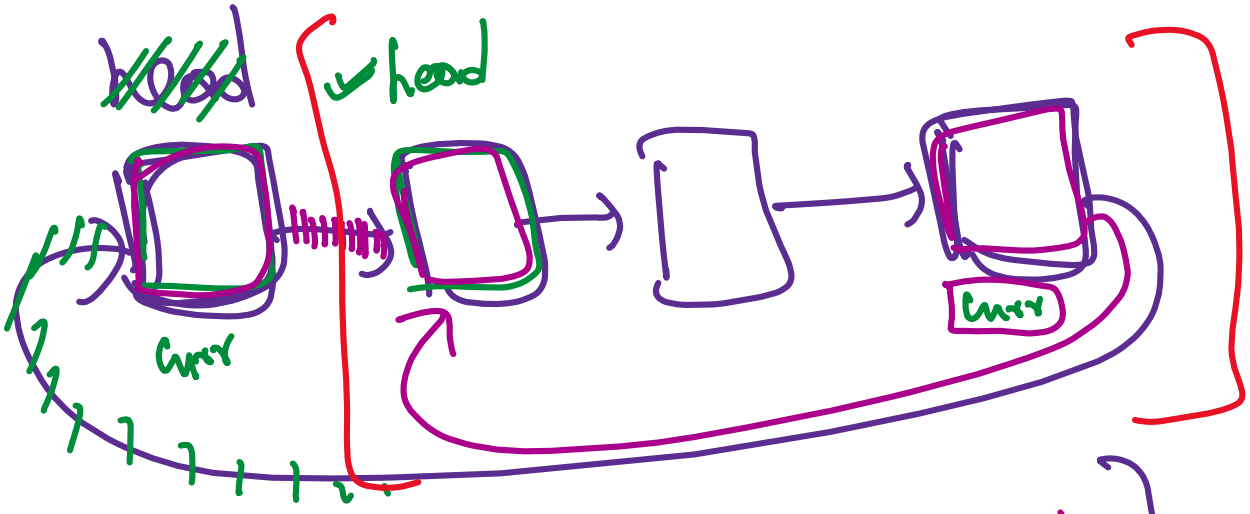
↳ cannot delete

ii) Not Empty



↳ head → None





while curr.next != head
curr = curr.next

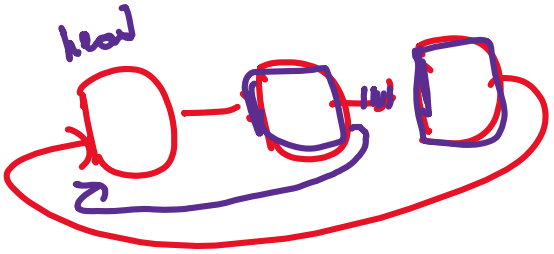
head → head.next
curr.next → head

length = length - 1

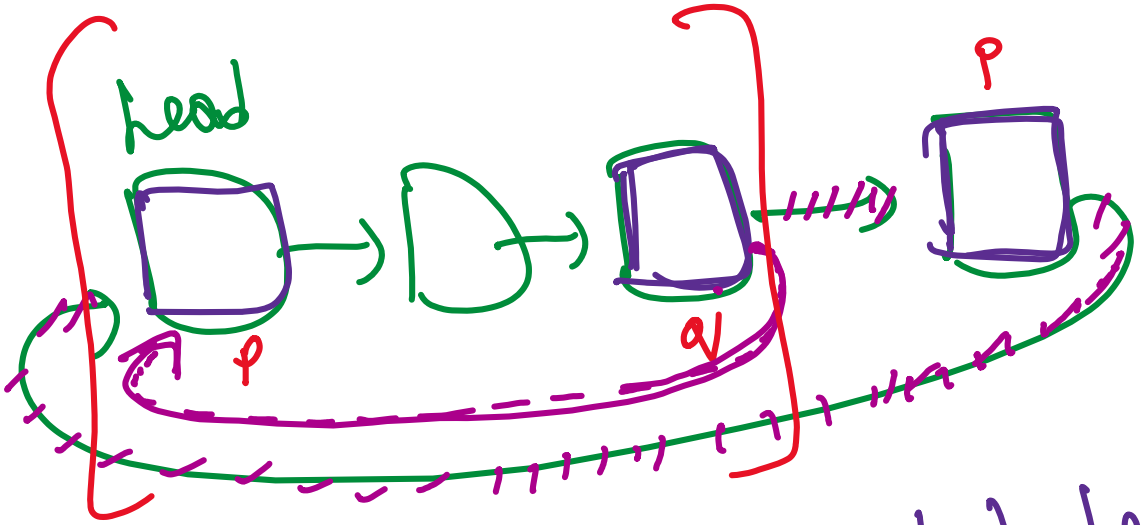
2. End

i) Single

ii) Multiple



head → None



q → None

while p.next != head:

q = p

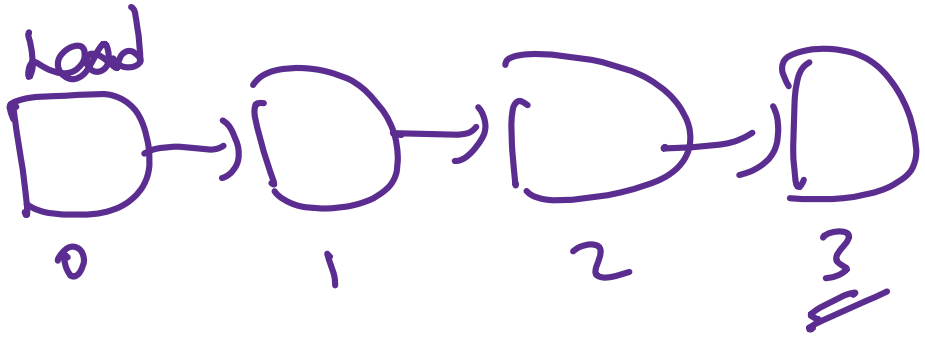
p = p.next

p2.next

q.next → head

p.next → None

3. Middle:



n=4

$0 \leq pos < n$

pos=0 → beginning

pos = n-1 → end

↳ middle

pos=2

range(pos)

range(pos)

C -> B.next

A, B

loop over range(pos)

q = p

p = p.next

q.next → p.next

p.next → None

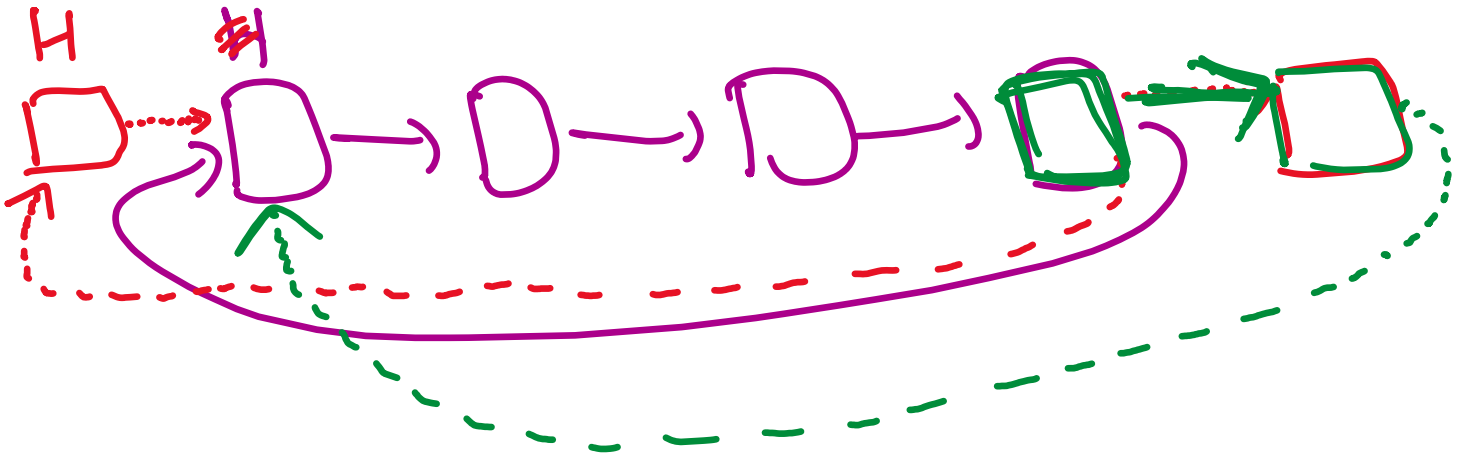
4.6.5 - CSLL Complexity Analysis

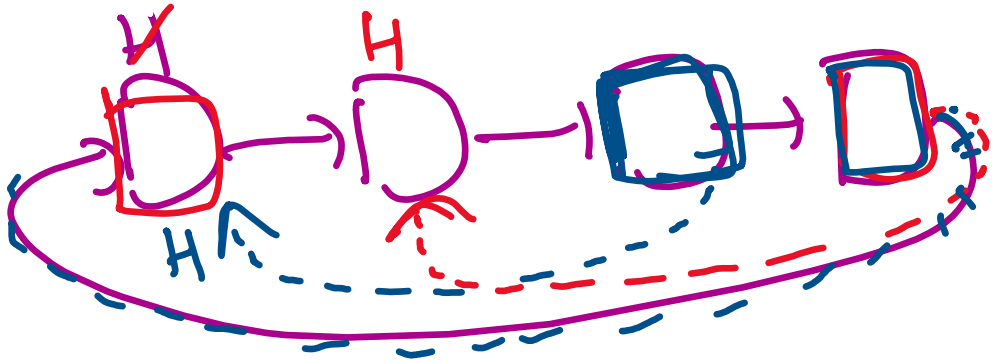
Tuesday, September 9, 2025 9:34 AM

OPERATION
Traversal
Search
Insert at Beginning
Insert at End
Insert at Middle
Delete at Beginning
Delete at End
Delete at Middle
Space per Node
Overall Space Usage

O(n)
O(n)
O(n)
O(n)
O(n)
O(n)
O(n)
O(n)
O(1)
O(n)

2xn
⇒ 2n
⇒ n
⇒ O(n)





Head

Tail

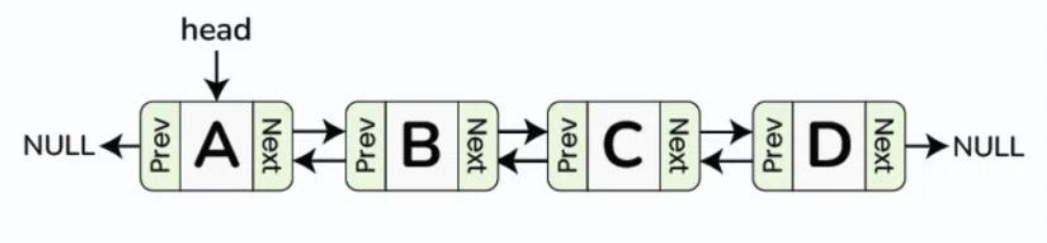
Head

Tail

4.7 - Doubly Linked List

Saturday, September 20, 2025 12:35 PM

- A **Doubly Linked List** is a type of linked list in which the nodes are linked in both directions
- Each node has three fields:
  - **Data:** stores the element
  - **Prev:** pointer/reference to the previous node
  - **Next:** pointer/reference to the next node
- The first node is called the **head** (its **prev** is None)
- The last node is called the **tail** (its **next** is None)
- Traversal is possible in both directions: forward (using **next**) and backward (using **prev**)

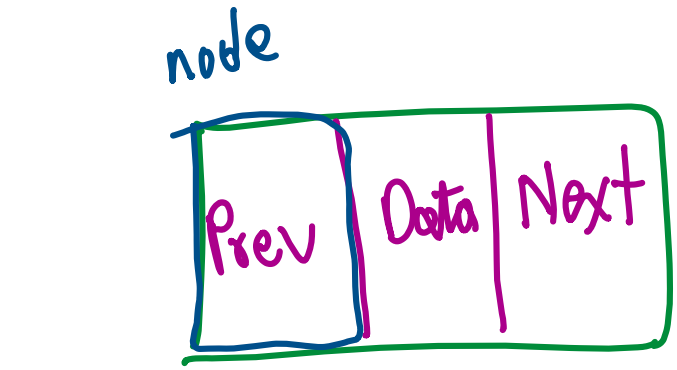


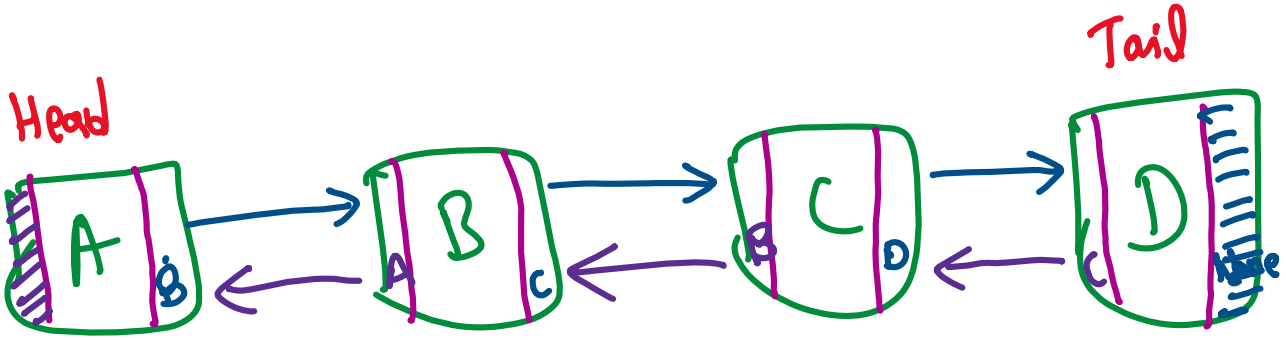
Pros:

- Allows bidirectional traversal
- Easier deletion and insertion at both ends (head and tail)
- Can implement more complex data structures like **deque** and **doubly-ended queues**

Cons:

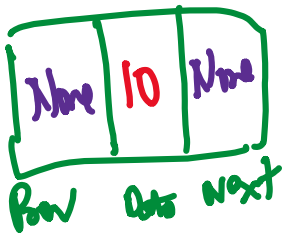
- Requires extra memory for storing the **prev** pointer
- More complex implementation compared to a singly linked list
- Insertion and deletion involve more pointer updates compared to singly linked lists
- Slower due to additional pointer handling
- Extra pointer can increase chances of pointer errors

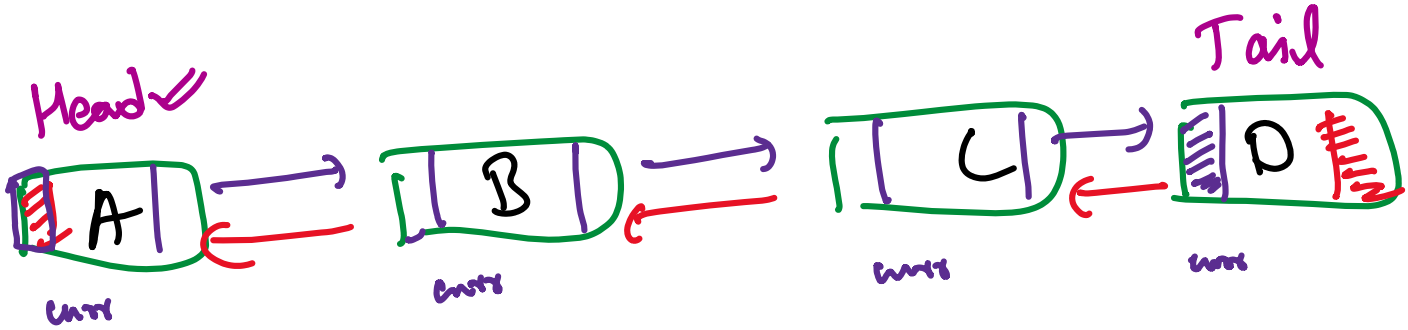

node
Prev
Data
Next


Head
Tail
A
B
B
A
B
C
C
B
D
D
C
Null

4.7.1 - DLL Initial Setup

Saturday, September 20, 2025 12:36 PM





curr → head

A B C D

curr → None

curr → tail

D C B A

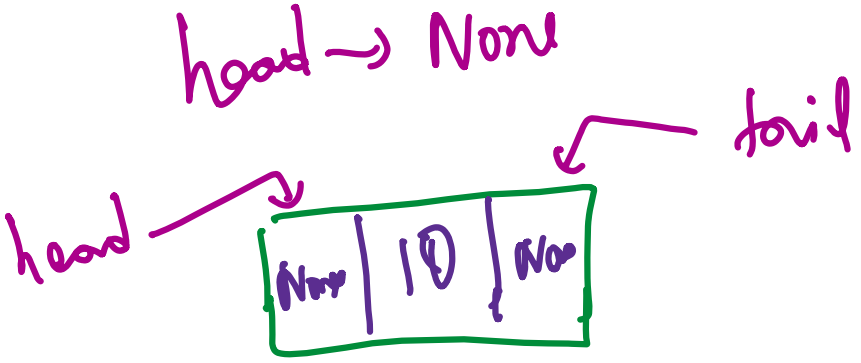
curr → curr.prev

4.7.2 - DLL Insertion

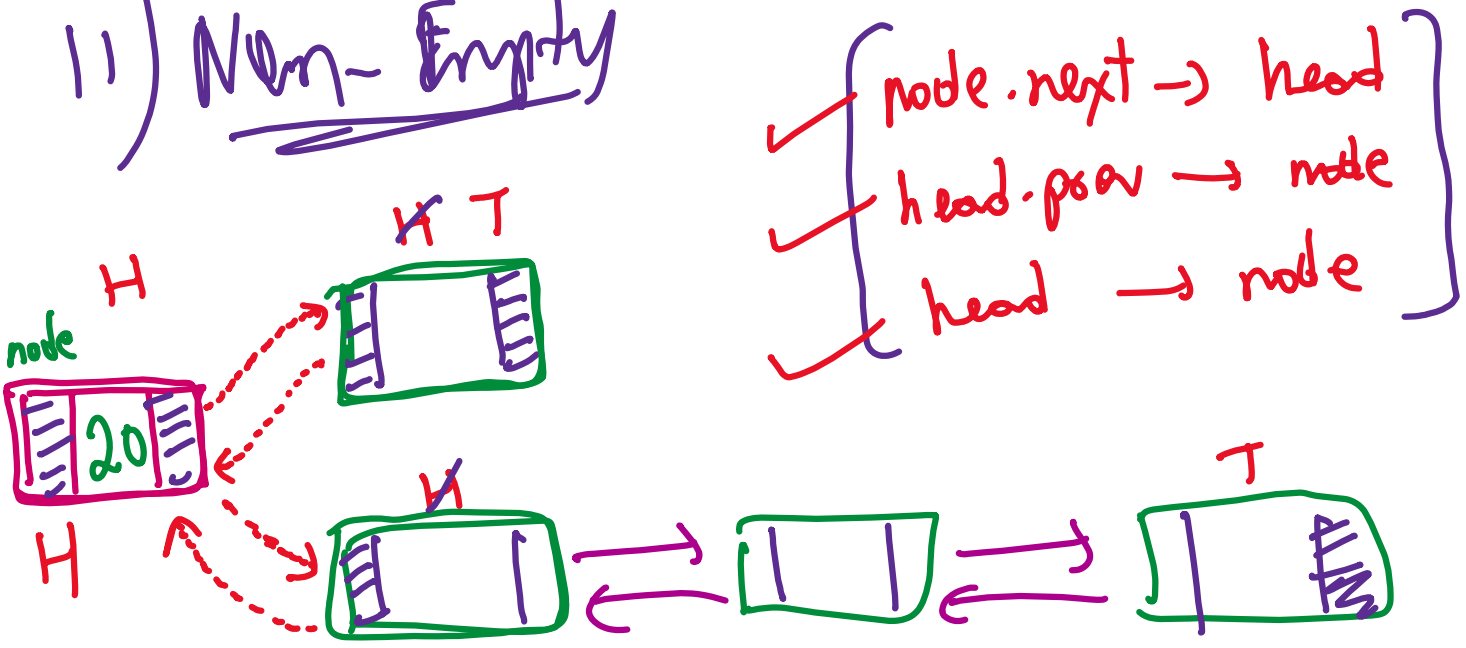
Saturday, September 20, 2025 12:36 PM

1. Beginning:

i) Empty



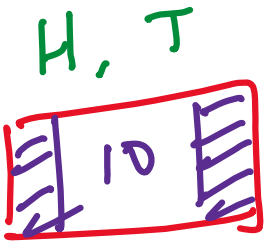
ii) Non-Empty



O(1)

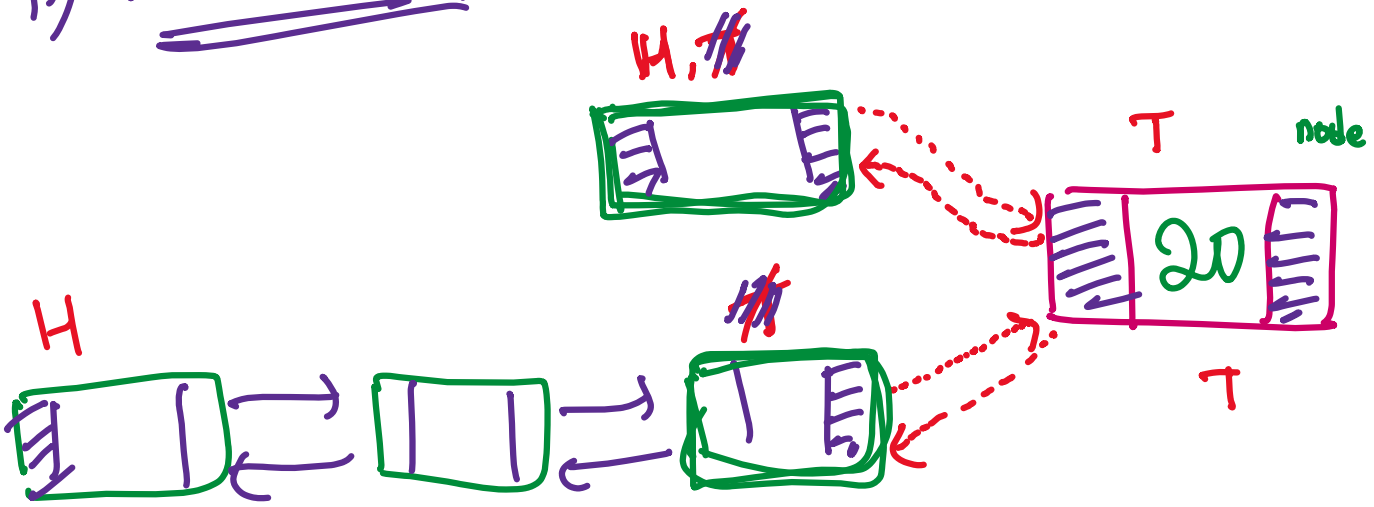
2. End:

i) Empty:




```
head → node
tail → node
```

ii) Non-Empty:



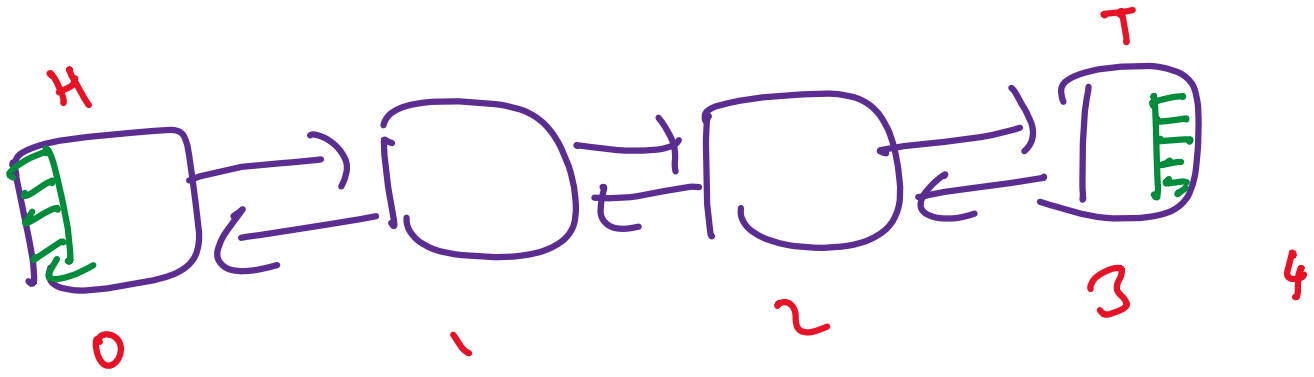

```
tail.next → node
node.prev → tail
```

node.prev → tail

tail → node

O(1)

3. Middle :



pos
data

$0 \le pos \le 3$

$n = 4$

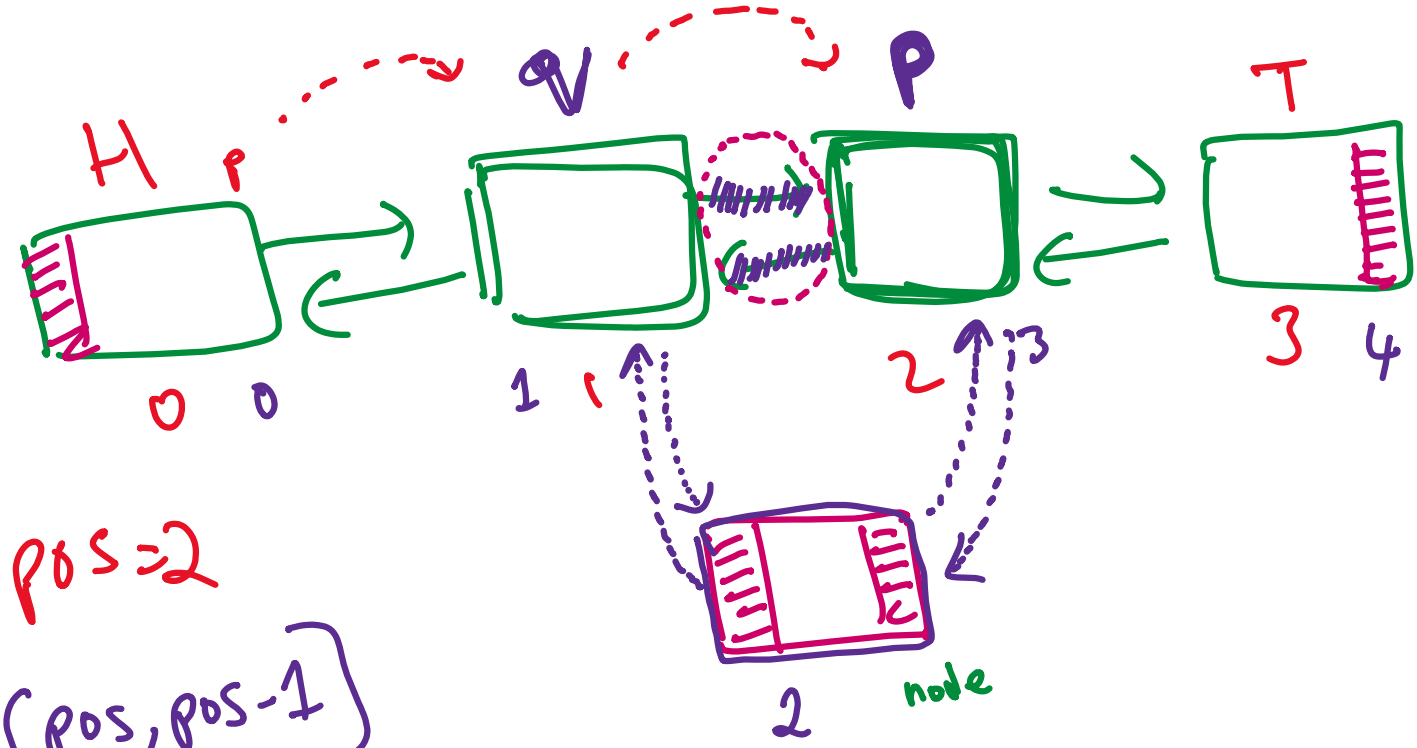
$0 \le pos \le n-1$

$(pos < 0), (pos > n)$

pos → 0

pos → n

pos ∈ (0, n)



pos=2

[pos, pos-1]

p = head

for range(pos):

p = p.next

p → p.next

q = p.prev ✓

q.next → node

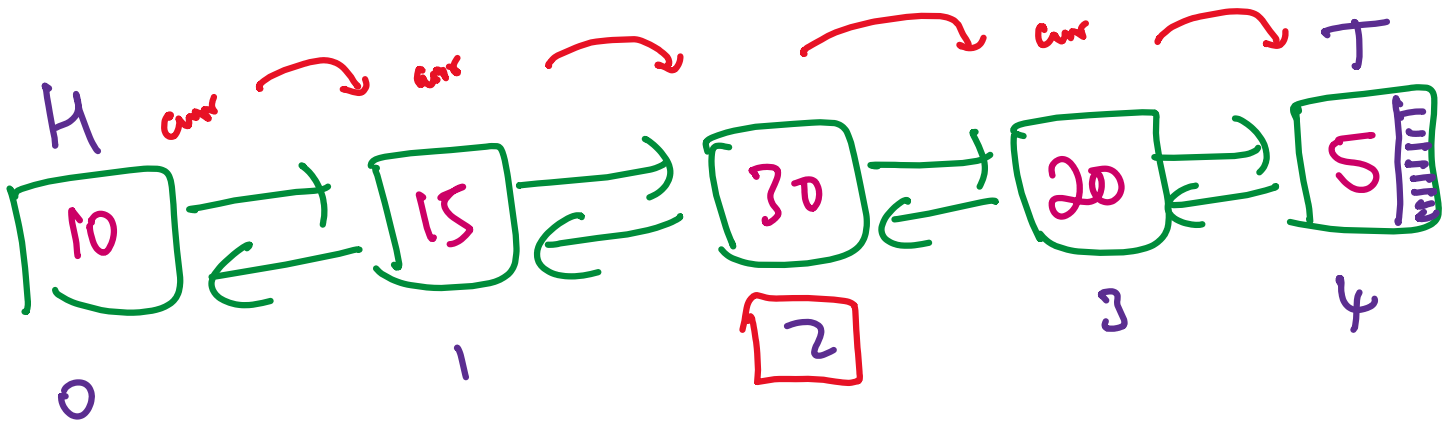
node.prev → q

node.next → p

p.prev → node

4.7.3 - DLL Search

Saturday, September 20, 2025 12:36 PM



list → empty ✓

key = 30

= 60

pos = 0

while current :

{ current.data == key

↳ pos

{ pos += 1

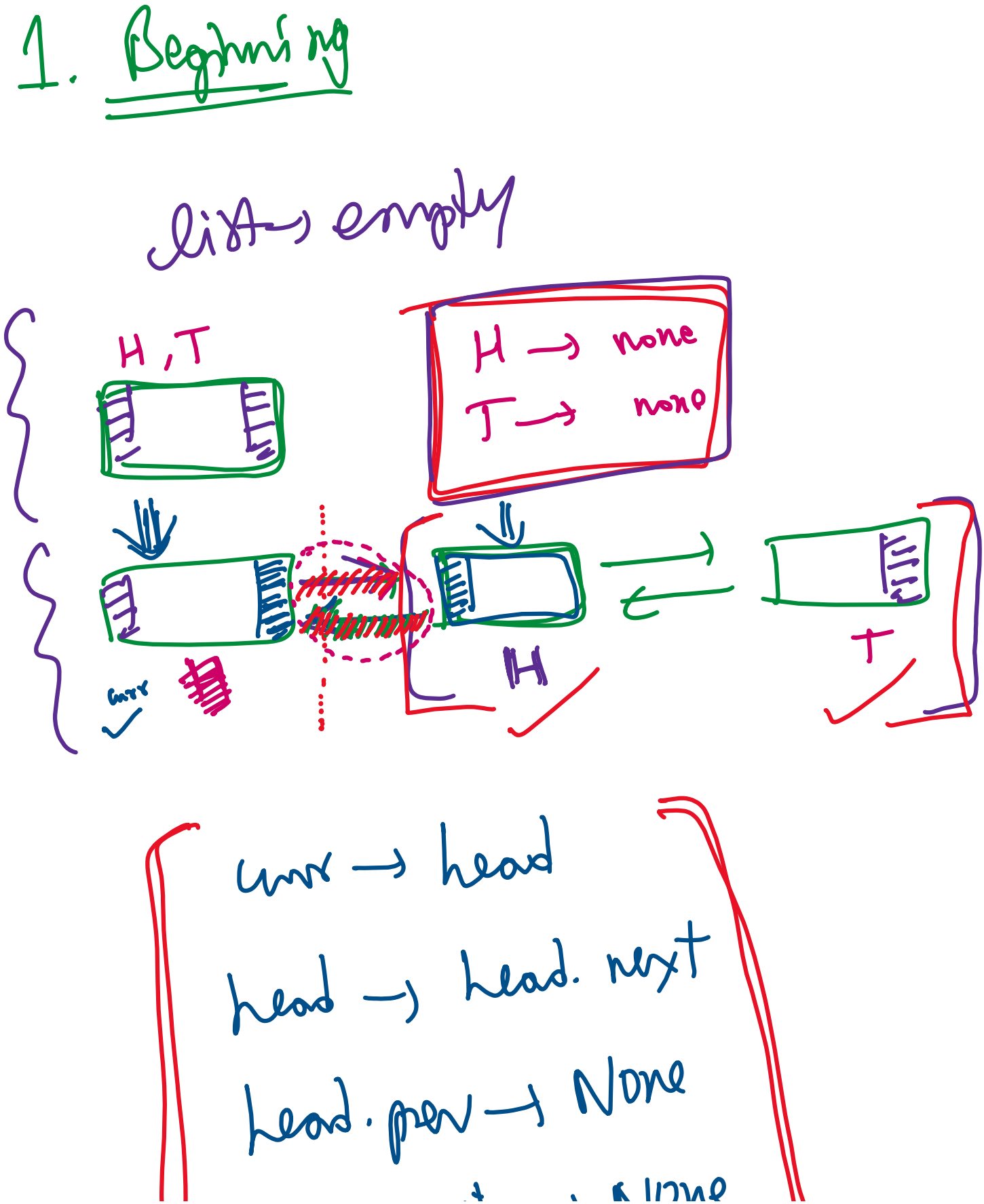
current → current.next

curr → None

curr → None

4.7.4 - DLL Deletion

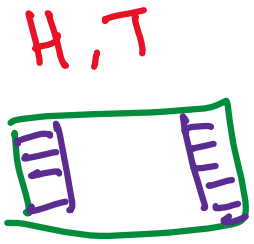
Saturday, September 20, 2025 12:36 PM

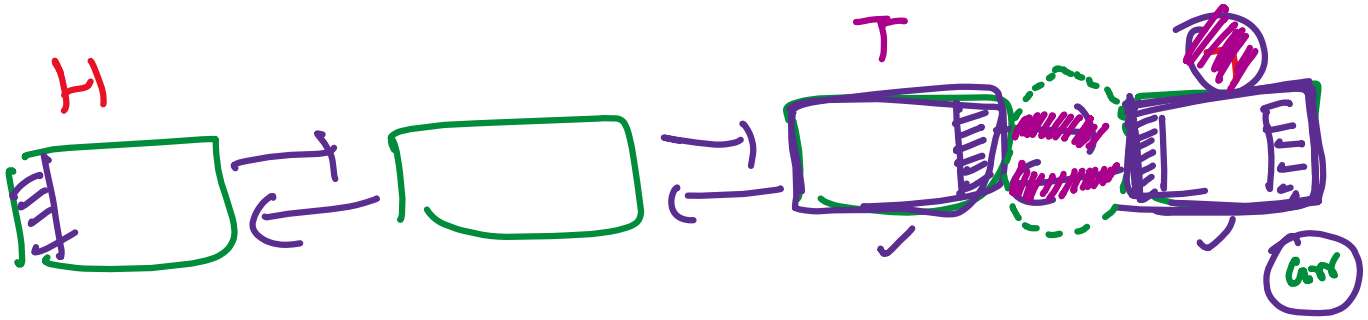


curr.next → None

2. End

list → empty





head → None
tail → None

curr → tail

tail → tail.prev

tail.next → None

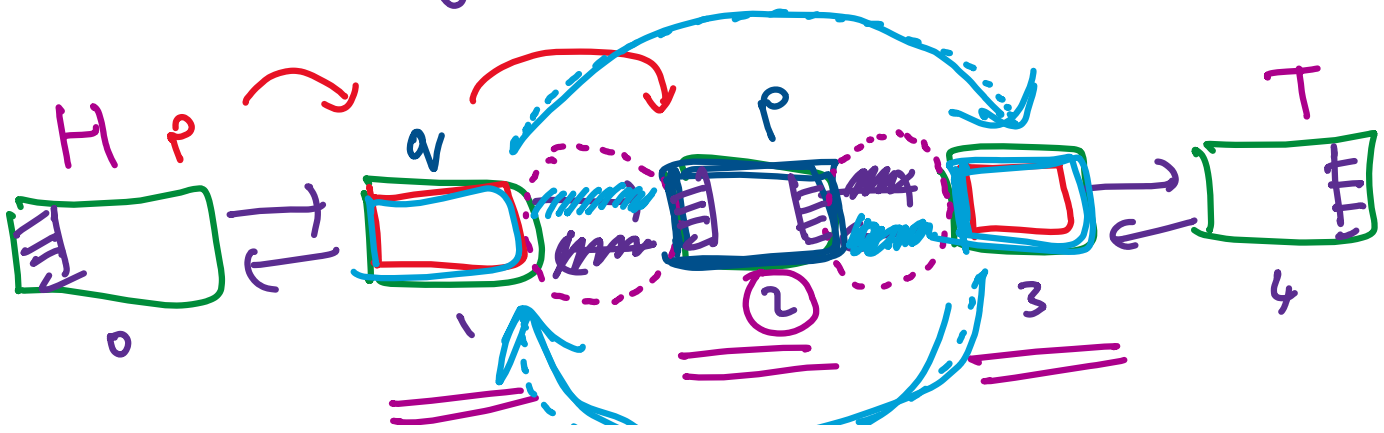
curr.prev → None

3. Middle:

n = 4

$0 \le pos \le n-1$

$0 \le pos < n$



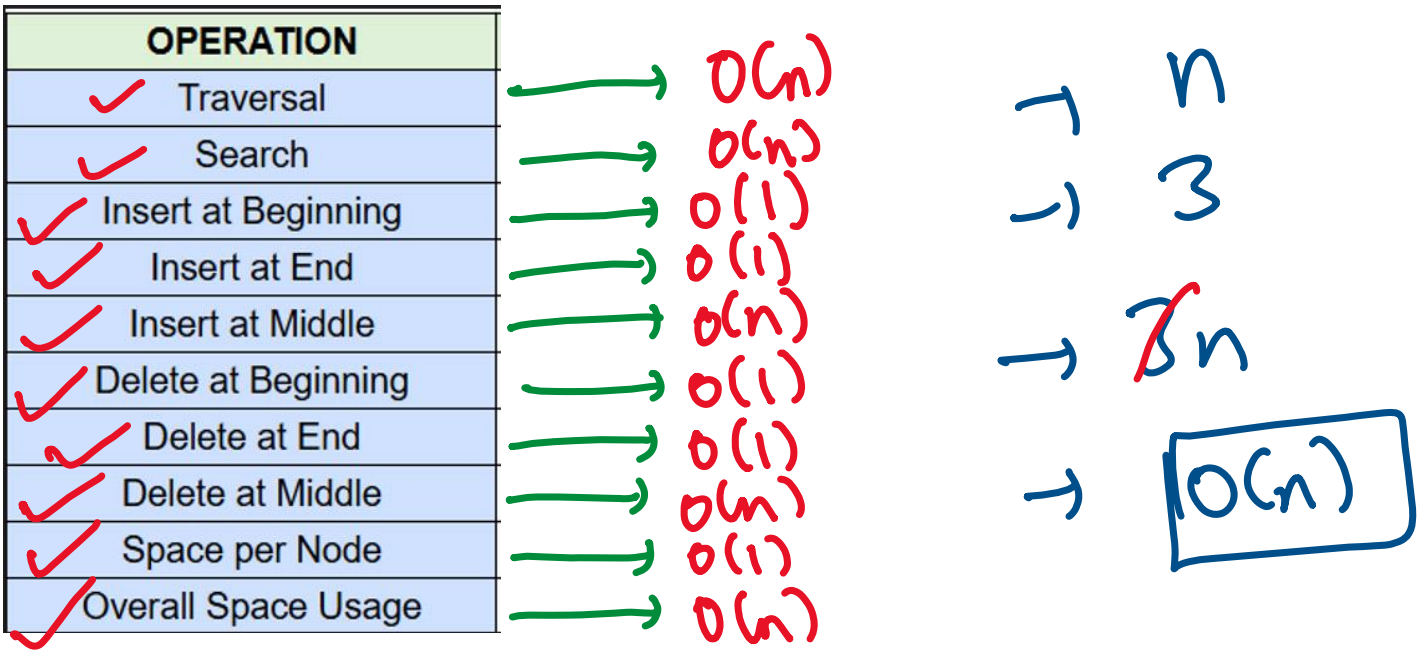
pos = 2

```
for range (pos)
    p = p.next
q = p.prev
```

```
q.next → p.next
p.next.prev → q
p.next → None
p.prev → None
```

4.7.5 - DLL Complexity Analysis

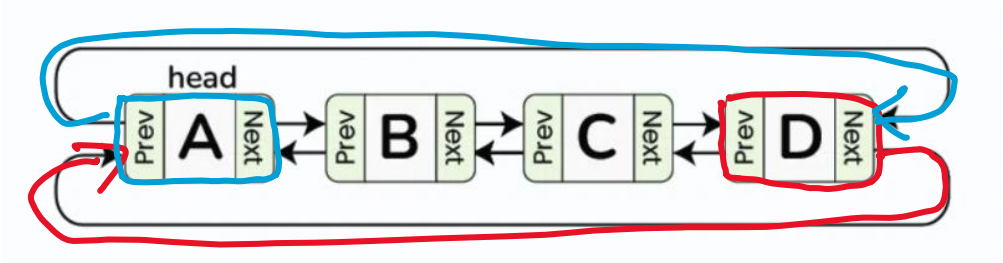
Saturday, September 20, 2025 12:36 PM



4.8 - Circular Doubly Linked List

Saturday, October 4, 2025 2:19 PM

- A **Circular Doubly Linked List** is a variation of the doubly linked list in which the last node is connected back to the first node, forming a circular structure
- Each node has three fields:
  - **Data** – stores the element
  - **Prev** – pointer/reference to the previous node
  - **Next** – pointer/reference to the next node
- The first node's **prev** points to the **last node**, and the last node's **next** points to the first node
- There is no **NULL** (or **None**) reference in the list since it is circular
- Traversal is possible in both directions:
  - Forward using the **next** pointer
  - Backward using the **prev** pointer
- Circular structure makes it easy to move continuously through the list without needing to reset at the **head** or **tail**



Pros:

- Can be traversed forward and backward easily using **next** and **prev** pointers
- No need to check for **None** — can loop through the list endlessly
- Nodes can be inserted or removed efficiently from both ends or any position

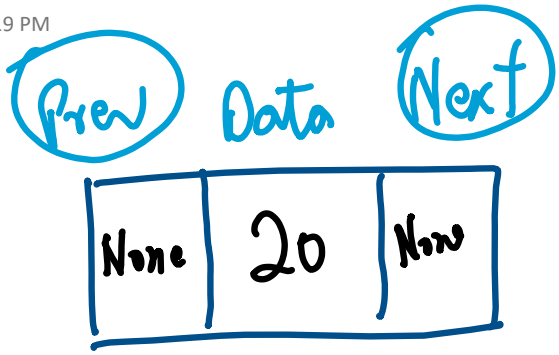
Cons:

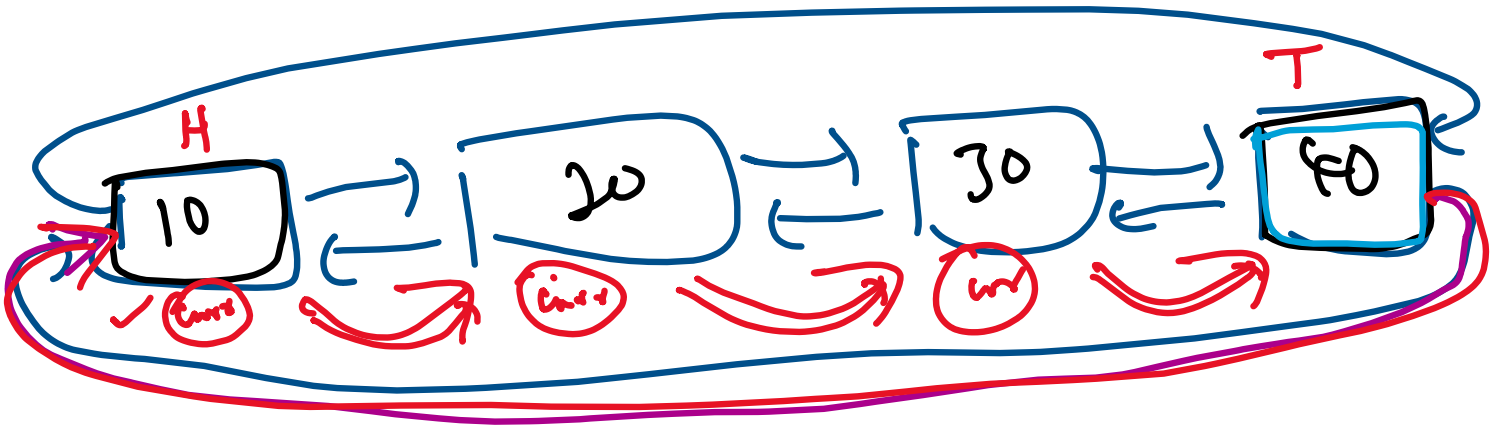
- Requires two pointers (**prev** and **next**) per node, increasing memory usage overall

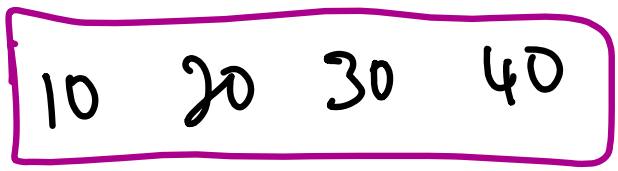
- More pointer adjustments are needed during insertion and deletion
- If termination conditions are not handled properly, traversals may become infinite
- Implementation is complex among all other types

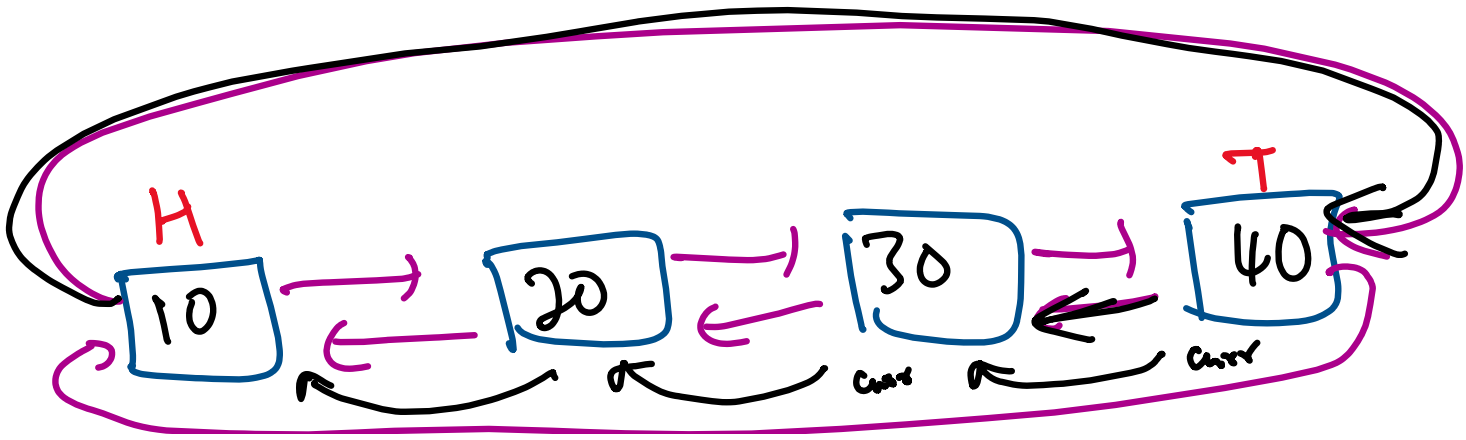
4.8.1 - CDLL Initial Setup

Saturday, October 4, 2025 2:19 PM









40 30 20 10

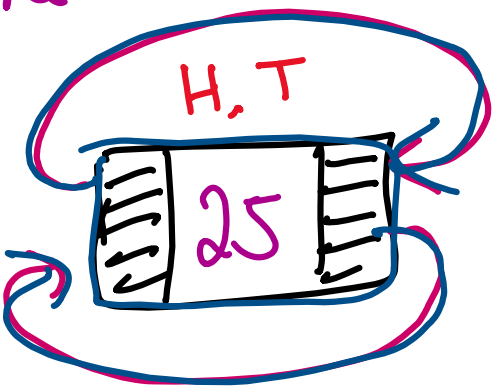
4.8.2 - CDLL Insertion

Saturday, October 4, 2025 2:19 PM

1. Beginning

i) Empty

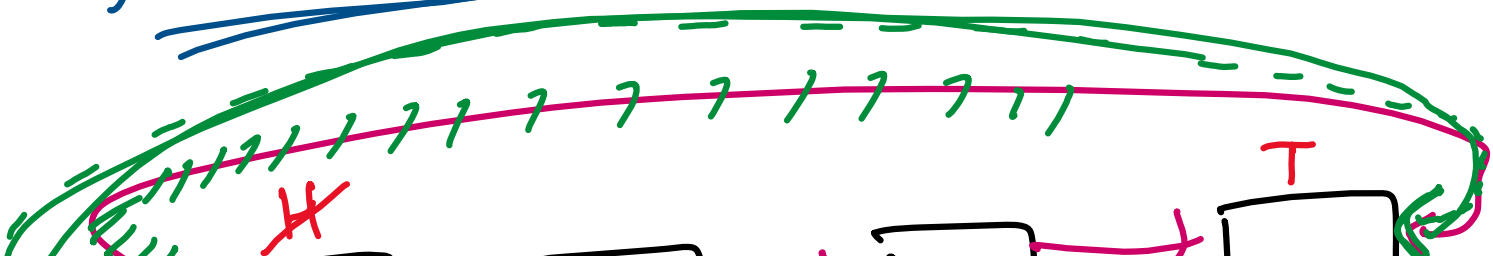
Head
Tail } — None

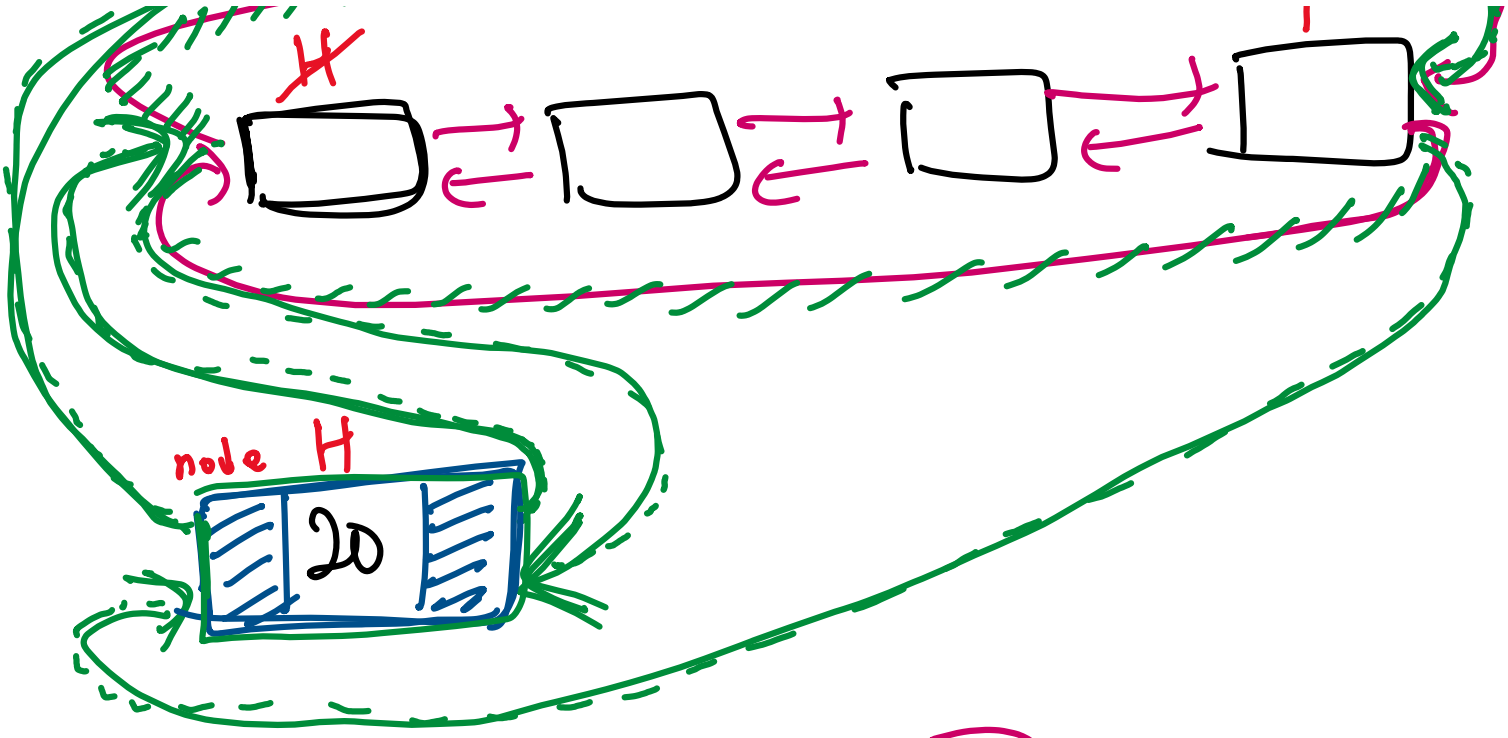


Head → Node
Tail → Node

node.next → Node
node.prev → Node

ii) Non-empty:

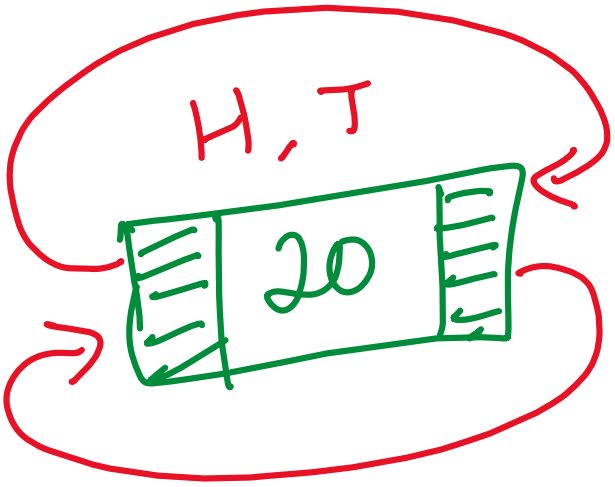




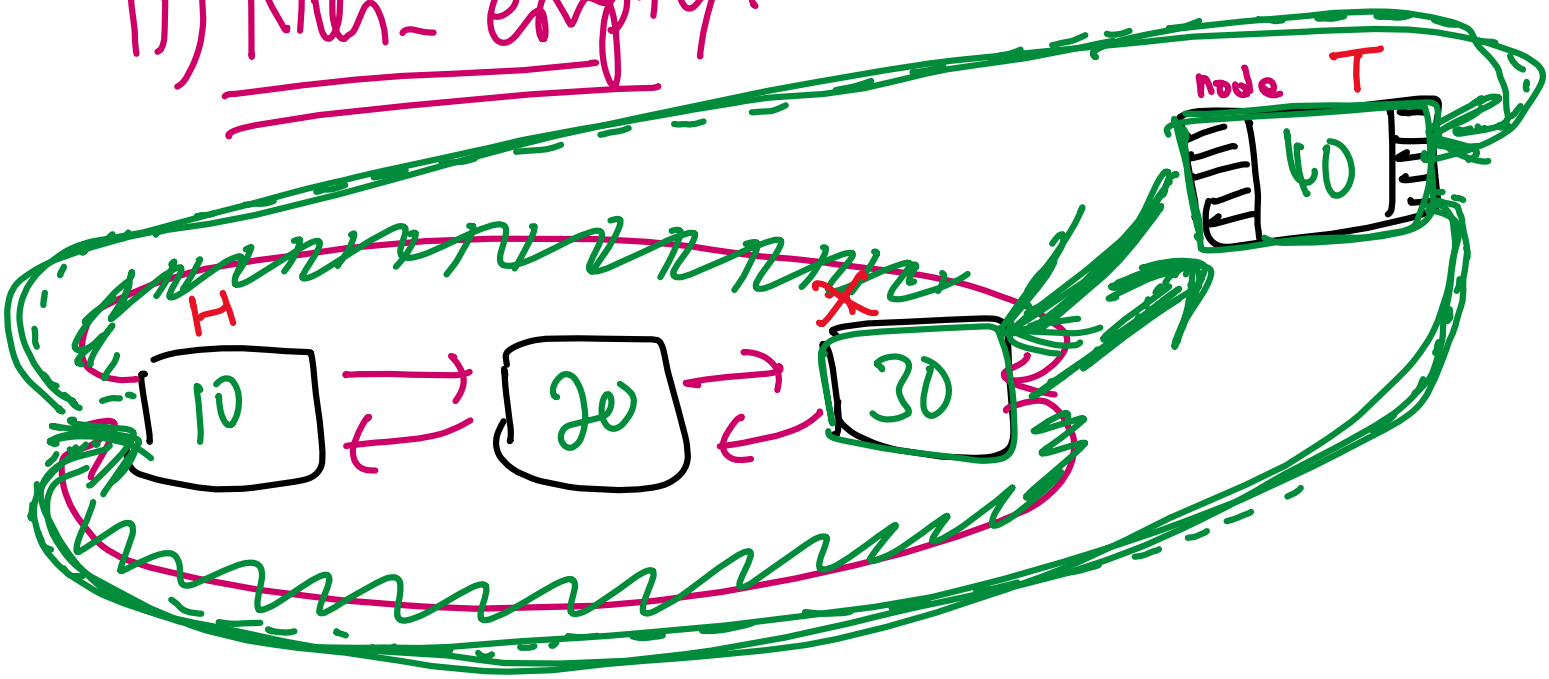
node.next → head
node.prev → tail
head.prev → node
tail.next → node
head → Node

2. End:

i) Empty



ii) Non-empty:



node.next → head

node.prev → tail

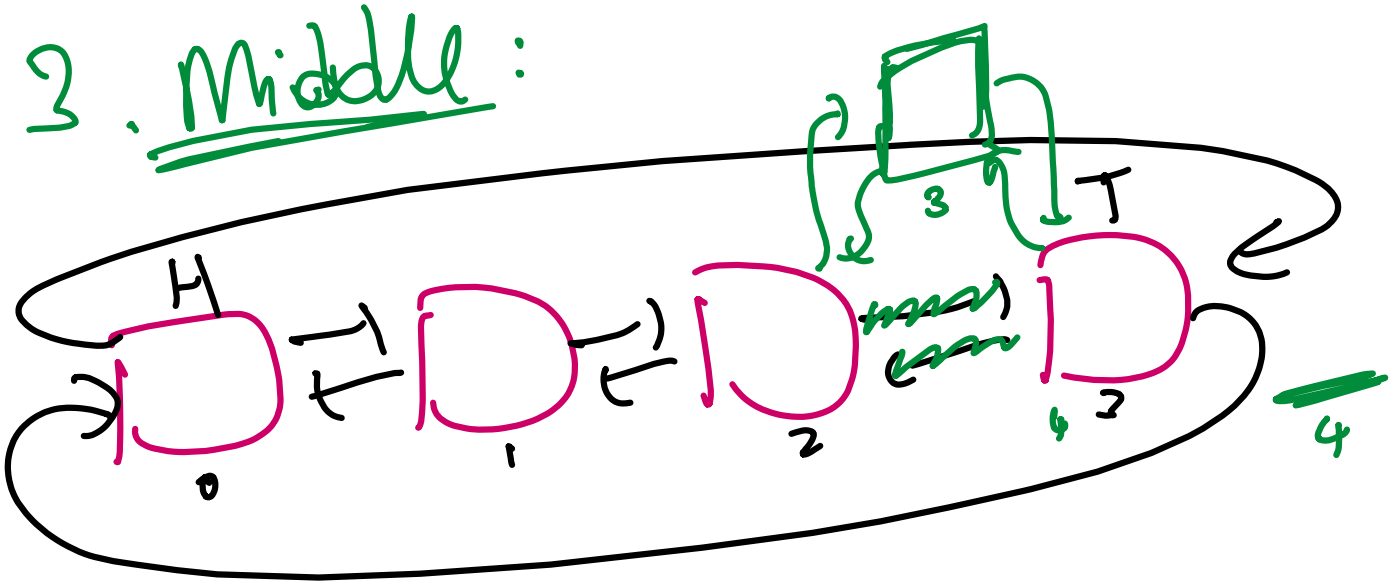
head.prev → node

tail.next → node

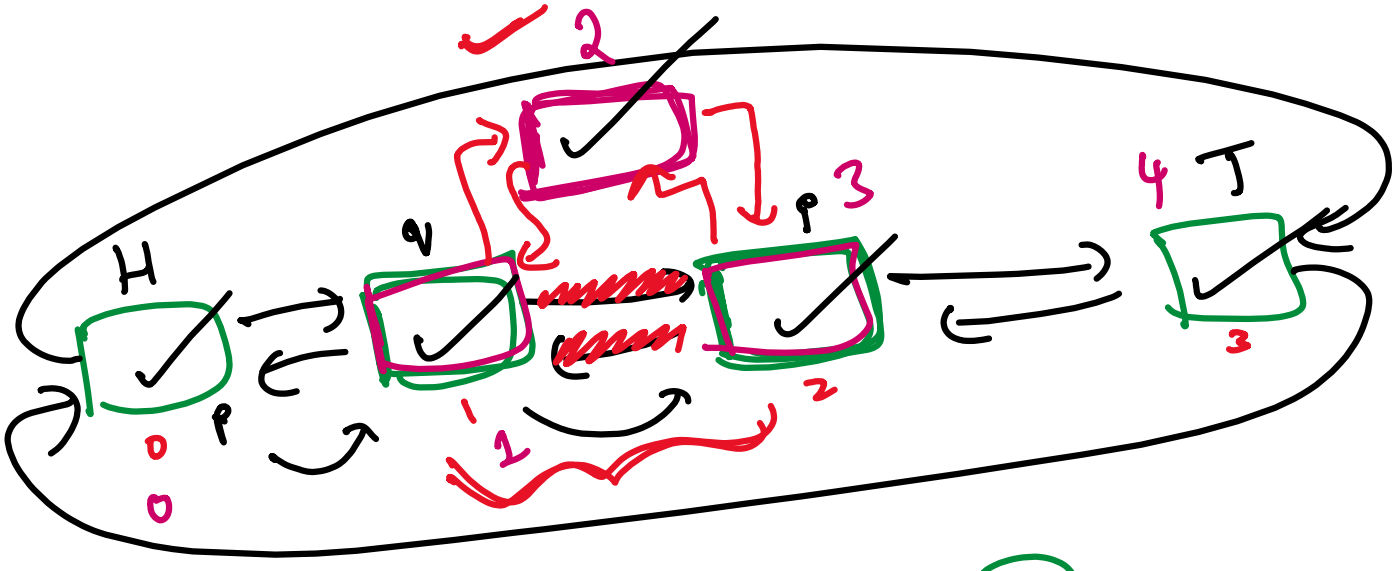
fail. next

fail ⟶ node

3. Middle:



$$0 \le pos \le n$$



pos = 2

range(2)
↳ 0, 1
↳ 2 times

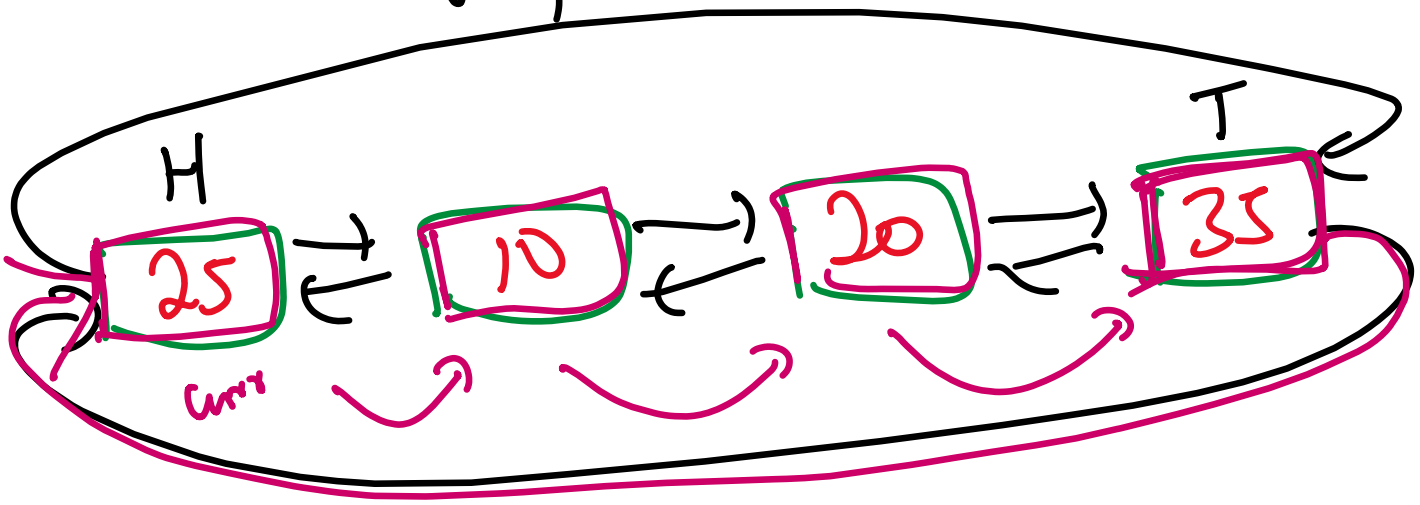
q.next → node
node.prev → q
node.next → p
p.prev → node

4.8.3 - CDLL Search

Saturday, October 4, 2025 2:19 PM

Empty → @empty print

Non-Empty →



key

curr → head

while True:

if curr.data == key:

print("found")

break

curr = curr.next

curr → curr.next

if curr == head:

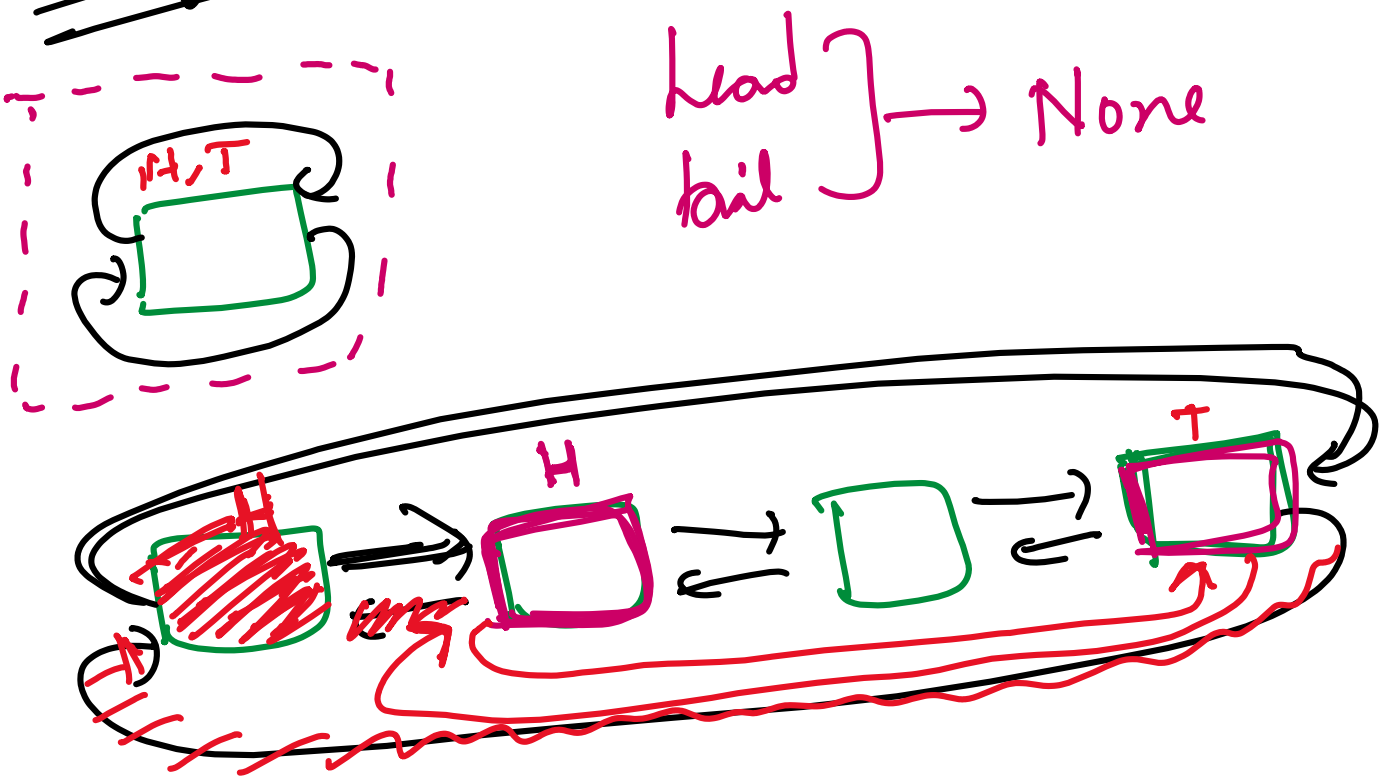
print("not found")

break

4.8.4 - CDLL Deletion

Saturday, October 4, 2025 2:19 PM

1. Beginning:



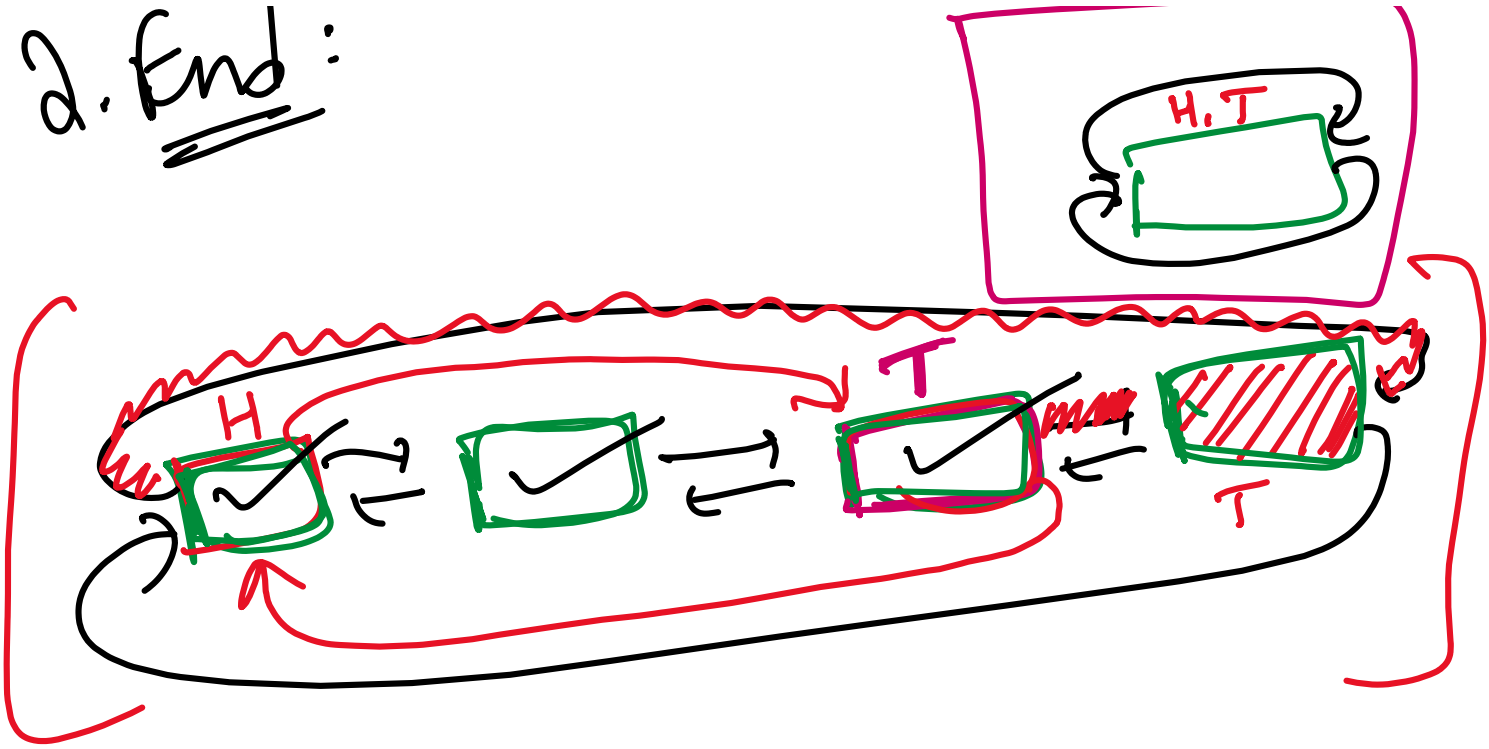
head → head.next

head.prev → tail

tail.next → head

2. End:

2. End:

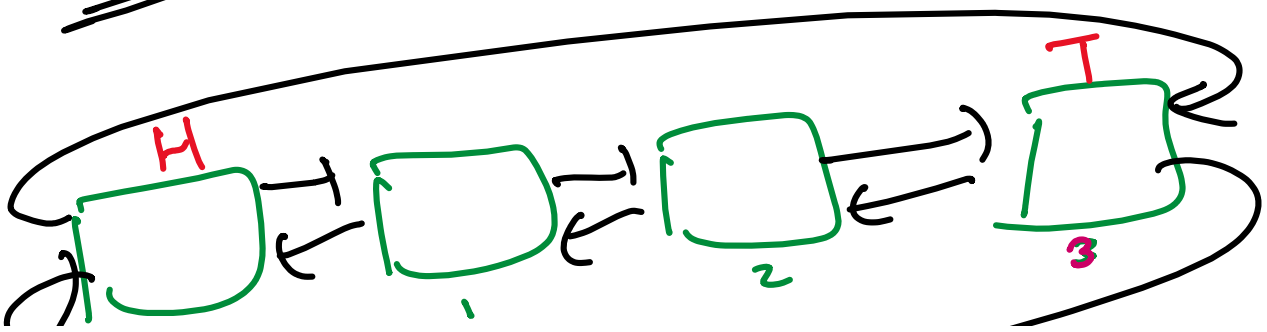


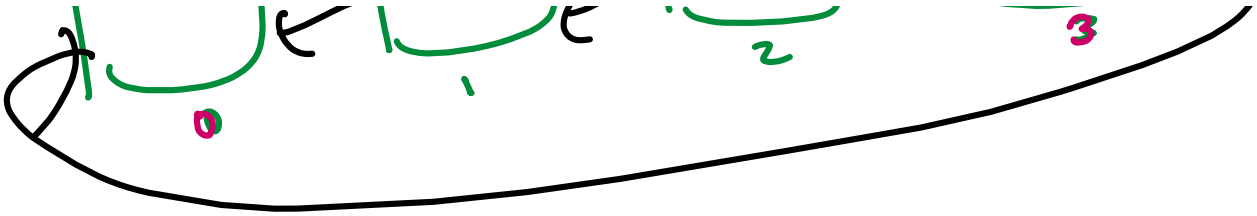
tail → tail.prev

head.prev → tail

tail.next → head

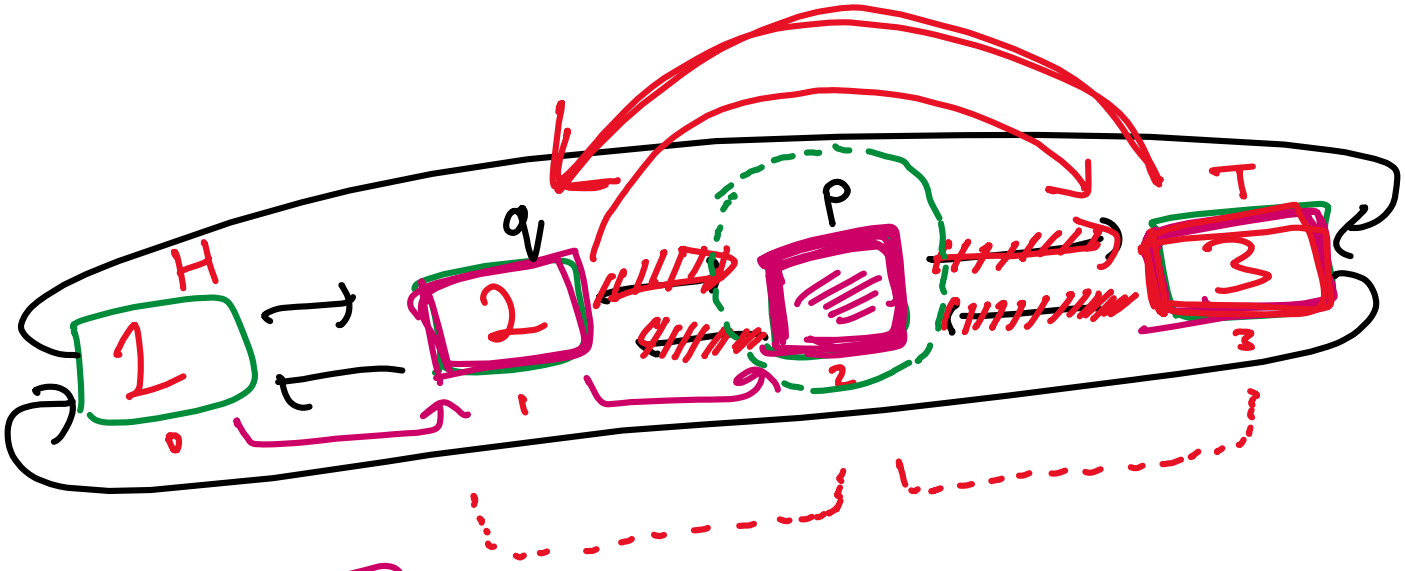
3. Middle:





pos

$0 \leq pos < n$



pos → 2

```
q.next → p.next
p.next.prev → q
p.next → None
p.prev → None
```

p.prev → None

4.8.5 - CDLL Complexity Analysis

Saturday, October 4, 2025 2:19 PM

OPERATION	
Traversal	O(n)
Search	O(n)
Insert at Beginning	O(1)
Insert at End	O(1)
Insert at Middle	O(n)
Delete at Beginning	O(1)
Delete at End	O(1)
Delete at Middle	O(n)
Space per Node	O(1)
Overall Space Usage	O(n)

node → 3 units

list → 'n' nodes

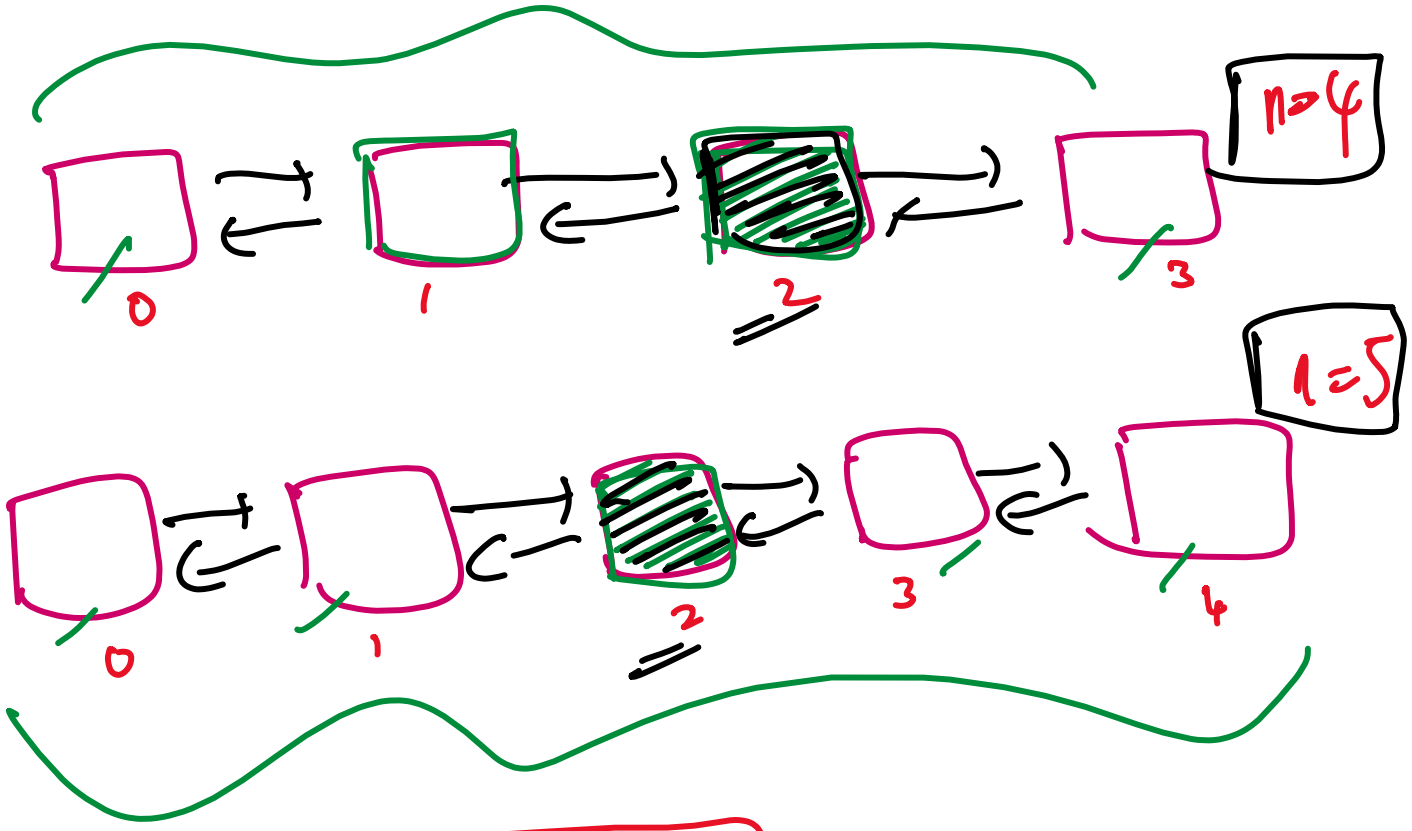
→ 3 × n

→ 3n

→ n

4.9 - Coding Problems

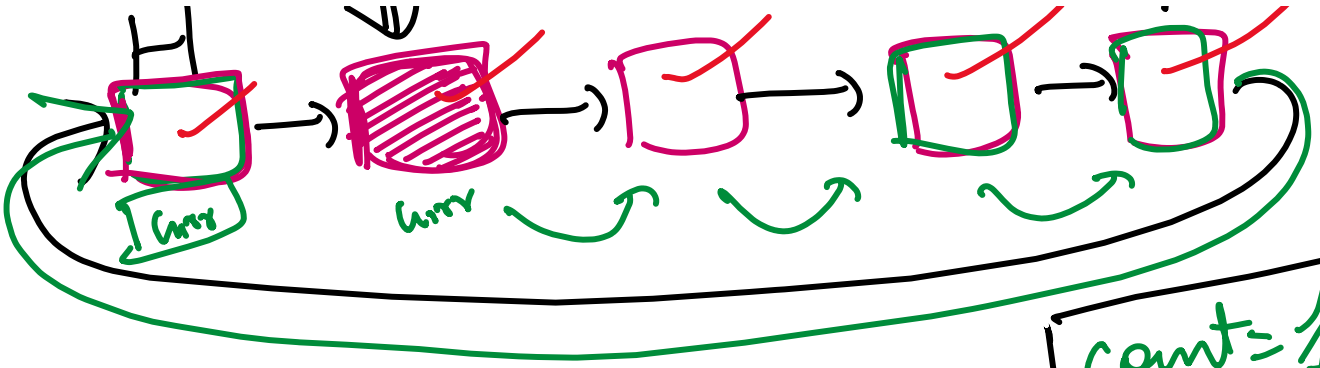
Saturday, October 18, 2025 11:24 AM



$n // 2$

$4 // 2 \rightarrow 2$

$5 // 2 \rightarrow 2$



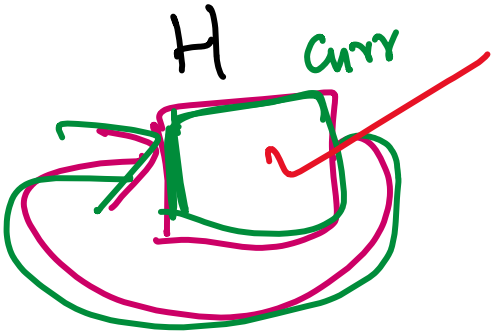
count = 12
345

return 0

curr → head

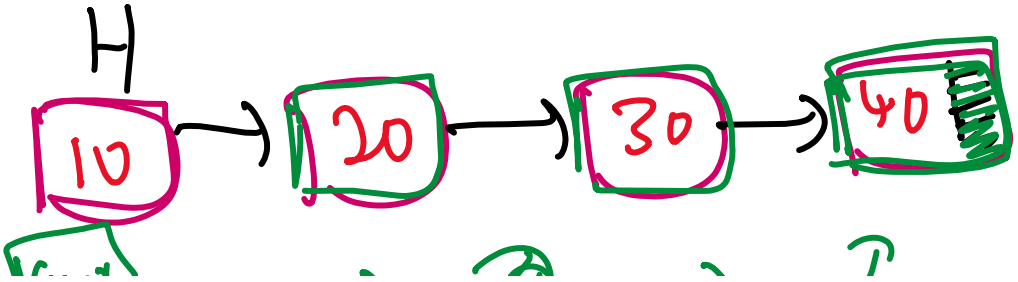
count = 1

curr → head.next



count = 1

curr → head.next



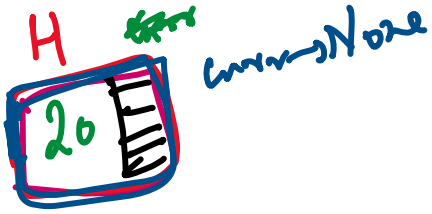
total = ~~0~~ ~~10~~ ~~30~~ 60 100

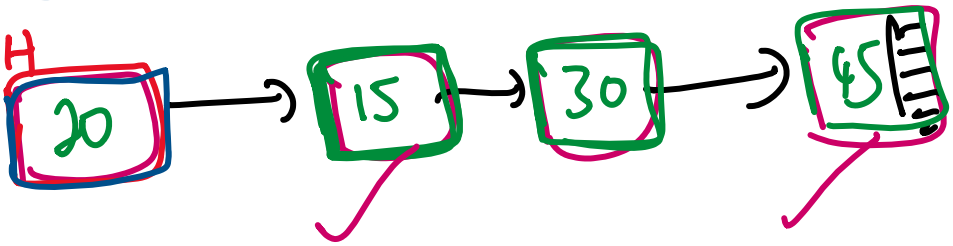
```
while curr:
    total = total + curr.data
    curr = curr.next
return total
```

None

max → 20, min → 20

curr → head.next





max, min → Head.data

n, 2n → O(n)

5s, 10s

max → 20 30 45 ✓

min → 20 15 ✓

if curr.data > max

max → curr.data

elif curr.data < min

min → curr.data

curr → curr.next

5. Stacks

Saturday, October 18, 2025 11:24 AM

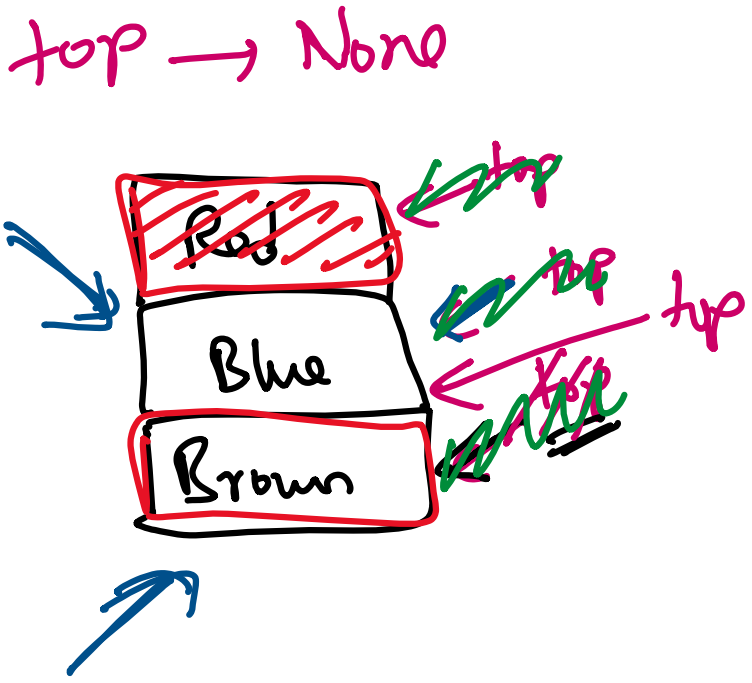
- **Introduction**
- **Common Operations**
- **Implementation**
- **Complexity Analysis**
- **Applications**
- **Pros & Cons**
- **Coding Problems**

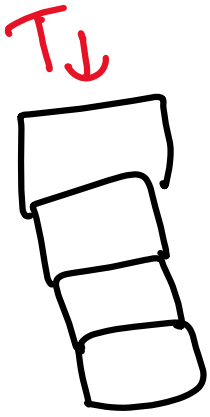
5.1 - Introduction

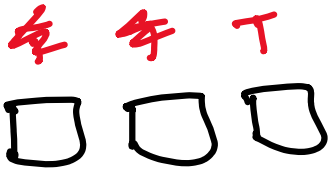
Saturday, October 18, 2025 11:24 AM

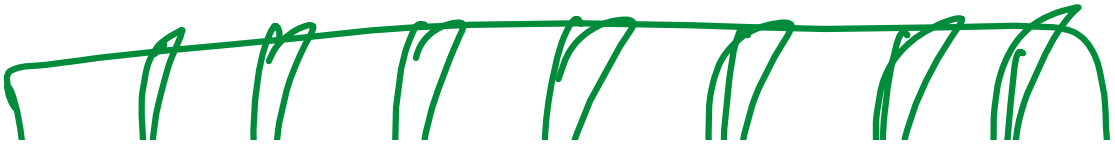
What is a Stack?

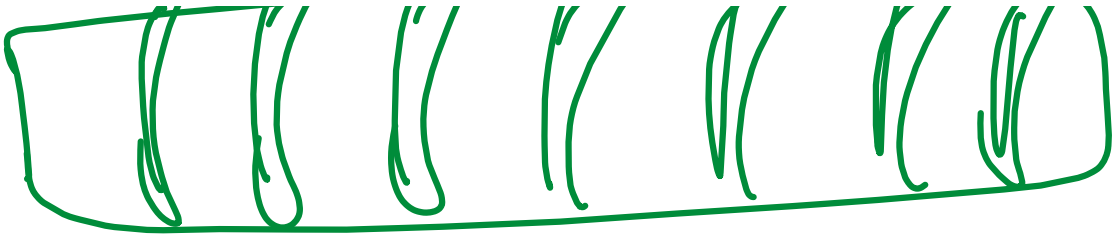
- A Stack is a linear data structure that stores elements in a sequential order
- Insertion and Deletion of elements happen only from one end, called the **top** of the stack
- Follows the **LIFO (Last In, First Out)** principle:
  - When an element is inserted, it is the last element in the stack
  - When an element is removed, it is the first one to be removed from the stack

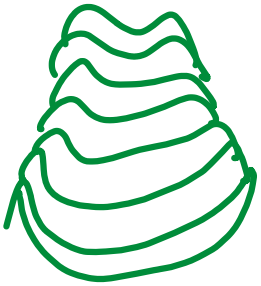








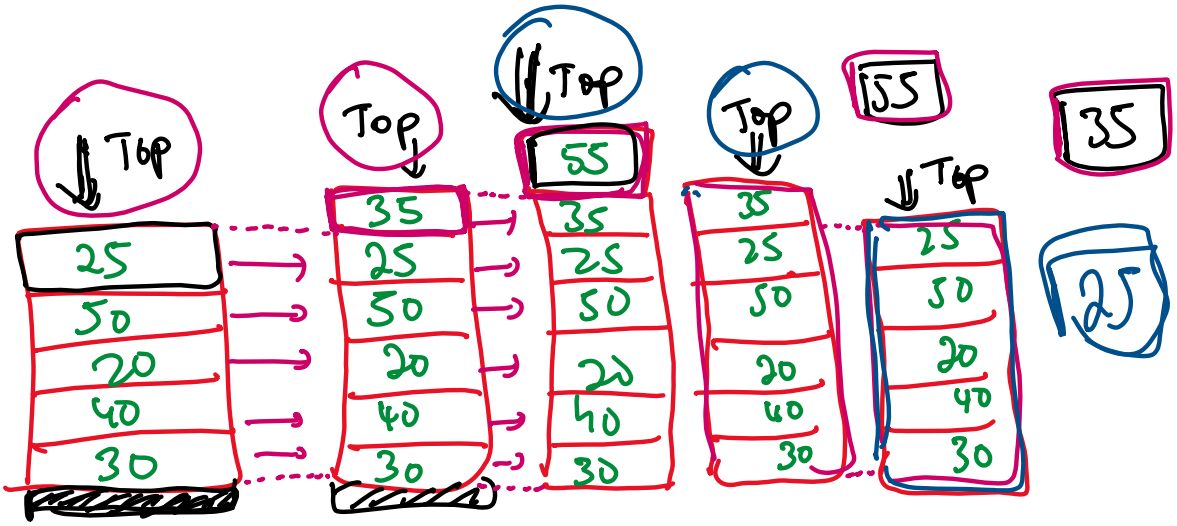




5.2 - Common Operations

Saturday, October 18, 2025 11:24 AM

- **Push** (Insert)
- **Pop** (Delete)
- **Peek** (Access)
- **isEmpty**



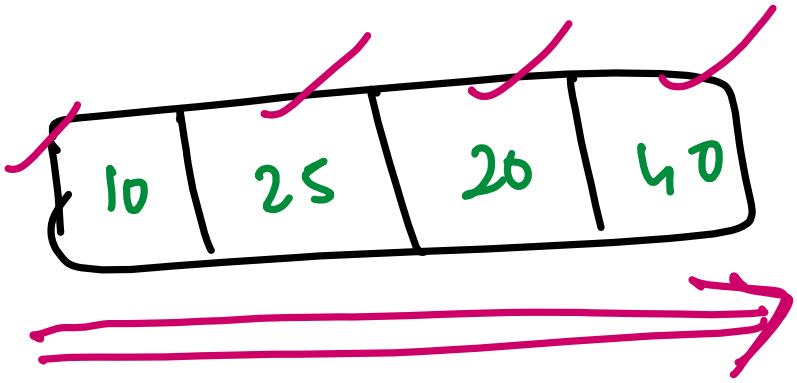
5.3 - Implementation

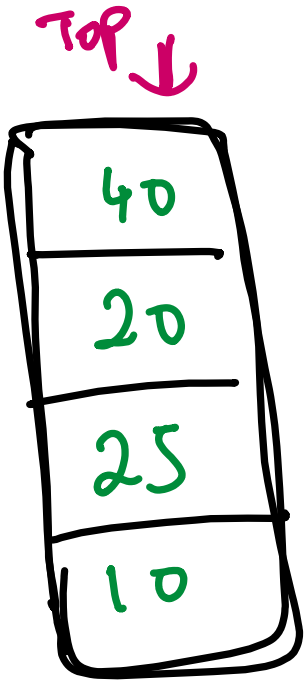
Saturday, October 18, 2025 12:08 PM

In Python, Stacks can be implement in 3 different ways:

5.3.1 - Using Lists

Saturday, October 18, 2025 12:27 PM





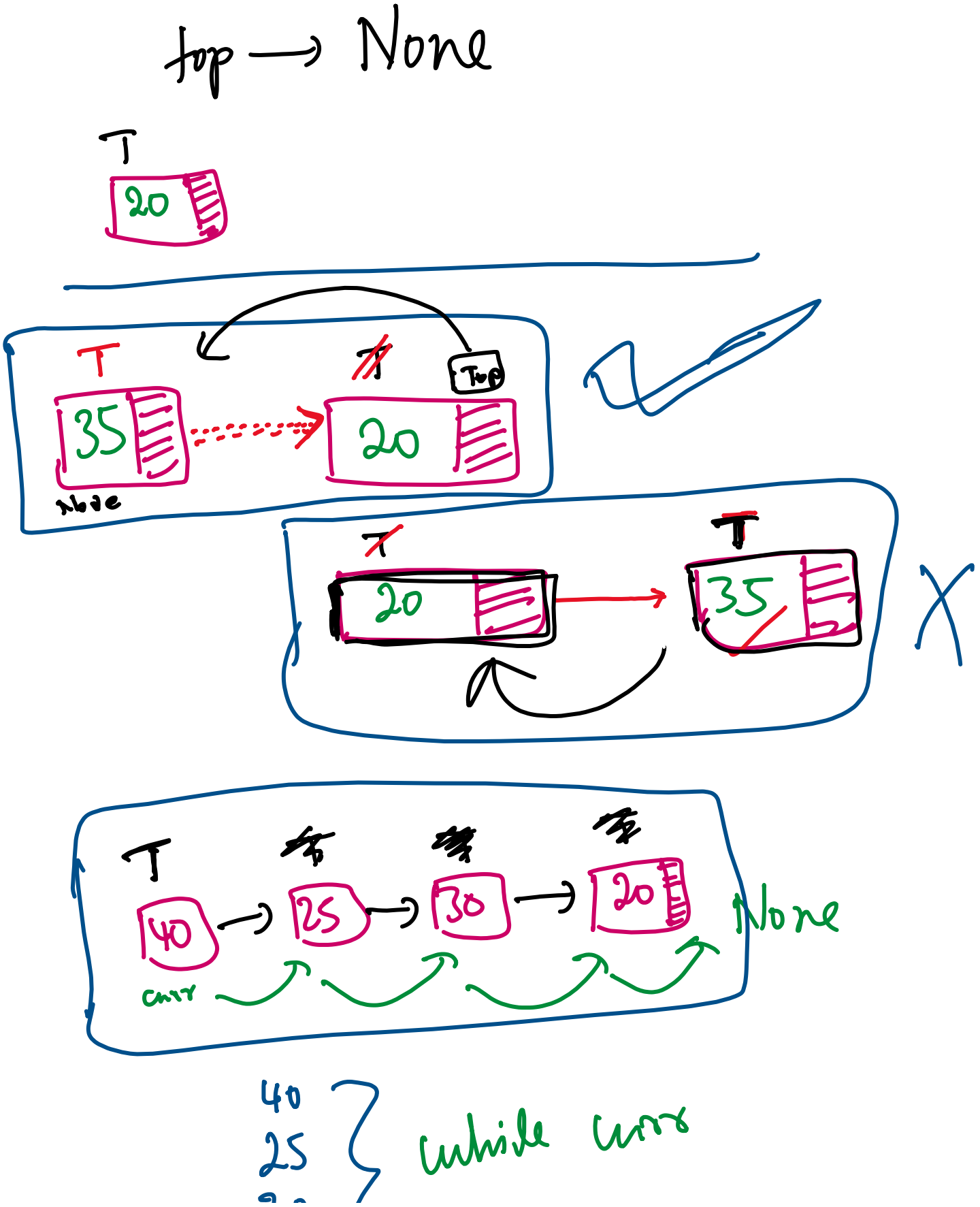
5.3.2 - Using collections

Saturday, October 18, 2025 12:27 PM

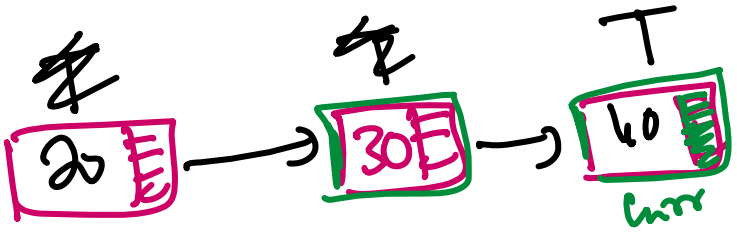
- Implemented using a high-performance data structure called **deque** from Python's built-in **collections** module
- A deque allows fast insertion and deletion of elements in O(1) time
- It's implemented as a **doubly-linked list** under the hood

5.3.3 - Using Linked Lists

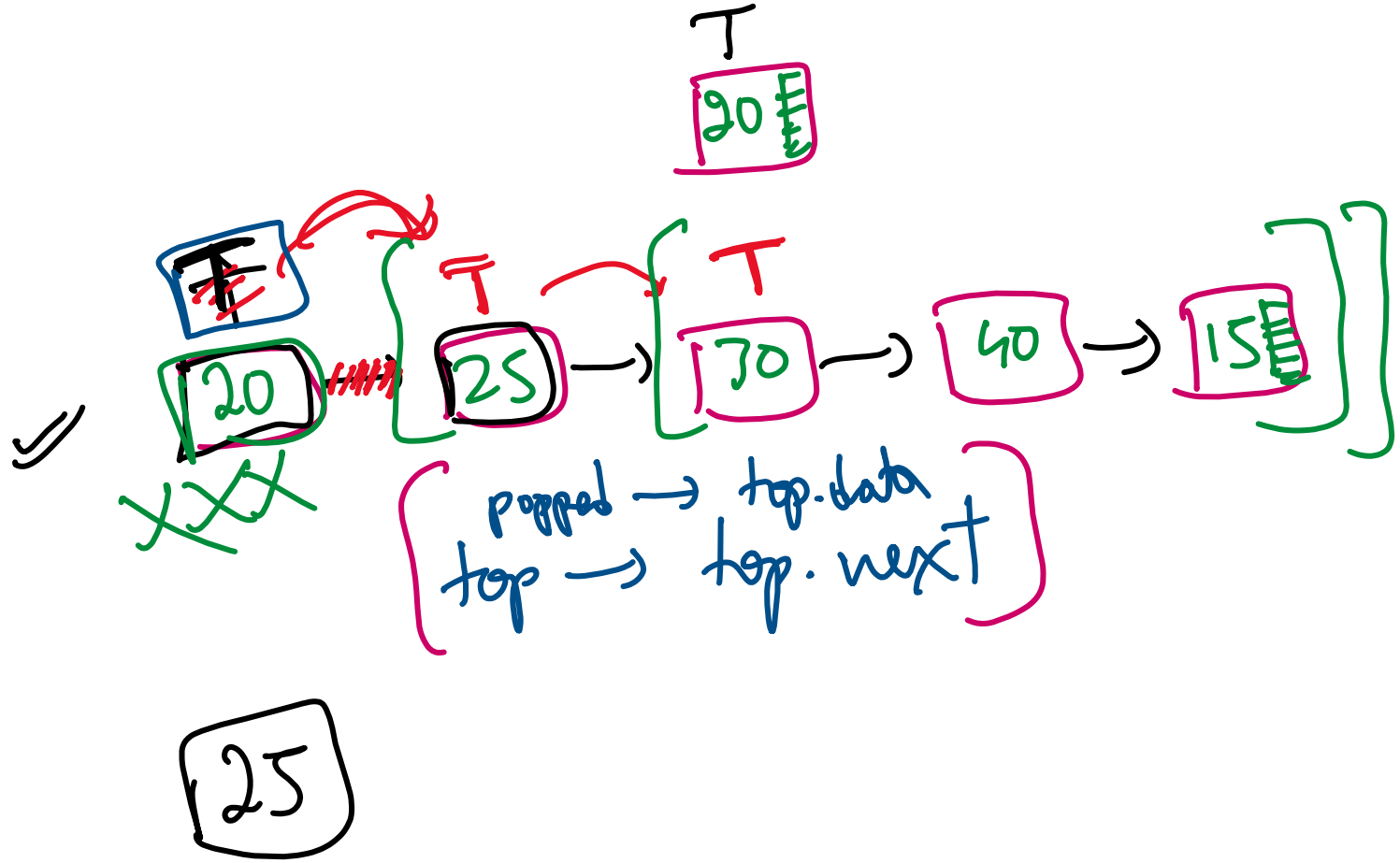
Saturday, October 18, 2025 12:32 PM



25
30
20



40



25

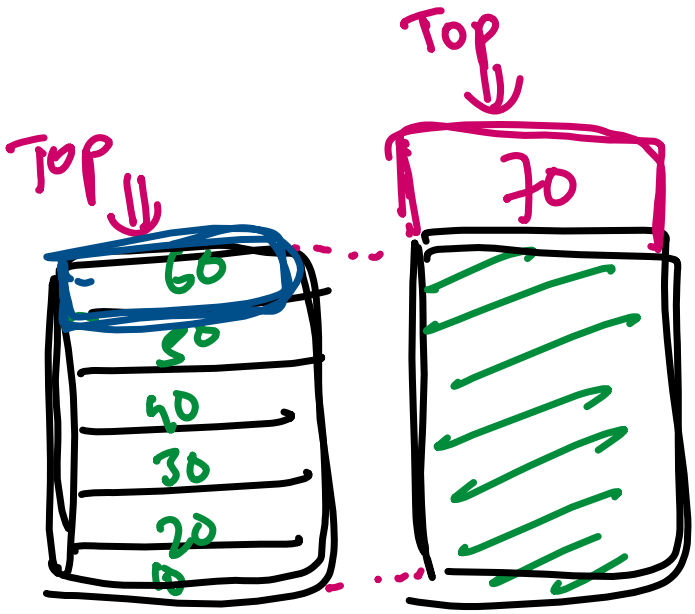
5.4 - Complexity Analysis

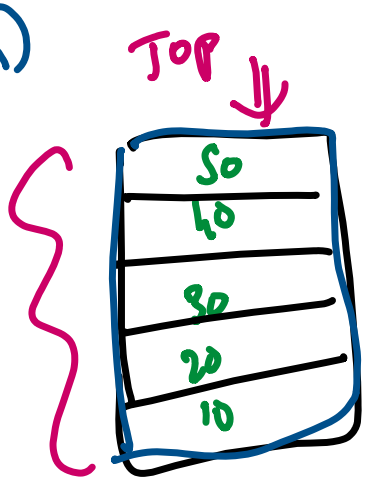
Saturday, October 18, 2025 12:08 PM

Operations:

- Push → O(1)
- Pop → O(1)
- Peek → O(1)
- isEmpty → O(1)

Space → O(n)

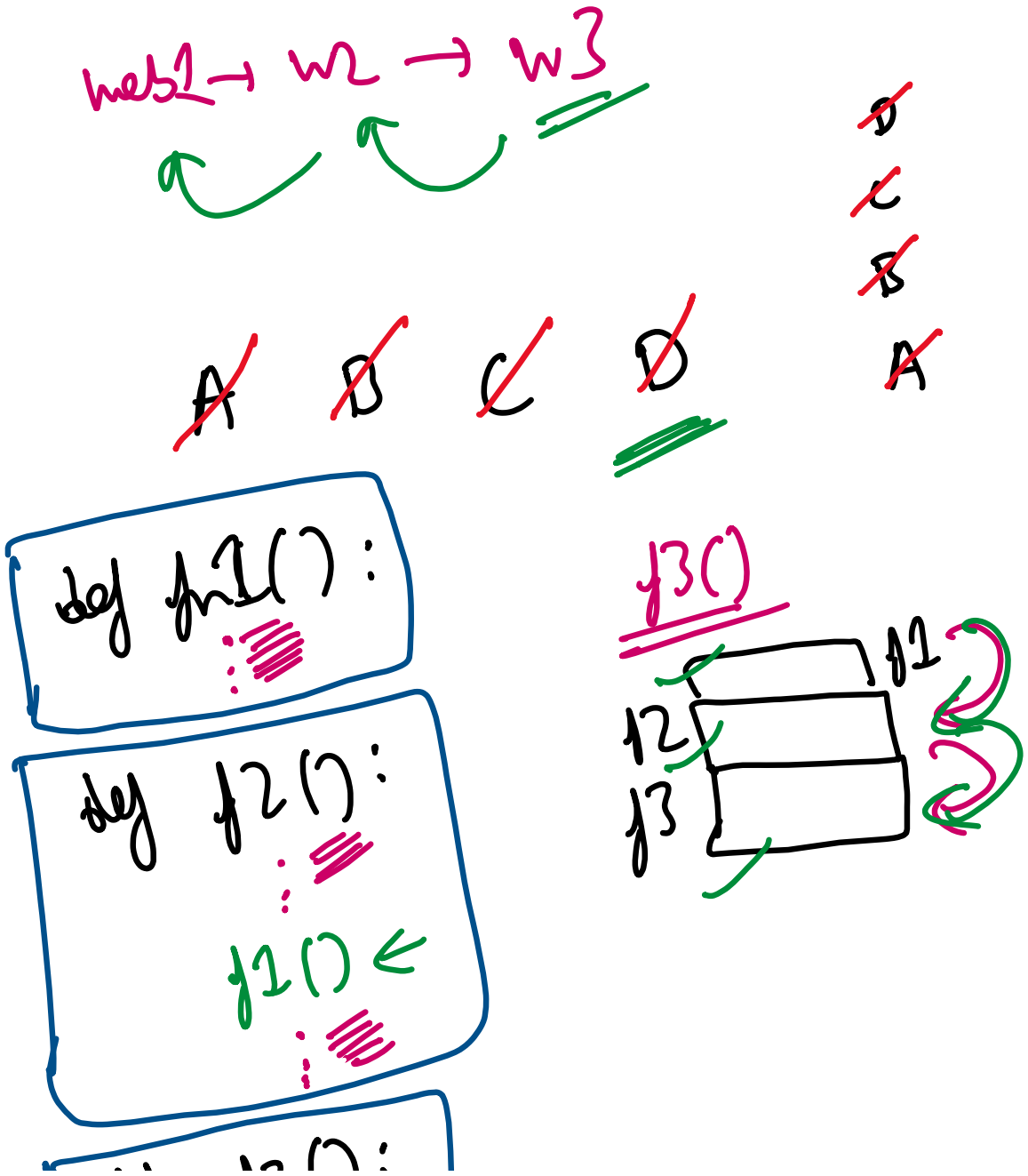


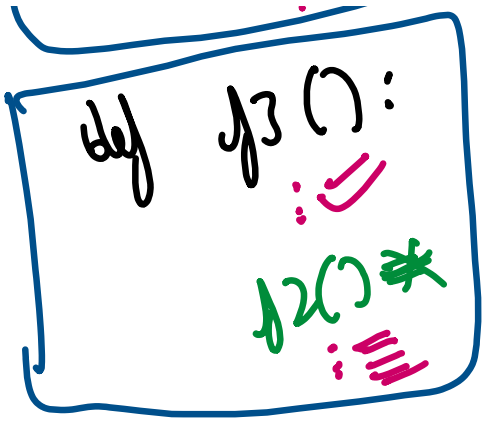


5.5 - Applications

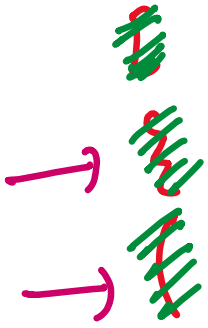
Saturday, October 18, 2025 12:08 PM

- **Undo/Redo functionality in Text Editors**
- **Browser navigation (forward/backward)**
- **Function calls in programming** ⟶ recursive
- **Parentheses/Bracket matching**





$(2+4\times\{5+[1+1]-4\}+3))$



Tree-based RNNs

5.6 - Pros & Cons

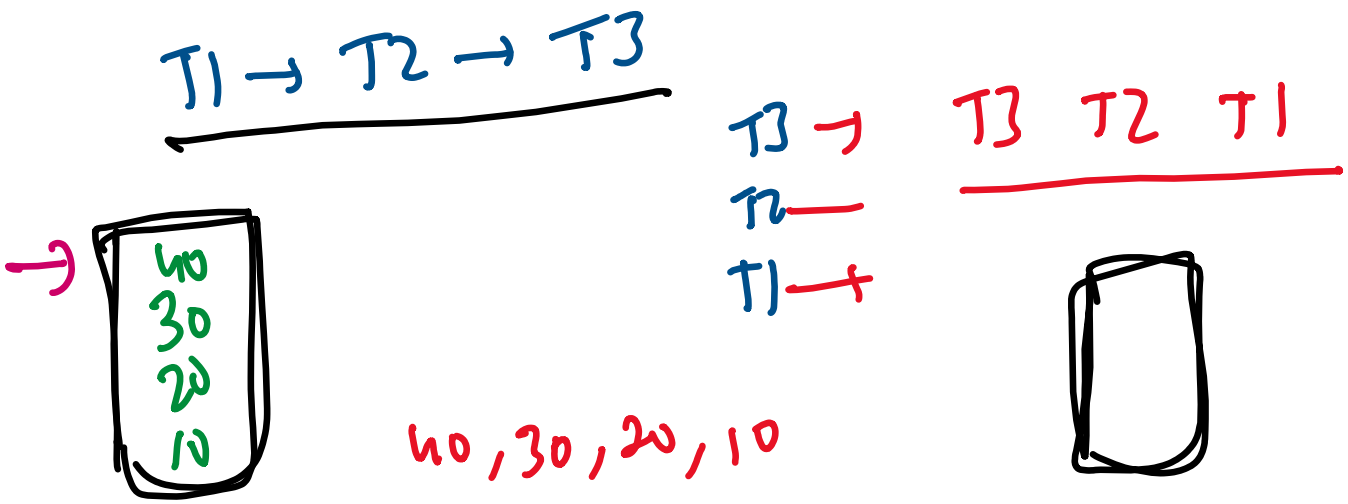
Saturday, October 18, 2025 12:11 PM

Advantages:

- **Efficient Memory Management:** Since data is added and removed only from one end, **top**
- **Useful in Backtracking Problems:** Allows navigation to previous states
- **Prevents Data Inconsistency**
- **Ideal for Reversing Data**
- **Helps in Expression Evaluation & Syntax Parsing:** Compilers use stacks to manage operator precedence and parenthesis matching

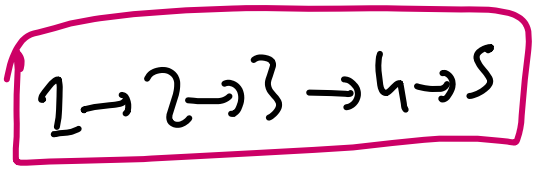
Disadvantages:

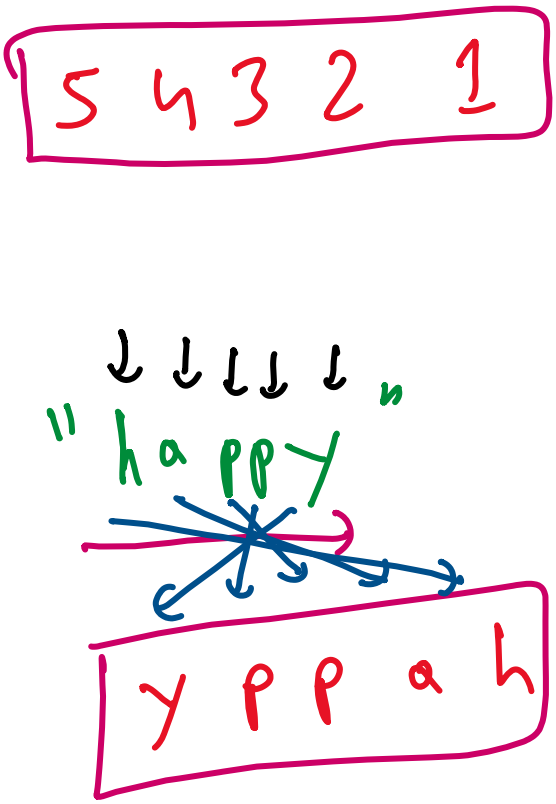
- **Limited Access:** Can't reach middle or lower elements since only **top** is available
- **Limited usability:** Not suitable for random access or sorting
- **Difficult to Traverse/Search:** Since only the **top** element is readily accessible

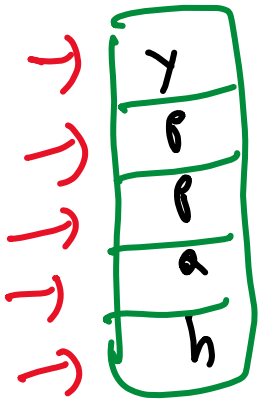


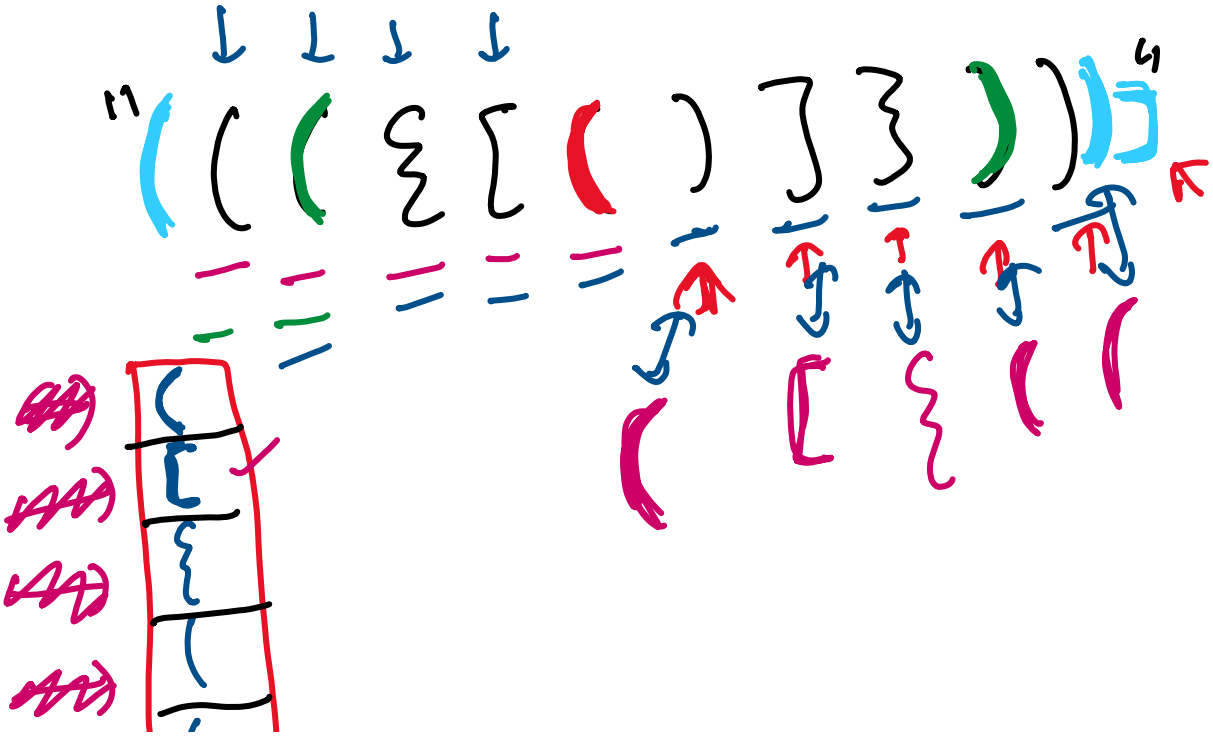
5.7 - Coding Problems

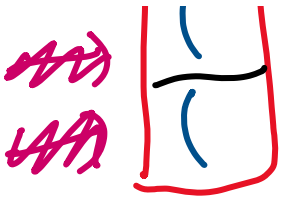
Saturday, October 18, 2025 12:09 PM

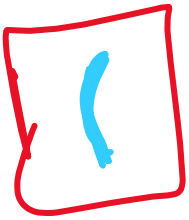












Imbalance

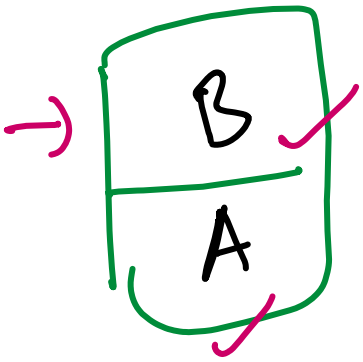
1. If recent closing bracket & popped bracket not match
2. String traversed, stack not empty
3. Stack empty, still closing bracket

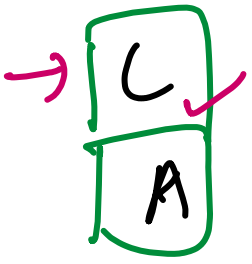
type A

type B

Imbo

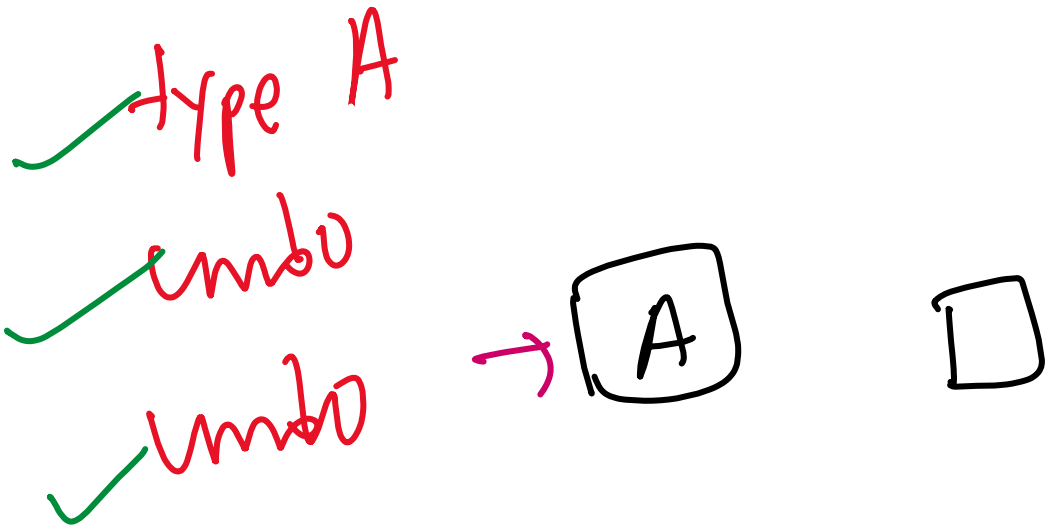
type C





AC

type A



5.8 - DS Context

Tuesday, December 16, 2025 2:00 AM

Data Science / ML Correlation:

- Neural network forward/backward pass

Libraries & Usage:

- PyTorch ***autograd***
- TensorFlow graph execution

6. Queues

Saturday, October 18, 2025 3:01 PM

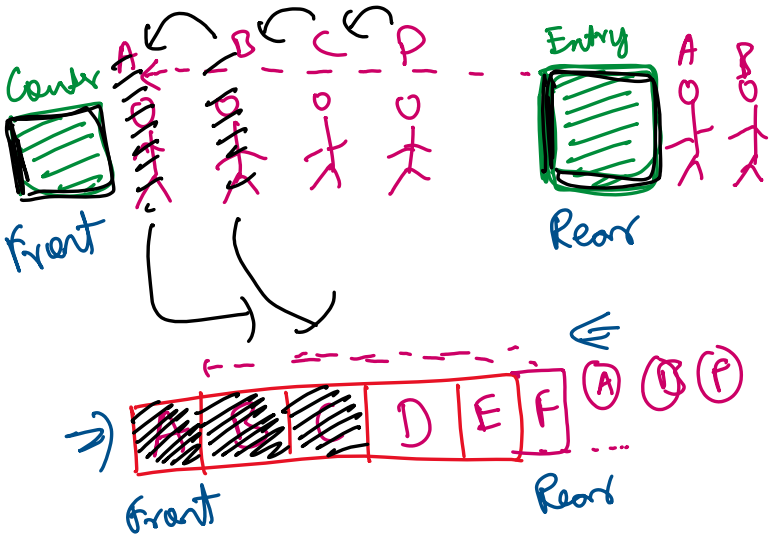
- **Introduction**
- **Common Operations**
- **Implementation**
- **Complexity Analysis**
- **Coding Problems**

6.1 - Introduction

Saturday, October 18, 2025 3:10 PM

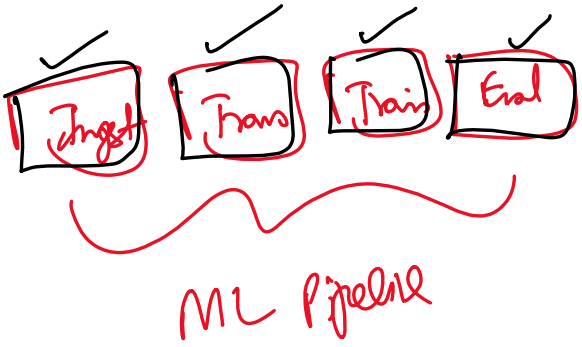
What is a Queue?

- A Queue is a linear data structure that follows the FIFO (First In, First Out) principle — the first element added to the queue is the first one to be removed



Real-World Applications:

- **Call Center Systems:**
  - Incoming calls are kept in a queue until an agent is free
  - The first caller waits the least, while others wait behind in line
  - Ensures fairness and orderly service
- **CPU / Process Scheduling:**
  - Operating systems use **ready queues** to manage processes waiting for CPU time
  - The process that enters first gets executed first
  - To manage multiple processes efficiently without conflicts
- **Data Preprocessing:**
  - During ML training, large datasets are processed in stages
  - Each stage can push processed data into a queue, from which the next stage consumes it
  - In TensorFlow, **tf.data.Dataset** uses iterators and queues internally to manage data pipelines efficiently
- **Streaming Data & Online Learning:**
  - In **real-time ML** or **stream processing,** incoming data is buffered using a queue
  - Data from the queue is then processed or fed into an online learning model
- **Model Serving & Inference:**
  - Deployed ML models receive multiple inference requests simultaneously
  - A queue manages the incoming prediction requests, ensuring FIFO (first-in-first-out) and system stability
  - **Ex: Redis Queue**, **RabbitMQ**



Advantages:

- Maintains Order via the **FIFO** Principle
- **Efficient for Sequential Processing:** Ideal when data needs to be handled one at a time
- **Prevents Overload in Systems:** By queuing requests, servers and routers can handle high loads gracefully instead of crashing
- **Wide-level Usage:** Essential for algorithms like **Breadth-First Search (BFS)**, **Level Order Traversal**, and **CPU scheduling**

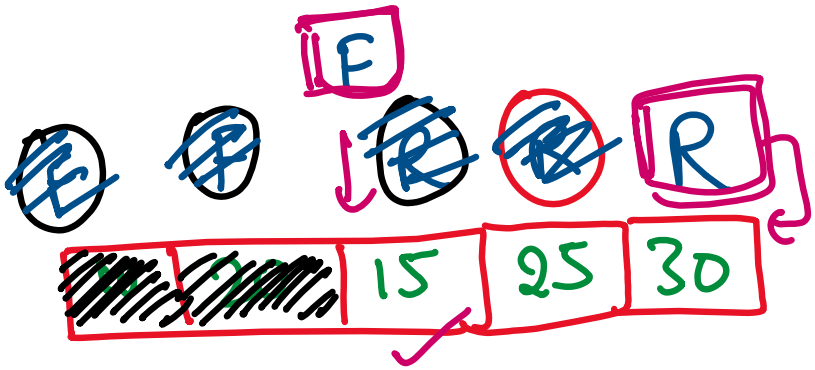
Disadvantages:

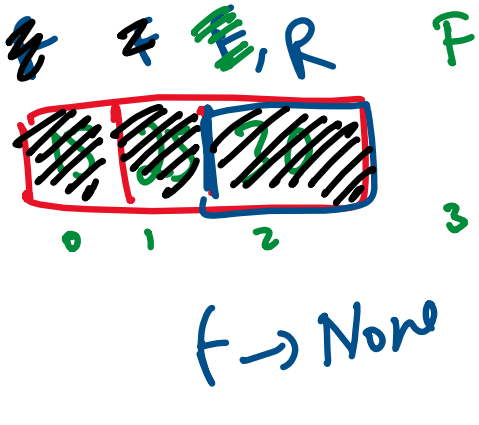
- **Limited Access:**
  - Only the **front** and **rear** elements can be accessed directly
  - Cannot access or modify elements in the middle easily
- **Inefficient in List-Based Queues:**
  - If the queue is large, deletion can take O(n) time
- **Not Suitable for Random Access:**
  - Queues don’t support random access or search operations efficiently

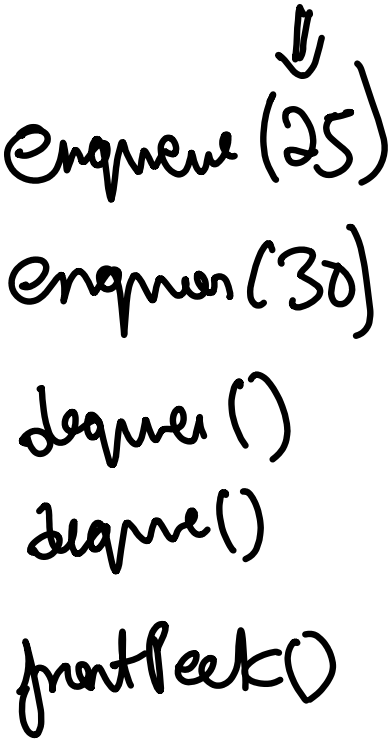
6.2 - Common Operations

Saturday, October 18, 2025 3:13 PM

- **Enqueue:** Insert element at **rear**
- **Dequeue:** Remove element from **front**
- **FrontPeek:** View first element
- **RearPeek:** View last element
- **isEmpty:** Check if queue is empty







6.3 - Implementation

Saturday, October 18, 2025 3:13 PM

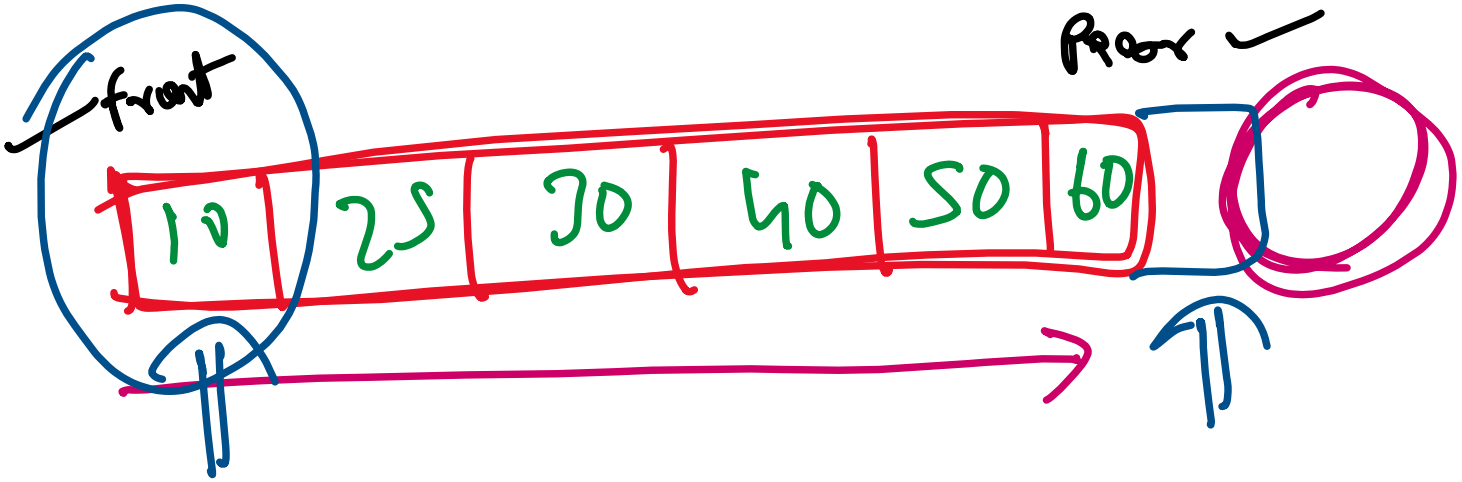
In Python, Queues can be implement in multiple ways:

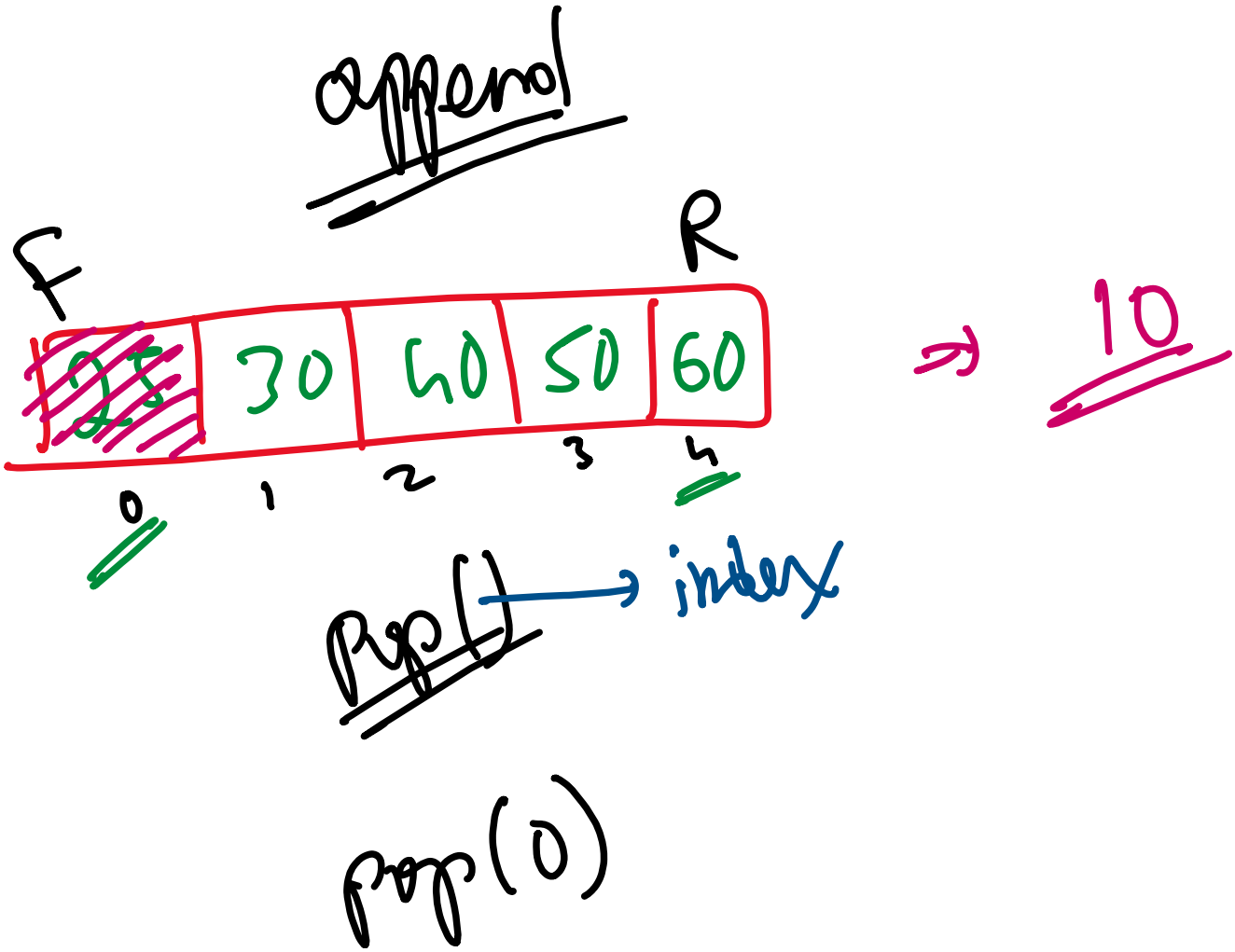
- **Lists**
- **Linked List**
- ***collections*** → deque
- ***heapq***: Used for implementing **Priority Queues**
- ***queue***: Handles locking and blocking; suitable for cross-thread communication
- ***asyncio***: Suitable for **async/await** contexts

NOTE: The modules - ***queue*** and ***asyncio*** implementations aren't much used in DSA/CP context

6.3.1 - Using List

Wednesday, October 22, 2025 9:19 PM





6.3.2 - Using Linked List

Wednesday, October 22, 2025 9:19 PM

front
rear
None

F,R
10

F
20
R
30

F
10
20
30
R
40

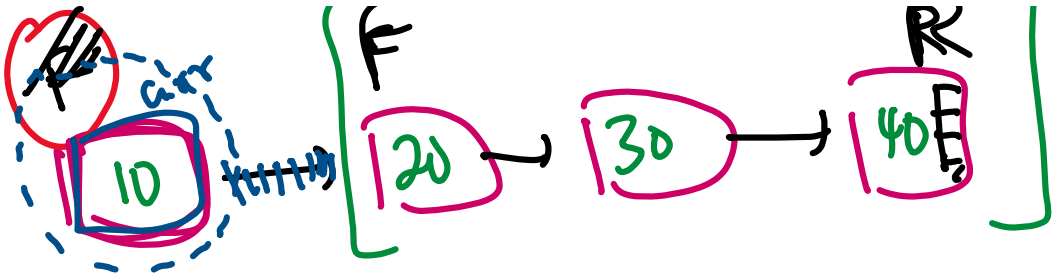
rear.next → node
rear → node

F, R → None

F, R
10

front = rear = node

1.



2.

curr = front

front → front.next

front → None

rear → None

return curr.data

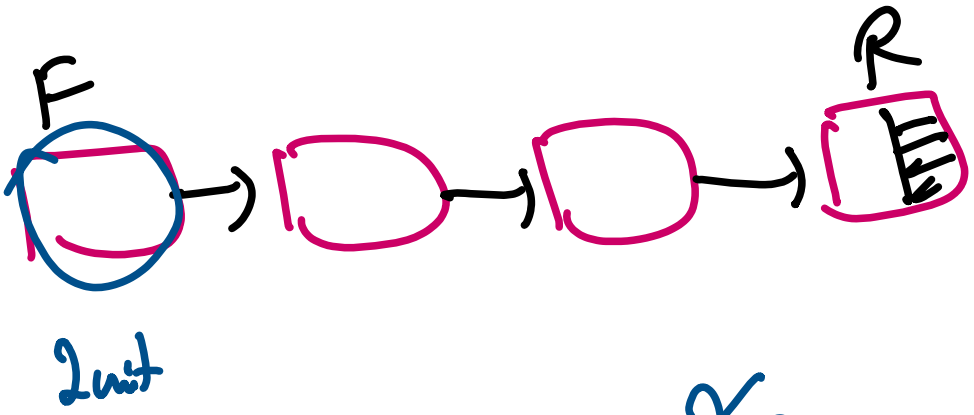
6.3.3 - Using collections

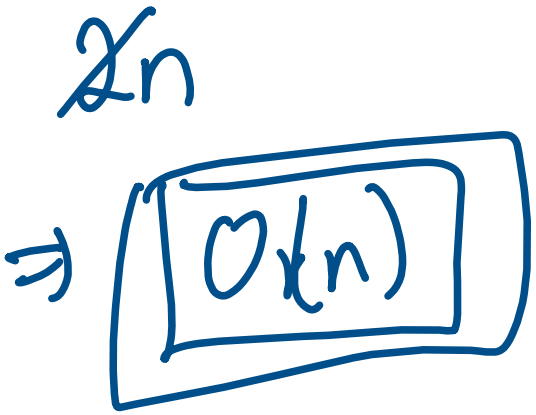
Wednesday, October 22, 2025 9:19 PM

6.4 - Complexity Analysis

Saturday, October 18, 2025 3:24 PM

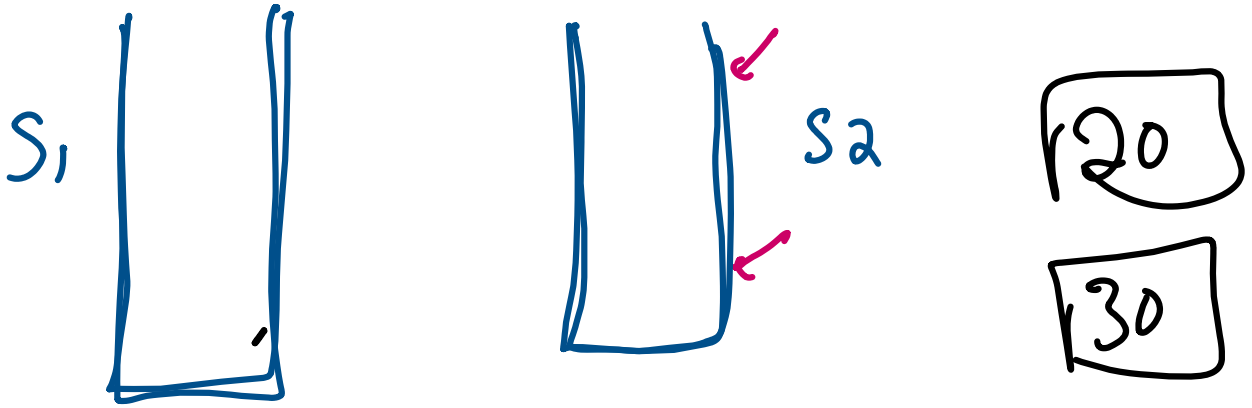
- **Enqueue:** O(1)
- **Dequeue:** O(1)
- **FrontPeek:** O(1)
- **RearPeek:** O(1)
- **isEmpty:** O(1)





6.5 - Coding Problems

Saturday, October 18, 2025 3:24 PM



Enqueue :

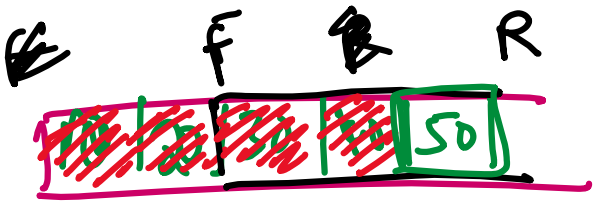
S1 → push

Dequeue:

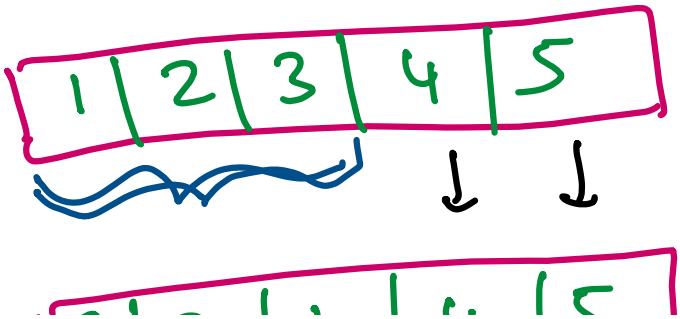
S1.pop() ⟶ S2.push()

S2.pop()

S1 → empty
S2 → empty
→ Queue empty
↓
Cannot Dequeue



q → queue
k → integer



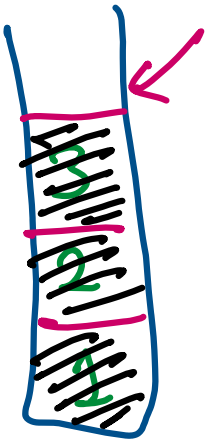
k = 3

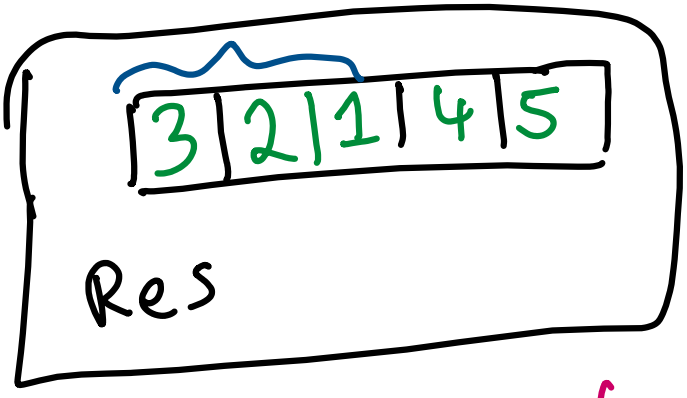
k=3

for range(k)

S.push(q.deque())

res.enque(S.pop())





res.enque(q.deque())

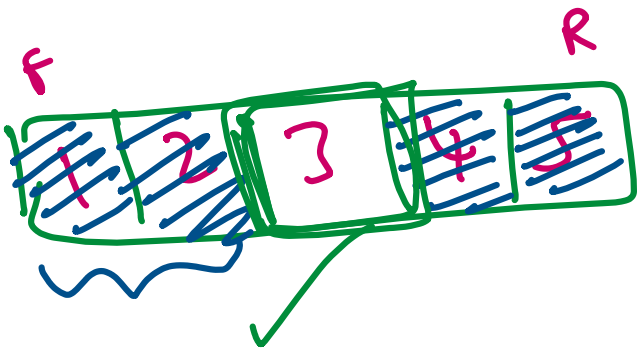
n

k

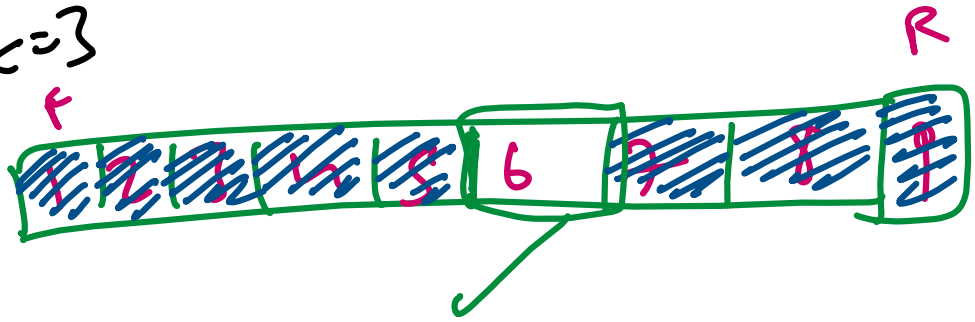
$k \leq n$

R

n=5
k=2



n=9
k=3



if n → 1:

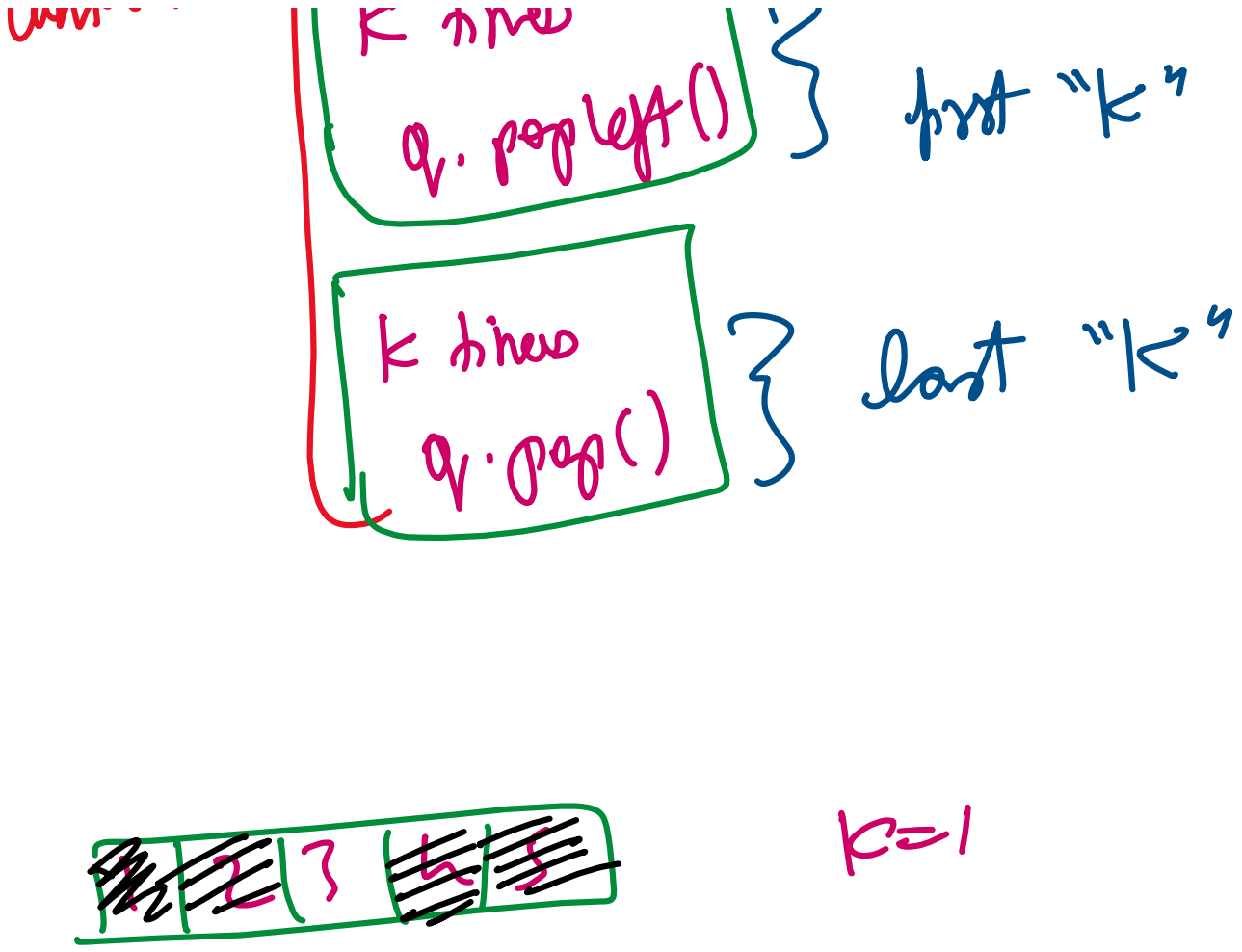
return n

q = deque()
for x in range(n):
q.append(x)

} create queue

while True: [k times

} Last "1"


k times
q.popleft()
first "k"
k times
q.pop()
last "k"

k=1

6.6 - DS Context

Tuesday, December 16, 2025 2:06 AM

Data Science / ML Correlation:

- Mini-batch training
- Stochastic Gradient Descent (SGD) batches

Libraries & Usage:

- TensorFlow Data API (***tf.data.Dataset***)
- PyTorch ***DataLoader***

7. Searching

Wednesday, October 29, 2025 10:09 PM

- **Linear Search**
- **Binary Search**
- **Exponential Search**

7.1 - Linear Search

Monday, November 3, 2025 9:30 PM

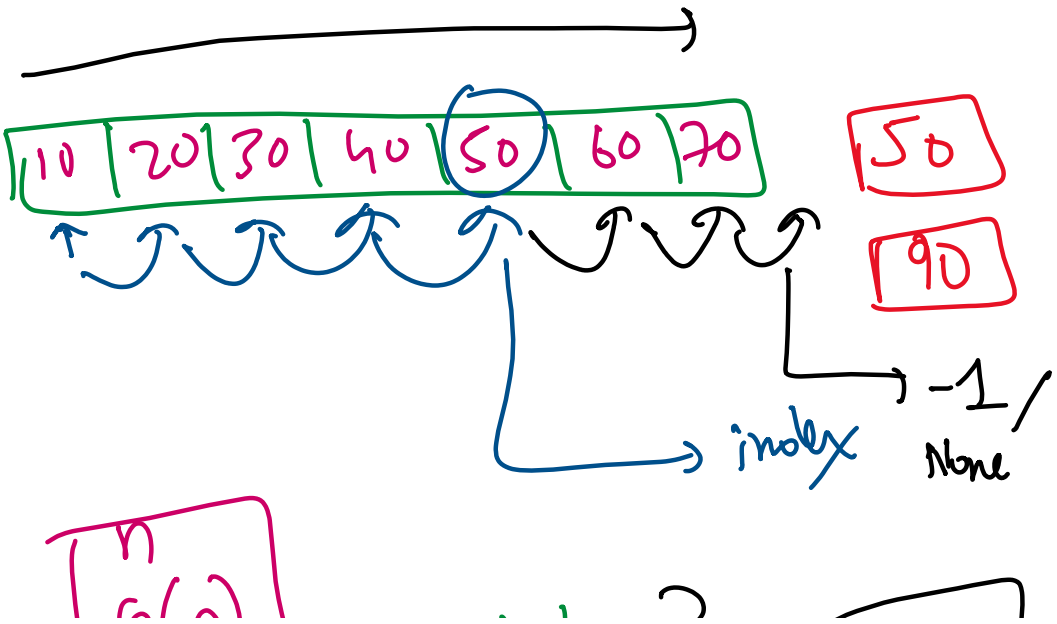
- Linear search, also known as sequential search, is a fundamental search algorithm used to find a specific element within a data structure (array, linked list, etc.)
- It operates by sequentially examining each element in the collection until either the target element is found or the end of the collection is reached

Working:

- The search begins at the first element of the data structure
- Each element is compared to the target value
- If a match is found, the index of the element is returned, and the search terminates
- If no match is found after checking all elements, the algorithm indicates that the element is not present

Characteristics:

- **Simplicity:** Linear search is easy to understand and implement
- **No Sorting Required:** It does not require the data to be sorted, making it suitable for unordered lists
- **Time Complexity:** In the worst-case scenario (element not present or at the very end), linear search has a time complexity of O(n)
- **Space Complexity:** It has a space complexity of O(1) as it only requires a constant amount of extra space



n

O(n)

index
temp

O(1)

7.2 - Binary Search

Monday, November 3, 2025 9:30 PM

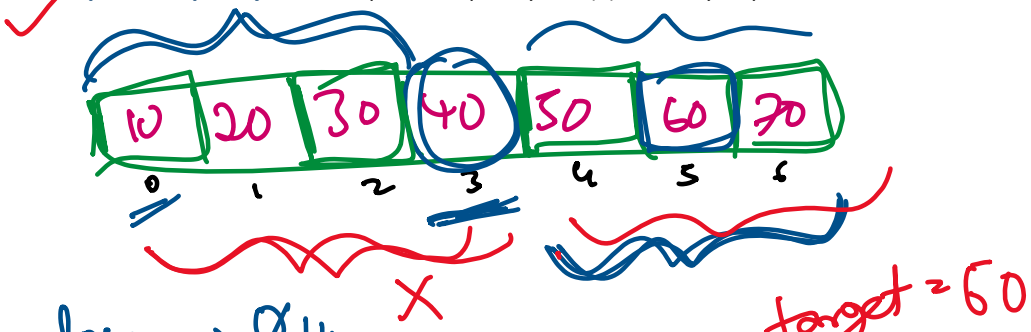
- Binary Search is an efficient algorithm used to find the position of a target value within a sorted array or list
- It works on the principle of "divide and conquer," repeatedly dividing the search interval in half

Working:

- **Initialize Pointers:** Set a low pointer to the beginning of the array and a high pointer to the end of the array
- **Calculate Midpoint:** Calculate the middle index mid as **low + (high - low) / 2** (to prevent potential integer overflow)
- **Compare and Adjust:**
  - If the element at **mid** is equal to the target value, the search is successful, and **mid** is returned
  - If the element at **mid** is less than the target value, the target must be in the right half of the array; Update **low** to **mid + 1**
  - If the element at **mid** is greater than the target value, the target must be in the left half of the array; Update **high** to **mid - 1**
- **Repeat:** Continue steps 2 and 3 until the target is found or the low pointer crosses the **high** pointer (indicating the target is not in the array)

Characteristics:

- **Sorted Data Requirement:** Binary Search is only applicable to data structures that are sorted (ascending or descending)
- **Time Complexity:** It has a time complexity of O(log N), making it significantly faster than linear search (O(N)) for large datasets
- **Space Complexity:** It has a space complexity of O(1) as it only requires a few variables to store pointers



low → 04

high → n-1 → 6

mid → low + (high - low)//2

$\frac{6-0}{2} \Rightarrow 3+0=3$

mid → 3

$4+\frac{(6-4)}{2} \rightarrow 5$

while low ≤ high :

mid = l + (h-l)//2

[mid] == target :

if arr[mid] == target:

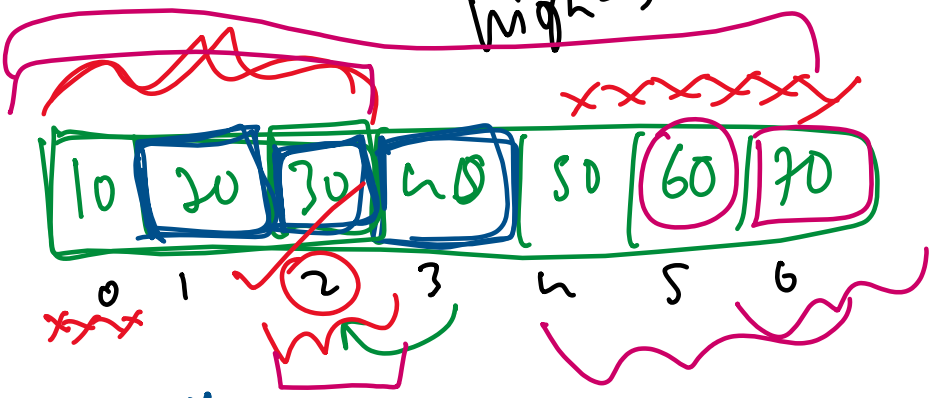
return mid

if arr[mid] < target:

low → mid + 1

else:

high → mid - 1



low → 2

high → 6
2

mid → $0 + \frac{(6-0)}{2} \rightarrow 0 + \frac{6}{2} \rightarrow 3$

$0 + \frac{(2-0)}{2} \rightarrow \frac{2}{2} \rightarrow 1$

$2 + \frac{(2-2)}{2} \rightarrow 2 + \frac{0}{2}$

$\rightarrow 2$

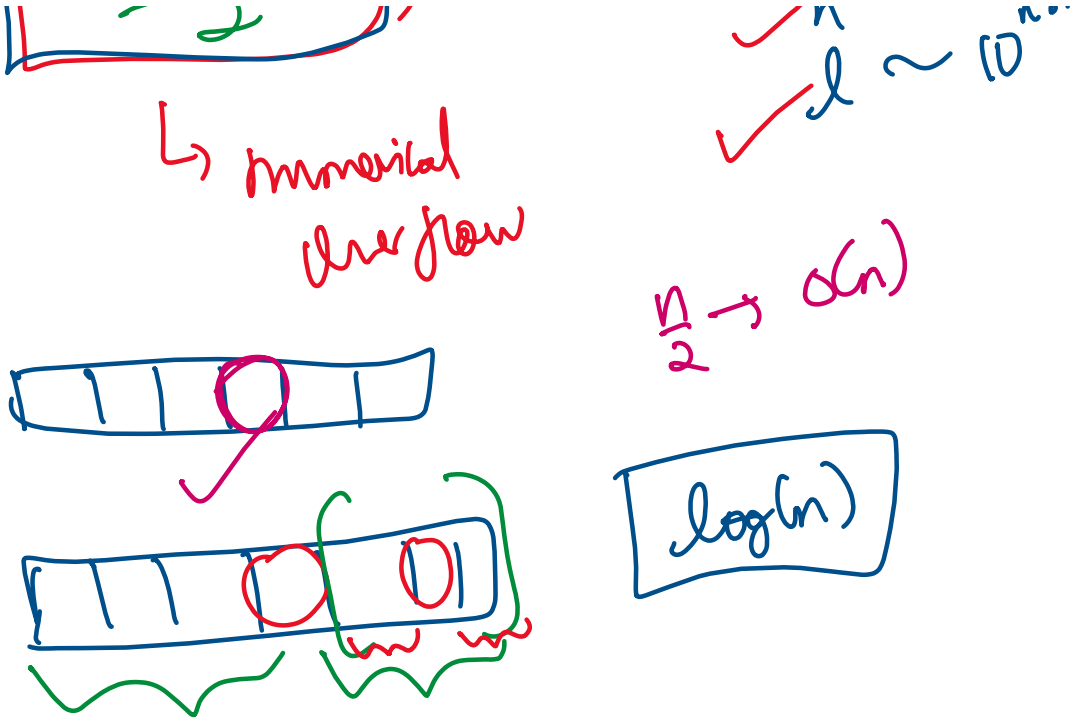
$low + \frac{(high - low)}{2}$

$\frac{(low + high)}{2}$ ✗

10^{31}

$h \sim 10^{31}$

$l \sim 10^{31}$


numerical
overflow
n/2 → O(n)
log(n)

7.3 - Exponential Search

Monday, November 3, 2025 9:30 PM

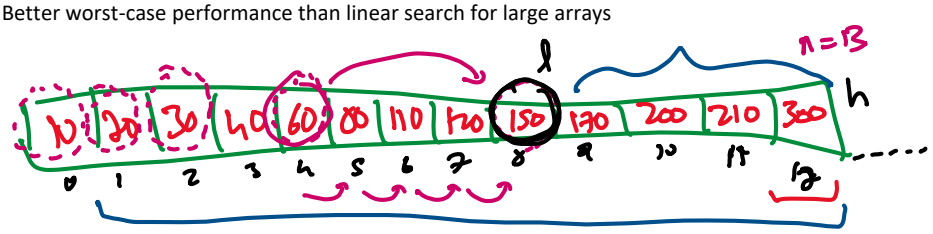
- Exponential Search, also known as Doubling Search or Galloping Search, is a search algorithm particularly efficient for sorted arrays
- Most useful when the array is very large or its size is unknown (unbounded arrays)

Working: *Operates in 2 main stages*

1. **Finding the Range:**
   - The algorithm starts by checking the first element of the array; If it's the target, the search ends
   - Otherwise, it iteratively doubles the index (**bound = 1, then 2, 4, 8, etc.**) as long as the element at the current bound index is less than the target value and the **bound** remains within the array limits
   - This stage aims to find a range **[bound/2, min(bound, n-1)]** where n is the array size, such that the target element, if present, must lie within this range
2. **Binary Search within the Range:**
   - Once the appropriate range is identified, a standard Binary Search algorithm is applied within this narrowed-down range to locate the exact position of the target element

Characteristics:

- Efficient for very large or unbounded sorted arrays, especially when the target is near the beginning
- Better worst-case performance than linear search for large arrays



Target → 170

i = 1 → 2 → 4 → 8 → 16 → 32 → 64, ...

i = 1 2 4 8 16 ✗

while (i < n):

    i *= 2

low → i//2 → 8

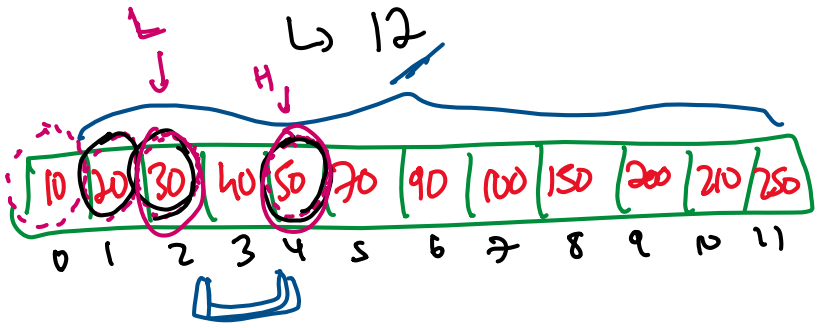
high → min(i, n-1)

↳ min(16, 13-1)

↳ min(16, 12)

↳ min(16, 12)

↳ 12



n = 12

target → 40

50 <= 40 ✗

i = 12 4

(i < n) & (arr[i] <= target)

low → 2 (i // 2)

high → min(4, 11) → 4

min(i, n-1)

7.4 - DS Context

Tuesday, December 16, 2025 4:15 AM

Data Science / ML Correlation:

- Hyperparameter tuning
- Threshold optimization
- Nearest neighbor search

Libraries & Usage:

- **scikit-learn:** ***GridSearchCV***
- **FAISS:** similarity search
- **scipy:** optimization

8. Sorting

Wednesday, November 5, 2025 10:11 PM

- **Introduction**
- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**
- **Merge Sort**
- **Quick Sort**
- **Counting Sort**
- **Complexity Analysis**

8.1 - Introduction

Tuesday, November 11, 2025 10:53 PM

What is Sorting?

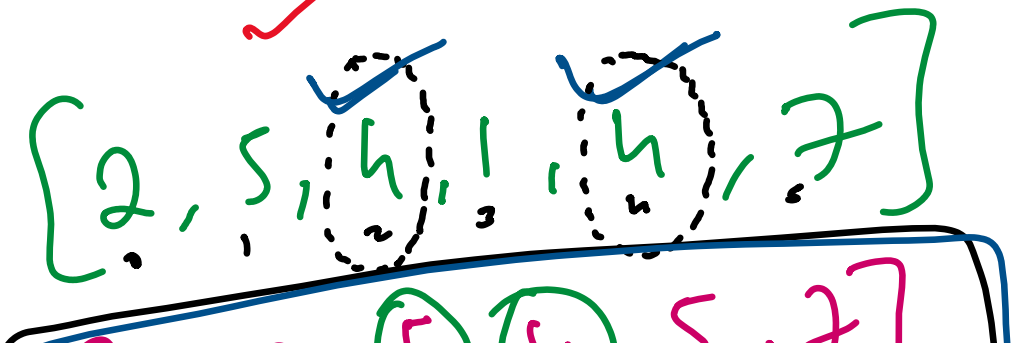
Sorting is the process of arranging data in a particular format or order (ascending or descending)

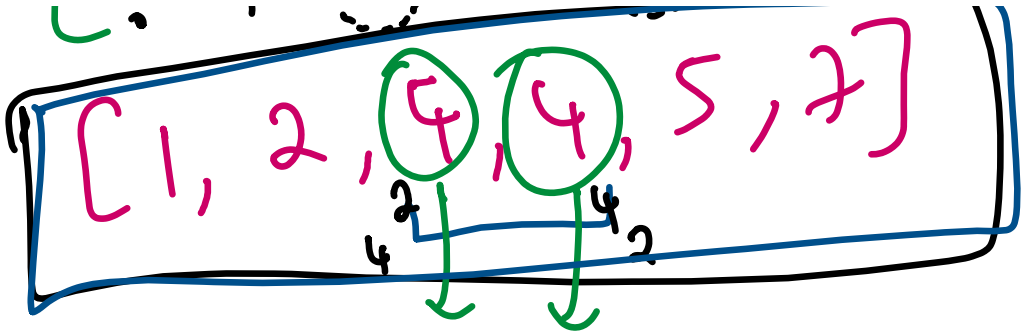
Why Sorting?

- Helps in efficient searching (e.g., Binary Search)
- Simplifies data visualization and analysis
- Essential in data processing, ranking, and optimization problems

Classifications of Sorting:

1. **Based on Implementation:**
   - **Internal Sorting:** Entire data fits in main memory (e.g., Quick Sort, Merge Sort)
   - **External Sorting:** Data stored in external memory (e.g., External Merge Sort)
2. **Based on Stability:**
   - **Stable:** Equal elements maintain their relative order (e.g., Merge Sort, Insertion Sort)
   - **Unstable:** Equal elements may change order (e.g., Quick Sort, Heap Sort)
3. **Based on Time Complexity:**
   - **Quadratic Time ($O(n^2)$):** Bubble, Selection, Insertion
   - **Log-linear Time (O(n log n)):** Merge, Quick, Heap




[1, 2, 4, 4, 5, 7]
2
4
4
2

8.2 - Bubble Sort

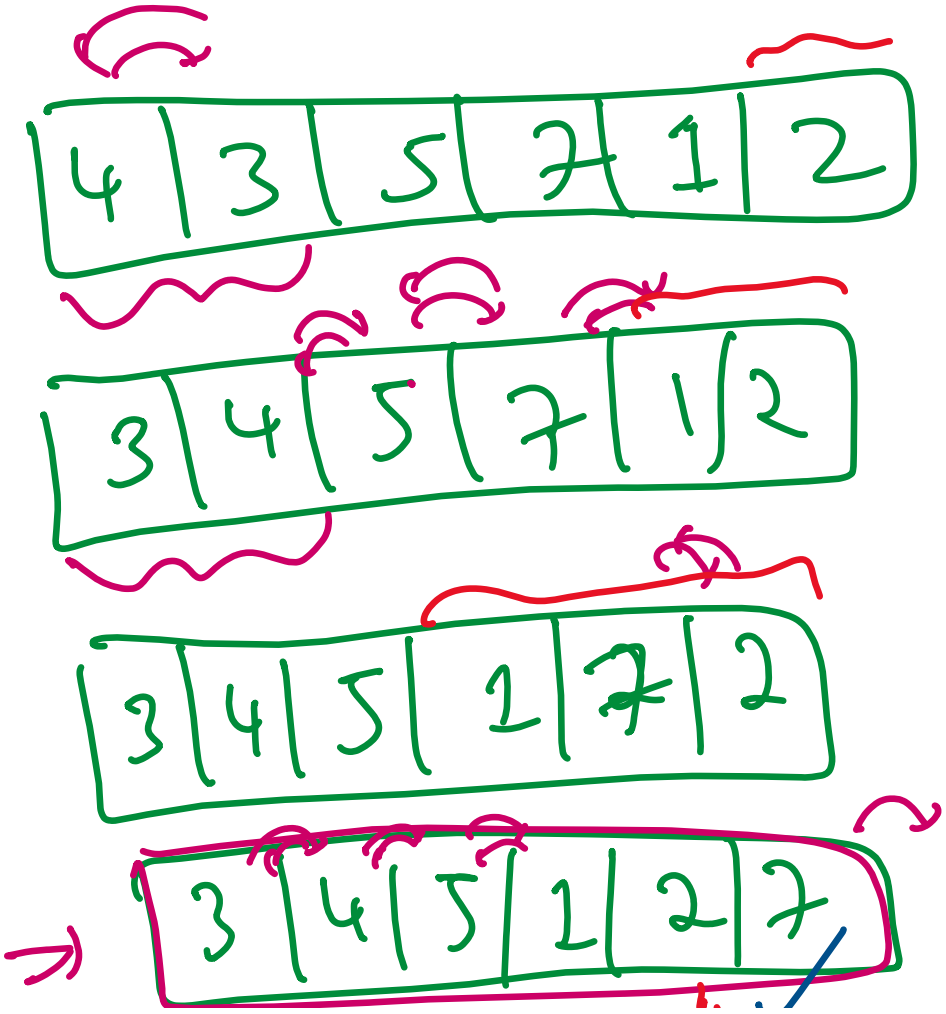
Tuesday, November 11, 2025 11:00 PM

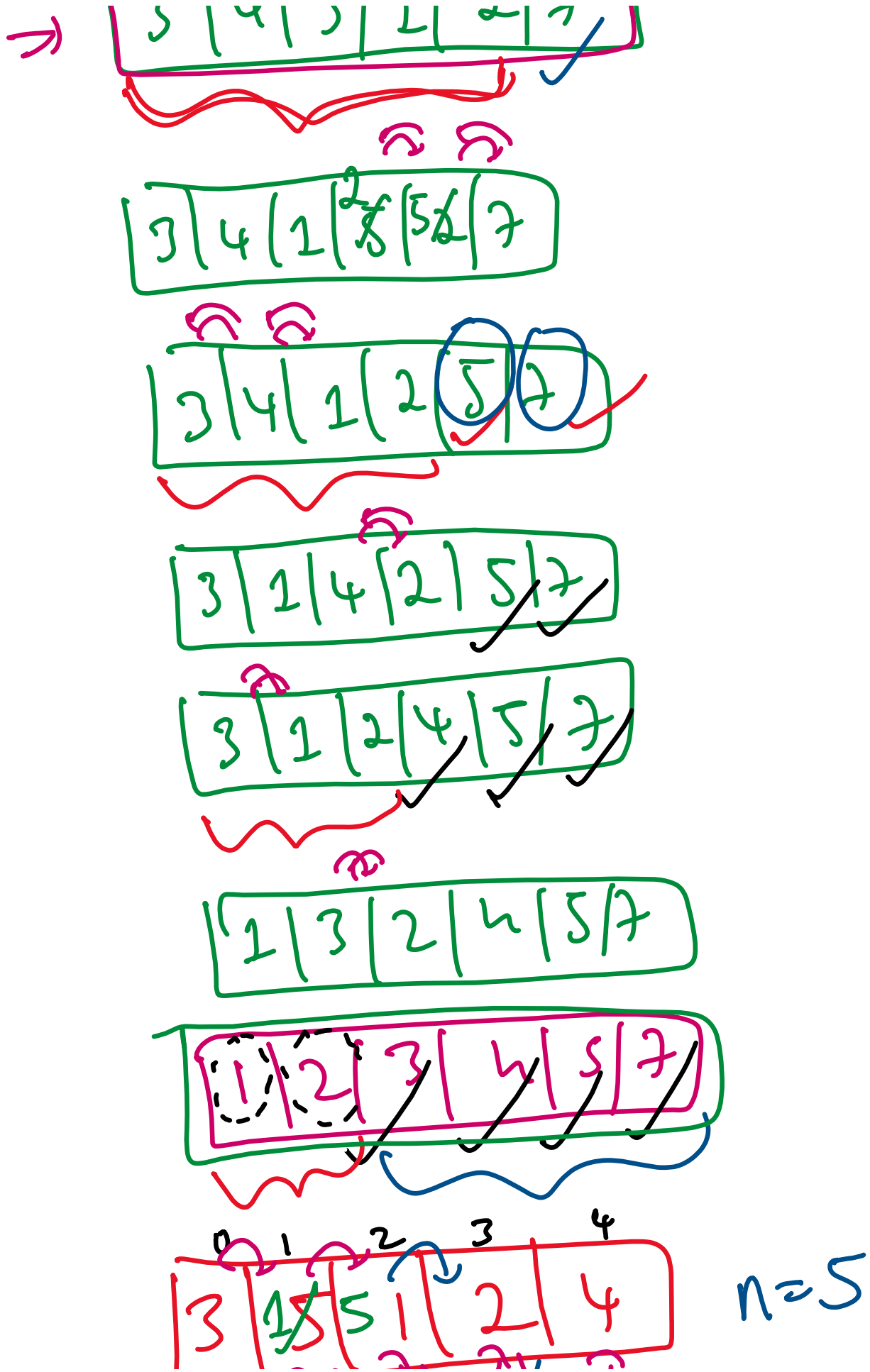
Main Ideas:

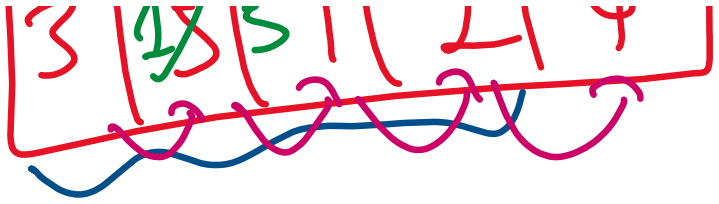
- Compare **neighboring pairs**
- Swap if they're out of order
- Keep repeating until no swaps are needed
- After each pass, the largest element moves to the end

Key Points:

- Very easy to understand and implement
- Inefficient for large lists




3 4 1 2 5 7
3 4 1 2 5 7
3 4 1 2 5 7
3 1 4 2 5 7
3 1 2 4 5 7
1 3 2 4 5 7
1 2 3 4 5 7
0 1 2 3 4
3 1 5 1 2 4
n=5



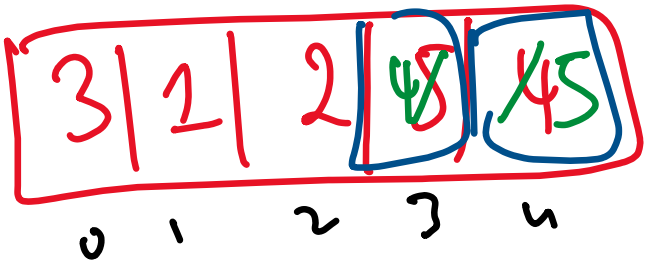
i = 1

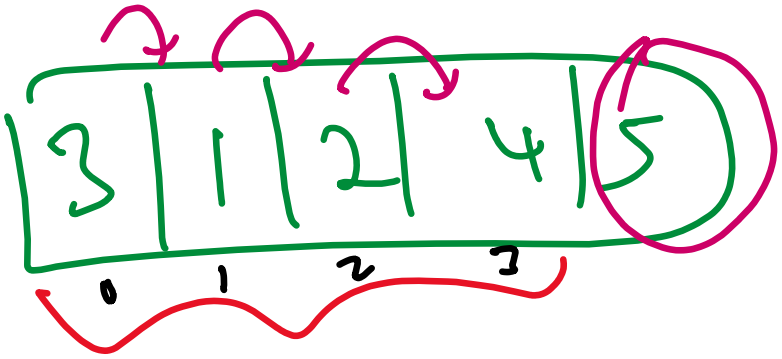
swapp → F

j → 0 → n-1-i

→ 5-1-0 → 4

→ 0 < 3





j → 5-1-1

→ 3

→ j (0, 3)

→ (0, 1, 2)

Applications:

- **Data Validation:** Quickly detect whether data is already sorted
- **Test Case Generation:** Verify correctness of complex sorting implementations

8.3 - Selection Sort

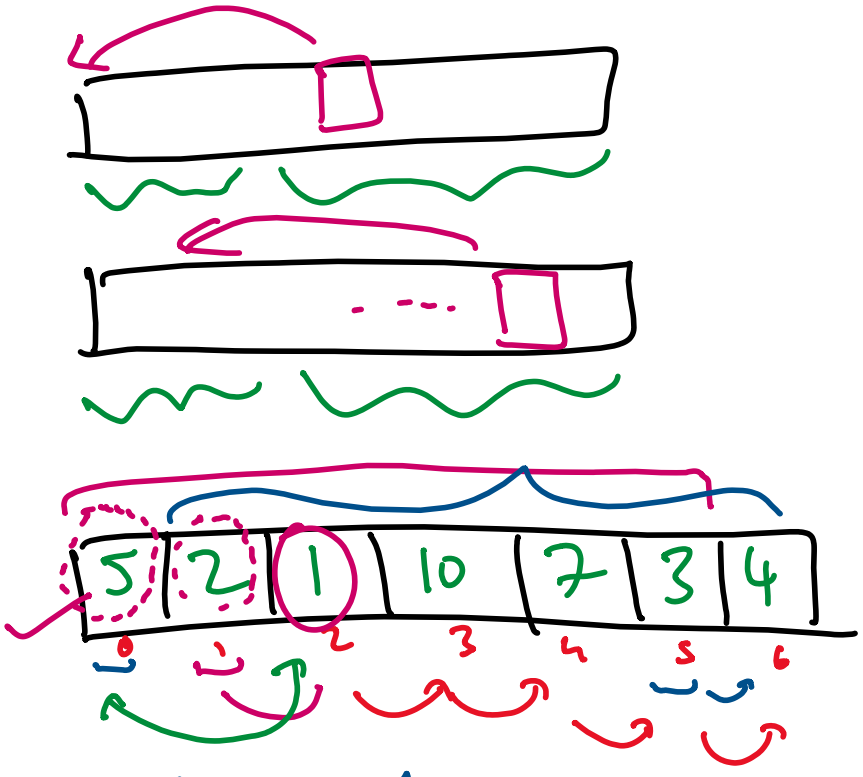
Tuesday, November 11, 2025 11:14 PM

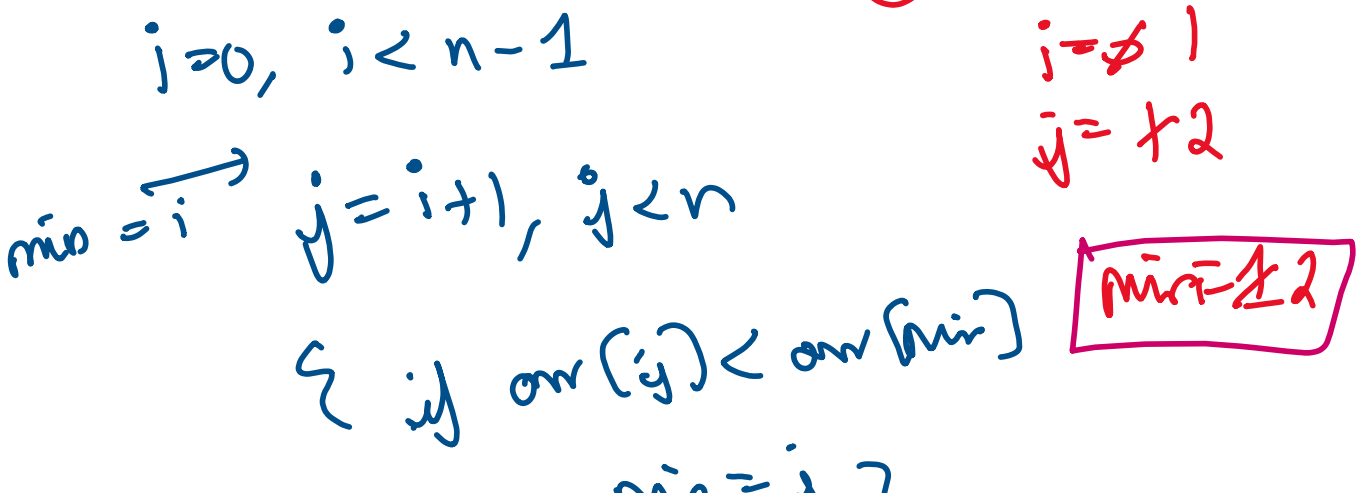
Main Ideas:

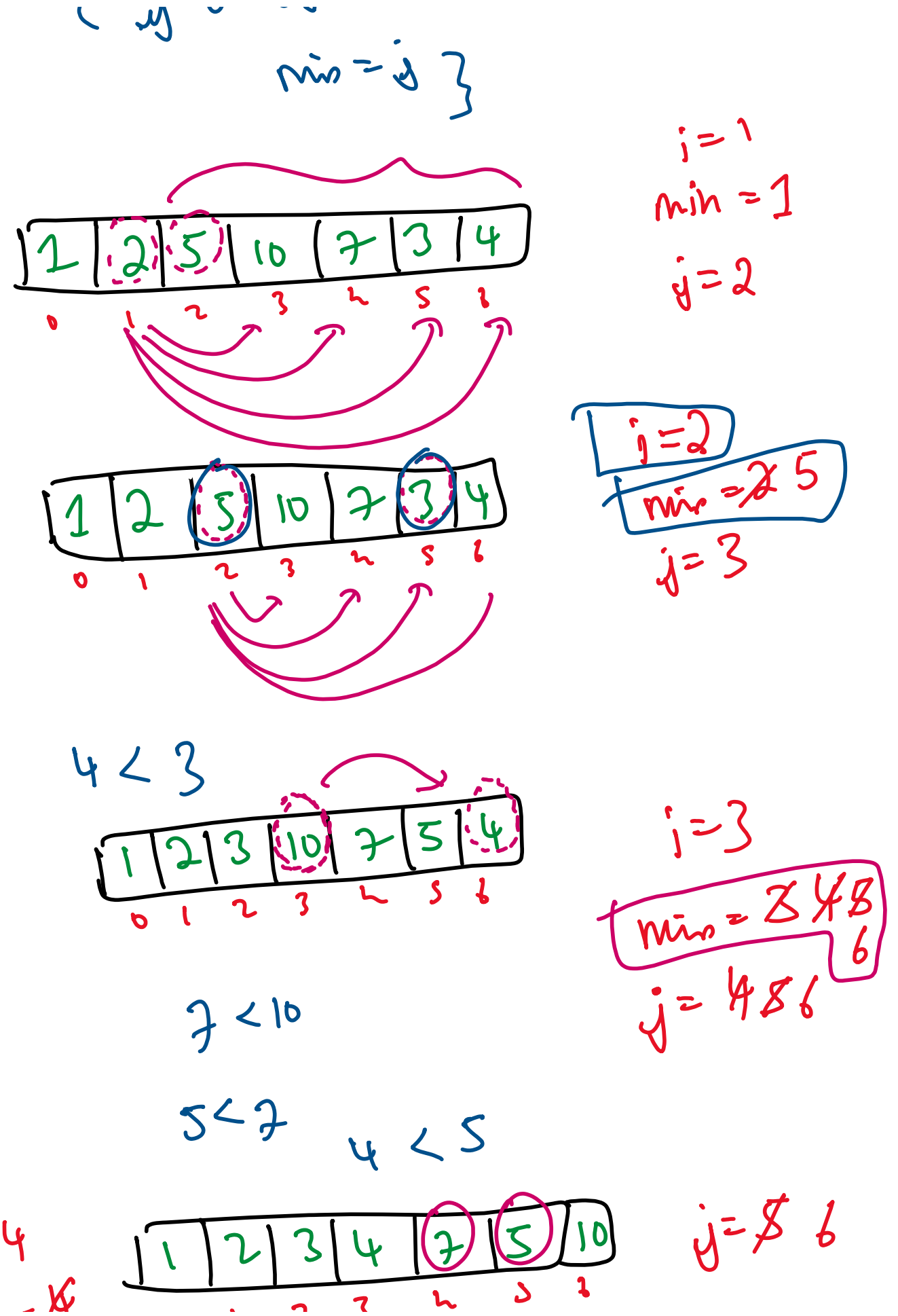
- Find the smallest element in the unsorted part of the list
- Swap it with the first element of the unsorted section
- Move the boundary between sorted and unsorted parts one step forward and repeat until the list is sorted

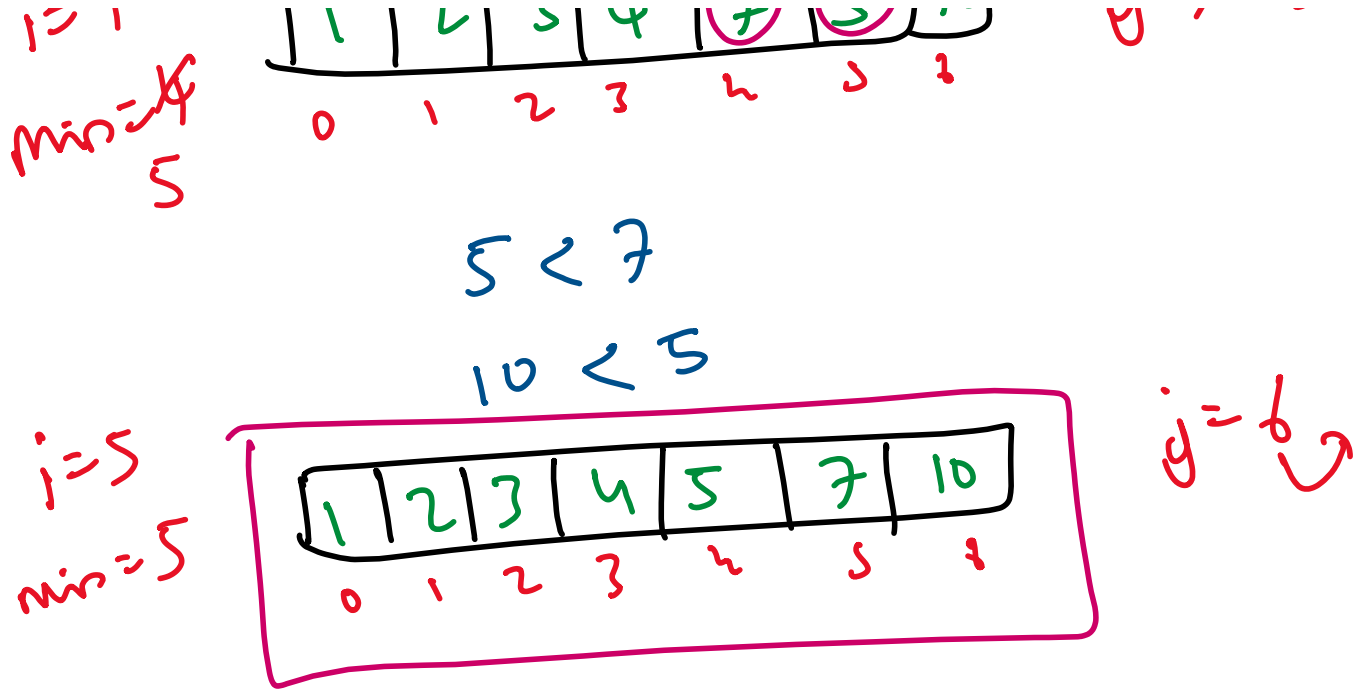
Key Points:

- Makes fewer swaps than Bubble Sort
- Inefficient for large lists since it needs $O(n^2)$ comparisons






min = j }
i = 1
min = 1
j = 2
1 2 5 10 7 3 4
0 1 2 3 4 5 6
i = 2
min = 2 5
j = 3
1 2 5 10 7 3 4
0 1 2 3 4 5 6
4 < 3
1 2 3 10 7 5 4
0 1 2 3 4 5 6
i = 3
min = 3 4 5
6
j = 4 5 6
7 < 10
5 < 7
4 < 5
i = 4
1 2 3 4 7 5 10
j = 5 6



Applications:

- **Flash Memory:** Writes degrade hardware lifespan
- **Firmware & IoT systems:** Sorting configuration tables

8.4 - Insertion Sort

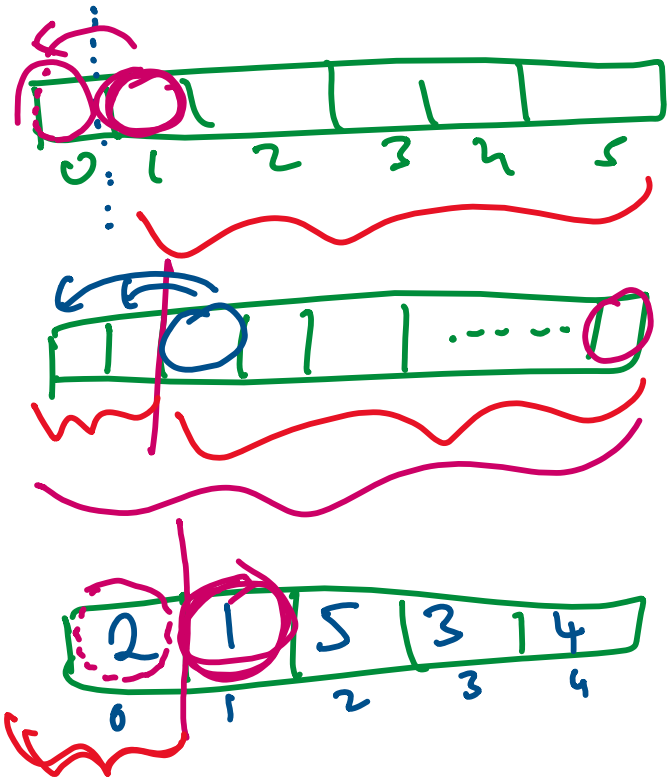
Tuesday, November 11, 2025 11:21 PM

Main Ideas:

- Start from the second element and compare it with the elements before it
- Insert the current element into its correct position in the already sorted part of the list
- Repeat this for all elements until the entire list is sorted

Key Points:

- Works well for small lists or lists that are almost sorted
- Sorting happens in-place (no extra memory needed)
- Inefficient for large lists



i=1. i<n, i++

i=1 key=arr[i]

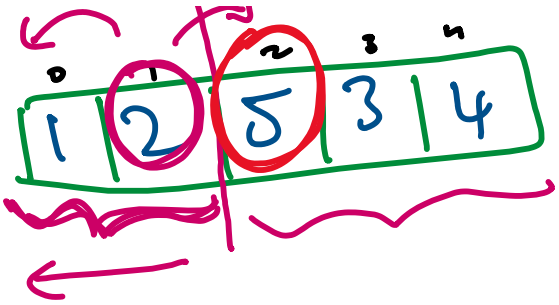
j=i-1

(j≥0 & arr[j]>key)

arr[j+1]=arr[j]

j=i-1
=0

arr[i]=1
arr[j]=2



arr[j+1] = arr[j]

j = j-1

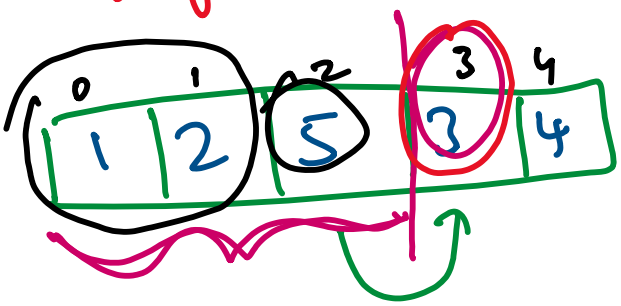
arr[j+1] = key

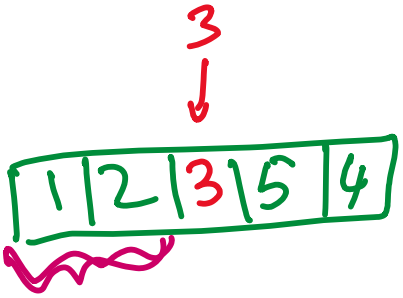
i=2

key=5

j=1

arr[j] = 2





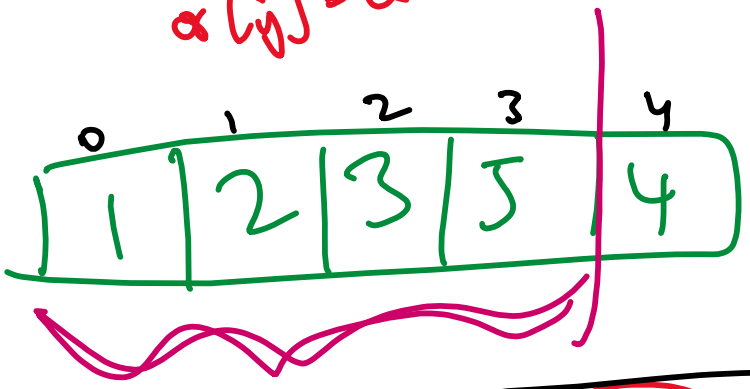
i=3

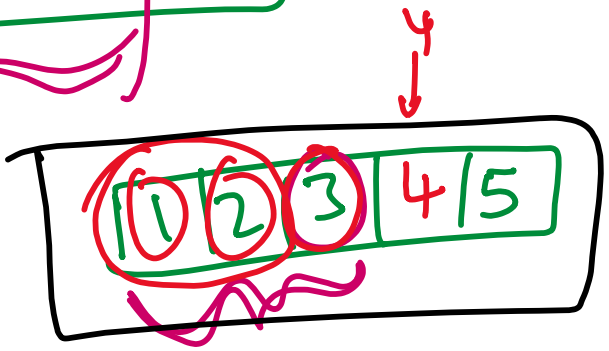
key = 3

j = 2 1

arr[j] = 5

arr[j] = 2





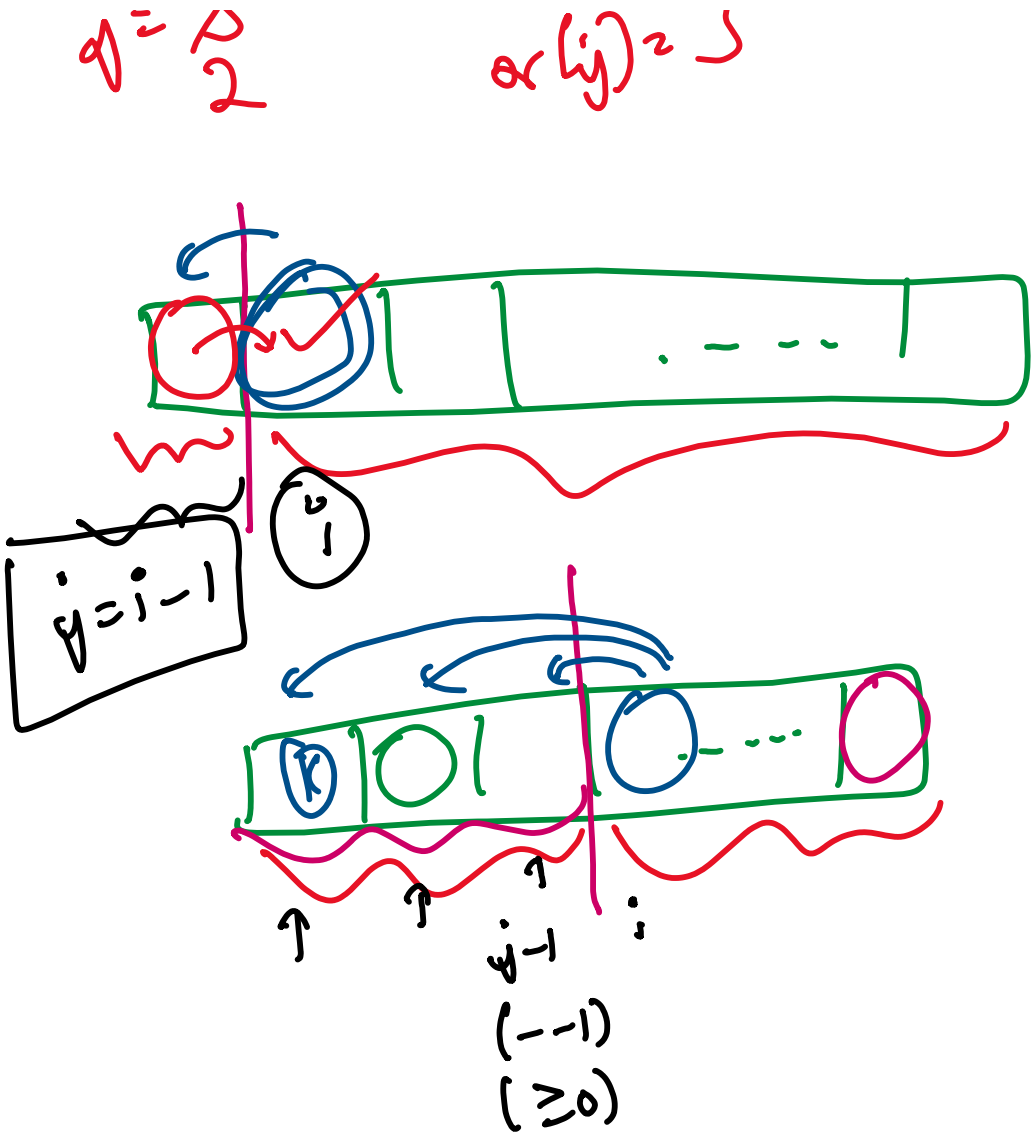
i=4

key = 4

j = 3 2

arr[j] = 5

arr[j] = 3



Applications:

- **Real-time streaming systems:** New data arrives continuously
- **Maintaining sorted order incrementally**

8.5 - Merge Sort

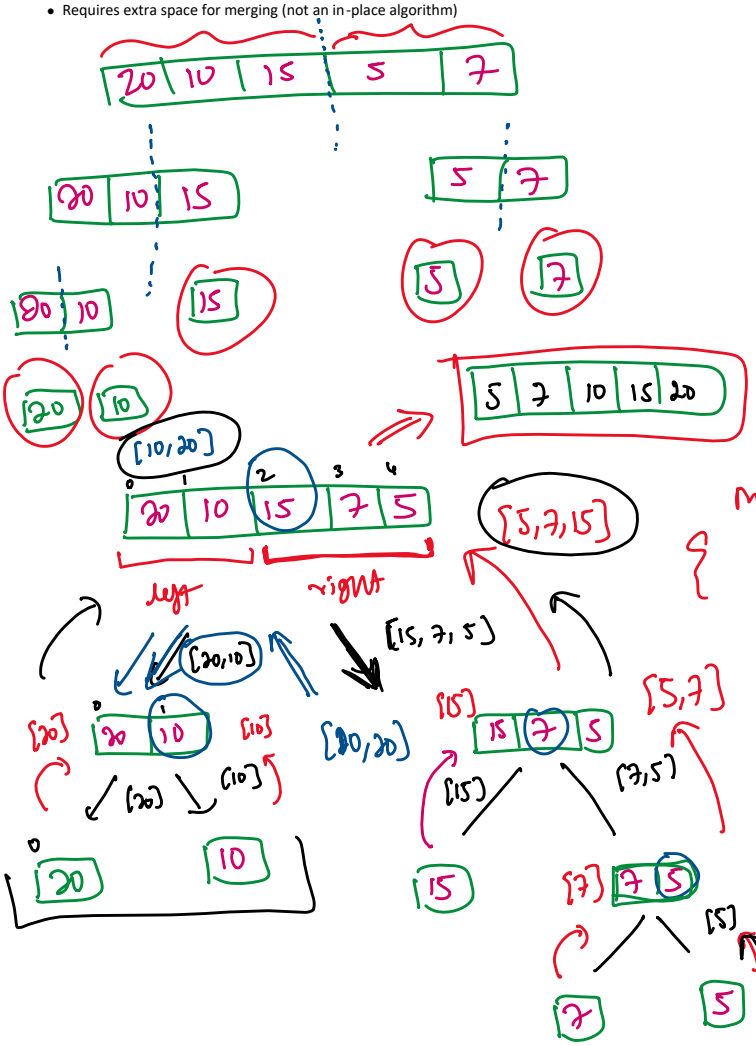
Tuesday, November 11, 2025 11:41 PM

Main Ideas:

- Divide the list into two halves until each sublist has only one element
- Merge the sublists by comparing elements and combining them in sorted order
- Repeat merging until one fully sorted list is formed

Key Points:

- Based on the Divide and Conquer strategy
- Very efficient for large datasets, with time complexity O(n log n)
- Requires extra space for merging (not an in-place algorithm)




```
mid → 2
result = [10,20
mergeSort ( )
{
if len(arr) == 1:
    retur arr
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
return merge(left,right)
}
```

merge ([15], [5,7])

left → [15] i=0, len(left)=1

right → [5,7] j=~~0~~ 1, len(right)=2

15 ~~&~~ 7 result = [5,7,15]

left right

merge ([10,20], [~~5~~,~~7~~, 15])

```
merge ( [7]  [5] )
{ result = []
  i = j = 0
  while (i < len(left)) & (j < len(right))
  {  if left[i] < right[j]
     { result.append(left[i])
       i++
```

[20] [10]
i=0 j=~~0~~ 1

[5, 7]

merge ([10,20], [5,7,15])

i → 01

j → 01 2 3

result → [5, 7, 10, 15, 20]

Applications:

- **External sorting** (data > RAM)
- **Distributed computing frameworks**
- **Stable sorting requirements**

```
{ result. appe
    i++
}
else
{   result.append (right[j])
    j++
}
if (left)
result. extend (left [i:])
if (right)
result. extend [right [j:])
return result
}
```

8.6 - Quick Sort

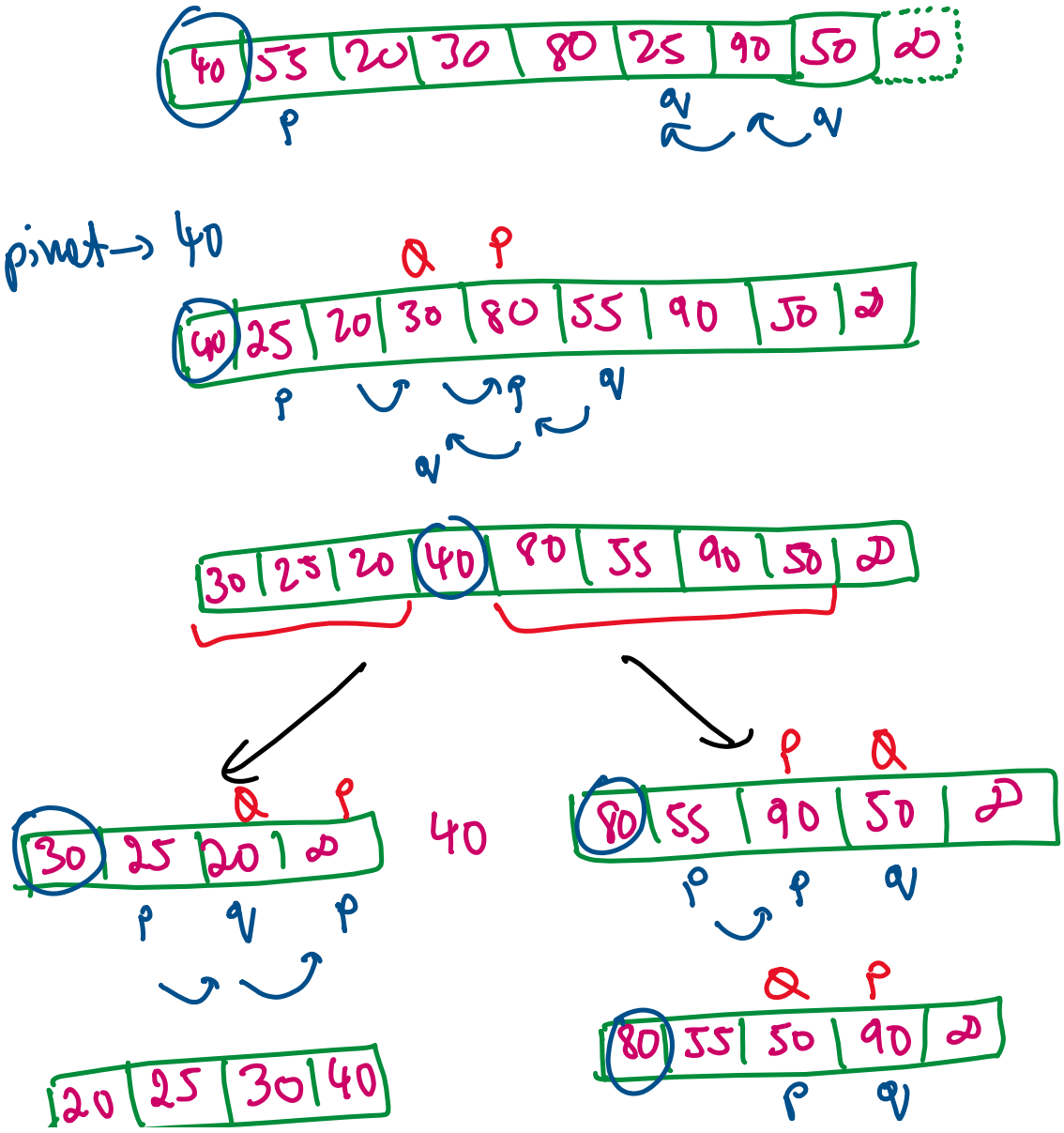
Tuesday, November 11, 2025 11:47 PM

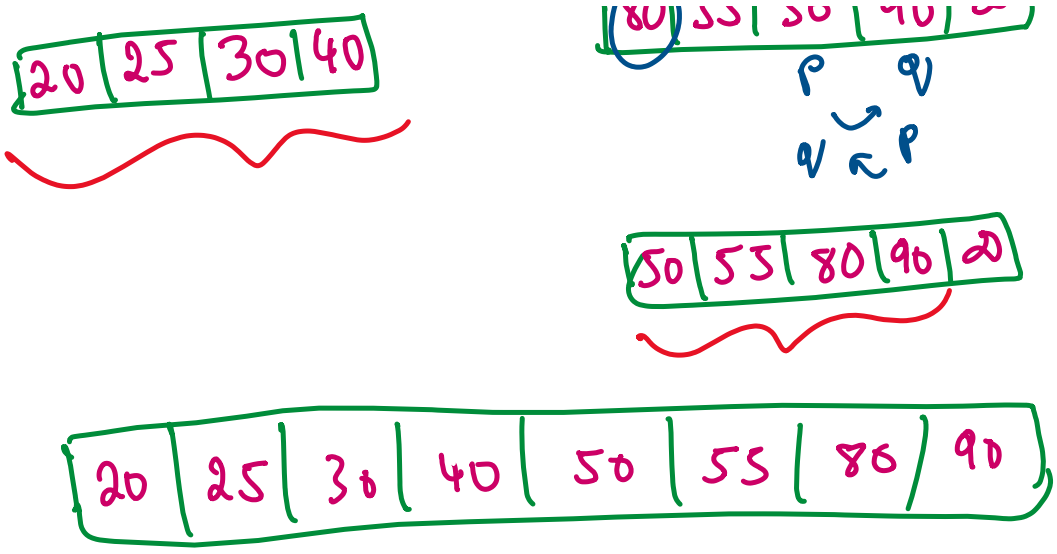
Main Ideas:

- Choose a pivot element from the list
- Rearrange (partition) the list so that all elements smaller than the pivot are on its left, and all greater elements are on its right
- Recursively apply the same process to the left and right sublists until the list is sorted

Key Points:

- Based on the Divide and Conquer strategy
- Faster than Merge Sort in practice for many datasets (average case: O(n log n))
- In-place sorting algorithm (needs little extra space)





Applications:

- **Better Applications**
- **System libraries**
- **Low-latency applications**
- **Real-time analytics**

8.7 - Counting Sort

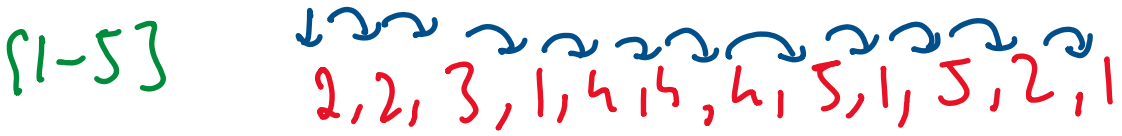
Tuesday, November 11, 2025 11:23 PM

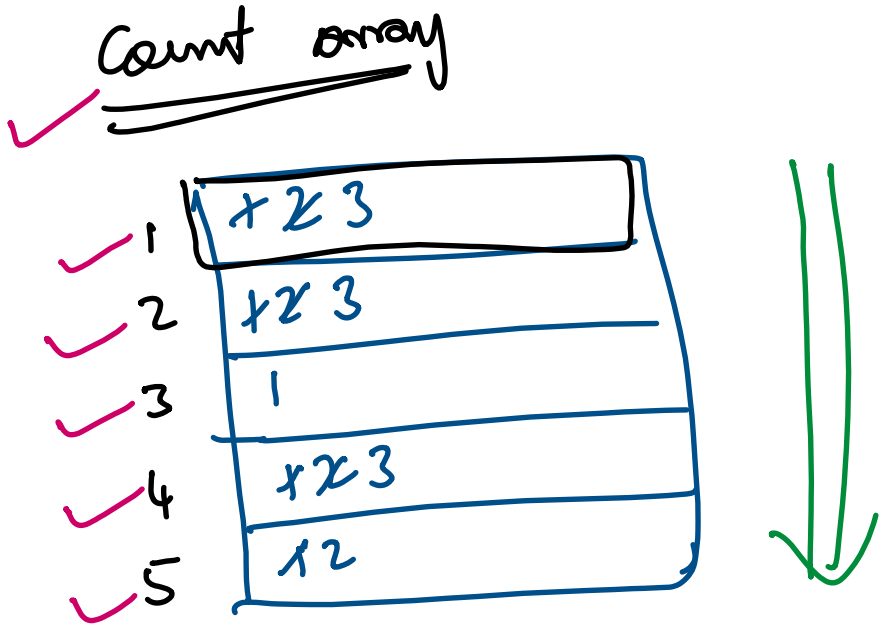
Main Ideas:

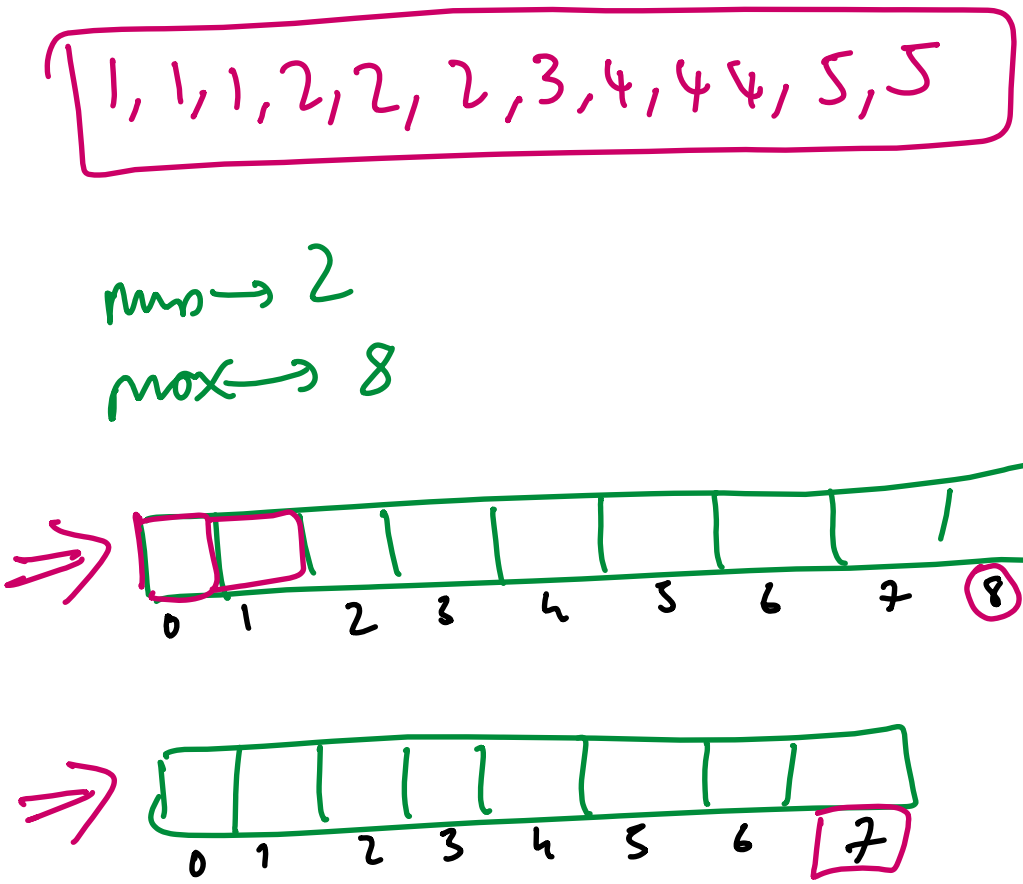
- Count the frequency of each unique element in the input list
- Store these counts in an auxiliary array (called the count array)
- Use the count array to place each element directly in its correct sorted position in the output array

Key Points:

- Works best for integers or discrete values within a small range
- Very fast when the range of input values is not much larger than the number of elements (time complexity O(n + k))
- Not comparison-based and stable (if implemented carefully)
- Requires extra space for the count array, so not ideal for large ranges







Applications:

- **Better Applications**
- **Voting systems**
- **Telemetry & sensor data**
- **Categorical data processing**
- **Image processing**

8.8 - Complexity Analysis

Tuesday, November 11, 2025 11:49 PM

ALGORITHM	TIME	SPACE	STABLE	TYPE
Bubble Sort	$O(n^2)$	O(1)	✅	Comparison
Selection Sort	$O(n^2)$	O(1)	❌	Comparison
Insertion Sort	$O(n^2)$	O(1)	✅	Comparison
Merge Sort	O(n log n)	O(n)	✅	Comparison
Quick Sort	O(n log n)	O(log n)	❌	Comparison
Counting Sort	O(n + k)	O(k)	✅	Non-Comparison

$K \Rightarrow$ $\boxed{n + K}$

8.9 - DS Context

Tuesday, December 16, 2025 4:21 AM

Data Science / ML Correlation:

- Ranking predictions
- Feature importance ordering
- Evaluation metrics (Top-K)

Libraries & Usage:

- **numpy.argsort()**
- **pandas.sort_values()**
- Recommendation systems

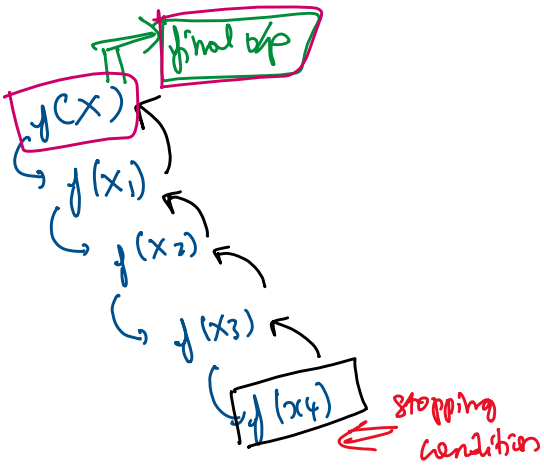
9. Recursion

Sunday, November 23, 2025 5:51 PM

- **Introduction to Recursion**
- **Internal Working of Recursion**
- **Common Mistakes**
- **Recursion vs Iteration**
- **Coding Problems**

9.1 - Introduction

Monday, November 24, 2025 8:53 PM



What is Recursion?

- Recursion is a programming technique where a function calls itself to solve smaller subproblems of the original problem
- This process continues until a specific base case (or stopping condition) is met
- At this point the recursion terminates and the results are combined back up the chain of calls to solve the original problem

Structure of Recursive Functions:

A recursive solution typically has two components -

1. **Base Case:**
   - Stops recursion from continuing infinitely
   - Should always be reachable
   - Returns the answer for the smallest version of the problem
2. **Recursive Case:**
   - Divides the problem into smaller versions
   - Should always progress toward base case

```
def recursive_function(parameters):
    if base_condition:
        return base_value
    else:
        return recursive_function(smaller_parameters)
```

Applications of Recursion:

- **Tree Traversals:** Algorithms like in-order, pre-order, and post-order traversal are inherently recursive
- **Graph Algorithms:** Depth-First Search (DFS) uses recursion to explore all possible paths in a graph
- **Sorting Algorithms:** Quick Sort and Merge Sort are divide-and-conquer algorithms that use recursion to sort subarrays
- **Mathematical Problems:** Calculating factorials, Fibonacci sequences, and the Tower of Hanoi puzzle are classic examples solved with recursion
- **Backtracking:** Problems like solving a Sudoku puzzle or the N-Queens problem often rely heavily on recursion to explore different possibilities

Advantages:

- Leads to shorter, cleaner, and more intuitive code
- Ideal for **Divide and Conquer** algorithms
- Good for **Backtracking**
- Helps model **Hierarchical Structures** (file system, XML, JSON)

Disadvantages:

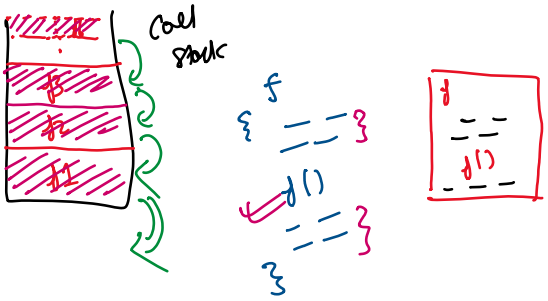
- High memory usage
- Risk of **Infinite Recursion**
- Harder to debug
- Not always efficient
- Limited by **Maximum Recursion Depth** (Python ~ 1,000 calls)

When to Use Recursion?

- Problem is naturally repetitive
- Tree/Graph traversal is needed
- Need for Divide and Conquer algorithm
- Backtracking/Search problems
- Code clarity is more important than raw speed

9.2 - Internal Working

Monday, November 24, 2025 8:53 PM



Recursion is managed using a Call Stack, which is a Last-In, First-Out (LIFO) data structure:

1. **When a function is called, a new stack frame (also known as an activation record) is created and pushed onto the call stack**
   - This frame stores the function's local variables, parameters, and the return address
2. **As the function recursively calls itself, new stack frames accumulate on top of the previous ones**
3. **Once the base case is reached, the functions start returning their results**
   - The top stack frame is popped off, and control returns to the function below it in the stack
   - The function uses the returned result to perform its remaining calculations and return its own result
   - This process is called ***stack unwinding*** or the ***ascending phase***

Examples:

- **Factorial**
- **Fibonacci Numbers**

$5! = 5\times4\times3\times2\times1$

$3! = 3\times2\times1$

$2! = 1$

$0! = 1$

$5! = 5\times4\times3\times2\times1$

$= 5\times[4\times3\times2\times1]$

$= 5\times4!$

$3! = 3\times2\times1$

$= 3\times2!$

$1! = 1\times0!$

fact(n) = n × fact(n-1)

n > n-1 ⇒ repetitive

fact(n-1) = (n-1) × fact(n-2)

1. Base condition
2. Recursive call

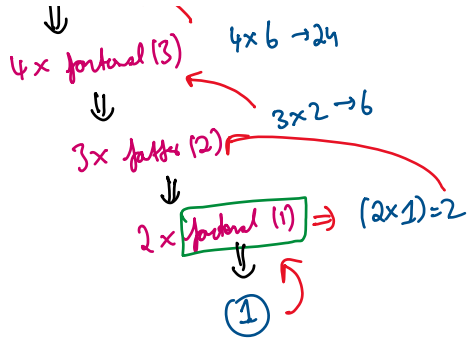
$n\times(n-1)\times(n-2)\times\ldots\times1$

```
def factorial(n):
    # base condition
    if (n==0) or (n==1):
        return 1
    # recursive call
    return n * factorial(n-1)
```

factorial(5) → 120

5 × factorial(4) — 5 × 24

4 × factorial(3) — 4 × 6 → 24



0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Let F → fibonacci

$$\begin{cases} F(0) = 0 \\ F(1) = 1 \end{cases} \Rightarrow \text{Base condition}$$

$F(2) = F(1) + F(0) = 0+1 = 1$

$F(3) = F(2) + F(1) = 1+1 = 2$

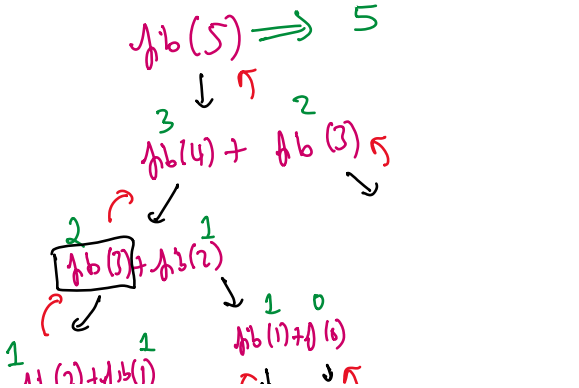
$F(4) = F(3) + F(2) = 2+1 = 3$

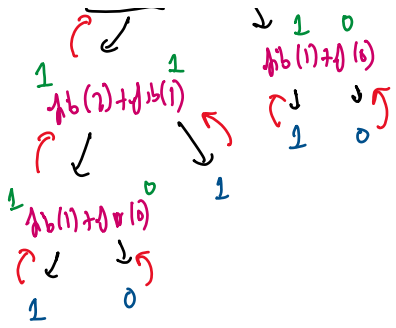
$F(5) = F(4) + F(3) = 3+2 = 5$

$$F(n) = F(n-1) + F(n-2)$$

$$n \geq 2$$

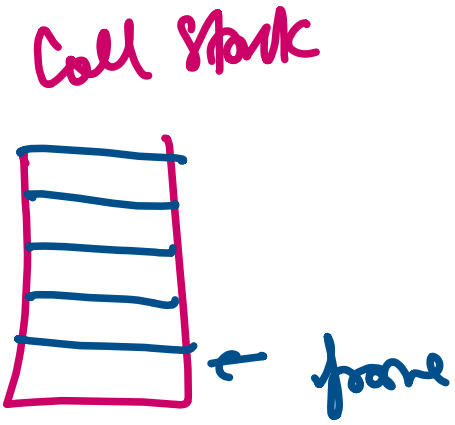
```
def fib(n):
    # base cond.
    if (n==0) or (n==1):
        return n
    # recursive call
    return fib(n-1) + fib(n-2)
```





9.3 - Common Mistakes

Monday, November 24, 2025 8:53 PM

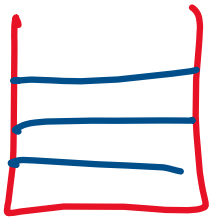


- **Forgetting the Base Case:**
  - Leads to infinite recursion
  - Overflow of the memory stack
- **Reducing the problem incorrectly:**
  - May skip base case
  - Could lead to infinite loop
- **Doing extra work in recursive return:**
  - Inefficient code

9.4 - Recursion vs Iteration

Monday, November 24, 2025 9:15 PM

RECURSION	ITERATION
Uses call stack ✓	Uses loops ✓
Elegant and mathematical ✓	Faster & memory efficient ✓
May be slower due to repeated calls ✓	No overhead of stack frames ✓
Easy for tree/graph and backtracking ✓	Hard for problems like DFS ✓



for, while

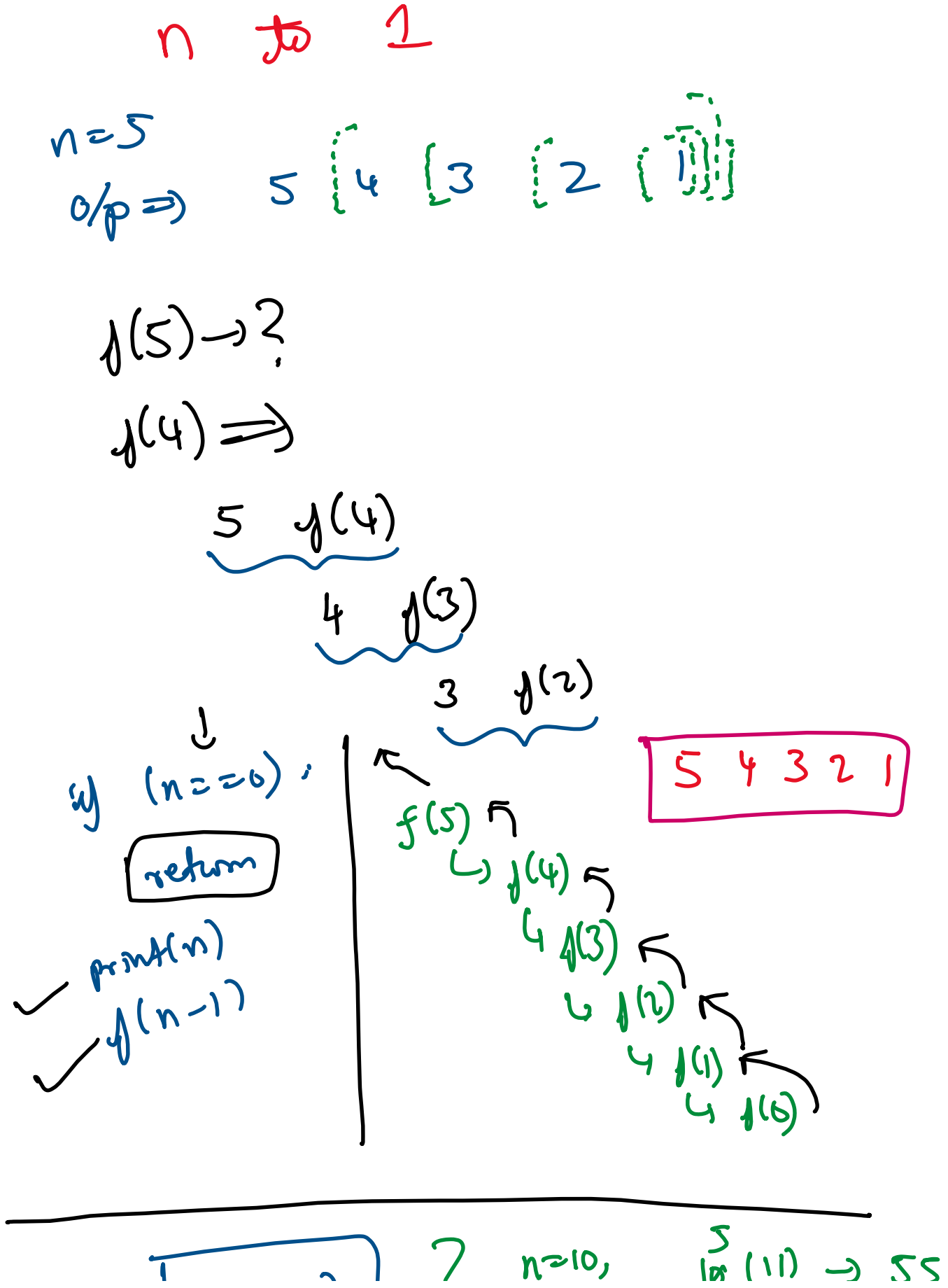
fact(n) = n x fact(n-1)

⇓

Recursive

9.5 - Coding Problems

Monday, November 24, 2025 9:31 PM



$y \Rightarrow \boxed{\frac{n(n+1)}{2}}$ $\Big\}$ $n=10$, $\frac{10(11)}{2} \to 55$

$n=5$, o/p : $5+4+3+2+1$

$= 15$

$n=7$, o/p : $7+6+[5+4+3+2+1]$

$= 28$

$n=6$, o/p : $6+[5+4+3+2+1]$

$f(6) = 6 + f(5)$

$f(7) = 7 + f(6)$

$f(3) = 3 + f(2)$

$f(2) = 2 + f(1)$

$f(1) = 1 + f(0)$

$\boxed{f(0) = 0}$

```
def fun(n):
    if n==0:
        return 0
    return n + f(n-1)
```

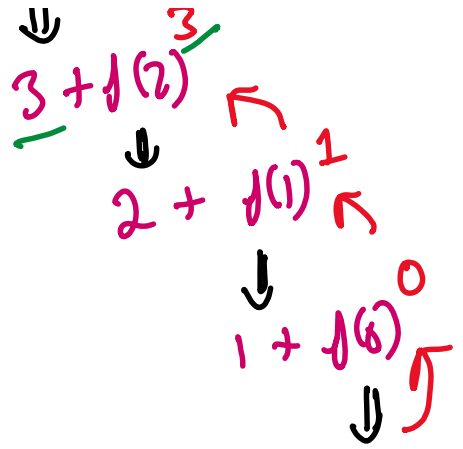
$f(4)$ 10 $\Rightarrow$ 10

$\Downarrow$ $\uparrow$

$4 + f(3)$ 6

$\Downarrow$ $\uparrow$

$3 + f(2)$ 3 $\leftarrow$



n = 4326

o/p → 4 n//10

count = 0 + 2 + 34

n = 4326

n // 10

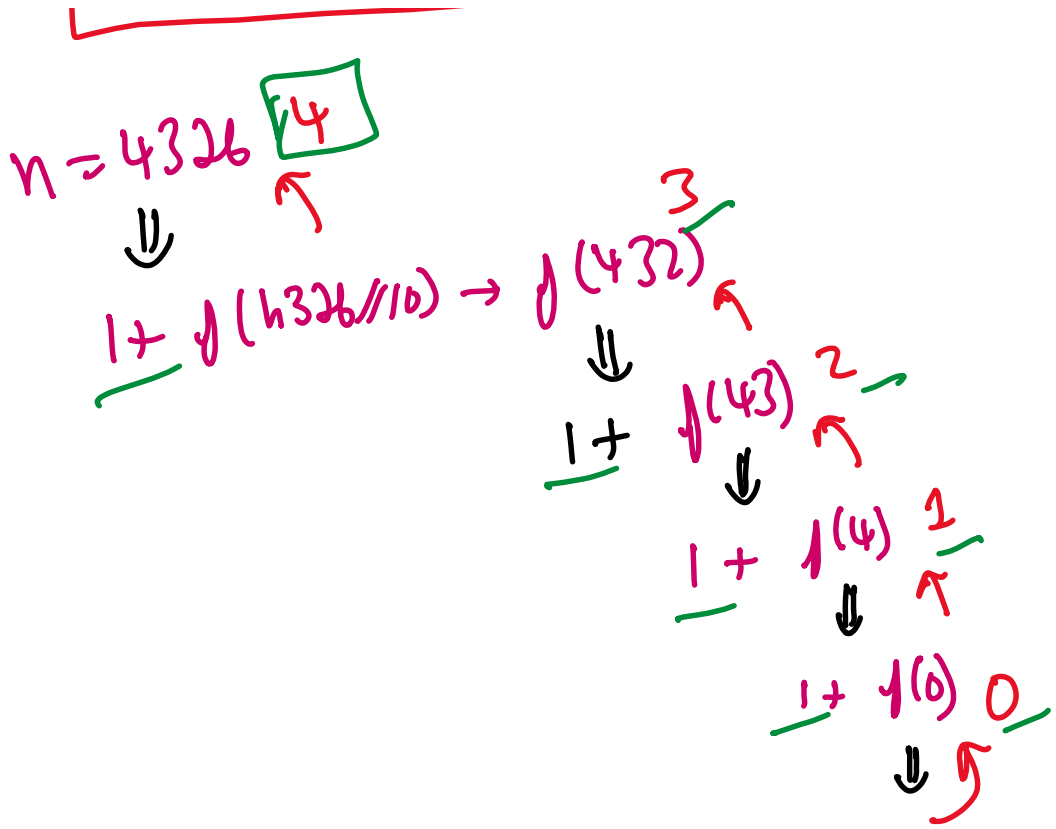
n → 4326//10 → 432

n → 432//10 → 43

n → 43//10 → 4

n → 4//10 → 0

1 + f(n//10)



$f(5,3) \rightarrow 5^3$

$\rightarrow 5 \times 5 \times 5$

$\rightarrow 5 \times 5^2$

$f(2,4) \rightarrow 2 \times 2 \times 2 \times 2$

$\rightarrow 2 \times 2^3$

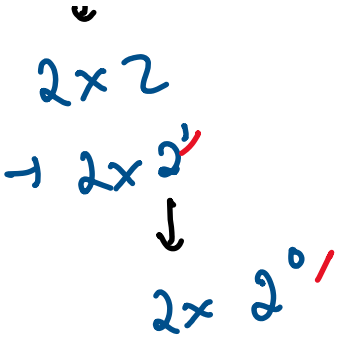
$2 \times 2 \times 2$

$\rightarrow 2 \times 2^2$

2×2

$f(n,k)$

f(n, k)
↳ n × f(n, k−1)



16

f(2,4) (2⁴)

2 x f(2,3) 8

2 x f(2,2) 4

2 x f(2,1) 2

2 x f(2,0) 1

1

9.6 - DS Context

Tuesday, December 16, 2025 4:27 AM

Data Science / ML Correlation:

- Tree-based models
- Hierarchical modeling

Libraries & Usage:

- **scikit-learn:** ***DecisionTree***, ***RandomForest***
- Recursive splitting in trees

10 - Hashing

Wednesday, November 26, 2025 11:51 PM

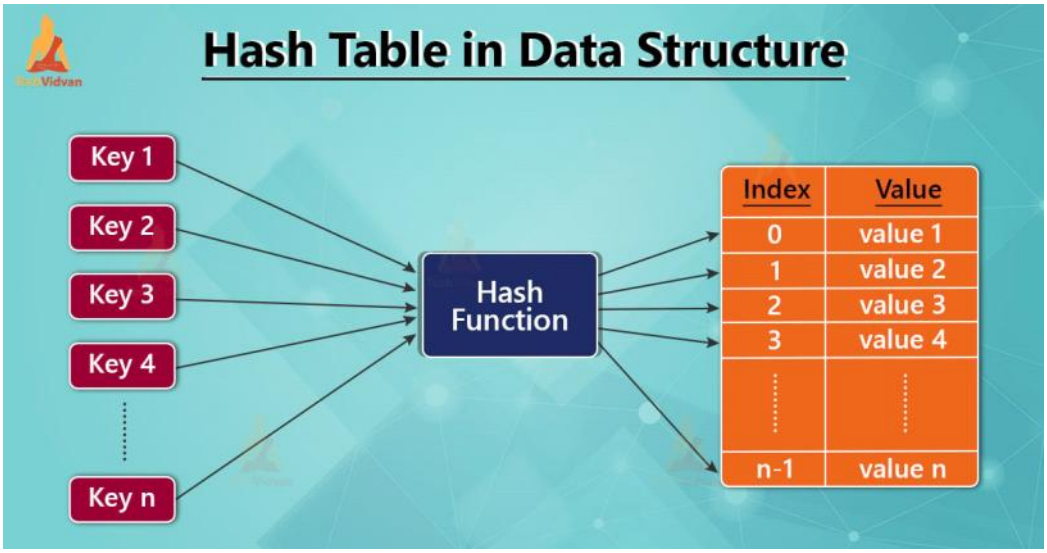
- **Introduction**
- **Hash Functions**
- **Hash Tables**
- **Hash Collisions & Collision Resolution**
- **Load Factor**
- **Common Operations**
- **Complexity Analysis**
- **Coding Problems**

10.1 - Introduction

Wednesday, November 26, 2025 11:52 PM

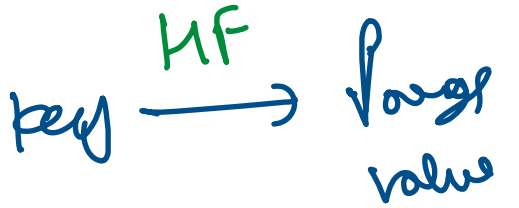
What is Hashing?

- Hashing is a technique that converts a key (string, number, object) into a fixed-size number called a **hash value**
- Makes use of a **hash function** to map data keys to a specific index in an array, known as a **hash table**



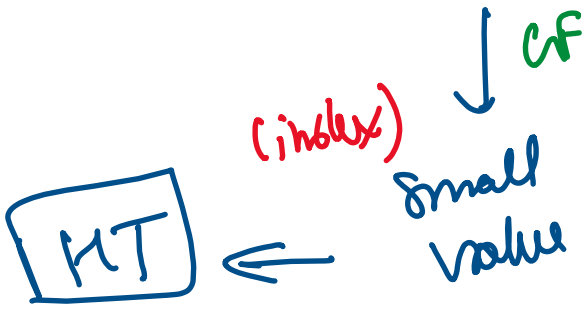
Major Components in Hashing:

- **Key**
- **Hash Function**
- **Hash Table**



Common Workflow:

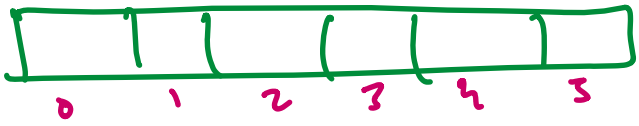
- Convert key into hash value (hash function)
- Map the hash value to a smaller integer (compression function)
- Insert the key into the index position in array (hash table)

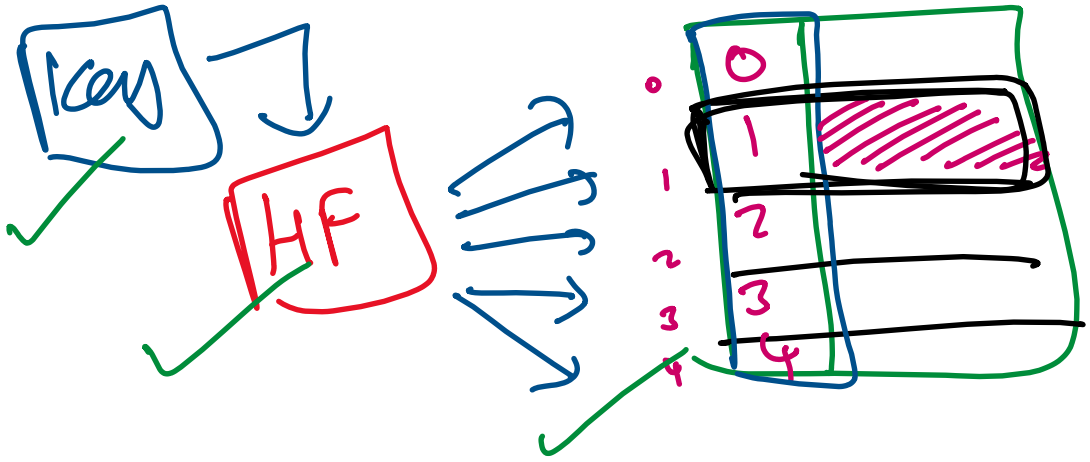


Applications:

APPLICATION	PURPOSE	EXAMPLE
Database Indexing	Fast lookup	MySQL hash index

APPLICATION	PURPOSE	EXAMPLE
Database Indexing	Fast lookup	MySQL hash index
Password Storage	Security	SHA-256, bcrypt
Data Deduplication	Avoid duplicates	Dropbox, cloud storage
Data Structures	Key-value storage	Python dict, Java HashMap
Data Integrity	Detect tampering	MD5/SHA checksums
Load Balancing	Distribute requests	Consistent hashing
Caching	Quick access	Web browsers
Plagiarism Detection	Detect duplicates	Document fingerprinting
Networking	P2P storage	BitTorrent DHT
Blockchain	Secure transactions	Bitcoin, Ethereum





10.2 - Hashing Function

Friday, November 28, 2025 7:44 AM

What is a Hashing Function?

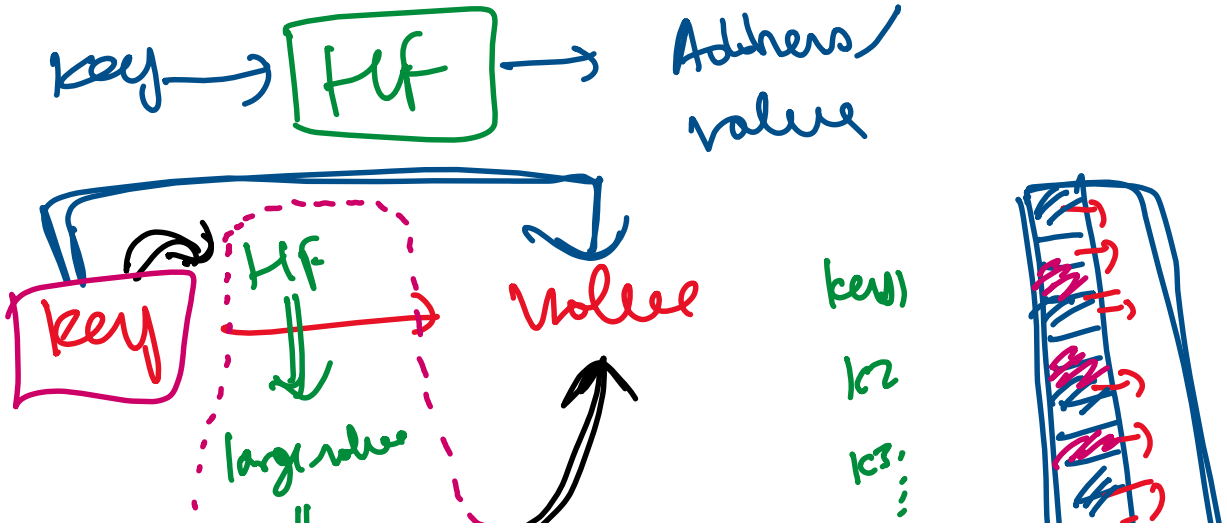
- A hash function is a function that maps keys to one of the values in the hash table
- Hash functions return the location/address where we can store the particular key
- We input the key to the hash function and the output is an address
- Internally, complex mathematical operations are implemented to generate the hash value

Properties of Hashing Function:

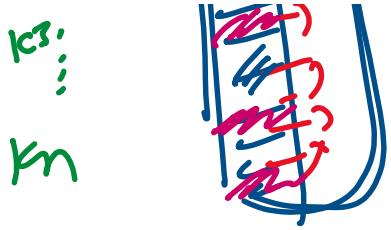
- **Deterministic:** For a particular input, the output is the same
- **Uniform Distribution:** Keys should spread across the table and not cluster
- **Fast Computation:** A hash function must be extremely fast since it is used for every insertion, deletion, and search
- **Low Collision:** Two different keys shouldn't/rarely end up with the same hash value

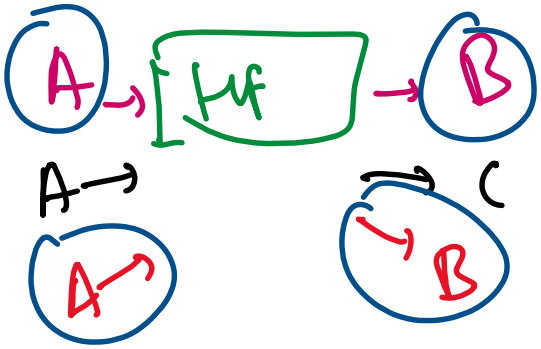
Common Hashing Functions:

1. **Simple Hash**
2. **Multiplicative Hash**
3. **DJB2 (very popular)**
4. **Built-in hash function**



large value
small value (index)


K3
Kn


A
f
B
A
A
B
C

10.3 - Hash Table

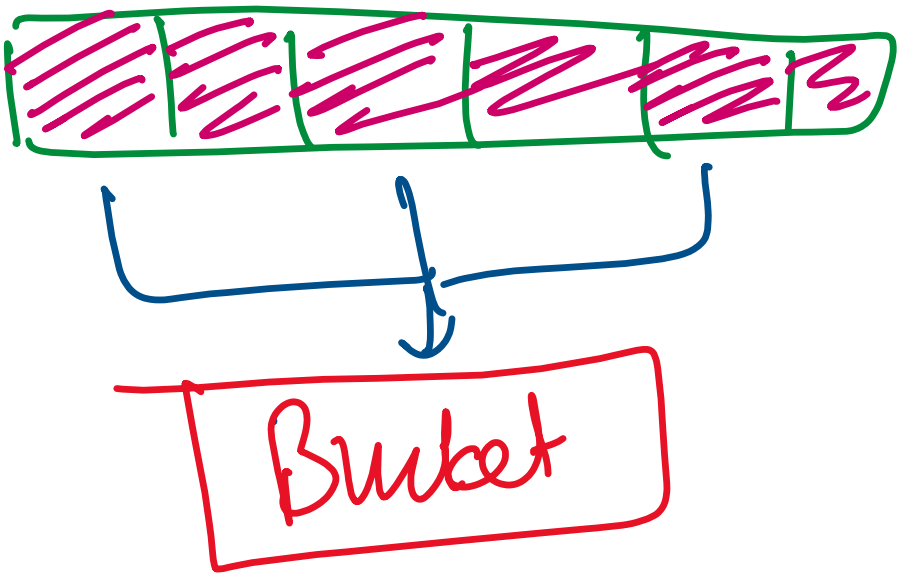
Friday, November 28, 2025 7:55 AM

What is a Hash Table?

- Type of data structure that makes use of the **hash function** to map values to the key
- Stores values in an associative manner i.e. each key is mapped to a specific value
- **Hash tables** make use of array-like data structures for storing values
- Each slot/cell in the table is also called as **bucket**
- The process of mapping keys to the values is known as **Hashing**
- The purpose of using hash tables as a data structure is to enable faster access

Python Implementation:

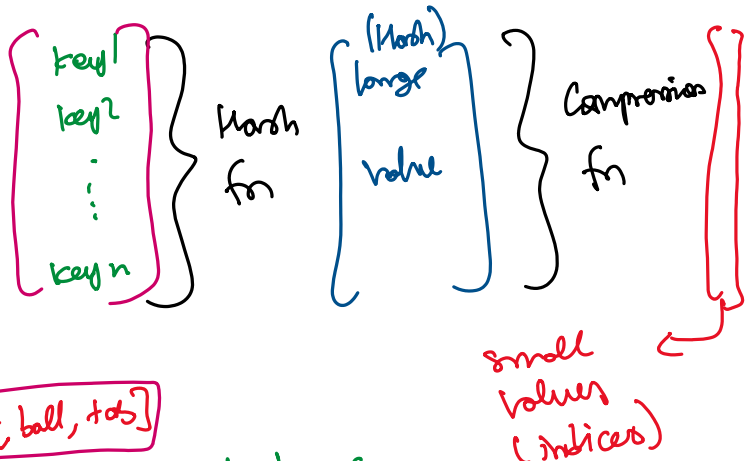
- **Sets (set)**
- **Dictionaries (dict)**



key → index → value

10.4 - Hash Collisions

Friday, November 28, 2025 8:42 AM

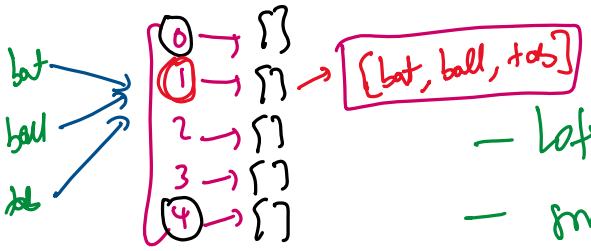


What is a Collision?

- Happens when two different keys produce the same table index
- This is the situation where multiple keys want the same index in the hash table
- Since, each index can typically hold only one key, this is called a Collision

Why do Collisions occur?

- Input space is infinite (all possible keys)
- Table size is finite (hash table)

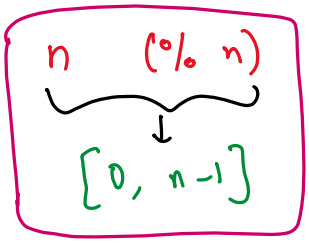


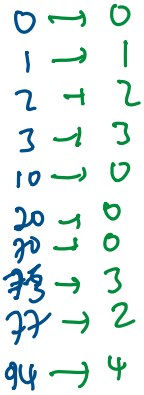
— lots of keys

— small Hash Table

Example:

- **Keys:** ['bat', 'ball', 'tab']
- **Hash Table size:** 5
- **Hashing Function:** Simple ASCII
- **Compression Function:** modulus (%)

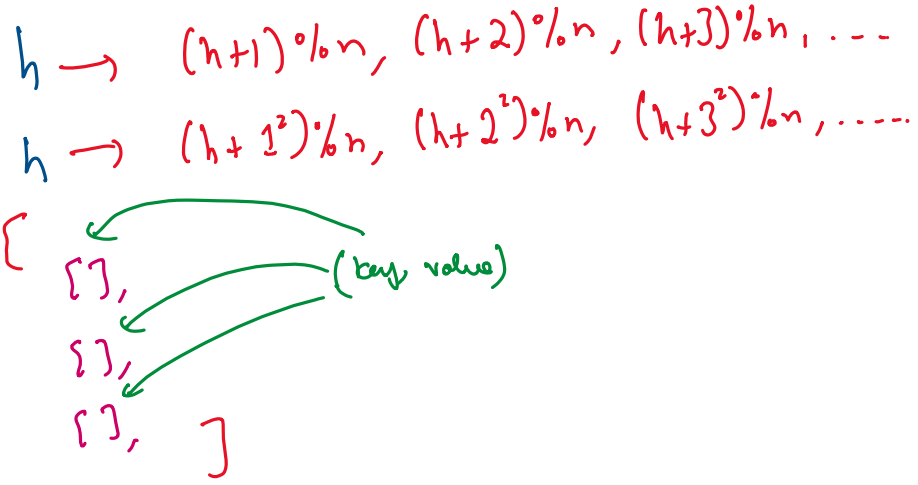


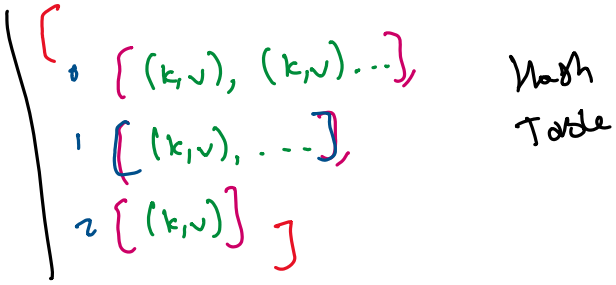


Collision Resolution:

Two main types of approaches to handle collisions:-

1. **Chaining:**
   - Each bucket is a list / linked list / dynamic array
   - On Insert, append they key into the bucket
   - On Search, scan they keys in the bucket
2. **Open Addressing:** If a bucket is occupied, we probe to find another slot; sequence must eventually visit empty buckets
   - **Linear Probing:** If index h is occupied, keep visiting *(h + 1) % size* until an empty bucket is found
   - **Quadratic Probing:** *h, (h + 1^2) % size, (h + 2^2) % size, (h + 3^2) % size*, etc.
   - **Double Hashing:** Uses 2 hashing functions





key → hash-fn → compress-fn → index (h)

h

$(h+0)\%n, (h+1)\%n, (h+2)\%n, \ldots$

n = 5

h = 2,

$(2+1)\%5 \rightarrow 3\%5 \rightarrow 3$ ✓

$(2+2)\%5 \rightarrow 4\%5 \rightarrow 4$ ✓

$(2+3)\%5 \rightarrow 5\%5 \rightarrow 0$ ✓

$(2+4)\%5 \rightarrow 6\%5 \rightarrow 1$ ✓

$(2+5)\%5 \rightarrow 7\%5 \rightarrow 2$ ✗

[None, None, None, ... None]

0 1 2 .... m-1

i) index is empty → insert value

ii) index is full, same key → nothing

iii) index is full, different key → next slot

[None, None, ... , None]

$(h+0^2)\%n, (h+1^2)\%n, (h+2^2)\%n, \ldots$

n = 7

h = 3

$(3+0^2)\%7 \rightarrow 3\%7 \rightarrow 3$

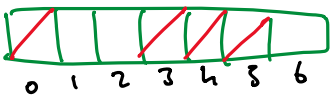
$(3+1^2)\%7 \rightarrow 4\%7 \rightarrow 4$

$(3+2^2)\%7 \rightarrow 7\%7 \rightarrow 0$

$(3+3^2)\%7 \rightarrow 12\%7 \rightarrow 5$

$(3+4^2)\%7 \rightarrow 19\%7 \rightarrow 5$

$\ldots \rightarrow 28\%7 \rightarrow 0$



(345) 27.7 → 39.7 → 4

(345) 27.7 → 28.7 → 0

(344) 27.7 ← 19.7 ← 5

10.5 - Load Factor

Friday, November 28, 2025 9:46 AM

What is Load Factor?

- The **Load Factor** is one of the most important concepts in hashing
- Used to indicate how full the hash table is
- Mathematically, it is expressed as:

$$\text{Load Factor} = \frac{\text{Number of elements inserted}}{\text{Total number of buckets (table size)}}$$

LF = # values / size

Interpretation of Load factor:

- **0.0** → completely empty
- **0.5** → half full
- **1.0** → completely full
- **>1.0** → impossible in open addressing tables, possible in chaining tables

n = 5, a = 4, LF = 4/5 = 0.8

n = 5, a = 5, LF = 5/5 = 1

a = 0, LF = 0/5 = 0

Why do we need Load Factor?

1. **How crowded the table is:** A high LF means more keys are trying to fit into a limited number of buckets
2. **How likely collisions are:** More items → more chances two keys hash to the same index
3. **Speed of operations:**
   - **Low load factor** → fewer collisions → faster operations
   - **High load factor** → many collisions → slow operations
4. **When to resize the hash table:** Most hash table implementations **automatically resize and rehash** when LF crosses a threshold

[[v1, v2, ...], [v1, v2, v3, ...], [v1, ...]]

n = 5

a ≥ n

⇒ LF ≥ 1

10.6 - Common Operations

Friday, November 28, 2025 10:19 AM

- **Insert**
- **Search**
- **Delete**
- **Update**
- **Membership**
- **Iteration**
- **Handling Collisions**

10.7 - Complexity Analysis

Friday, November 28, 2025 10:28 AM

Common Operations:

OPERATION	DICTIONARY	SET	AVG TIME (theta)	WORST TIME (O)
Insert	✔	✔	O(1)	O(n)
Search	✔	✔	O(1)	O(n)
Delete	✔	✔	O(1)	O(n)
Update	✔	✘	O(1)	O(n)
Membership	✔	✔	O(1)	O(n)
Iteration	✔	✔	O(n)	O(n)

10.8 - Coding Problems

Friday, November 28, 2025 10:36 AM

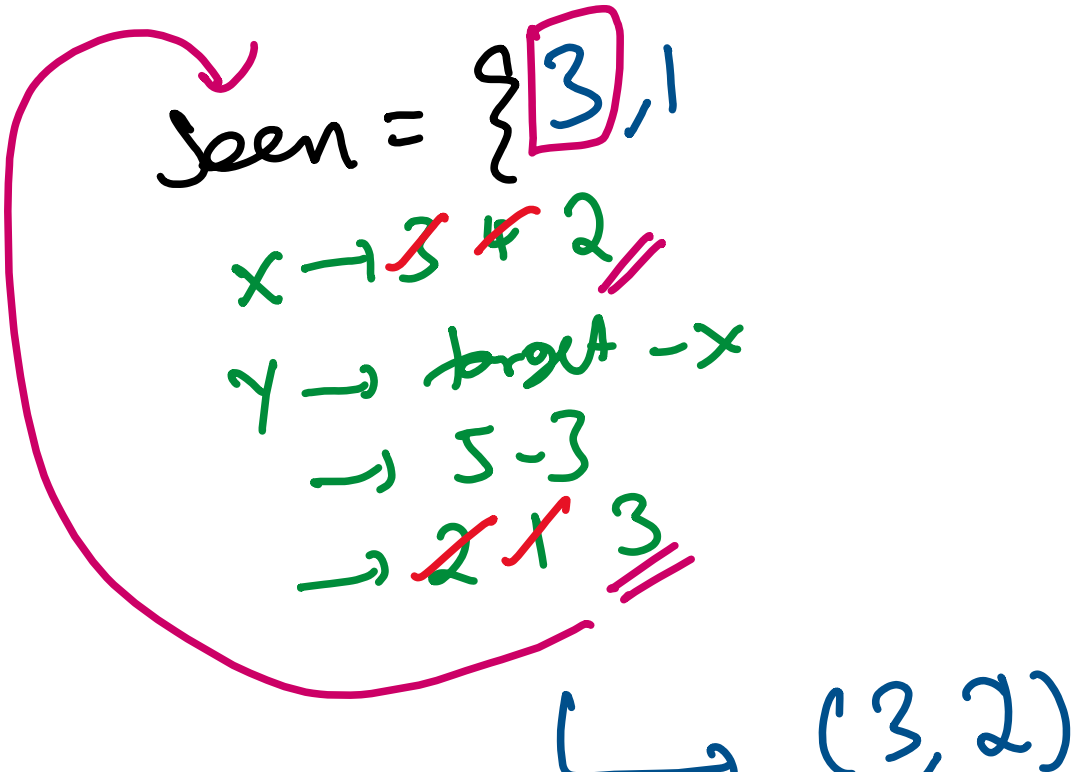
seq (.....)

target

seen = set()

for x in seq:

target - x



10.9 - DS Context

Tuesday, December 16, 2025 4:34 AM

Data Science / ML Correlation:

- Feature encoding
- Caching & memoization
- Deduplication

Libraries & Usage:

- ***sklearn.feature_extraction.FeatureHasher***

11 - Problem Solving

Thursday, December 4, 2025 9:17 AM

- **Sliding Window**
- **Two Pointers**
- **Fast & Slow Pointers**
- **Prefix Sum**
- **Hashing**
- **Binary Search Pattern**
- **Greedy Approach**

11.1 - Sliding Window

Tuesday, December 9, 2025 1:16 AM

What is Sliding Window?

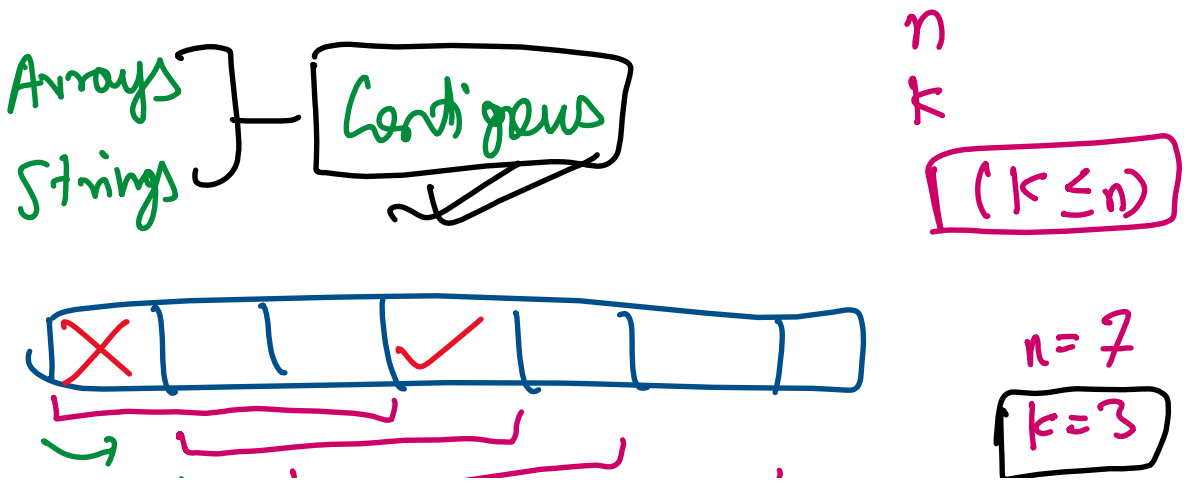
A technique used to process contiguous subarrays or substrings by maintaining a ***running window*** and updating it efficiently as the window moves through the input

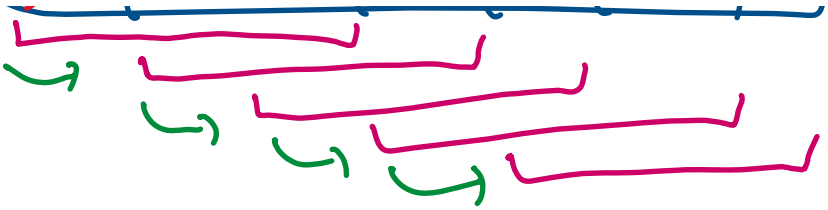
Why Use Sliding Window?

- Traditional brute-force approaches check **all possible subarrays** in $O(n^2)$ or worse
- Sliding Window uses **constant-time operations** by updating:
  - running sum
  - frequency
  - count
  - max/min length
- Time reduces to **O(n)** because each element is added and removed from the window at most once

Key Operations:

- **Expand window:** add elements to the right
- **Shrink window:** remove elements from the left
- **Check & maintain constraints** (sum, duplicates, frequencies)
- **Update result** (max/min length, sum, indices)

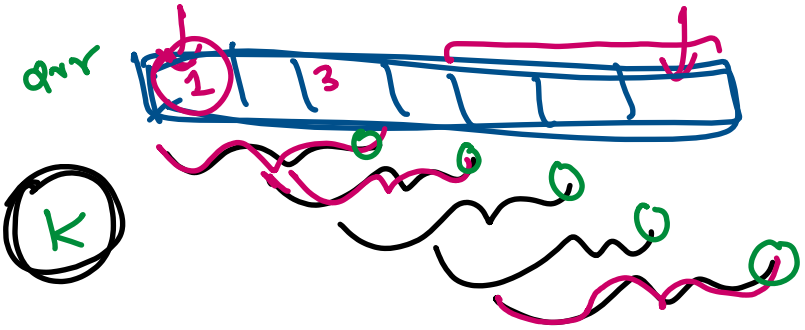




k=3

$O(n^2)$ → worse

O(n)



(right - left + 1) == k

6 - 4 + 1

↳ 2+1 → 3 == 3

left, right

max_sum/res = −inf

window_sum = 0

loop right < len(arr)

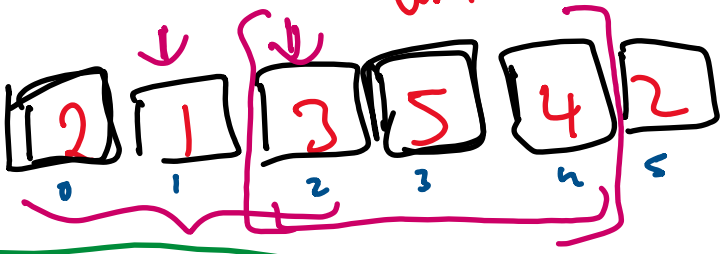
win_sum += arr[right]

if right - left + 1 == k (window size)

if right - left + 1 == k

compare(win_sum, res)

win_sum -= arr[left]



n = 6

k = 3

res = -inf 6 9 12

w_sum = 6 2 3 6 4 5

left = 0 1 2 3 4

right range (len(arr))

1 - 0 + 1 → 2 == 3 ✗

2 - 0 + 1 → 3 == 3 ✓

res = max(res, w_sum)

-inf,	6
6,	9
9	12
12	11

left += 1

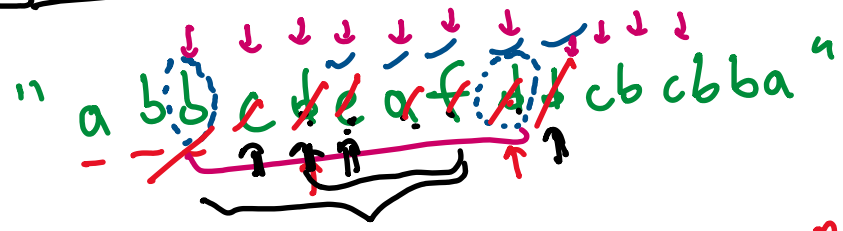
3 - 1 + 1 → 3 == 3

4 - 2 + 1 → 3 == 3

4 - 2 + 1 → 3

5 - 3 + 1 → 3 == 3

return res



substring → non-repeating

↓

length

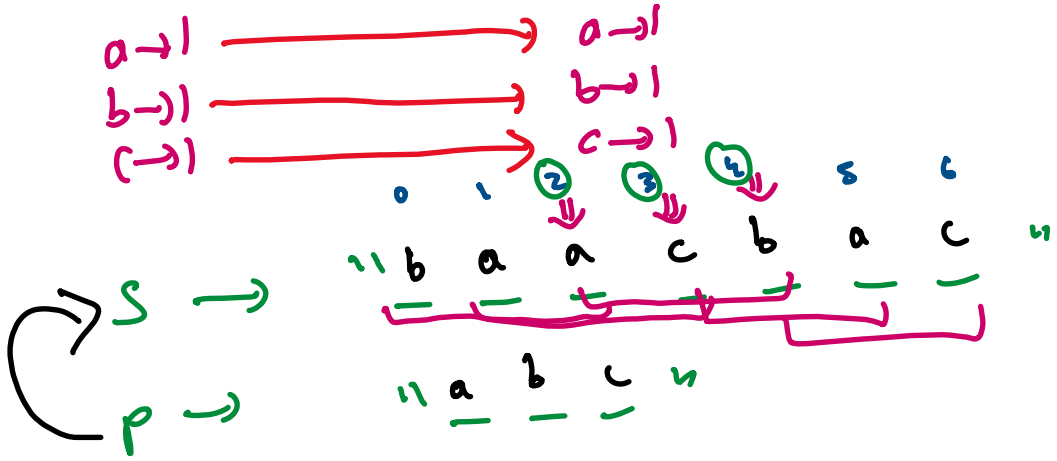
res → ~~2~~ 6

~~a b~~ ab ✓

ab b

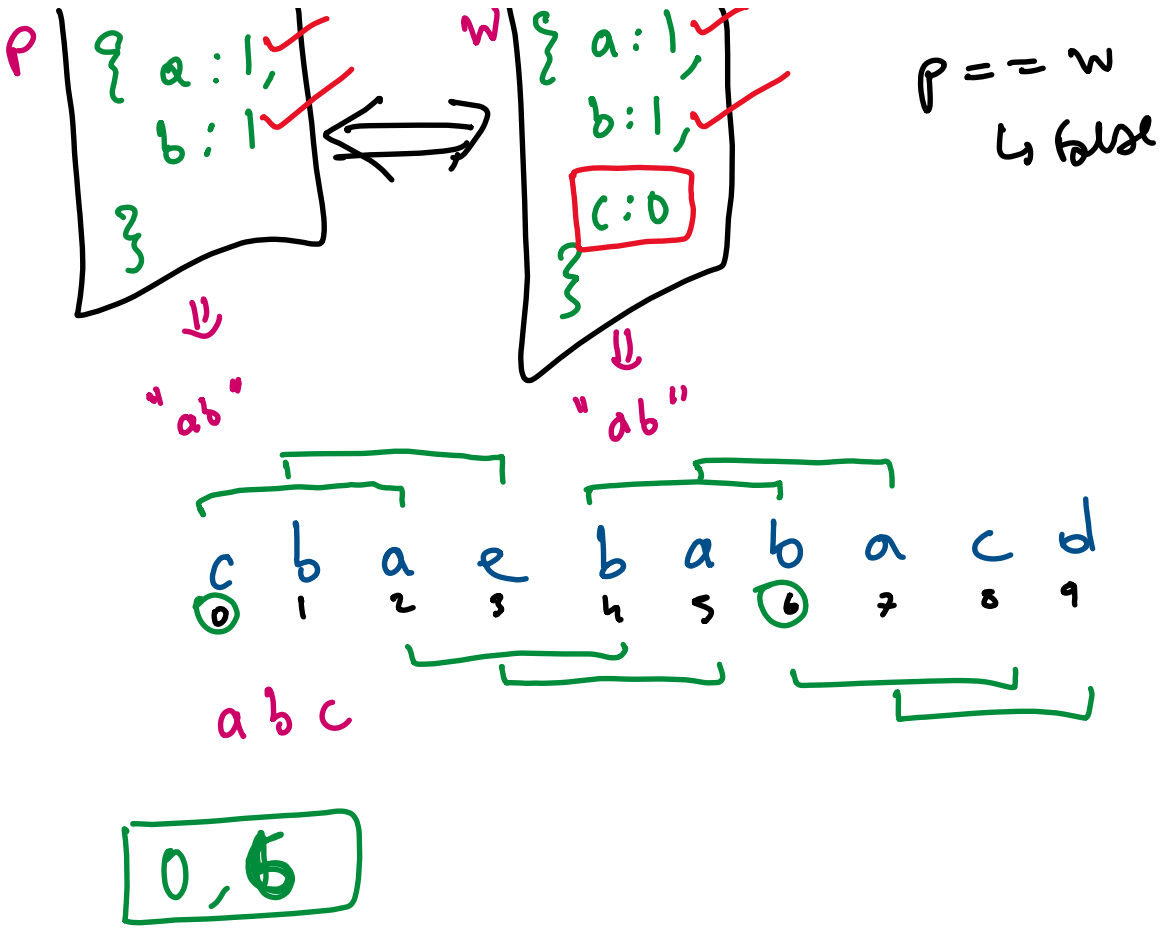
---

abc cba



collections → Counter

P { a : 1 } ✓ W { a : 1 } ✓ P == W



p == w
↳ false

0, 6

11.2 - Two Pointers

Tuesday, December 9, 2025 1:16 AM

What is the Two Pointer Technique?

A pattern where two indices (pointers) traverse the array from different directions (left–right) to solve problems efficiently in **O(n)** or **O(n log n)** instead of **O(n^2)**

When to Use Two Pointers?

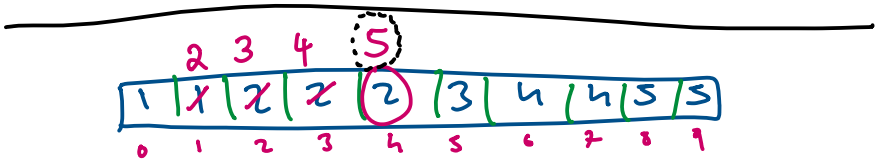
- Input is sorted
- Problem involves comparing or matching two ends of an array
- Need in-place modification without extra space

2 pointers

move in opp. directions

towards each other

$O(n^2) \longrightarrow O(n), O(n \log n)$



i → 0 1 2 3 4

j → 1 2 3 4 5 6 7 8 9

n = 10

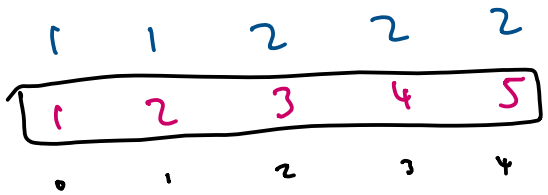
j → range (1, n)

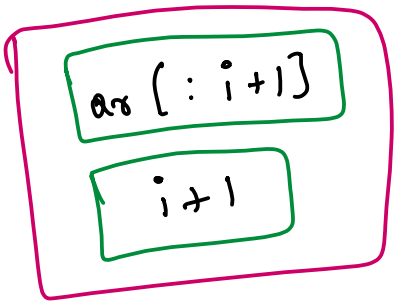
arr[i] → 1 2 3 4 5

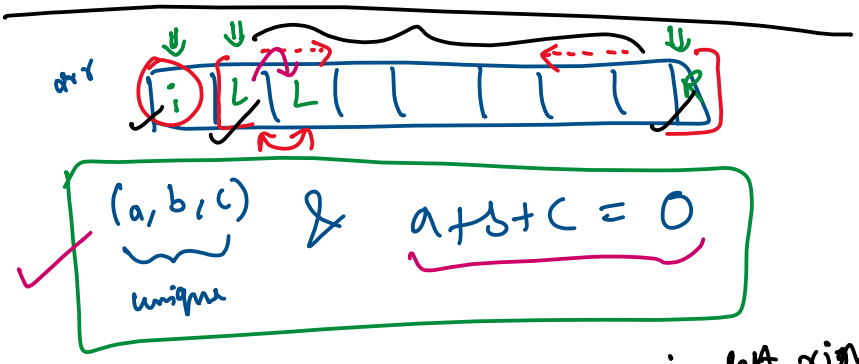
arr[j] → 2 3 4 5

```
arr[i] != arr[j]:
    i += 1
    arr[i] = arr[j]
```

arr[0] → arr[4]







i, left, right

i) 2 pointer
ii) duplicates handling

```
arr.sort()
res = [ ]
for i in range(n-2):
    if duplicate (i):          } duplicates
        continue

    left = i+1                 } 2-pointers
    right = n-1
    while left < right:
        s = arr[i] + arr[left] + arr[right]

        if s == 0:
            res.append ([i, left, right])
            left += 1
            right -= 1
```

```
        right -= 1

    if s < 0:
        left += 1
    else:
        right -= 1

return res
```

11.3 - Fast & Slow Pointers

Tuesday, December 9, 2025 1:16 AM

What is the Fast & Slow Pointer technique?

Uses two pointers that move through a sequence (linked list / array) at **different speeds**; mainly used for detecting cycles in the sequence

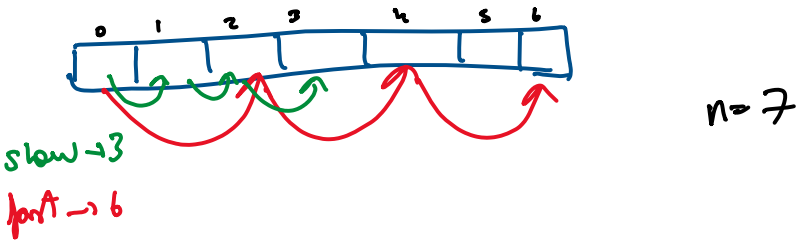
- **Slow pointer:** moves ***one step*** at a time
- **Fast pointer:** moves ***two steps*** at a time

Why It Works?

- If there is a **cycle**, the fast pointer eventually ***laps*** the slow pointer
- If there is **no cycle**, the fast pointer reaches the end (null)

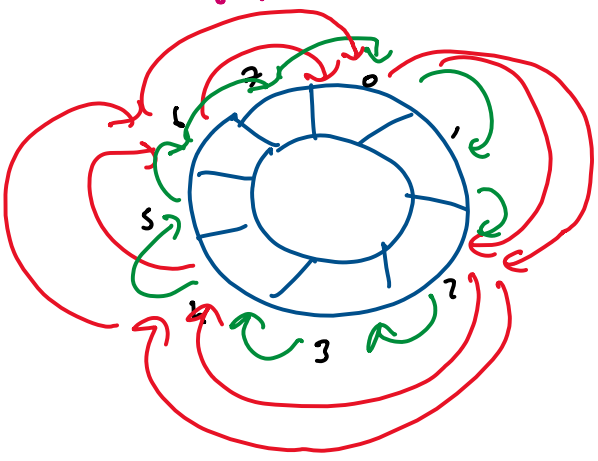
Common Use-Cases:

- Cycle detection
- Finding cycle start
- Detecting middle of linked list
- Finding palindrome linked list
- Detect duplicate numbers in an array

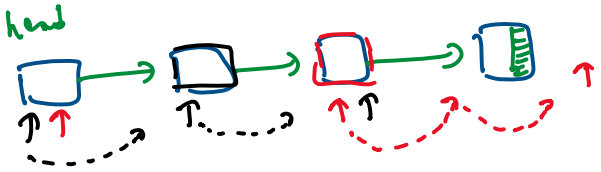


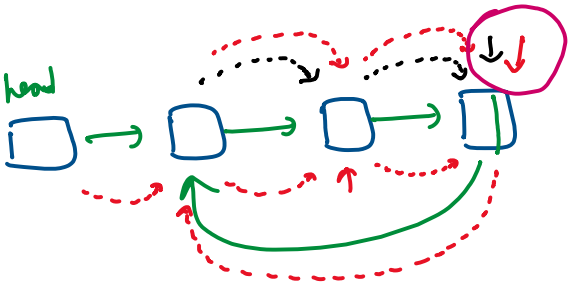
* if "fast" reaches the end, no cycle

* if "fast" reaches the end "slow" will be at middle



* If there's cycle, "fast" & "slow" will meet again



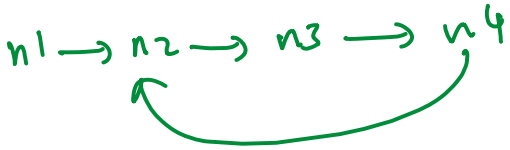


slow = fast = head

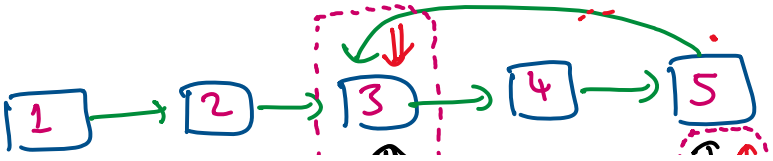
while fast & fast.next:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        return True

return False



a1 → a2 → a3 → a4



slow
fast
→ Lead

* detect cycle

```
while slow != fast
    slow = slow.next
    fast = fast.next
return slow.value
```

Positions:

1. Head
2. meeting pt. (cycle)
3. start cycle

Steps:

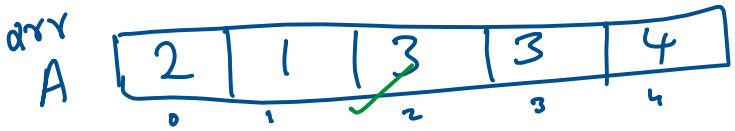
1. Detect cycle
2. Reposition "slow", keep "fast"
3. Move "slow" & "fast" one step
4. Converge at cycle starting pt.

Linked List Array

Linked List → Array

Node → Index

Next pointer → Value at Index



slow → 23023 → 3

fast → 3

slow → A[0]

fast → A[A[0]]

} initial

while slow != fast:

slow → A[slow]

fast → A[A[fast]]

} meeting pt.

slow = 0

while slow != fast:

slow → A[slow]

fast → A[fast]

} duplicate

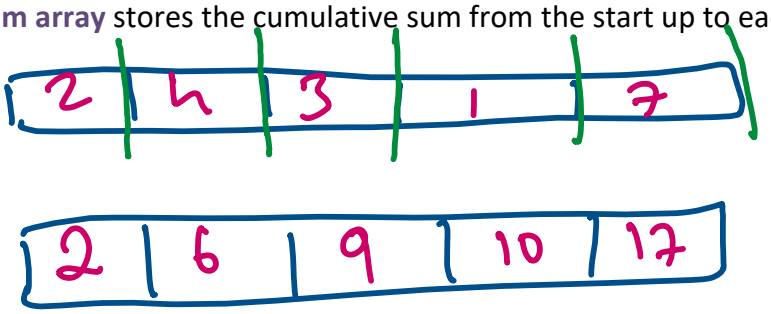
return slow

11.4 - Prefix Sum

Tuesday, December 9, 2025 1:16 AM

What Is a Prefix Sum?

A **prefix sum array** stores the cumulative sum from the start up to each index



Key Benefits:

- Fast subarray sum calculation
- Efficient range queries
- Helpful for frequency-based problems

Most Common Use Cases:

USE CASE	EXAMPLE
Subarray sum ✓	sum = K, divisible by K
Longest subarray ✓	with 0/1 balance
Count of subarrays ✓	satisfying a condition
Range sum queries ✓	prefix differences
Range update queries ✓	using difference array

---

arr, k

arr → [1, 2, 3] , k = 3

↓

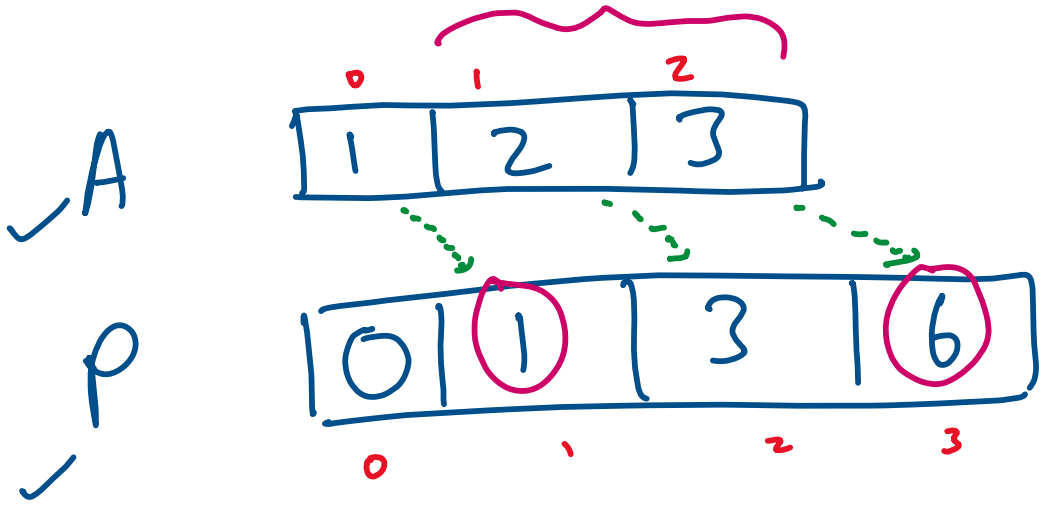
[1], [2], [3]

[1,2], [1,3], [2,3]

ans = 2

[1,2], [1,3], [2,3]

[1,2,3]



$$p[i] = p[0] + A[0]$$

$$sum(A[1..2]) = p[3] - p[1] = 5$$

$$sum(A[i..j]) = p[j+1] - p[i]$$

$$\Rightarrow sum(A[i..j]) = k$$

$$\Rightarrow p[j+1] - p[i] = k$$

$$\Rightarrow p[i] = p[j+1] - k$$

freq = {

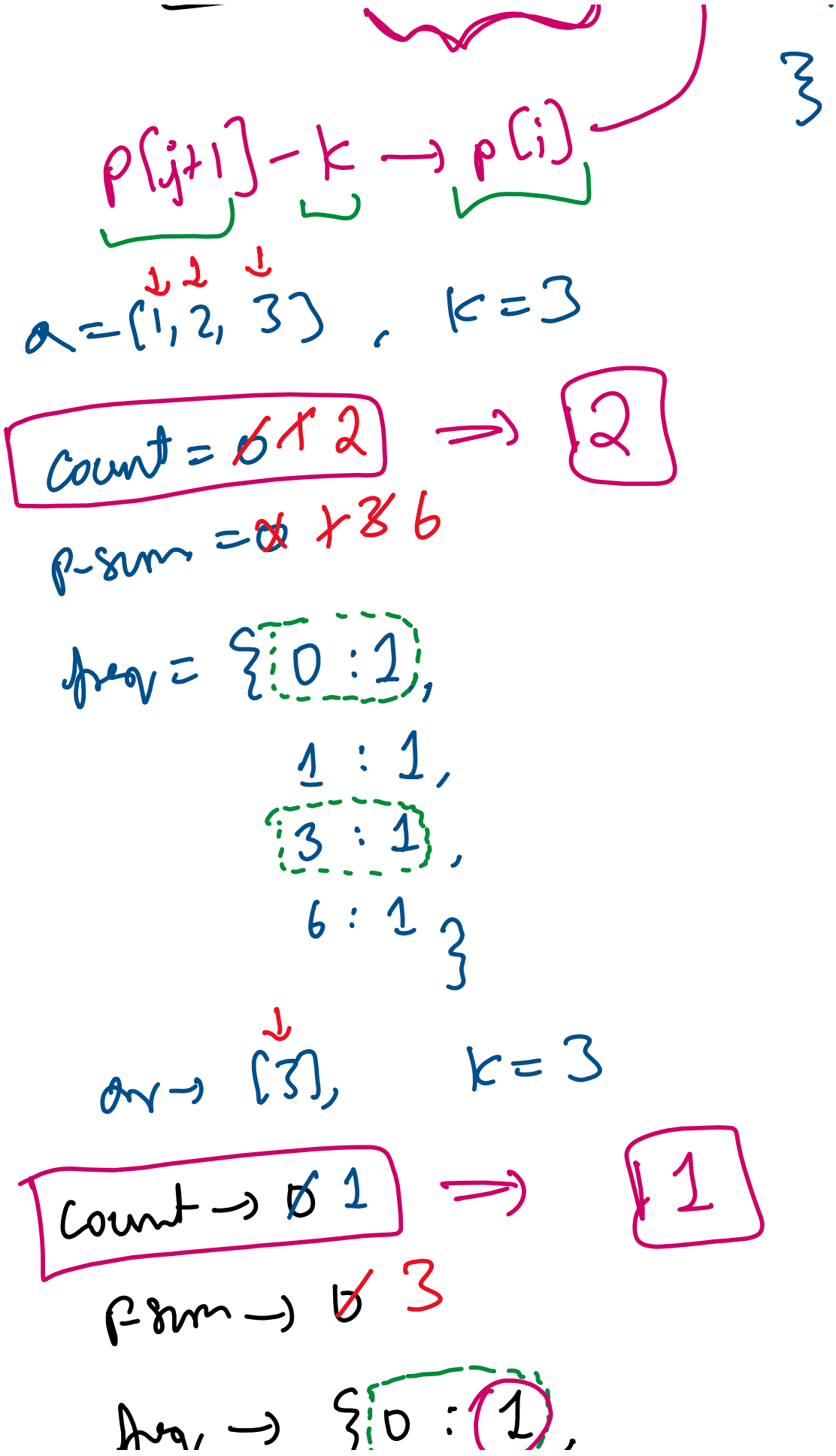
p[0] :

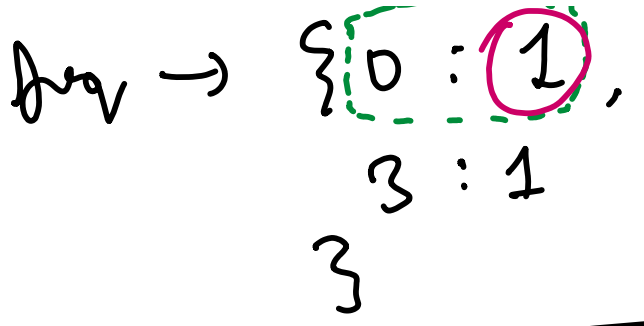
p[1] :

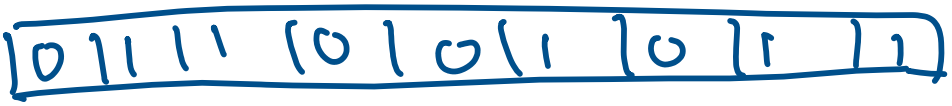
p[2] :

⋮

2


}
p[j+1] - k → p[i]
a = [1, 2, 3] , k = 3
count = 0 1 2 ⇒ 2
p-sum = 0 1 3 6
freq = { 0 : 1,
1 : 1,
3 : 1,
6 : 1 }
ans → [3], k = 3
count → 0 1 ⇒ 1
p-sum → 0 3
Ans → { 0 : 1,



arr 

$$sum(A[i \ldots j]) = P[j+1] - P[i]$$

1 → +1

0 → -1

0

2

#1 → 2 ✓

#-1 → 2 ✓

#1 → 3

#-1 → 1

1. long subA equal # 1s & 0s

2. long subA equal # 1s & -1s

2. Lg subA equal # 1s & -1s

3 Long subA sum 0

$$sum(A[i \ldots j]) = p[j+1] - p[i] = 0$$

$$\Rightarrow \boxed{p[j+1] = p[i]}$$

4. Long subA pref sum is equal

p_sum = 0

longest = 0

first_occ = { 0 : -1, }

for i, num in enum(arr):

prefix += 1 if num > 1

prefix += 1 if num > 0

-1 if num < 0

if pre_sum in first_occ:

longest = max(longest, i - first_occ[pre_sum])

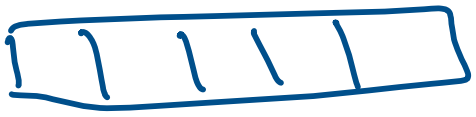
else:

first_occ[pre_sum] = i

return longest

---

arr, k



{ sum of subarrays % by k == 0 }

sum(A[i..j]) = P[j+1] - P[i]

$sum(A[i..j]) = P[j+1] - P[i]$

$\#\left[sum(A[i..j]) \% K \right] = 0$

$\Rightarrow (P[j+1] - P[i]) \% K = 0$

$\Rightarrow \boxed{P[j+1] \% K = P[i] \% K}$

p_sum = 0

count = 0

freq = {0 : 1}

p % K

```
for num in A:
    p_sum += num
    mod = p_sum % K
    if mod in freq:
        count += freq[mod]
```

freq[num] = freq.get(num, 0) + 1

return count

11.5 - Hashing

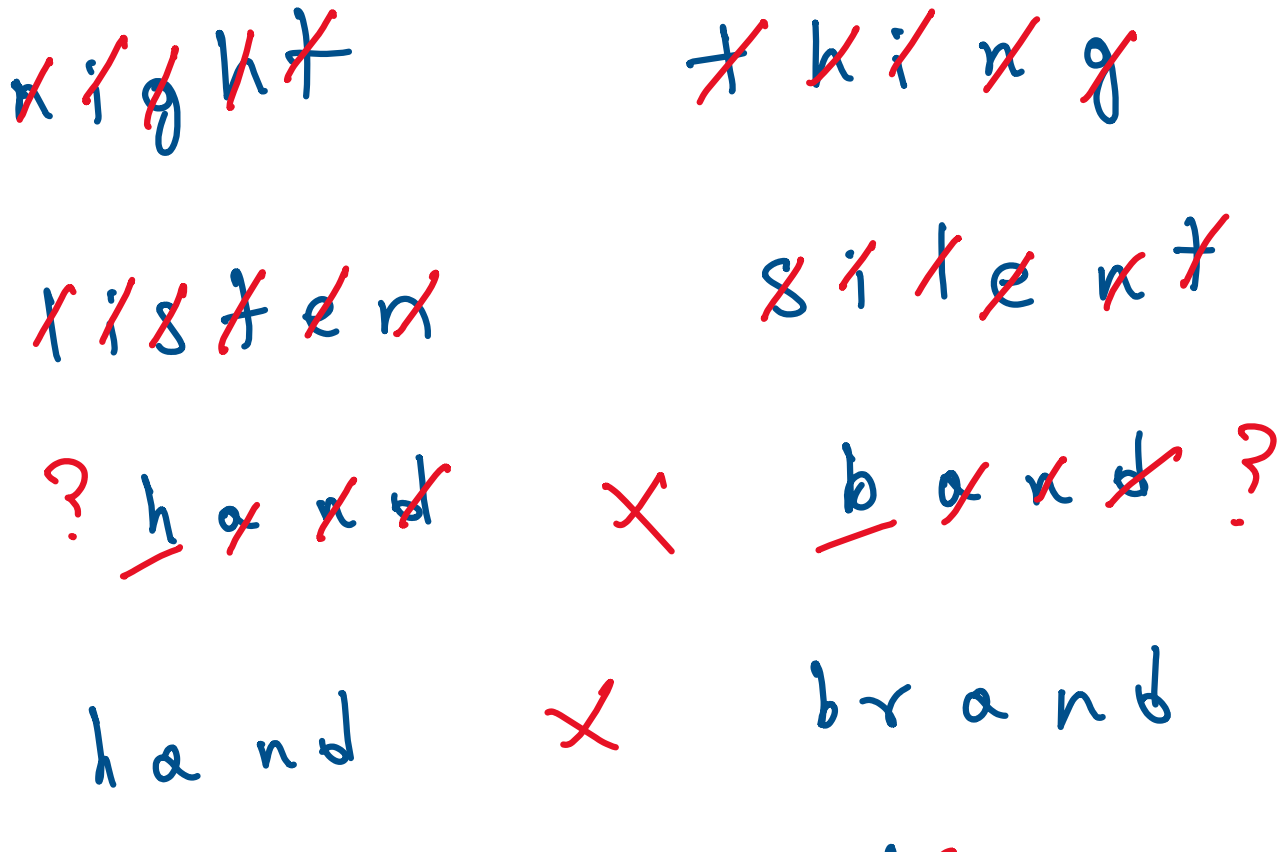
Tuesday, December 9, 2025 1:16 AM

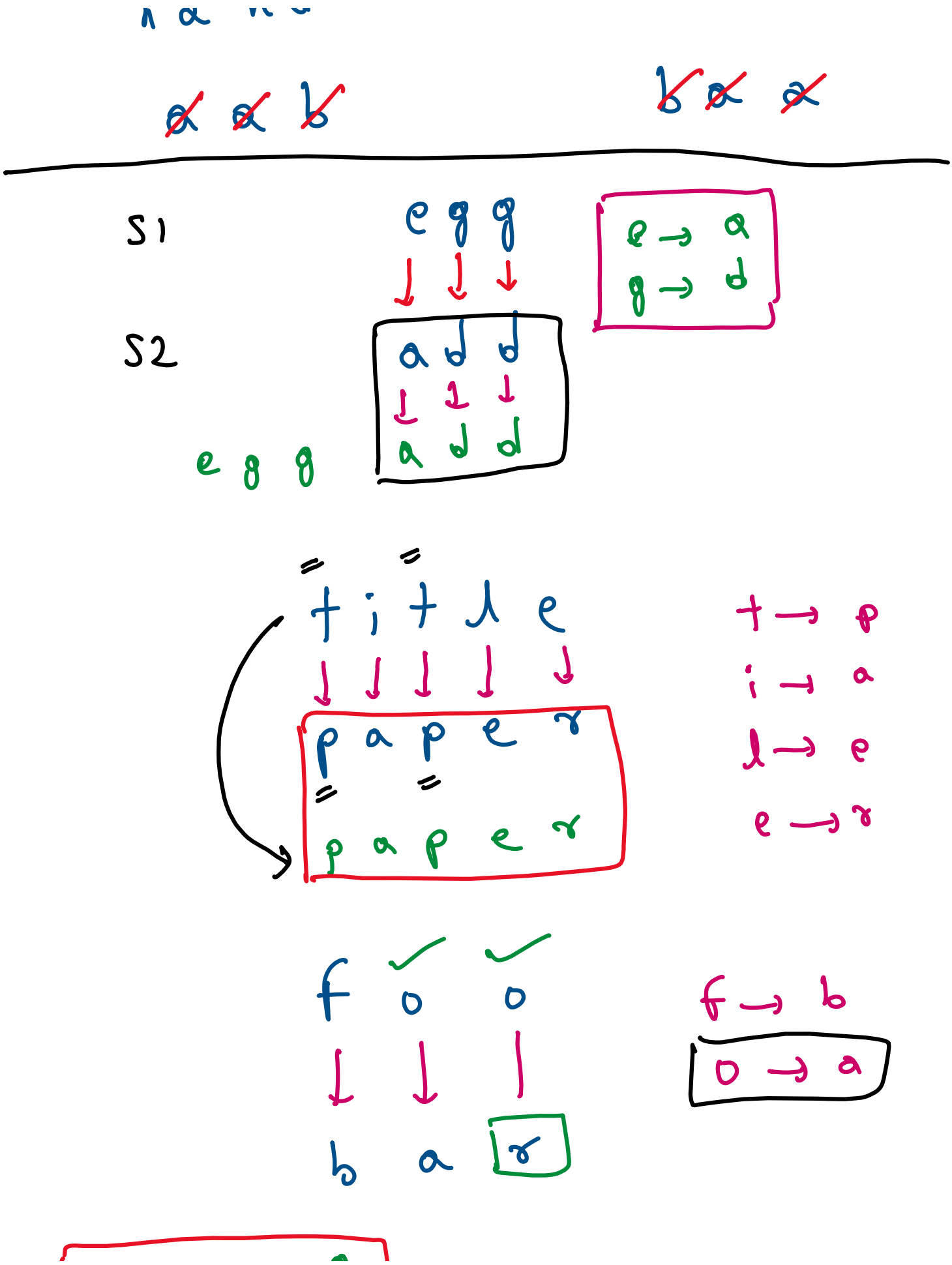
Most Useful For:

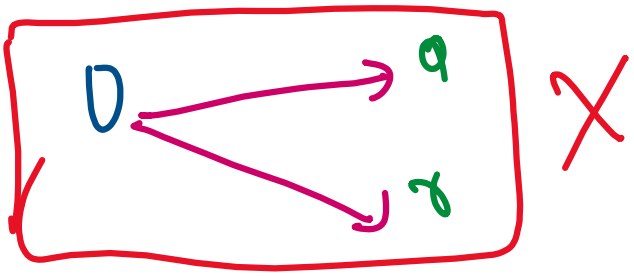
- **String problems**
- **Frequency counting**
- **Grouping problems**
- **Character transformations**
- **Duplicate detection**
- **Top-K queries**

Data Structure Implementation:

- **Dictionary (*dict*)**
- **Set (*set*)**
- **Counter (*collections*)**




S1
S2
e g g
a d d
1 1 1
a d d
e g g
e → a
g → d
t i t l e
p a p e r
p a p e r
t → p
i → a
l → e
e → r
f o o
b a r
f → b
o → a


D
q
γ
X

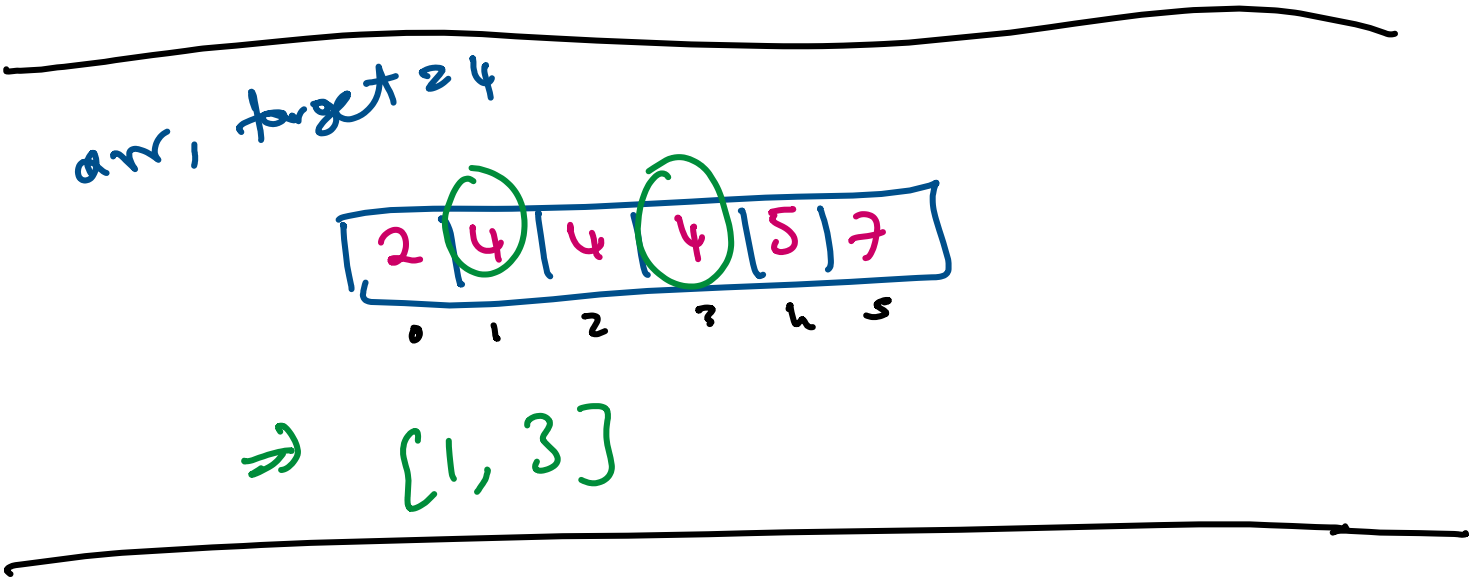
11.6 - Binary Search Pattern

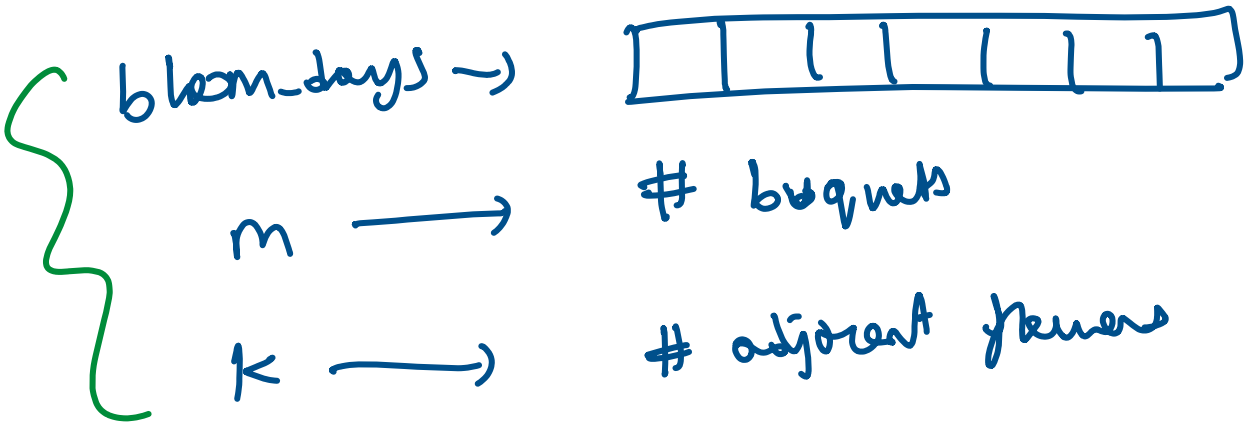
Tuesday, December 9, 2025 1:16 AM

Binary Search is not just an algorithm — it is a pattern used to solve a wide range of problems, including arrays, matrices, and even ***answer-search problems***

When to use Binary Search:

1. **The search space is monotonic:**
   - Non-decreasing / non-increasing
   - True → False transition
   - False → True transition
2. **Yes/No for Candidate value**
3. **Optimization of Values**

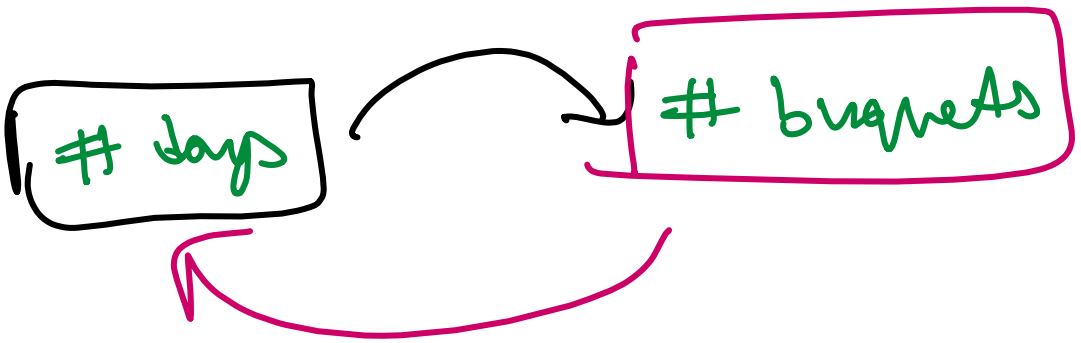




$n * k = \text{len}(\text{bloom_days})$

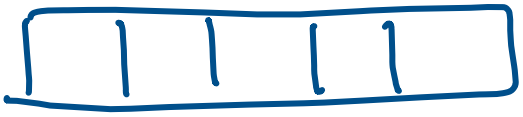
given # days, how many bouquets

min days required make bouquets

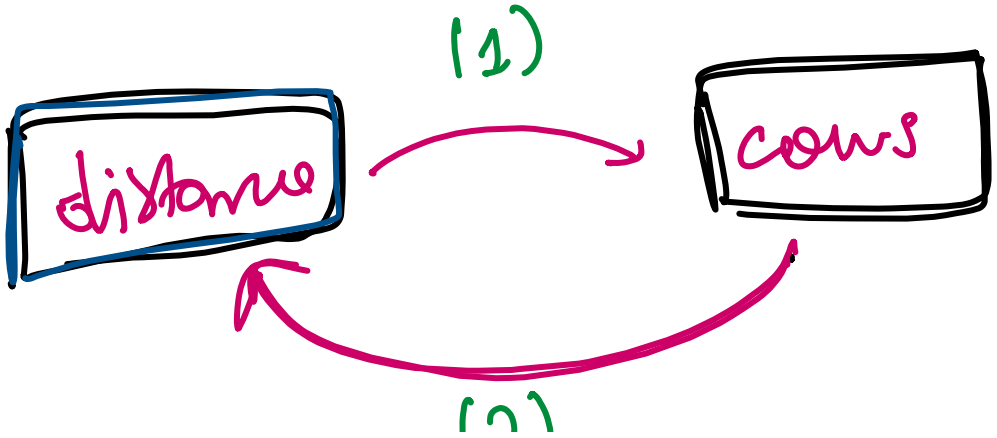


---

stalls →



cows →



(2)

11.7 - Greedy Approach

Wednesday, December 10, 2025 3:03 AM

What is the Greedy Approach?

Strategy which involves making the locally optimal choice at each step, hoping it leads to a global optimum

When to Use Greedy:

- The problem has **optimal substructure**
- Each local choice does not affect future optimality
- Sorting the input helps in picking best decisions
- DP solution exists but can be optimized using Greedy

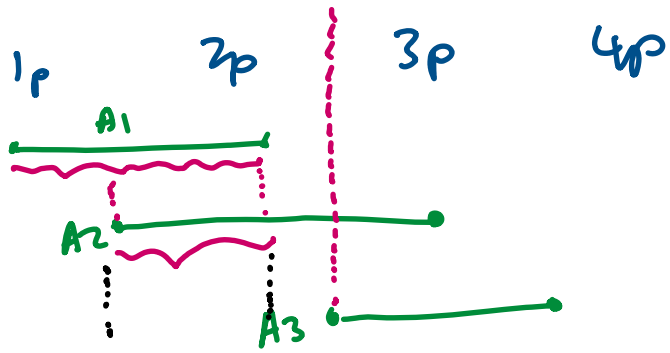
Common Use-Cases:

- Interval scheduling
- Scheduling with deadlines
- Minimizing/maximizing cost
- Choosing smallest/largest values
- Picking shortest path in weighted graph (Dijkstra)

$$\theta_{new} = \theta_{old} - \gamma \cdot \nabla J(\theta)_{old}$$

At each step → best choice right now

without consideration of future optimality



2p 3p
(s_1, e_1)

2:30p 3:30p
(s_2, e_2)

overlap

$s_2 \leq e_1$

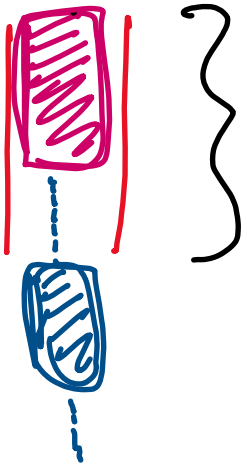
Activity (s_n, e_n)

---

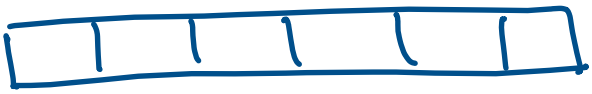
arrivals
departures
} → min no. of platforms required, so train waits

arr <= dep.

(1:30p) (2p)



arr



↳ jump lengths

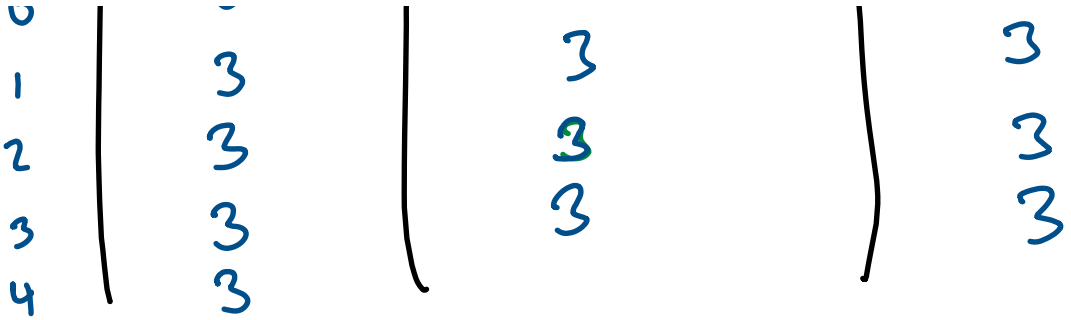
2, 3, 1, 1, 4

i	farthest	i + nums[i]	farthest_updated
0	0	2	2
1	2	4	4
2	4	3	4
3	4	4	4
4	4	8	8

True

3, 2, 1, 0, (4)

i	farthest	i + nums[i]	farthest_upd
0	0	3	3
1	3	3	3



false

12 - Trees

Saturday, December 20, 2025 12:09 AM

- **Introduction**
- **Common Terminology**
- **Properties of Trees**
- **Traversal Algorithms**
- **Binary Search Trees**
- **N-ary Trees**
- **Dynamic Programming in Trees**
- **Coding Problems**

12.1 - Introduction

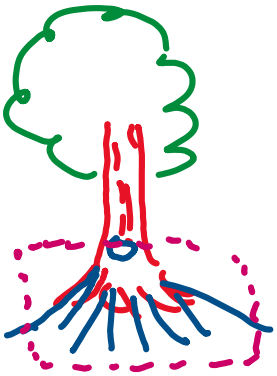
Saturday, December 20, 2025 12:18 AM

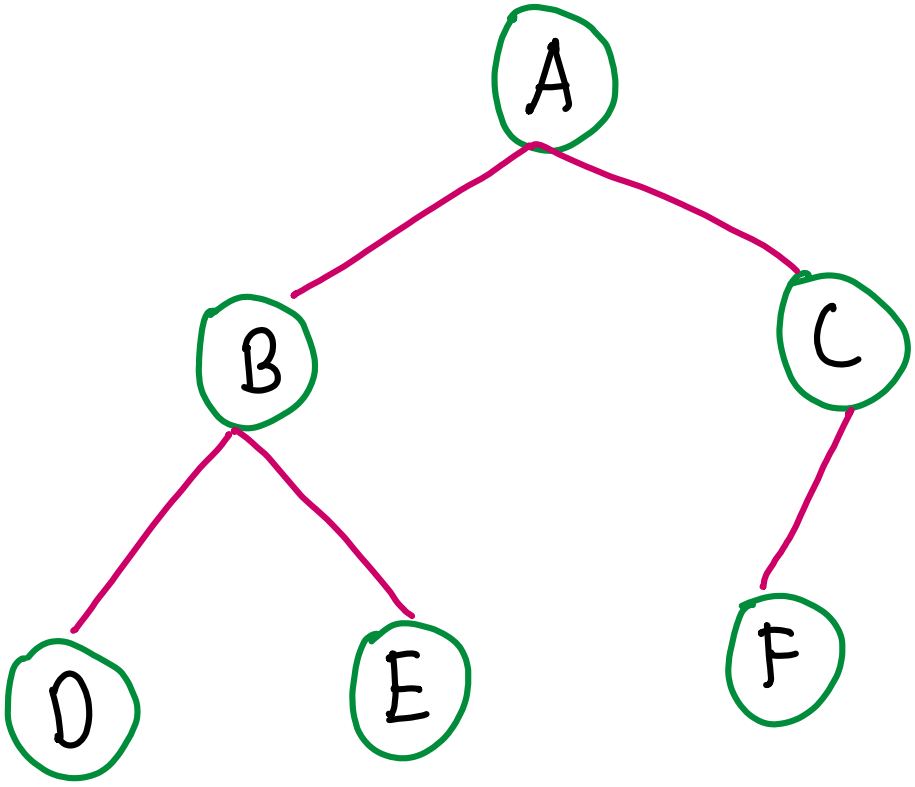
non-linear

hierarchical

What is a Tree?

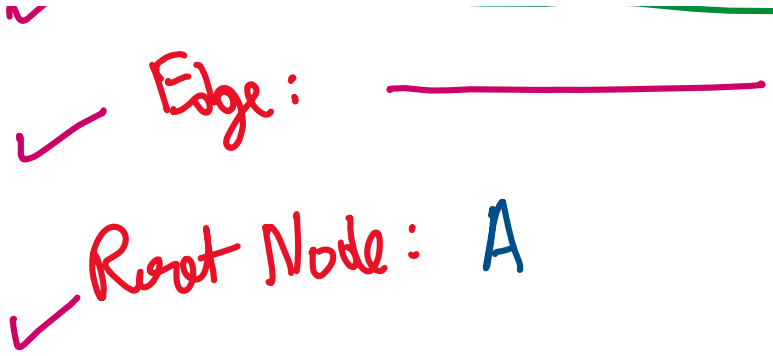
- A tree is a non-linear hierarchical data structure
- Consists of **nodes** connected by **edges**
- Data is organized in a **parent–child** relationship
- A tree consists of:
  - *A finite set of nodes*
  - *Connected by edges*
  - *With a single root node*
  - *And no cycles*





Nodes: A, B, C, D, E, F

Edges: _____


Edge :
Root Node : A

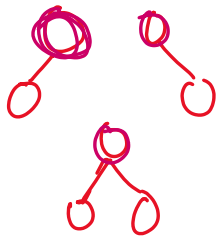
12.2 - Terminology

Saturday, December 20, 2025 12:21 AM

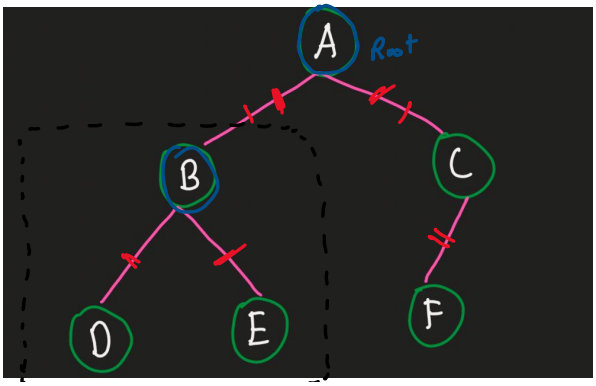
Tree Terminology:

TERM	MEANING
Root	Topmost node
Parent	Immediate predecessor
Child	Immediate successor
Leaf	Node with no children
Depth	Distance from root (edges count)
Height	Longest path to a leaf
Level	Depth + 1
Subtree	Tree formed from any node

1. **Root:**
   - The topmost node of the tree
   - Has no parent



2. **Parent:**
   - A node that has one or more children
   - Immediate predecessor of a node
3. **Child:**
   - A node that is a direct descendant of another node
4. **Leaf:**
   - A node with no children
   - Also called external node
5. **Depth:**
   - Number of edges from the root to the node
   - Root always has depth = 0
   - Depth answers "***how far am I from root?***"
6. **Height:**
   - Longest path from a node to a leaf
   - Measured in edges
   - Height of a leaf node = 0
   - Height answers "***how far can I go down?***"
7. **Level:**
   - Is a mathematical representation
   - **Depth + 1**
8. **Subtree:**
   - Any node along with all its descendants
   - Every node forms a subtree
   - Crucial in **recursion-based** tree problems



Node	Depth	Height	Level
A	0	2	1
B	1	1	2
C	1	1	2
D	2	0	3
E	2	0	3
F	2	0	3

E 2 0 3
F 2 0 3

Parent: A, B, C

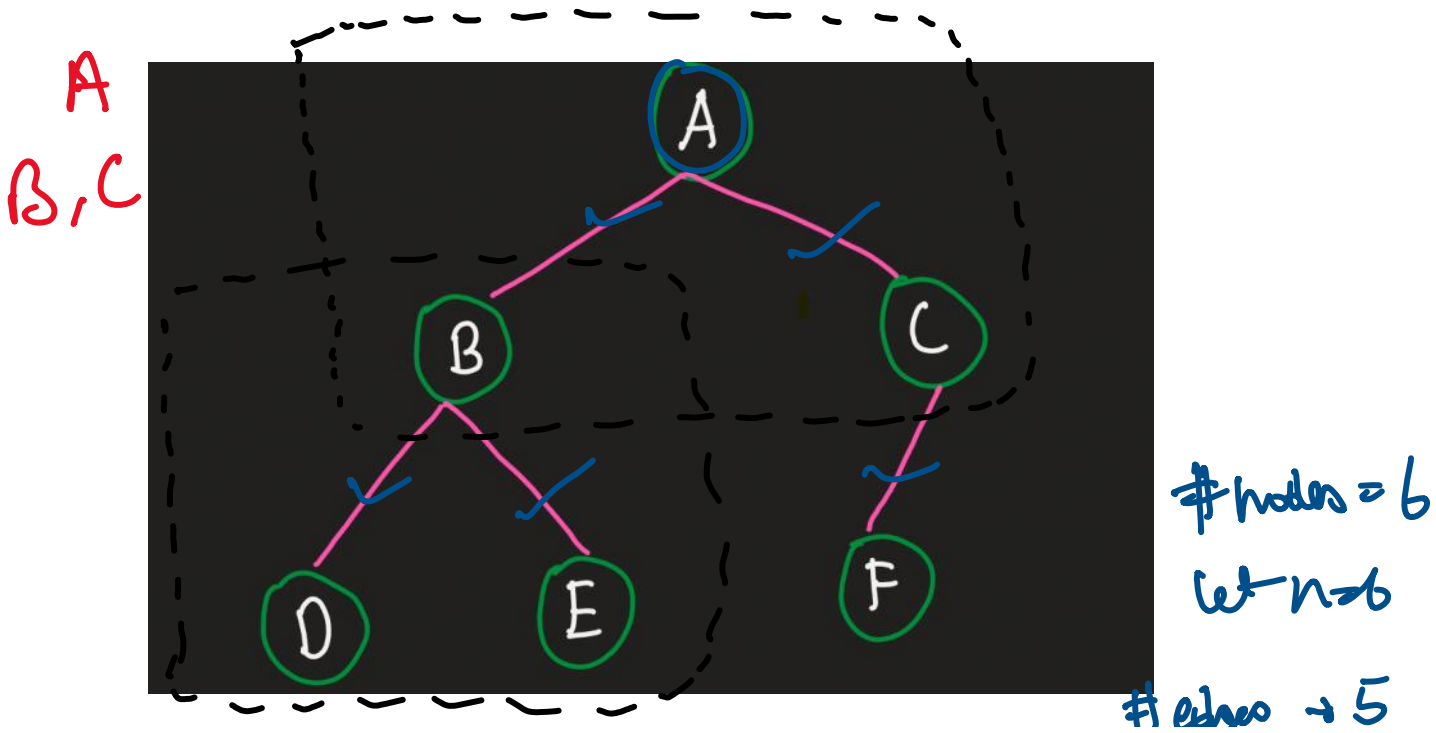
Leaf: D, E, F

12.3 - Properties

Saturday, December 20, 2025 12:20 AM

Key Properties of a Tree:

- **Exactly One Root:**
  - The root is the starting point of the tree
  - All nodes originate from this root
- **Parent–Child Relationship:**
  - Every node (except root) has exactly one parent
  - A node can have zero or more children
- **No Cycles:**
  - You can never return to a node once you move down the tree
  - This property distinguishes trees from graphs
- **Connected Structure:**
  - Every node is reachable from the root
- **Edges Count Rule:**
  - If a tree has **n** nodes, it will have exactly (**n - 1**) edges



#edges → 5

12.4 - Traversal

Saturday, December 20, 2025 2:13 AM

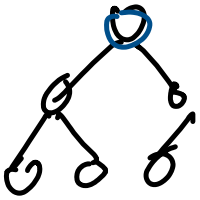
What is Tree Traversal?

- Tree traversal is the process of visiting every node of a tree exactly once in a systematic way
- Since a tree is non-linear, there is no single natural order like arrays or linked lists
- We define multiple traversal strategies depending on problem requirements
- **Traversal answers:**
  - *How do we visit every node exactly once?*
  - *In what order should nodes be processed?*

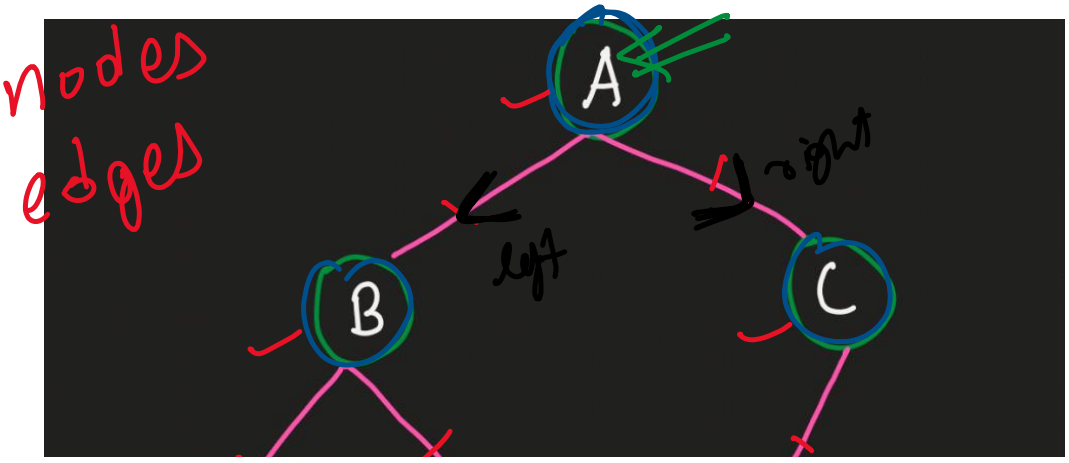
Types of Tree Traversals:

Tree traversals are broadly classified into two categories:-

1. **Depth First Search (DFS):** *uses recursion or stack; explores one branch completely before moving to another*
   - Preorder
   - Inorder
   - Postorder
2. **Breadth First Search (BFS):** *uses a queue; explores each node level by level*
   - Level Order Traversal

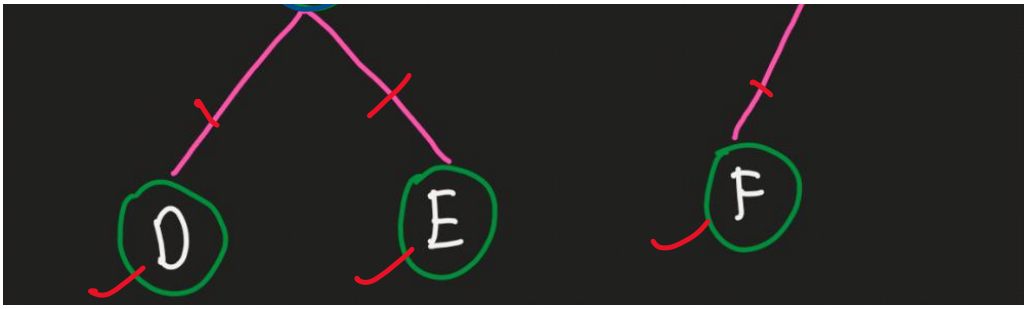


Q { R,



class Node:

value
left
right

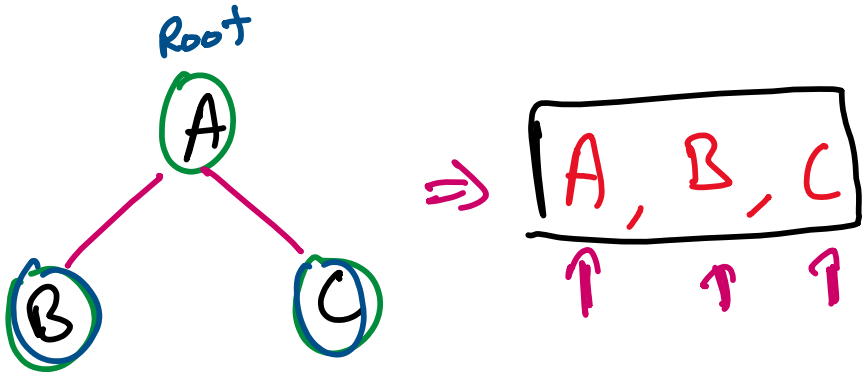

D
E
F

12.4.1 - Preorder

Saturday, December 20, 2025 11:42 AM

Order of Traversal:

Root → Left → Right

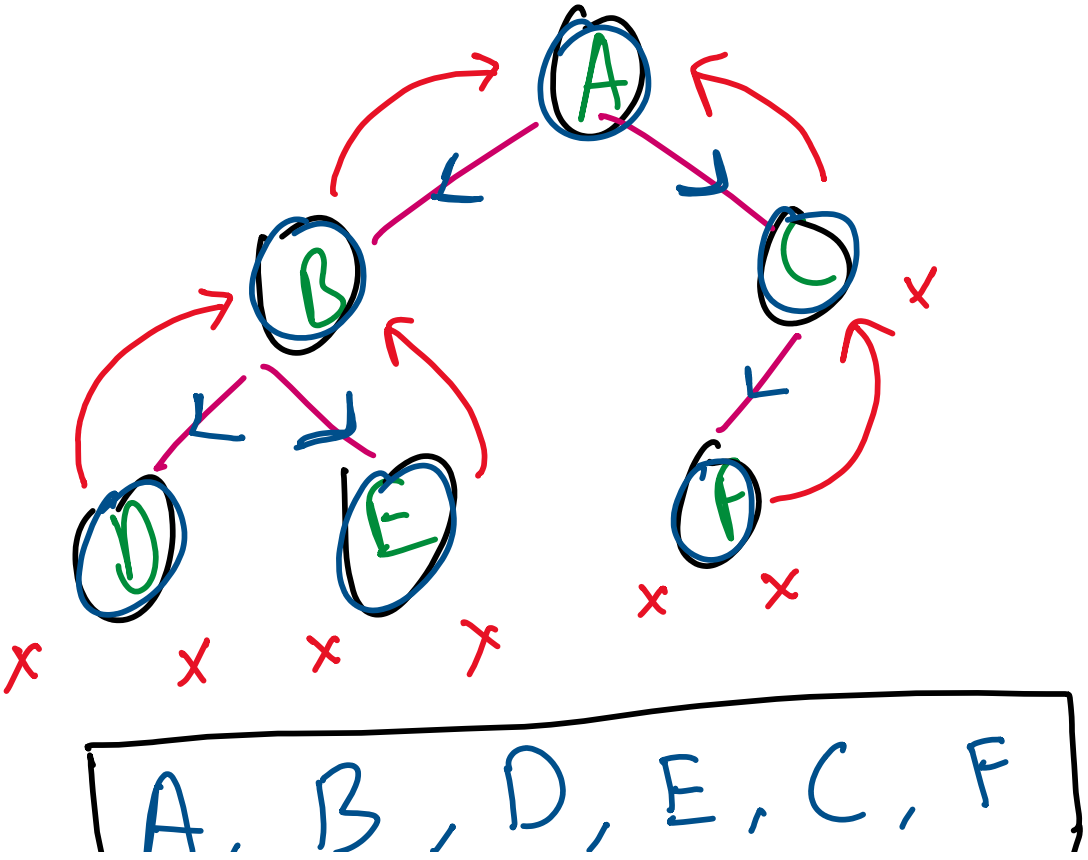


Intuition:

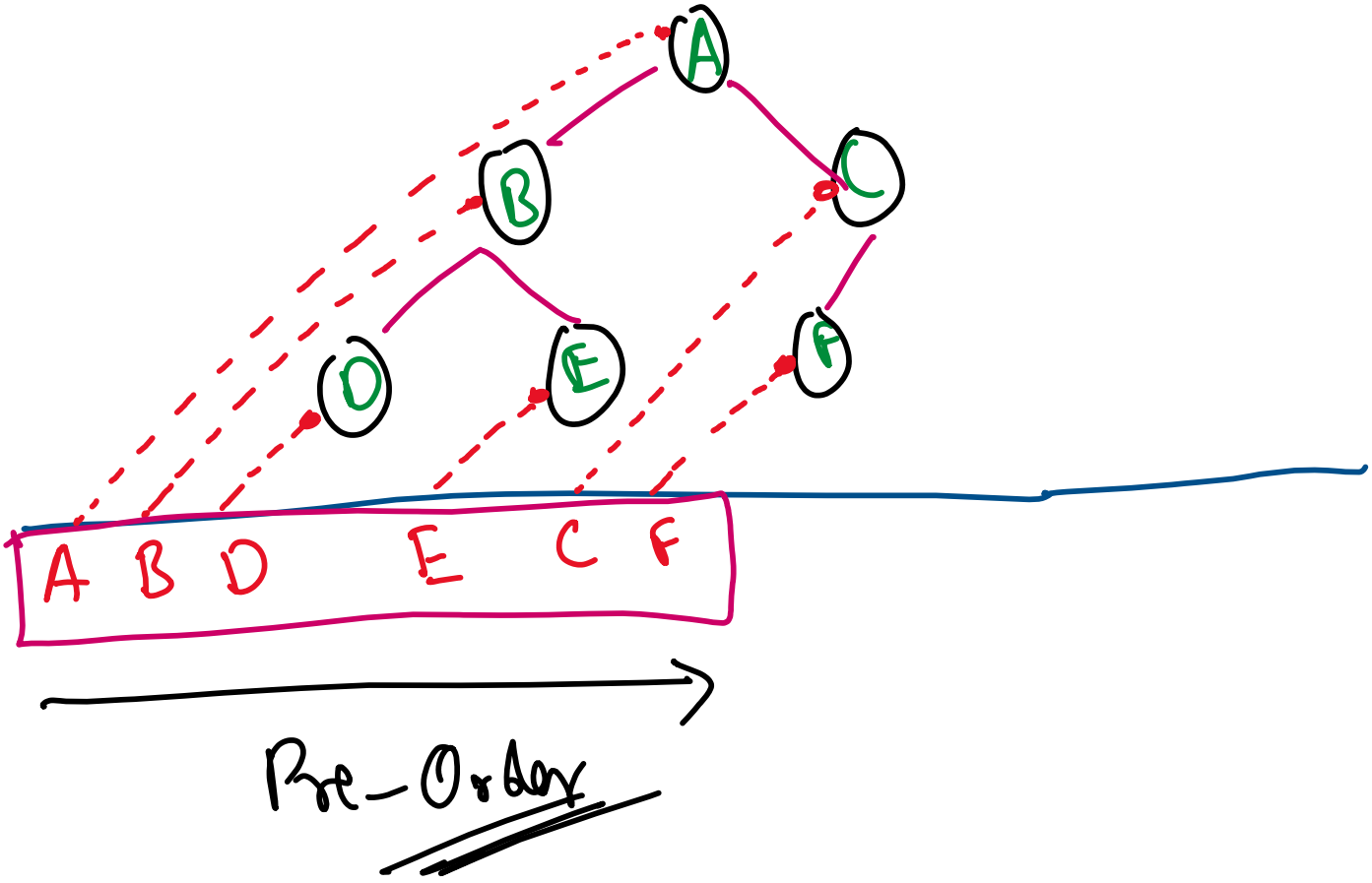
- Visit a node before its children
- Process current node, then go to left node, then right node

Usecases:

- **Copying / cloning a tree**
- **Prefix expression evaluation:** Operators appear before operands
- **Serialize a tree:** Convert a tree into a string or list
- **Creating directory structures:** Representing folders and files as a tree

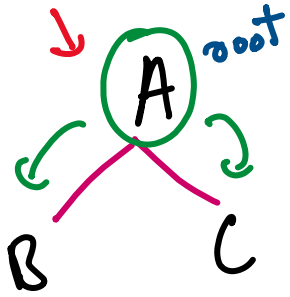


A, B, D, E, C, F



12.4.2 - Inorder

Saturday, December 20, 2025 11:50 AM



Order of Traversal:

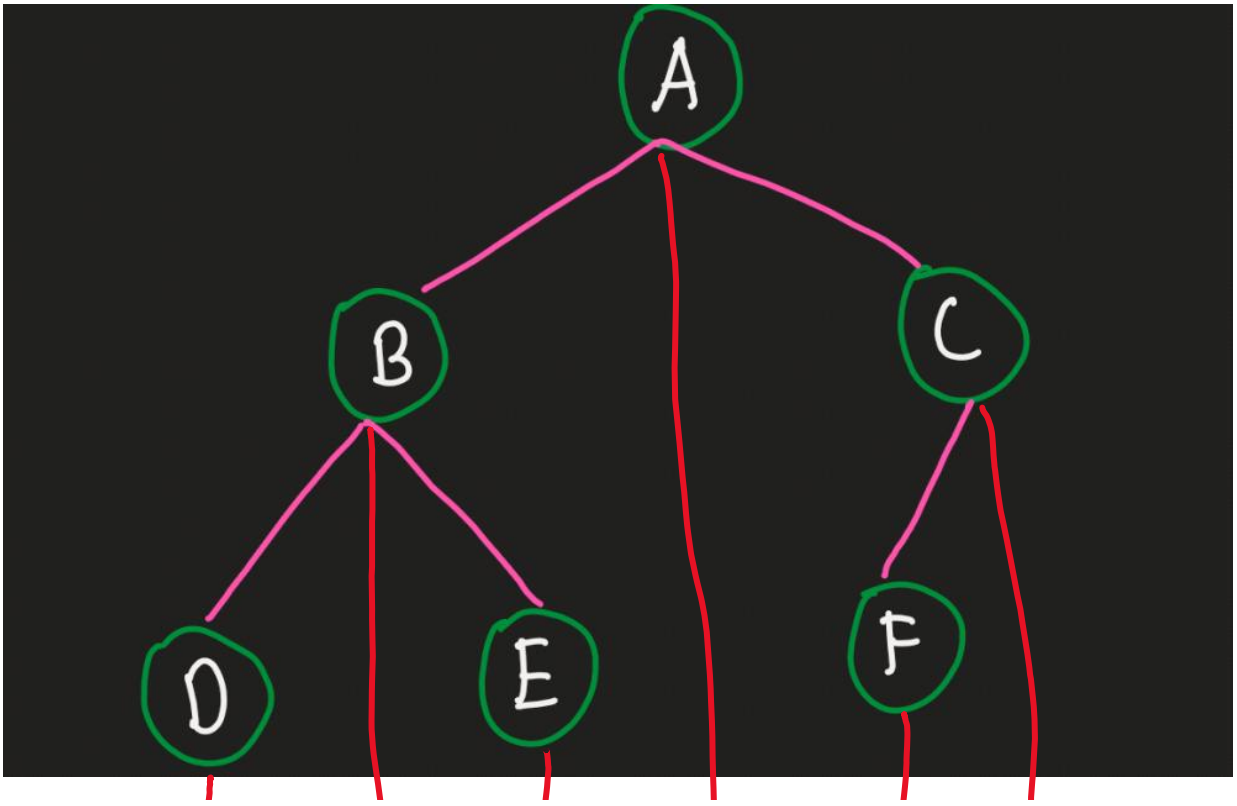
Left → Root → Right

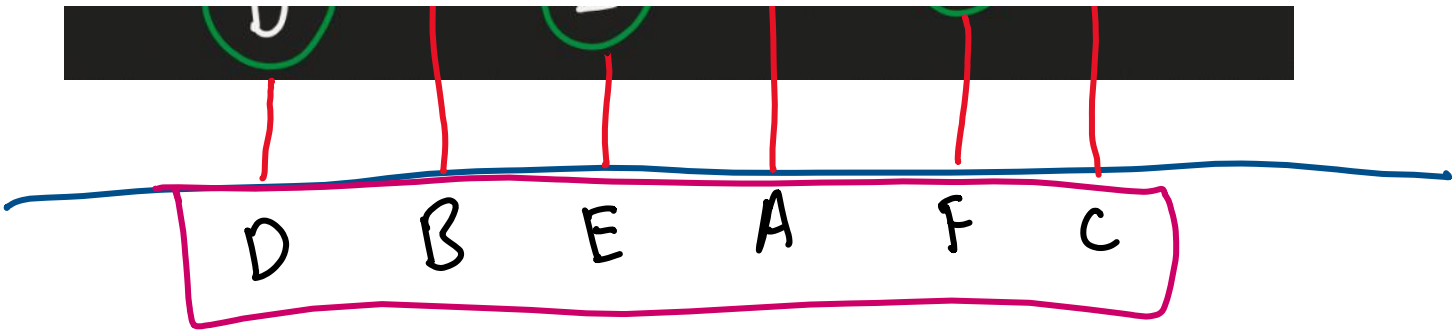
Intuition:

- Visit left child first
- Root comes in between
- Then visit right child

Use Cases:

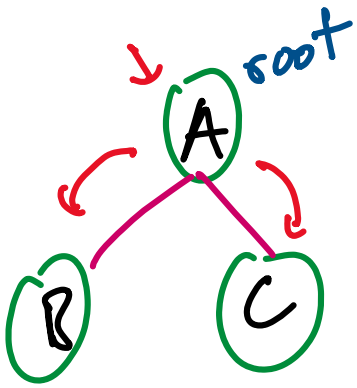
- Validate a BST
- Retrieve sorted data from BST
- Expression trees (infix notation)
- Finding kth smallest / largest




D B E A F C

12.4.3 - Postorder

Saturday, December 20, 2025 11:51 AM



Order of Traversal:

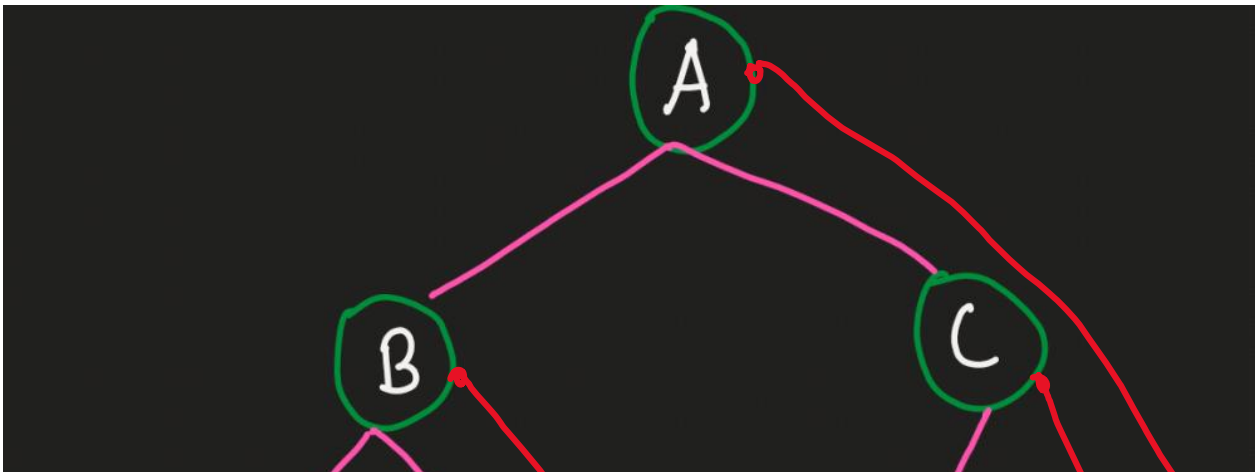
Left → Right → Root

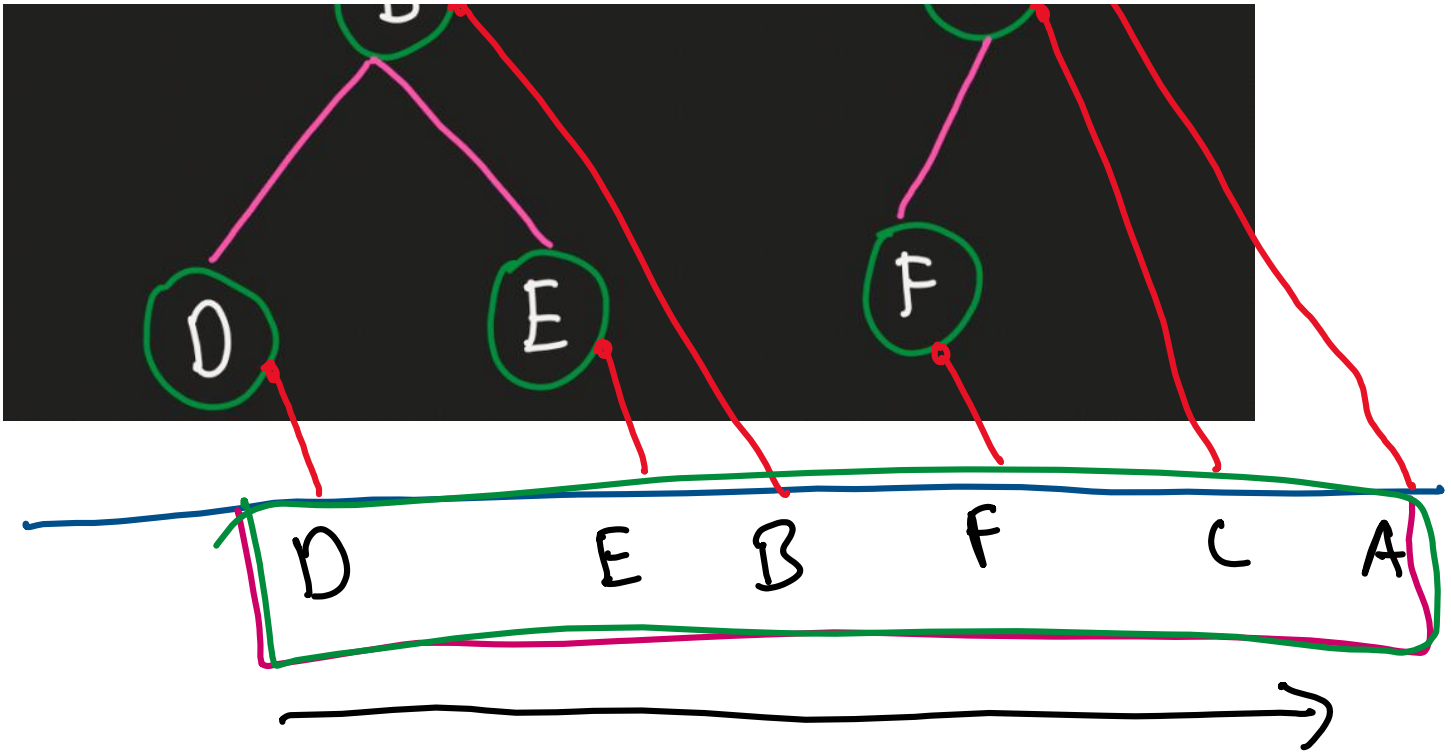
Intuition:

- Process children before parent
- Root is visited last
- Left node first, followed by right node

Use Cases:

- Deleting a tree
- Calculating height
- Postfix expression evaluation
- Bottom-up computations (DP on trees)




D
E
F
D
E
B
F
C
A

12.4.4 - Level Order

Saturday, December 20, 2025 11:52 AM

Order of Traversal:

- Visits nodes level by level
- Uses a queue (FIFO)

Intuition:

- Visit all nodes at depth 0
- Then depth 1
- Then depth 2
- Repeat until tree depth is reached

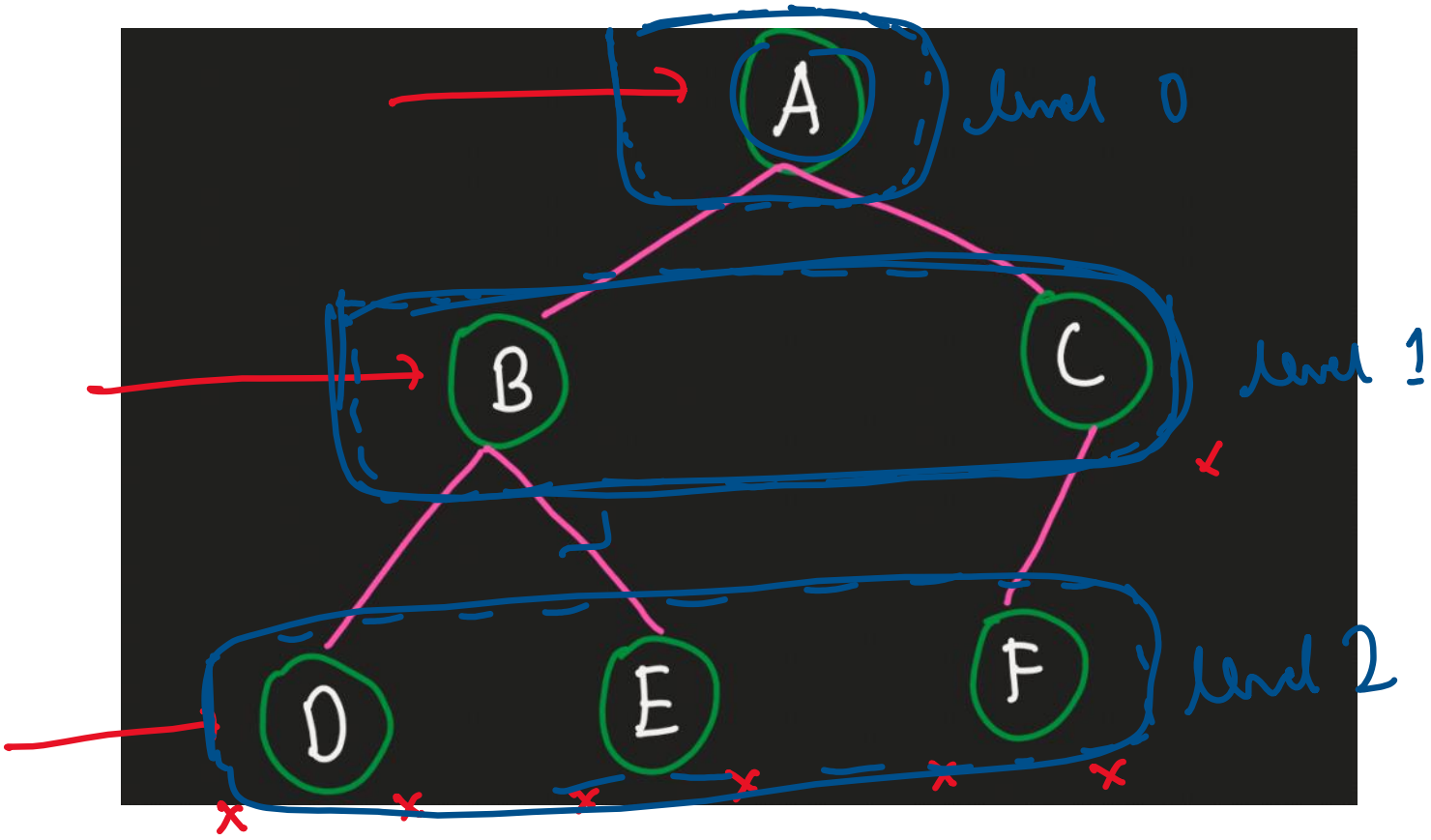
Why use a Queue?

- Must visit all nodes at level **L** before moving to level **L+1**
- Insert root into queue
- Remove (dequeue) the oldest node
- Add its children at the end of the queue
- Parents enter the queue before children

Use Cases:

- Level-based problems
- Tree views (top, bottom, left, right)
- Shortest distance in trees

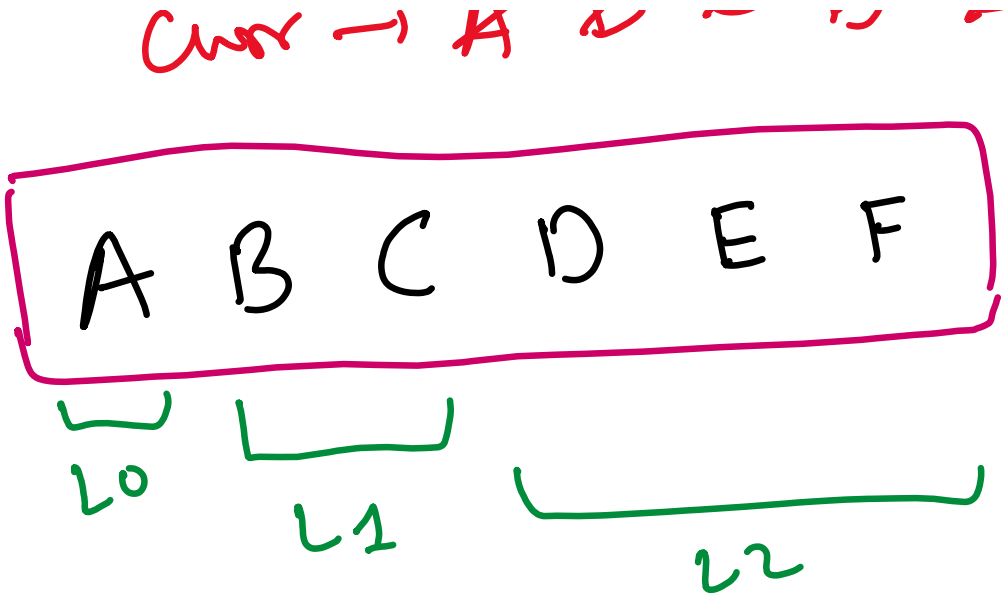
- Checking tree completeness
- Zig-zag traversal



A, B, C, D, E, F

Q = [A, B, C, D, E, F] [ ]

curr → A B C D E F


A B C D E F
L0
L1
L2

12.4.5 - Choose Traversal

Saturday, December 20, 2025 12:09 PM

How to Choose the Right Traversal?

PROBLEM	TRAVERSAL
Sorted output	Inorder (BST)
Root-first logic	Preorder
Child-first logic	Postorder
Level-wise	Level Order
Height / diameter	Postorder
Views / distance	Level Order

12.5 - BST

Saturday, December 20, 2025 12:15 PM

What is a Binary Search Tree?

A Binary Search Tree (BST) is a binary tree with a special ordering property that enables efficient searching, insertion, and deletion

Key Properties:

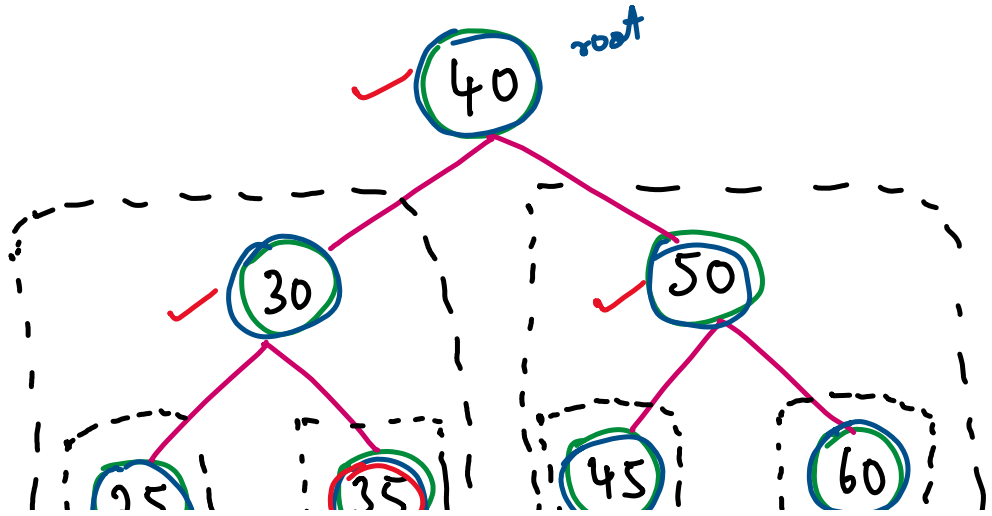
- Each node has zero, one, or two children
- All values in the left subtree are less than the node's value and all values in the right subtree are greater than the node's value
- Both, the left and right subtrees must also be Binary Search Trees themselves

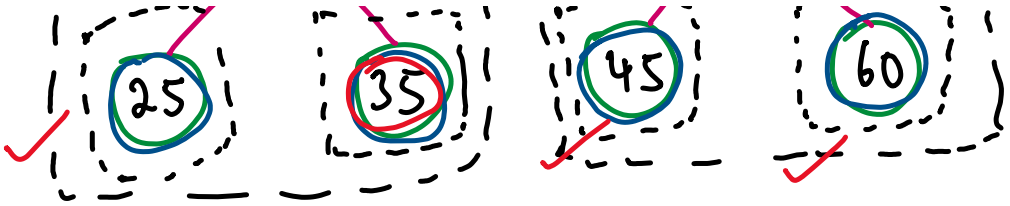
Why use a BST?

OPERATION	TIME COMPLEXITY
Search	O(log n)
Insert	O(log n)
Delete	O(log n)

Key Applications:

- Database Indexing
- Dictionary and Spell Checkers
- Dynamic Sorting
- Priority Queues
- 3D Graphics and Game Engines




25
35
45
60

12.5.1 - Insertion

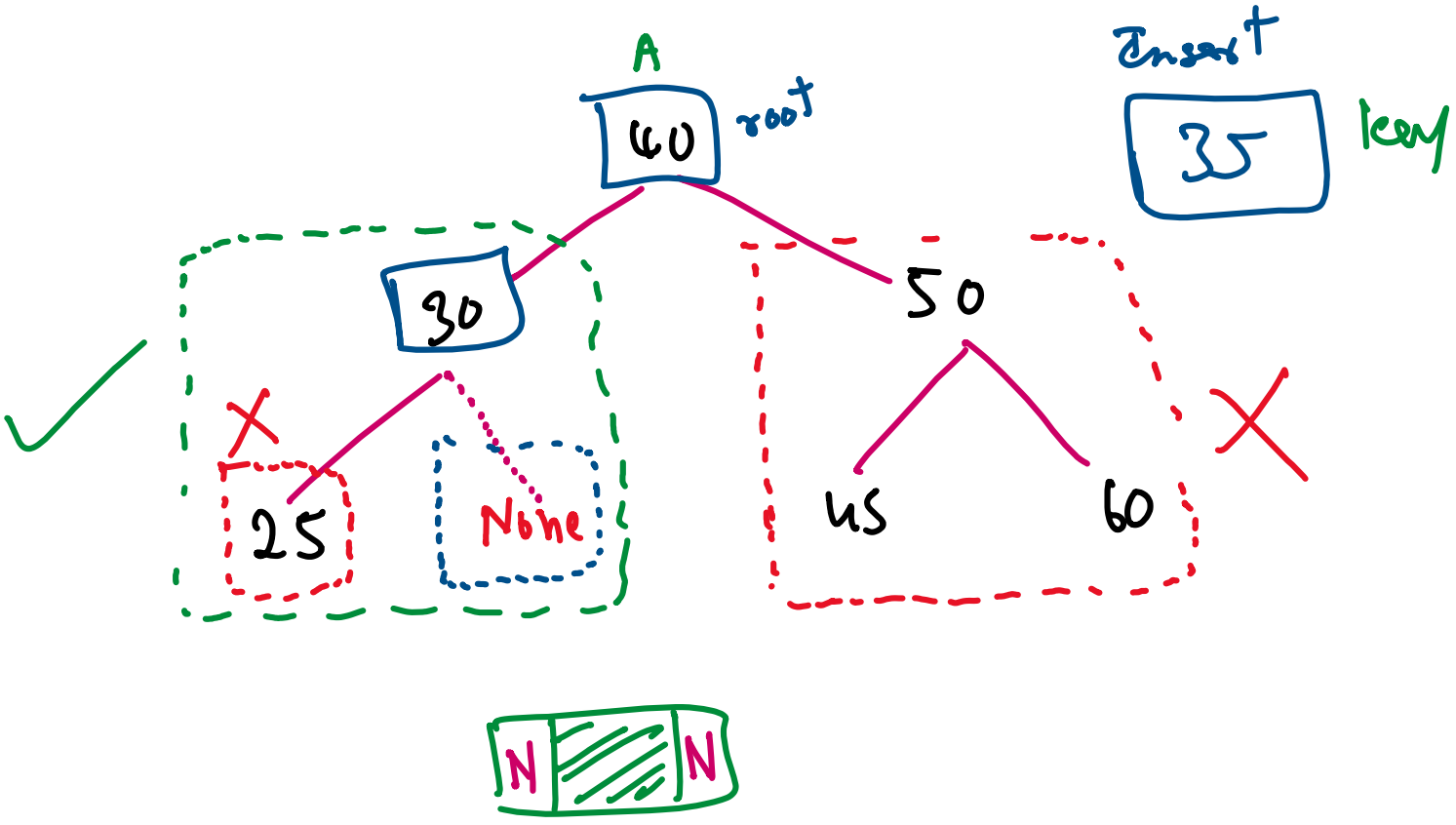
Saturday, December 20, 2025 12:29 PM

Steps:

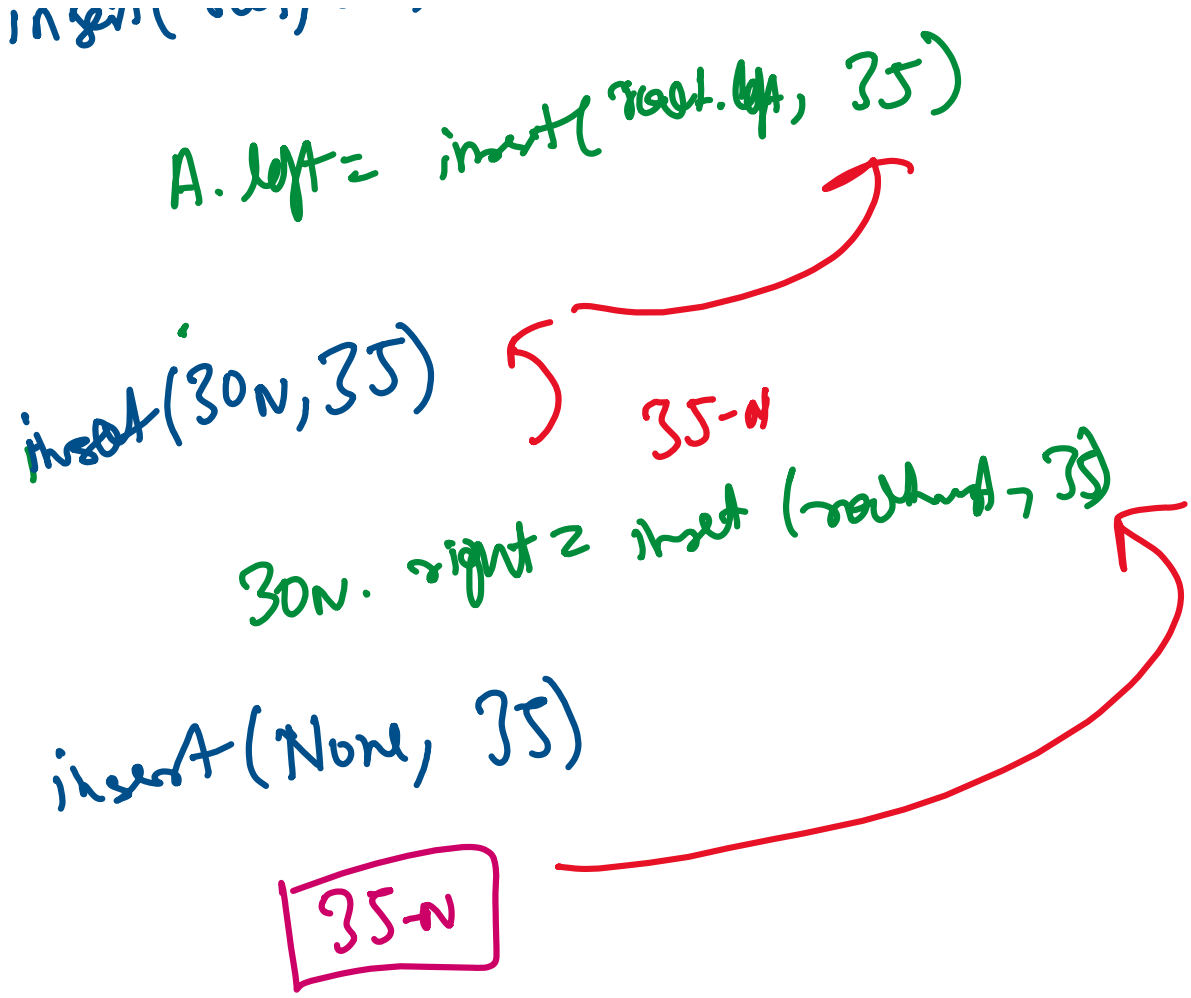
- Start from ***root***
- If **key < root.val** → go left
- If **key > root.val** → go right
- Insert at first ***None*** position

Time Complexity:

- **Average:** O(log n)
- **Worst case (skewed tree):** O(n)



insert(root, 35)


A.left = insert(root.left, 35)
insert(30N, 35)
35-N
30N.right = insert(rootright, 35)
insertA(None, 35)
35-N

12.5.2 - Searching

Saturday, December 20, 2025 12:29 PM

Steps:

- Compare key with **root.val**
- Go left or right accordingly
- Stop when found or reach ***None***

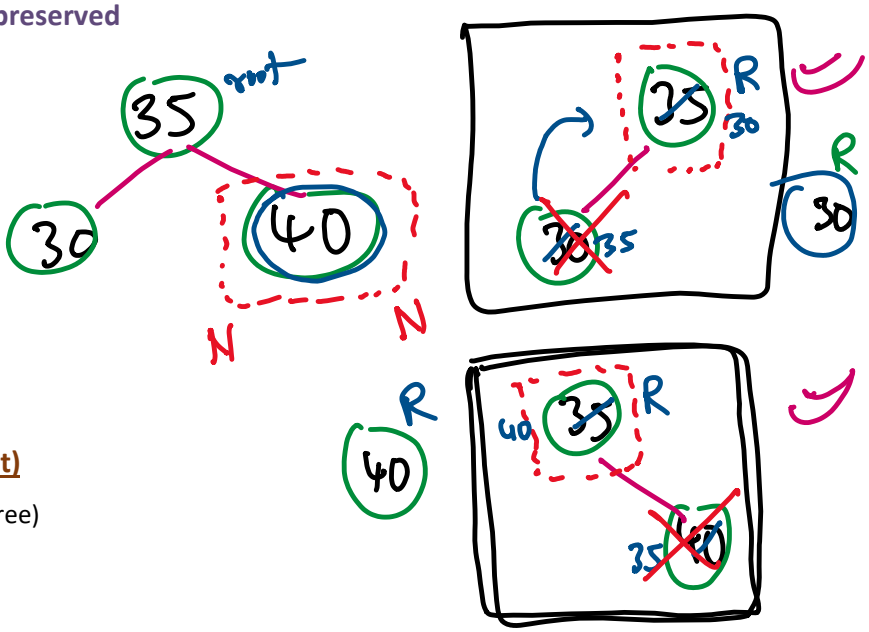
Time Complexity:

- **Average:** O(log n)
- **Worst case:** O(n)

12.5.3 - Deletion

Saturday, December 20, 2025 12:29 PM

Deletion is tricky because **BST property must be preserved**



Case 1: Leaf Node (No Children)

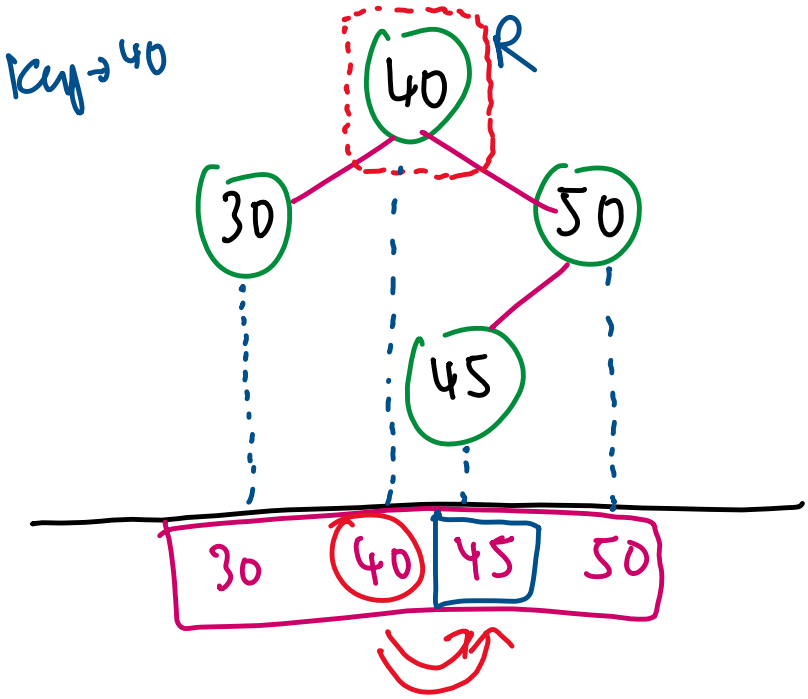
- Simply remove the node

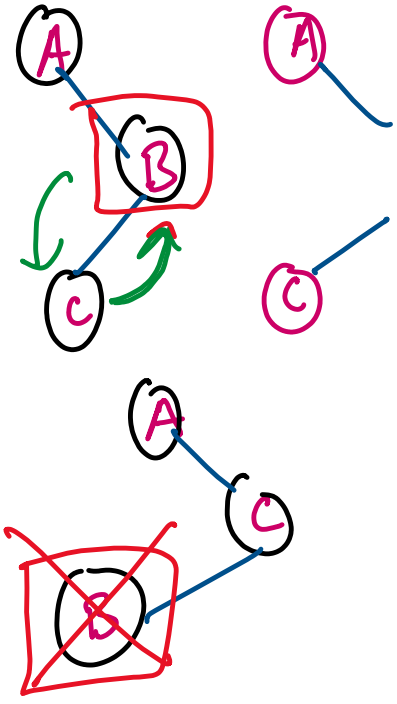
Case 2: Node with One Child

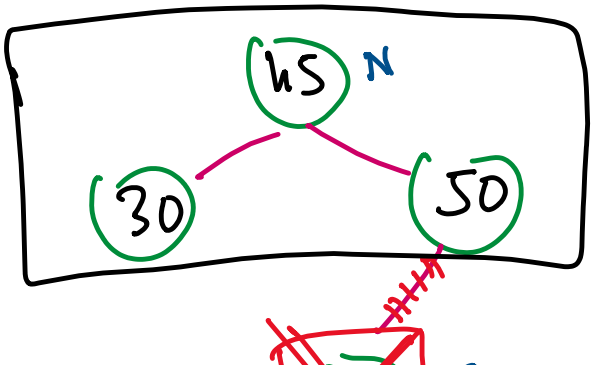
- Replace node with its child

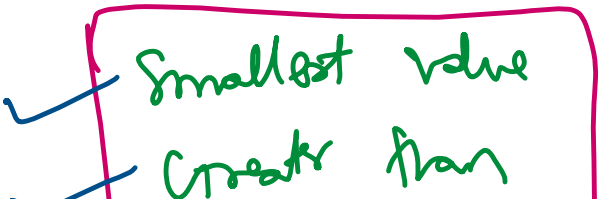
Case 3: Node with Two Children (Most Important)

- Find **Inorder successor** (smallest value in right subtree)
- Replace node's value with successor's value
- Delete successor node



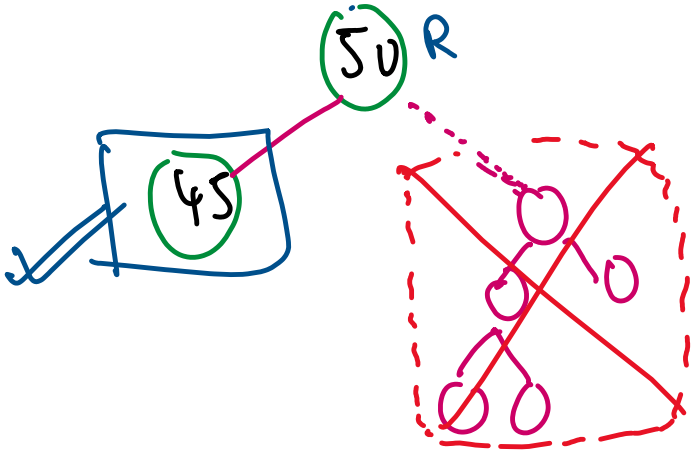






greater than the node

1. Given the node / key
2. Visit / explore the right subtree
3. Visit the left-most node



12.6 - N-ary Tree

Saturday, December 20, 2025 11:59 PM

What is an N-ary Tree?

An **N-ary tree** is a tree where **each node can have 0 to N children**

- **Binary Tree:** max 2 children
- **N-ary Tree:** any number of children

N-ary Tree
↳ max no of children any node can have

Traversal:

- **Preorder** (***Root → Children***)
- **Postorder** (***Children → Root***)
- **Level Order**

Node Structure:

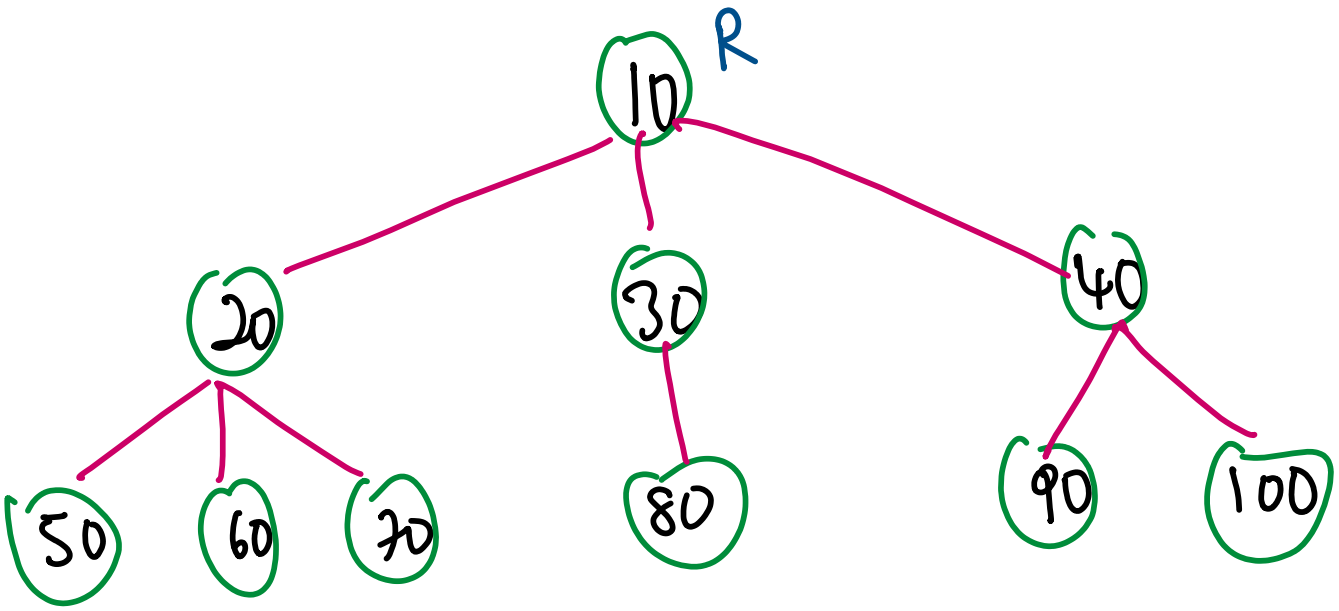
```
class Node:
    def __init__(self, val):
        self.val = val
        self.children = []
```

- ***children*** is a **list**
- No fixed left/right
- Order of ***children*** may or may not matter

Key Applications:

- **File system** (folders → many subfolders)
- **Organization hierarchy**
- **XML / HTML DOM tree**
- **Trie (special N-ary tree)**

3-Ary Tree


R
10
20
30
40
50
60
70
80
90
100

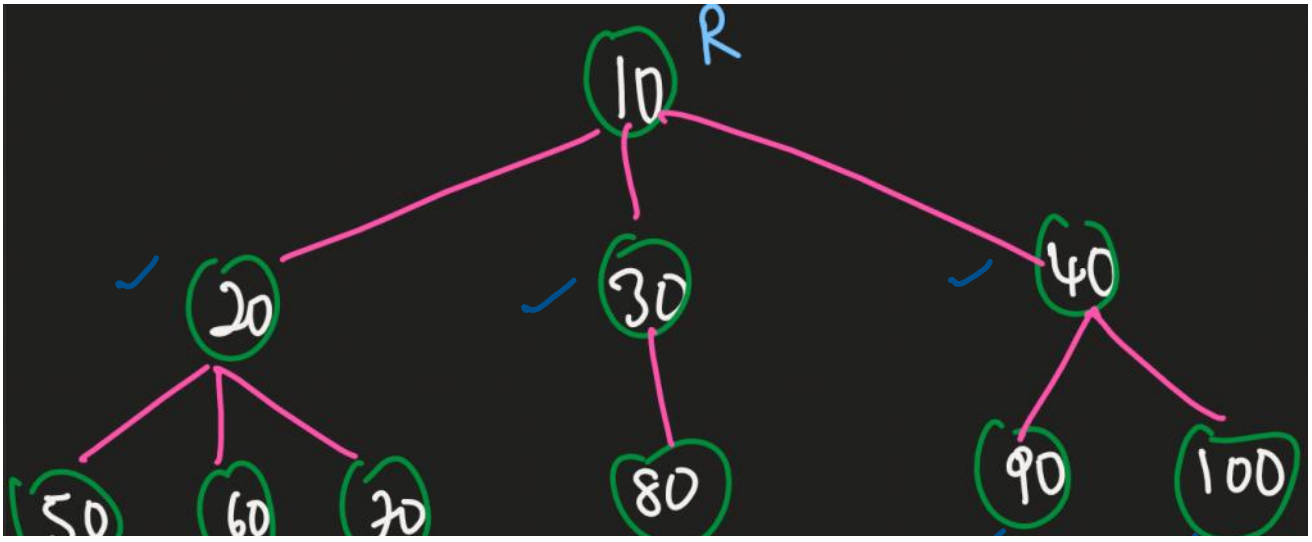
12.6.1 - Traversals

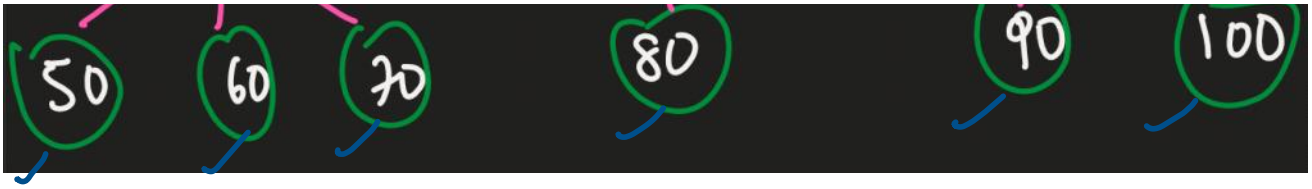
Sunday, December 21, 2025 12:10 AM

1. **Preorder:**
   - Visit the root first
   - Then, visit all children, one at a time
   - **Use cases:**
     - Serialize tree
     - Copy tree
     - Prefix expressions

2. **Postorder:**
   - Visit the children first
   - Then, visit the root node
   - **Use cases:**
     - Deleting tree
     - Aggregation problems (sum, height, DP)

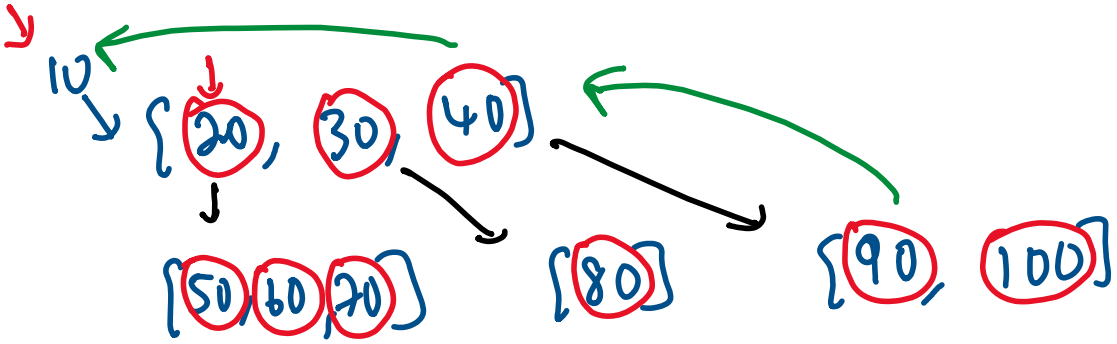
3. **Level Order:**
   - Natural ***level by level*** expansion
   - Used in hierarchy / shortest path problems





10 20 50 60 70 30 80 40 90 100

Pre-Order Traversal



Postorder Traversal

50, 60, 70, 20, 80, 30, 90, 100, 40, 10

Q = { ~~10~~ ~~20~~, ~~30~~, ~~40~~, ~~50~~, ~~60~~, ~~70~~, ~~80~~, ~~90~~, ~~100~~]

front

rear

curr → ~~10~~ ~~20~~ 30 40

~~50~~ 60 ~~70~~

~~80~~ ~~90~~ 100

80 90 100

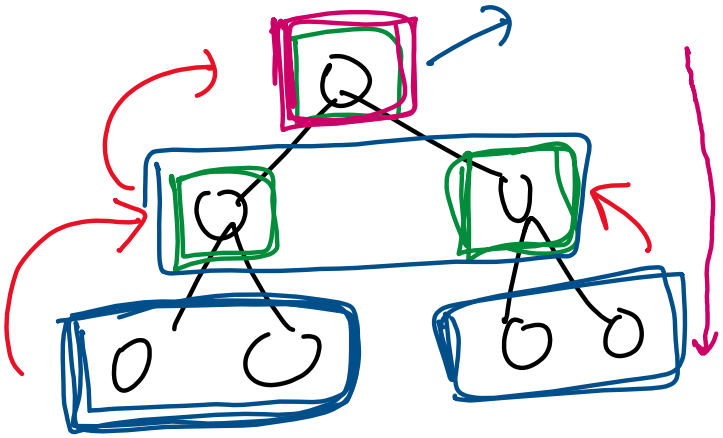
10, 20, 30, 40, 50, 60, 70, 80, 90, 100

Level-Order Traversal

1. Check if Queue is empty
2. Dequeue
3. Process the dequeued node
4. Enqueue it's child nodes

12.7 - Tree DP

Sunday, December 21, 2025 12:14 AM



What is Tree DP?

- Each node computes an answer using its children's answers

Key Properties:

- **No cycles** → recursion can be used
- **Each subtree is an independent problem**
- **Postorder traversal is the backbone of Tree DP**

Why only Postorder for Tree DP?

- Tree DP needs the children's results before the parent can be computed
- Postorder is the only traversal that guarantees this

When to use Tree DP?

- Does the answer for a node depend on the children?
- Are there multiple choices at each node?
- Is there a need to return more than one value?

Tree DP Template:

```python
def dfs(node):
    # 1. Base case
    if not node:
        return BASE_VALUE

    # 2. Compute results from children
    child_results = []
    for child in node.children:
        child_results.append(dfs(child))

    # 3. Combine child results
    result = combine(child_results, node)

    return result
```

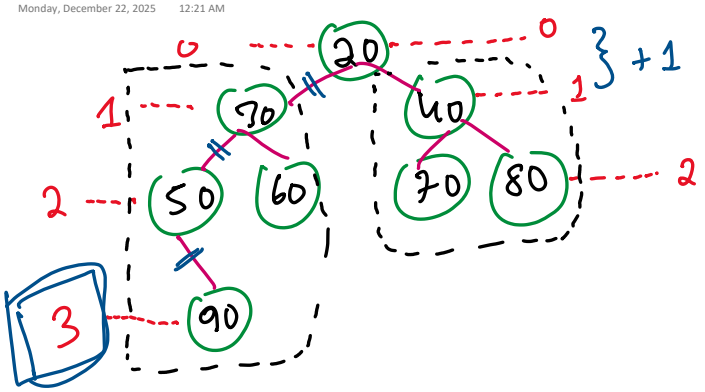
Base Condition

Process child nodes

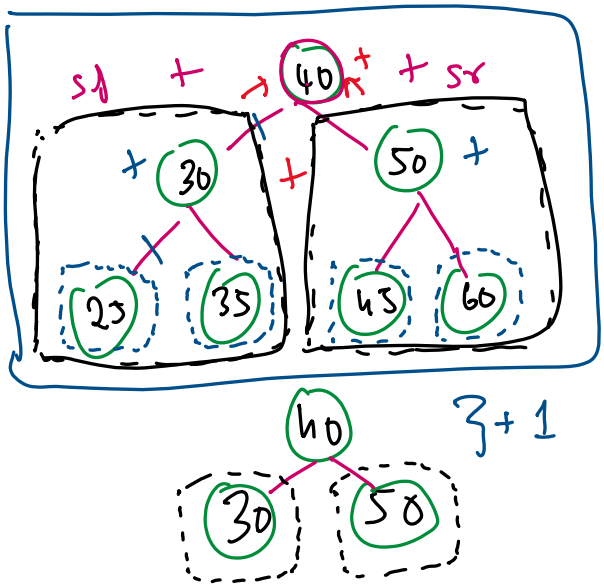
Combine results

1. Base Condition
2. Visit/Process child nodes

2. Visit/Process child
3. Console parents child notes
4. Return parent



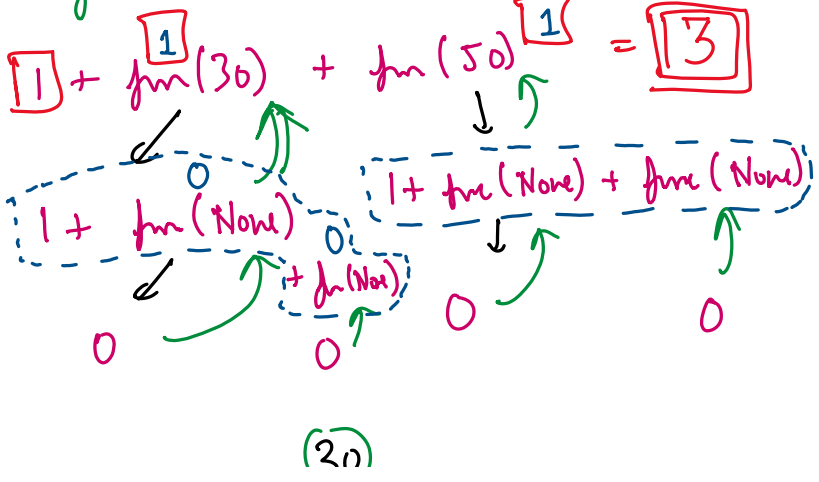
1. Observe left & right subtrees
2. At root, depth is zero
3. Obtain max (left, right) + 1

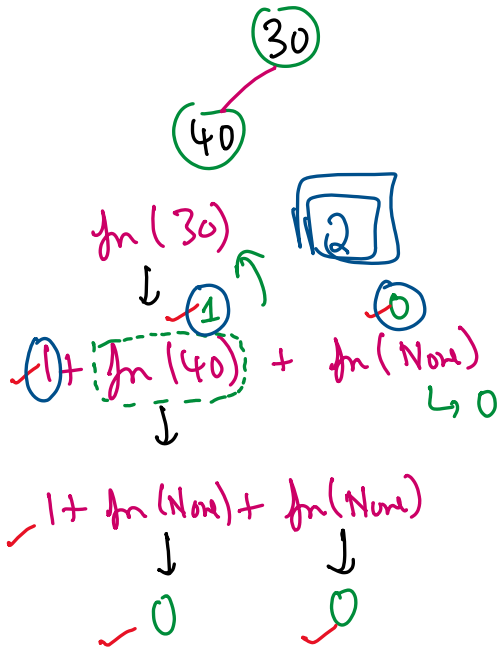


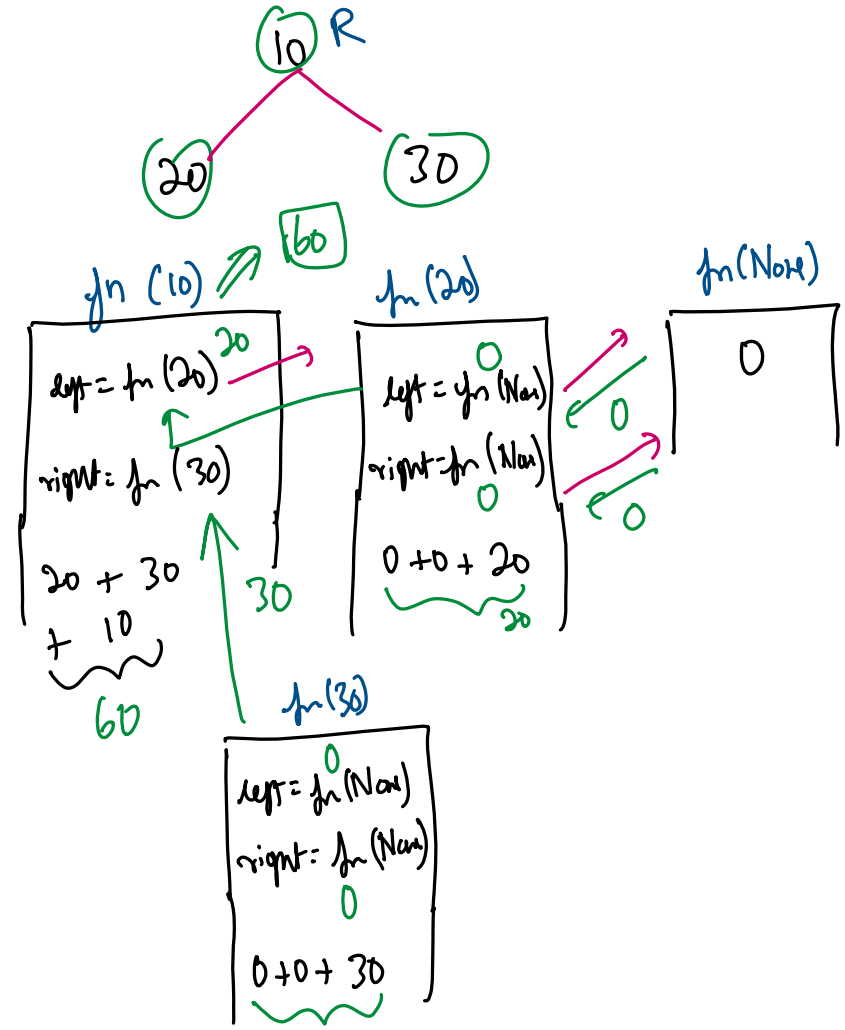
sum_left + sum_right + node.value

return 1 + count(left) + count(right)

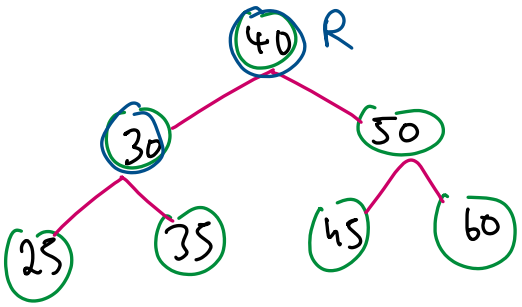
func(40)







left-tree < node.val < right-tree



left_tree < node.val < right_tree

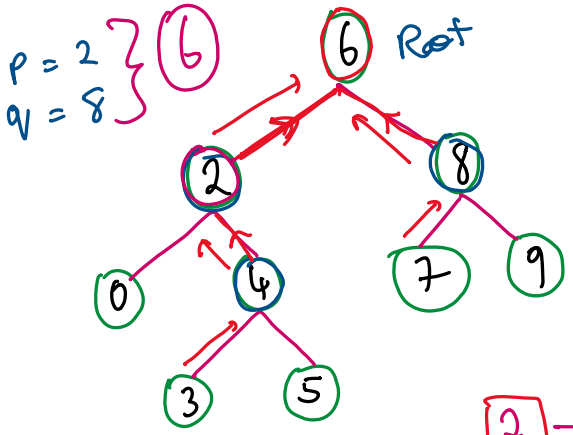
node.val
left (low)
right (high)

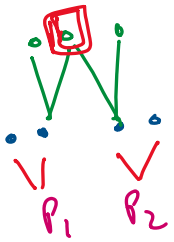
node → node.left
low → low
high → node.value

left

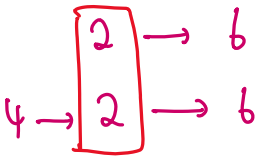
node → node.right
low → node.value
high → high

right





p=2
q=4
2



if p < node & q < node :

left sub tree

if p > node & q > node:

right sub tree

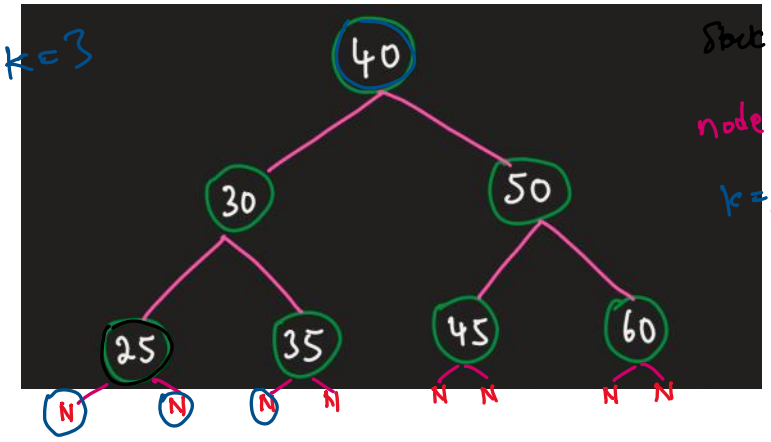
right sub tree

return node

3 → 4 → 2 → 6

7 → 8 → 6

(6)



stack = [40, 35]

node = 25 None 30 35

k = 3 2 1 0

None 35

10 17 23 25 34

BST → Inorder Traversal → Ascending Order

Recursive X

Iterative ✓

Inorder Traversal

Left, Root, Right

20 R

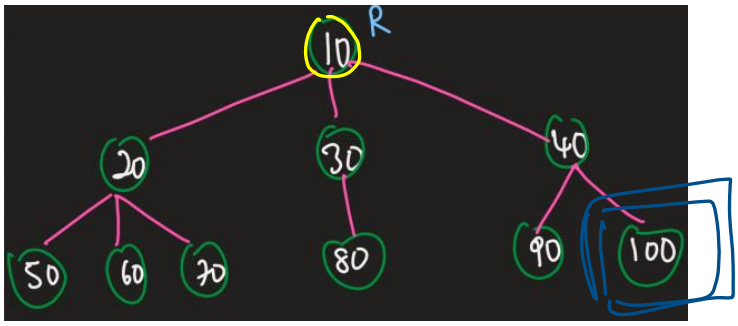
10 30 ⇒ 10, 20, 30

0 0 0 0

Stack!!

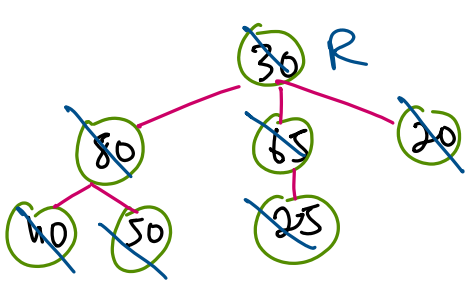
Stuck!..

K = 3 2 1 0



Base Condition → -inf

```
max_val → node.val
for child in node.children:
    max_val = max(max_val, fn(child))
return max_val
```



80

fn(30)

res = ~~30~~ 80

[80, 65, 20]

max(30, fn(80))

80

max(80, fn(65))

fn(80)

res = 80

[40, 50]

res = max(80, fn(40))

40

res = max(80, fn(50))

50

fn(40)

res = 40

40

fn(50)

res = 50

50

max(80, f(65))
65
max(80, f(20))
20
80

80

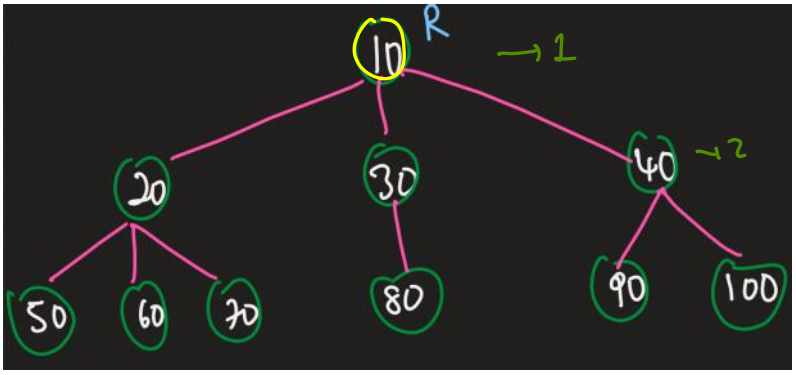
res = max(80, f(50))
50
80

50

f(20)
res = 20
20

f(65)
res = 65
[25]
max(65, f(25))
25
65

f(25)
res = 25
25

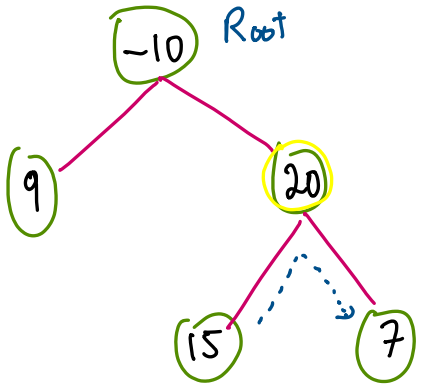


Base Condition → 0

heights = [ ]
for each child:
heights.append(f(child))
return 1 + max(heights)

-10 Root

15+20+(-10) = 25



15+20+(-10) = 25

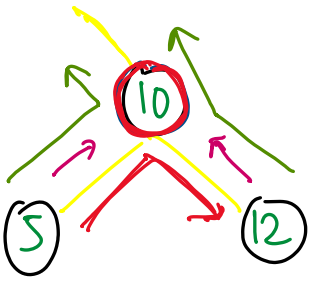
7+20+(-10) = 17

15+20 = 35

7+20 = 27

15+7+20 = 42

Path → Can start & end at any node



i) Path ends at node

ii) Path goes through node upwards

5 → 10

12 → 10

fn(node) → res of tree/node

fn(node.left) → res of left subtree

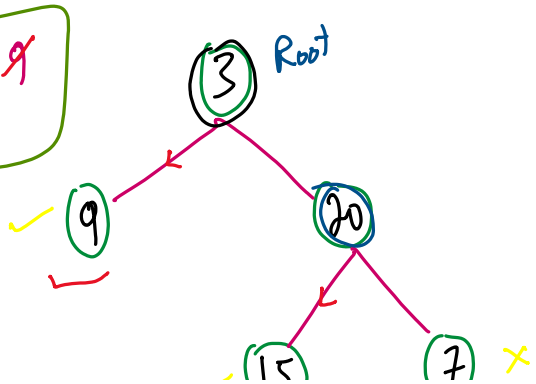
fn(node.right) → res of right subtree

recursive order

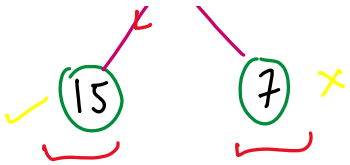
res → node.value + left gain + right gain

res → node.value + max(left gain, right gain)

sum = 0 ✗ 24



9+15 → 24



1. Node should be leaf node
2. Node must be left child

fn(3)

total = ~~0~~ ~~9~~ 24

total += f(9) } 0

total += f(20) } 15

24

fn(9)

total = 0

total += f(Null) } 0

total += f(Null) } 0

0

f(Null)

0

fn(20)

total = ~~0~~ 15

total += f(15) } 0

total += f(7) } 0

15

fn(15)

total = 0

total += fn(Null) } 0

0

fn(7)

total = 0

total += fn(Null) 0

total += fn(Null) 0

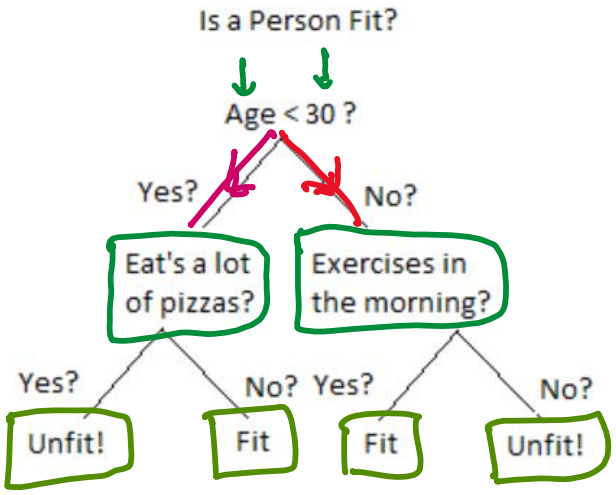
0

12.9 - DS Context

Saturday, December 27, 2025 11:52 AM

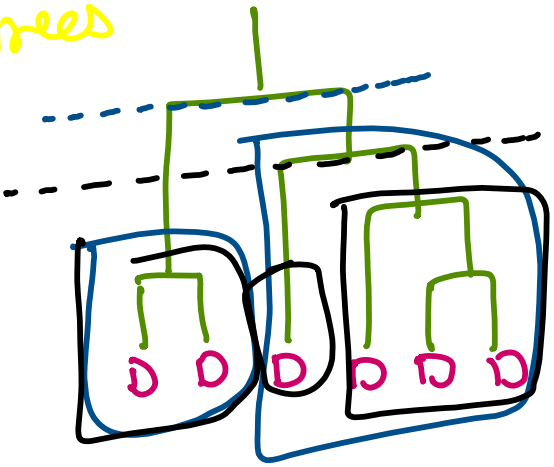
Applications of Tree Data Structures in Data Science & Machine Learning:

- **Decision Trees:**
  - These can be binary or multi-branch trees depending on problem statement
  - Each node results from a feature-value split
  - Each edge corresponds to an outcome/decision
  - Leaf nodes state the final prediction value



- **Random Forests**
- **Gradient Boosted Trees**

collection multiple decision trees



- **Hierarchical Clustering:**
  - Forms a tree-like structure called a ***Dendrogram***
  - Clusters of elements forms the nodes
  - Cutting the tree at a certain heights gives different clusters

- **NLP Parse Trees & Syntax Trees:**

- Grammar-based NLP
- Semantic analysis
- Syntax-aware transformers

- **Data Indexing & Search in ML Systems:**
  - Nearest Neighbor Search
  - Recommendation systems
  - Similarity search

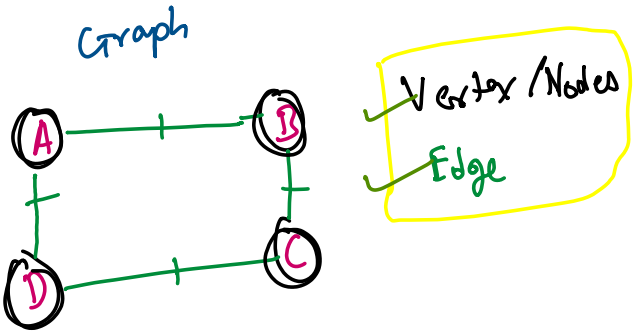
13 - Graphs

Sunday, December 28, 2025 12:21 AM

- **Introduction**
- **Types of Graphs**
- **Degree of Graph**
- **Graph Connectivity**
- **Graph Representation**
- **Graph Traversal**
- **Traversal Variants & Optimizations**
  - ***Multi-source BFS***
  - ***0-1 BFS***
  - ***BFS vs DFS decision making***
- **Shortest Path Algorithms**
  - ***Dijkstra's Algorithm***
  - ***Bellman–Ford Algorithm***
  - ***Floyd–Warshall***
- **Advanced Graph Concepts**
- **State-Space Graphs**
- **Directed Acyclic Graph (DAG) Algorithms**
- **Coding Problems**

13.1 - Introduction

Sunday, December 28, 2025 12:21 AM



What is a Graph?

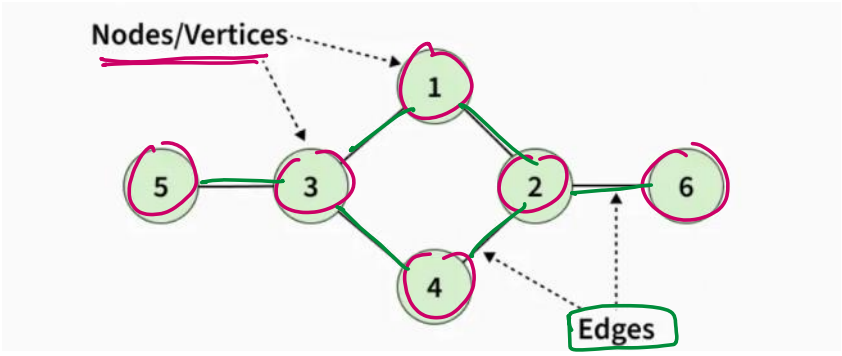
- A graph is a non-linear data structure used to model relationships between entities
- Unlike arrays, linked lists, or trees, graphs do not impose a strict hierarchy or order
- Mathematically, a graph is defined as → ***G = (V, E)***
  - **V (Vertices / Nodes)** → A finite set of points
  - **E (Edges)** → A set of connections between pairs of vertices
- ***Example:***
  - **V = {A, B, C, D}**
  - **E = {(A, B), (B, C), (C, D)}**

Vertices (Nodes):

- Vertices are the fundamental units of the graph
- Vertices are also known as vertex or nodes
- Vertices represent **entities** in a system
- Can store **values or labels**
- Can have **attributes** (weight, color, metadata)
- Total number of vertices = **|V|**

Edges (Connections):

- Edges represent **relationships between vertices**
- Edges are used to connect two nodes of the graph
- Edges can connect any two nodes in any possible way, there are no rules
- Every edge can be labelled/unlabelled
- Edges are also known as **arcs**



Applications:

- **Social Networks:** Represent users and their connections; used to find mutual friends, suggest new connections, and detect communities
- **Computer Networks:** Model routers and data links; used for efficient routing, fault detection, and network optimization
- **Transportation Networks:** Represent cities and routes; used to find shortest or fastest paths and plan optimal travel routes
- **Neural Networks:** Represent neurons and synapses; used to simulate learning, brain behavior, and data processing
- **Compilers:** Represent data dependencies and control flows; used for optimization, register allocation, and code analysis
- **Robot Path Planning:** Represent states and transitions; used to compute the safest or shortest route for autonomous movement
- **Project Dependencies:** Represent tasks and dependencies; used in topological sorting to determine the correct execution order

- **Network Optimization:** Represent network nodes and links; used to minimize cost, reduce latency, and improve efficiency

Advantages of Graphs:

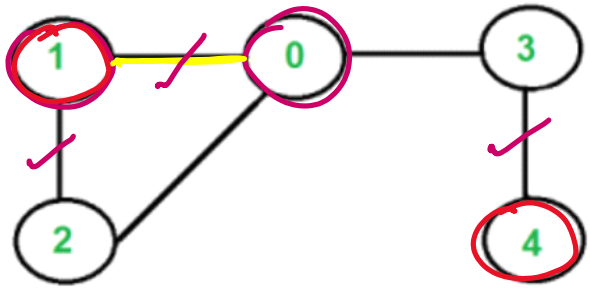
- **Flexibility:** Unlike arrays, linked lists, or trees, graphs have no restrictions and can represent any type of relationship
- **Model real-world problems:** Useful for pathfinding, data clustering, network analysis, and machine learning
- **Represent items and relationships:** Any set of items and their connections can be modeled as a graph
- **Simplifies complex data:** Graphs make complex relationships easy to visualize and understand

13.2 - Types of Graphs

Sunday, December 28, 2025 10:05 AM

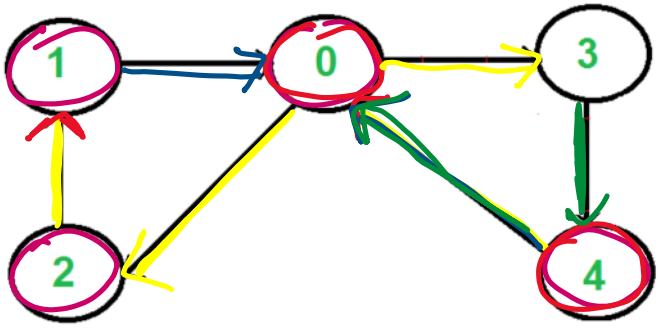
1. Undirected Graph:

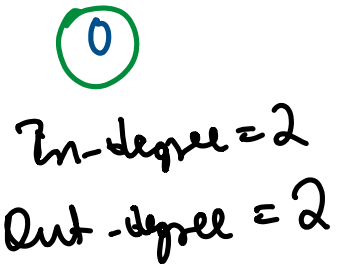
- A graph in which edges do not have any direction
- Nodes are unordered pairs in the definition of every edge
- Edges in an undirected graph are ***bidirectional*** in nature
- There is no concept of a "parent" or "child" vertex as there is no direction to the edges
- An undirected graph may contain loops, which are edges that connect a vertex to itself



2. Directed Graph:

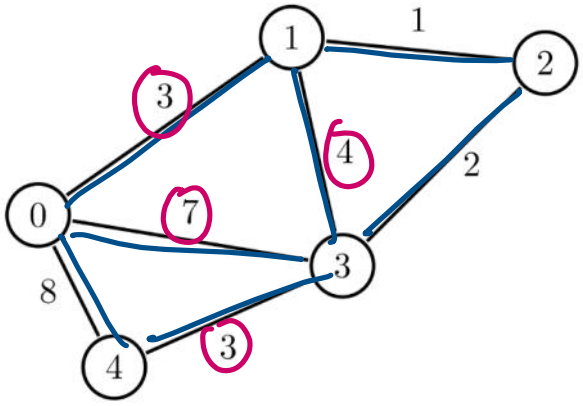
- A graph in which edges have direction
- One-way relationship between vertices
- A directed graph can contain cycles
- Paths in a directed graph follow the direction of the edges





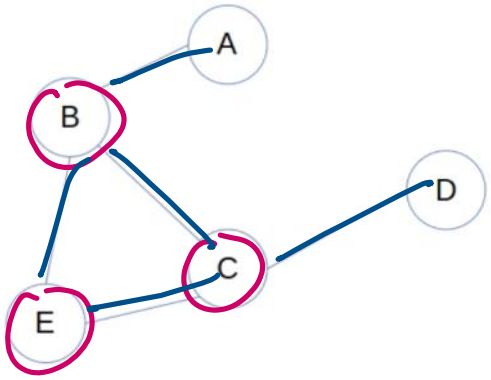
3. Weighted Graph:

- A weighted graph is one where the edges are assigned some weights
- Weights can represent cost, distance, or any other relative measuring unit



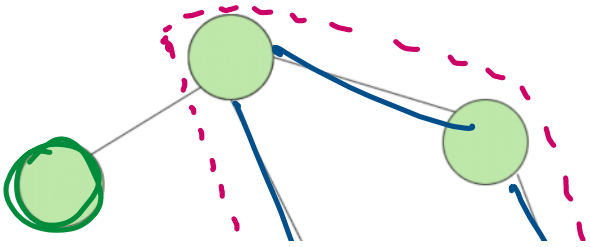
4. Unweighted Graph:

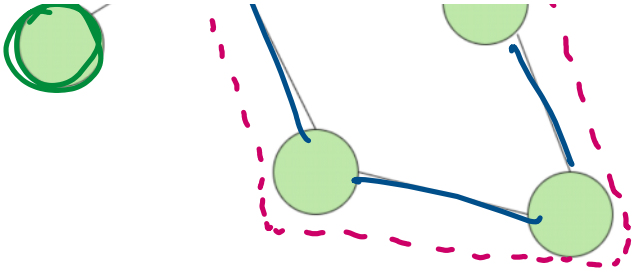
- Graph in which the edges do not have weights or costs associated with them
- Simply represent the presence of a connection between two vertices



5. Cyclic Graph:

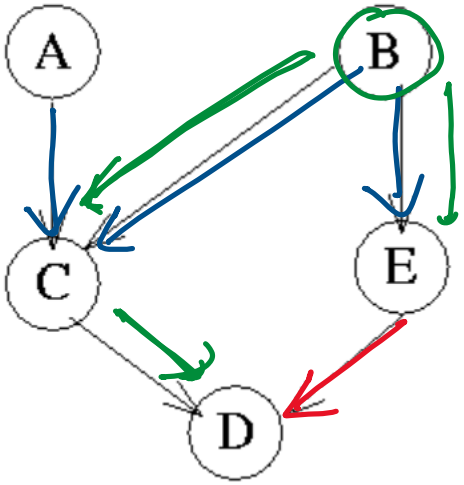
- A cyclic graph contains one or more cycles or closed paths
- A cyclic graph can be either directed or undirected
    - **Directed Cyclic Graph:** edges have a direction, and the cycle must follow the direction of the edges
    - **Undirected Cyclic Graph:** edges have no direction, and the cycle can go in any direction
- A cyclic graph may have multiple cycles of different lengths and shapes; some cycles may be contained within other cycles





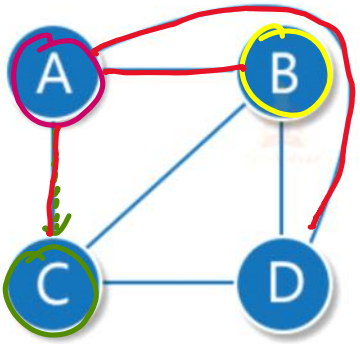
6. Acyclic Graph:

- An acyclic graph is a graph that contains no cycles or closed loops



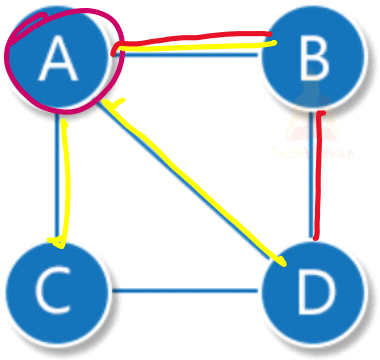
7. Complete Graph:

- A complete graph is a graph in which every node is adjacent to every other node
- A complete graph has **nC2 = n(n-1)/2** edges



8. Incomplete Graph:

- An incomplete graph is a graph in which every node is not adjacent to every other node



13.3 - Degree

Sunday, December 28, 2025 10:14 AM

- In a graph, degree is defined with respect to a vertex
- It is the **number of edges incident** (connected) to that vertex

Degree in an Undirected Graph:

- Number of edges connected to a vertex

Degree in a Directed Graph:

- In directed graphs, degree is split into two types:
  - **In-degree:** Number of edges ***coming into*** a vertex
  - **Out-degree:** Number of edges ***going out of*** a vertex

Degree Sequence:

- A degree sequence is a **list of degrees of all vertices** in a graph
- Usually written in descending order (largest to smallest)

13.4 - Connectivity

Sunday, December 28, 2025 10:39 AM

What is Graph Connectivity?

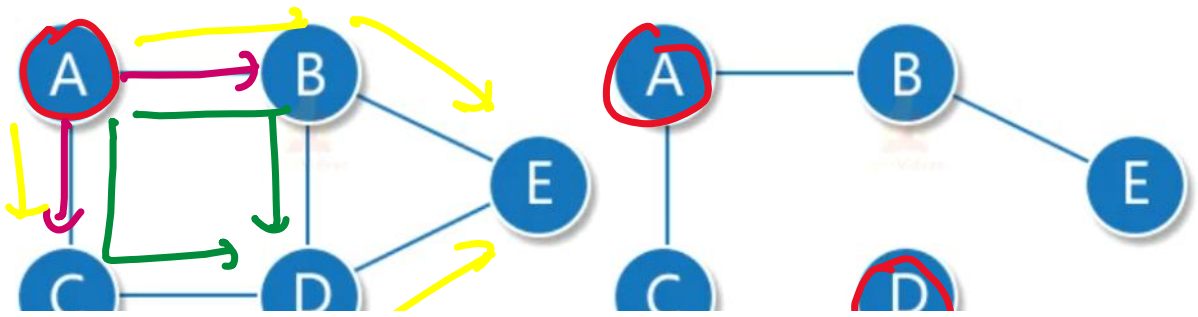
- Connectivity dictates **how vertices are linked together** in a graph
- There are two types of graphs based on this:
  - ***Connected Graph***
  - ***Disconnected Graph***

1. Connected Graph:

- A graph is connected, if every vertex is reachable from every other vertex
- There is at least one path between any two vertices in the graph
- The path between vertices can be direct or indirect

2. Disconnected Graph:

- A graph is disconnected if at least one pair of vertices cannot reach each other
- This occurs when the graph is split into separate parts with no edges between them



Connected graph

Disconnected graph

13.5 - Representation

Sunday, December 28, 2025 10:48 AM

What is Graph Representation?

- A graph representation is how we store a graph in computer memory
- Representation enables implementation of algorithms like BFS, DFS, Dijkstra, etc.

Why Do We Need Graph Representation?

- Different representations help us:
  - ***Save memory***
  - ***Access neighbors quickly***
  - ***Run graph algorithms efficiently***
- There is no single best representation—it depends on:
  - Number of vertices (V)
  - Number of edges (E)
  - Type of graph (dense or sparse)

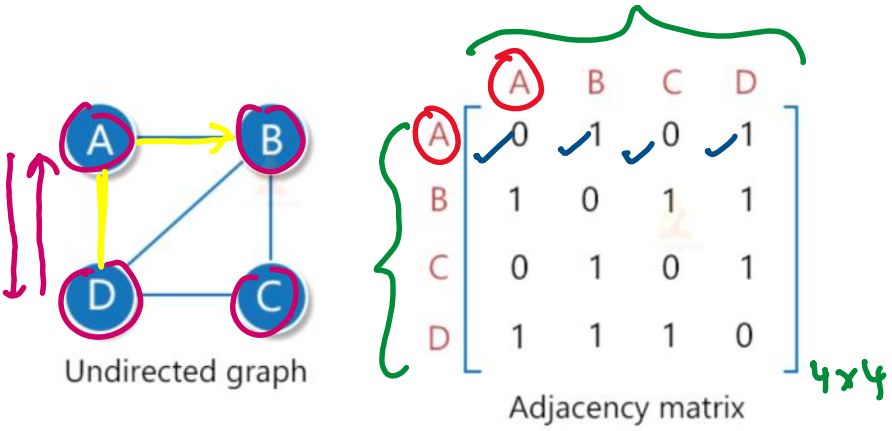
Most Common Graph Representations:

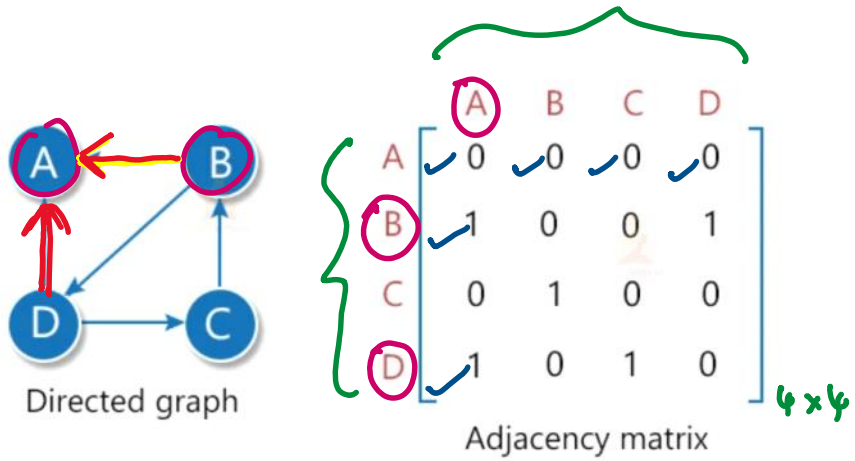
- ***Adjacency Matrix***
- ***Adjacency List***

13.5.1 - Adjacency Matrix

Sunday, December 28, 2025 10:52 AM

- An adjacency matrix is a way of representing a graph as a boolean matrix of (0's and 1's)
- The matrix represents the mapping between various edges and vertices
- The order of the matrix is given as **n*n** where n is the number of nodes in the graph
- In the matrix, each row and column represents a vertex and the values determine the presence of edges
- The representation varies for directed and undirected graphs, despite similar structure





Advantages of Adjacency Matrix:

- Performing operations on a matrix are easier as compared to performing them on the list or other data structure
  - Adding and removing edges
  - Checking if the edges are present in the graph
- Can perform heavy matrix operations easily on modern GPUs

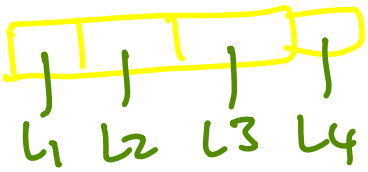
- Efficient even if the graph is dense and the number of edges present in the graph is large

Disadvantages of Adjacency Matrix:

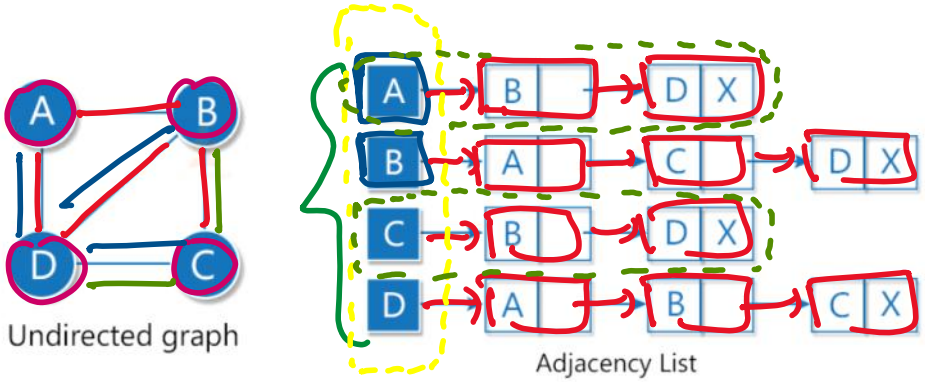
- In the adjacency matrix method, certain operations are computationally heavy and time consuming to perform
- Matrix representation requires a large amount of additional memory even if the number of edges in a graph is less

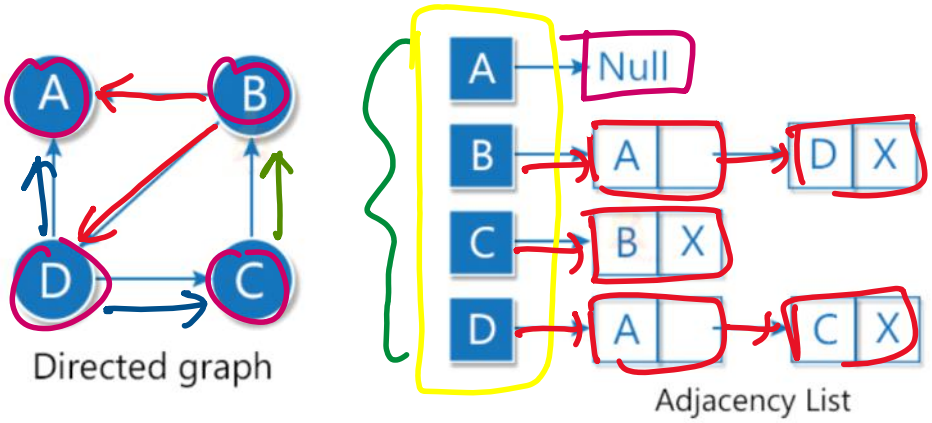
13.5.2 - Adjacency List

Sunday, December 28, 2025 10:52 AM



- The adjacency list is an array of linked lists
- The array denotes the total vertices/nodes in the graph
- Each linked list denotes the vertices connected to a particular node





13.6 - Traversal

Tuesday, December 30, 2025 12:57 AM

What is Graph Traversal?

Graph Traversal is the process of visiting all vertices (nodes) of a graph systematically

Applications of Graph Traversal:

- ***Searching paths***
- ***Detecting cycles***
- ***Finding connected components***
- ***Shortest path algorithms***
- ***Topological sorting***
- ***Real-world problems (maps, networks, recommendations)***

Fundamental Graph Traversal Techniques:

1. **Depth First Search (DFS)**
2. **Breadth First Search (BFS)**

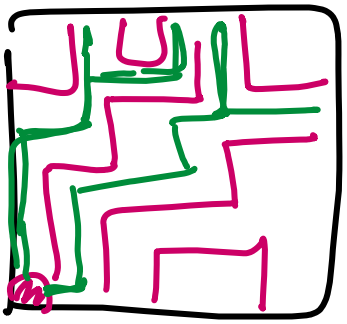
TECHNIQUE	DATA STRUCTURE	NATURE	USAGE
DFS	Stack / Recursion	Deep first	Cycle detection, Topological sort
BFS	Queue	Level by level	Shortest path (unweighted graphs)

13.6.1 - DFS

Tuesday, December 30, 2025 1:01 AM

What is DFS?

- Graph traversal algorithm that explores as deep as possible along one path before backtracking
- Starts from a source vertex, visits a neighbor, then a neighbor's neighbor, and so on, until no unvisited neighbors remain; then it backtracks
- DFS works best with ***Adjacency List***



Intuition:

- Exploring a maze
- Going deep into one corridor before trying others

Algorithm:

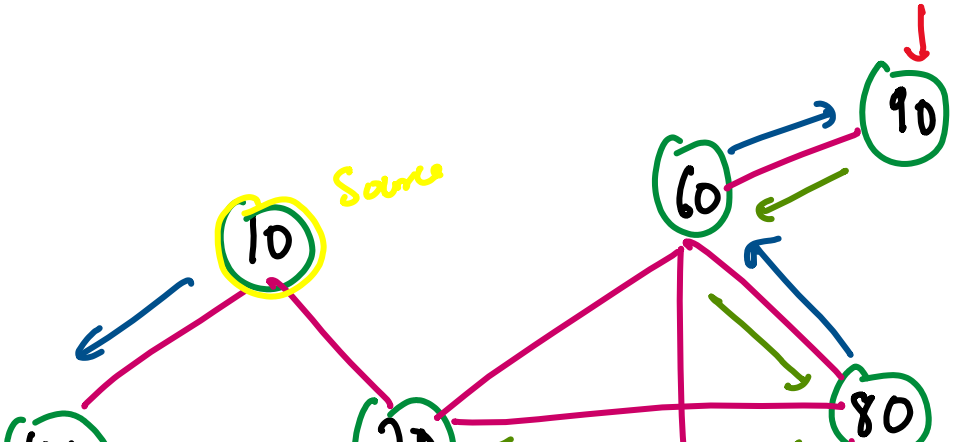
1. Create a visited set/array
2. Start DFS from a source node
3. Mark the current node as visited
4. For each adjacent node:
    - If not visited → perform DFS on it

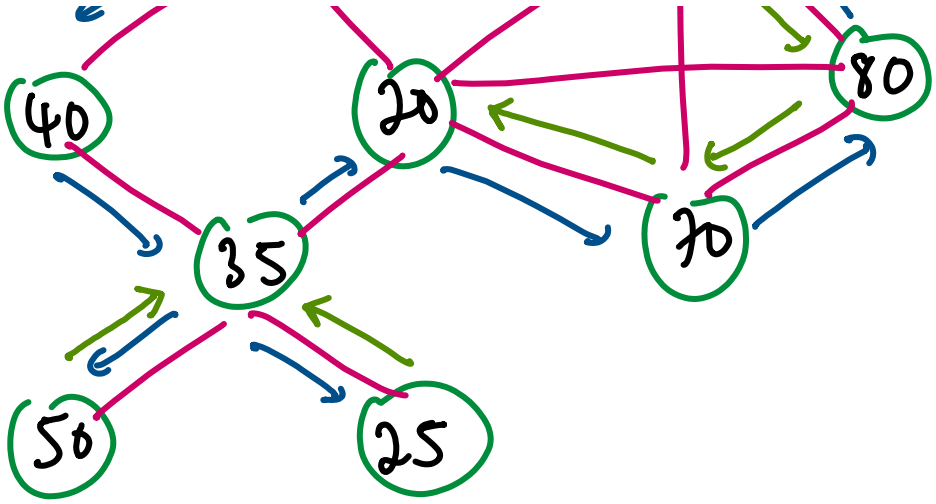
Time and Space Complexity:

- **Time:** **O(V + E)** ✓
- **Space:** **O(V)** ✓

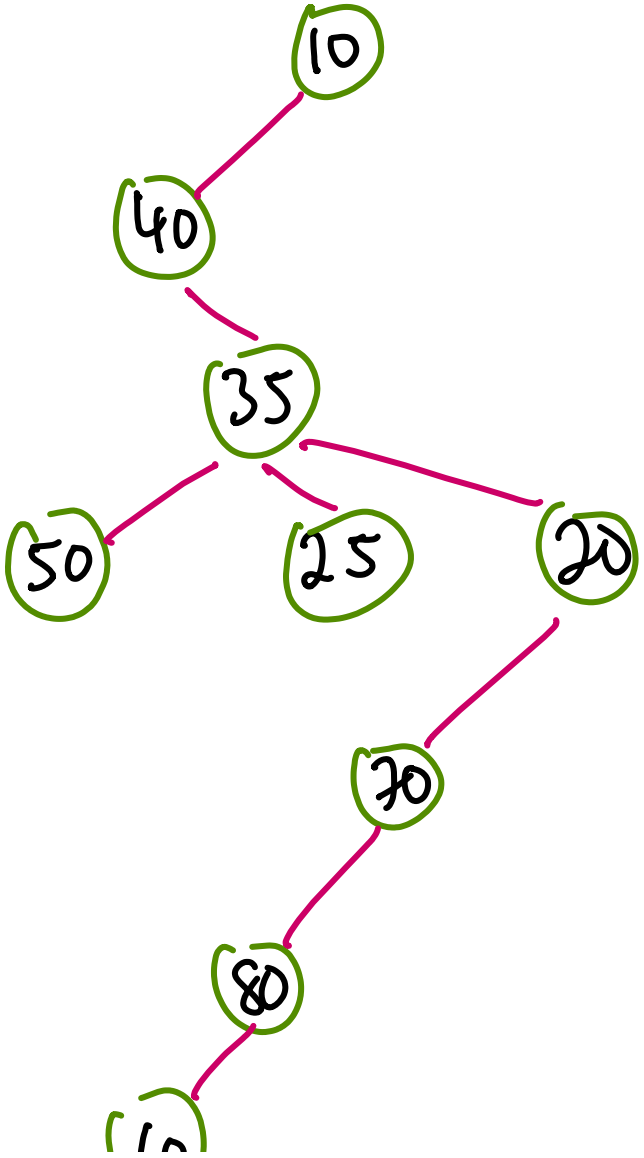
V → vertex

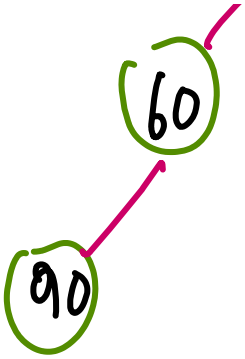
E → edge





Path: 10, 40, 35, 50, 25, 20, 70, 80, 60, 90




60
90

13.6.2 - BFS

Tuesday, December 30, 2025 1:01 AM

What is BFS?

- Graph traversal algorithm that explores all neighbors of a node first, then their neighbors
- Traversal happens level by level starting from a source node
- Involves a Queue (FIFO), which ensures level-order traversal
- Ideal for shortest path problems in unweighted graphs

Algorithm:

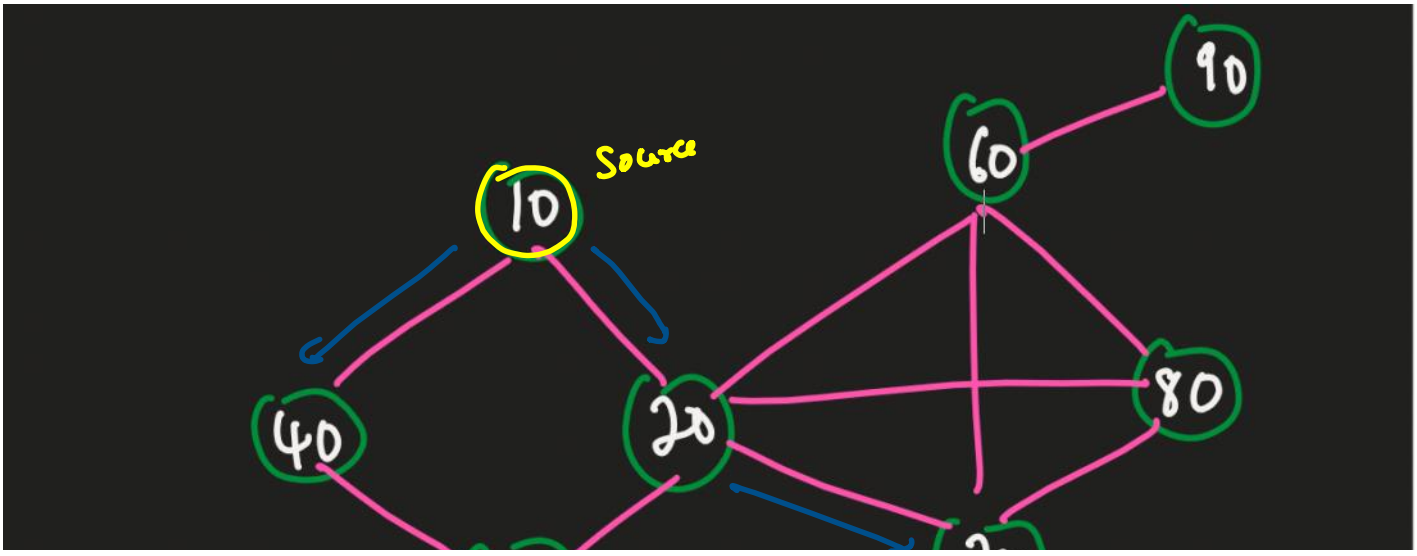
1. Start from a source node
2. Mark it as visited
3. Push it into a queue
4. While the queue is not empty:
    - Dequeue a node
    - Visit all its unvisited neighbors
    - Mark them visited and enqueue them

} Initial step

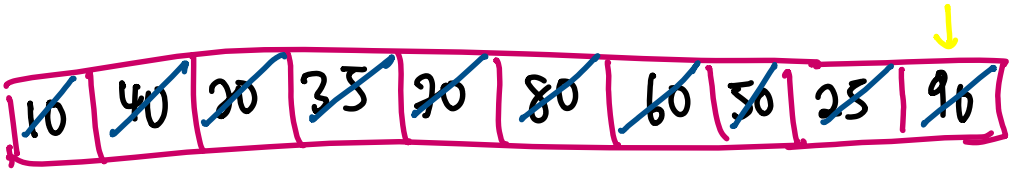
} Iterative steps

Time and Space Complexity:

- **Time:** **O(V + E)**
- **Space:** **O(V)**



Queue:



front

rear

Path: 10, 40, 20, 35, 70, 80, 60, 50, 25, 90

Curr: 10 40 20 35 70 80 60 50 25 90

13.6.3 - Multi-source BFS

Wednesday, December 31, 2025 12:16 AM

What is Multi-Source BFS?

In normal BFS, we start from one source node whereas in Multi-Source BFS, we start BFS from multiple sources at the same time

When to use Multi-Source BFS?

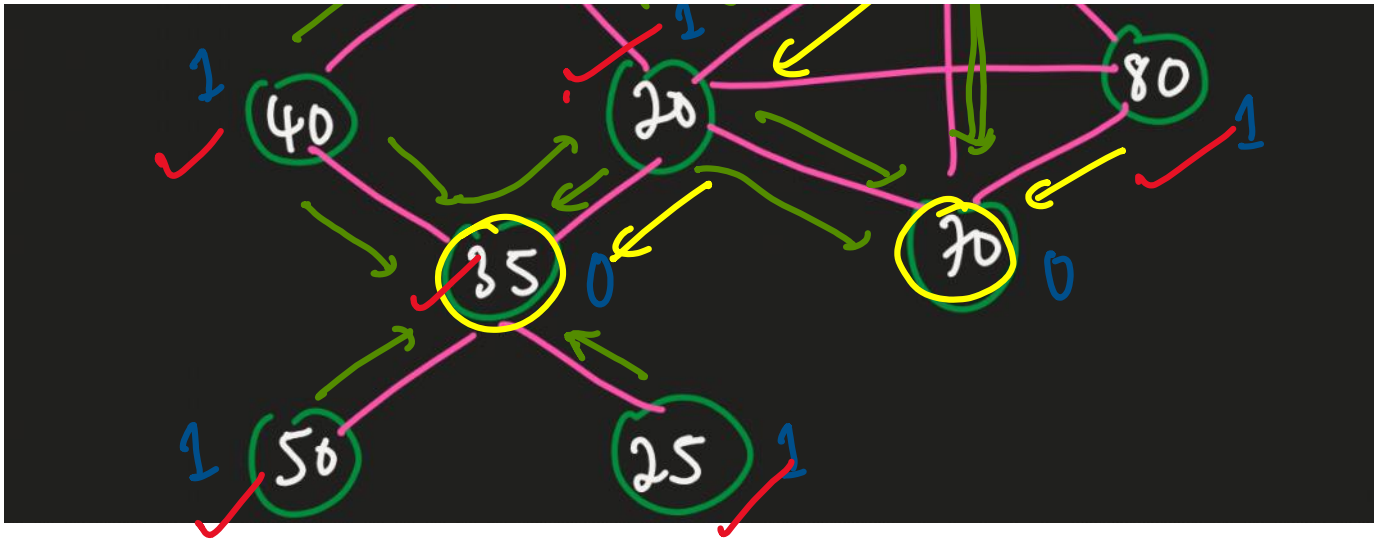
- There are multiple starting points
- We want the minimum distance to the nearest source

Algorithm:

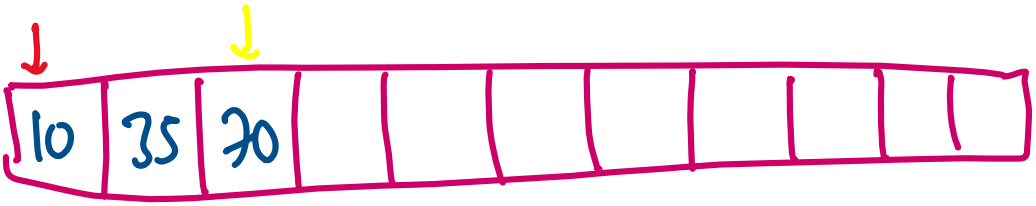
1. Identify all source nodes
2. Add all sources to queue
3. Mark all sources visited
4. Run normal BFS

Time and Space Complexity:

- **Time:** **O(R x C)**
- **Space:** **O(R x C)**



Queue :



rear

front

{ n1 :
n2 :
n3 :
⋮
}

Min. distance b/w node & its nearest source

$a \xleftarrow{+1} n1$

13.6.4 - 0-1 BFS

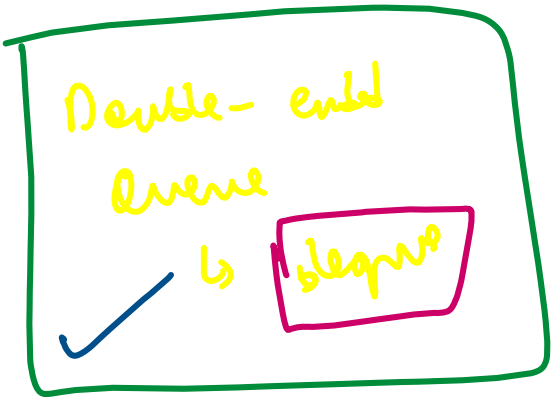
Wednesday, December 31, 2025 12:16 AM

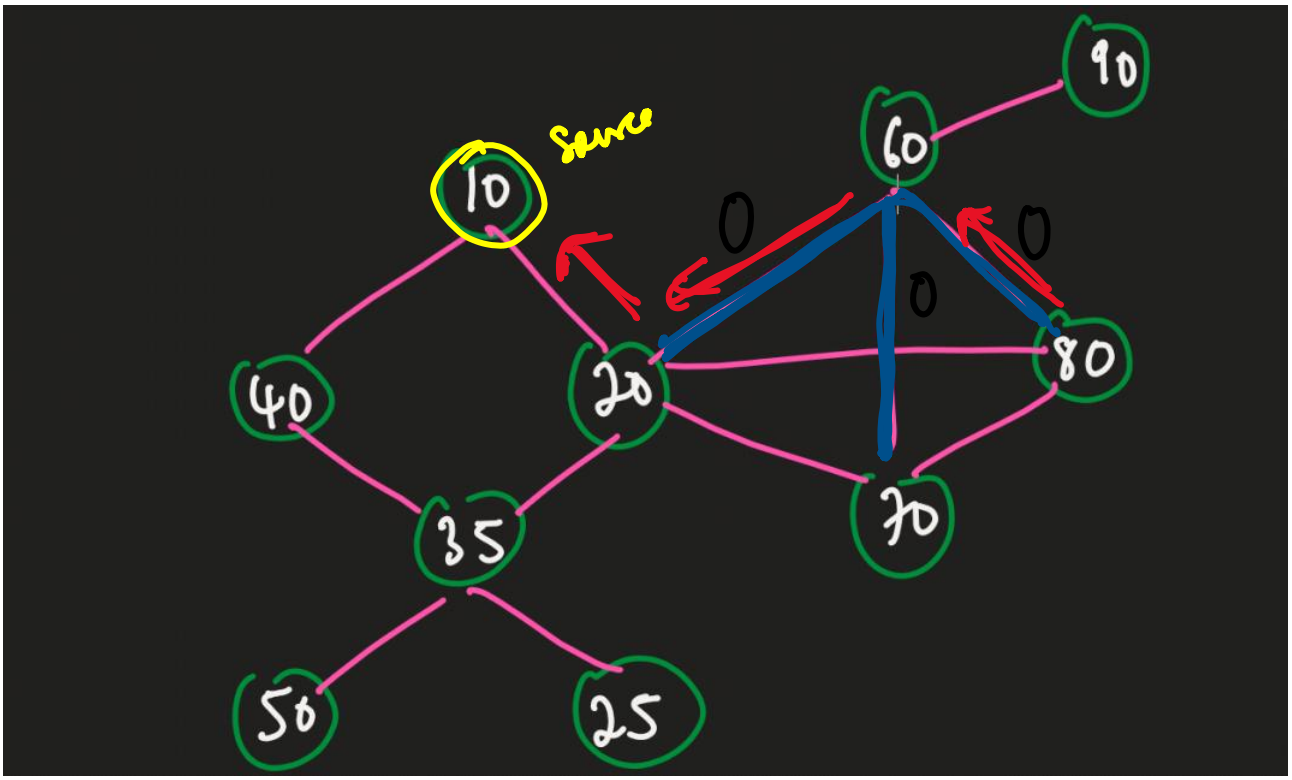
What is 0-1 BFS?

- Traversal algorithm which is used when the edge weights are only 0 or 1
- Most widely used to determine minimum cost / minimum distance / minimum operations

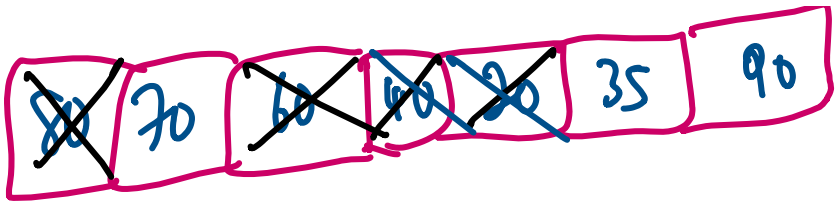
Intuition:

- Use a deque
- For each edge:
  - **Weight 0** → push to front
  - **Weight 1** → push to back
- This keeps nodes with lower cost processed first





Queue



distance {

10 : 0,

40 : inf 1,

20 : inf 1,

35 : inf 2,

60 : ~~inf~~ 1,

70 : ~~inf~~ 1,

80 : 1

90 : ~~inf~~ 2

curr ⇒ 10

1 1

40 20

[0 + 1 < inf] → [1 < inf]

[0 + 1 < inf
1 < inf]

10 → [1 + 1 < 0
2 < 0]

40 → 35 → [1 + 1 < 2
2 < 2]

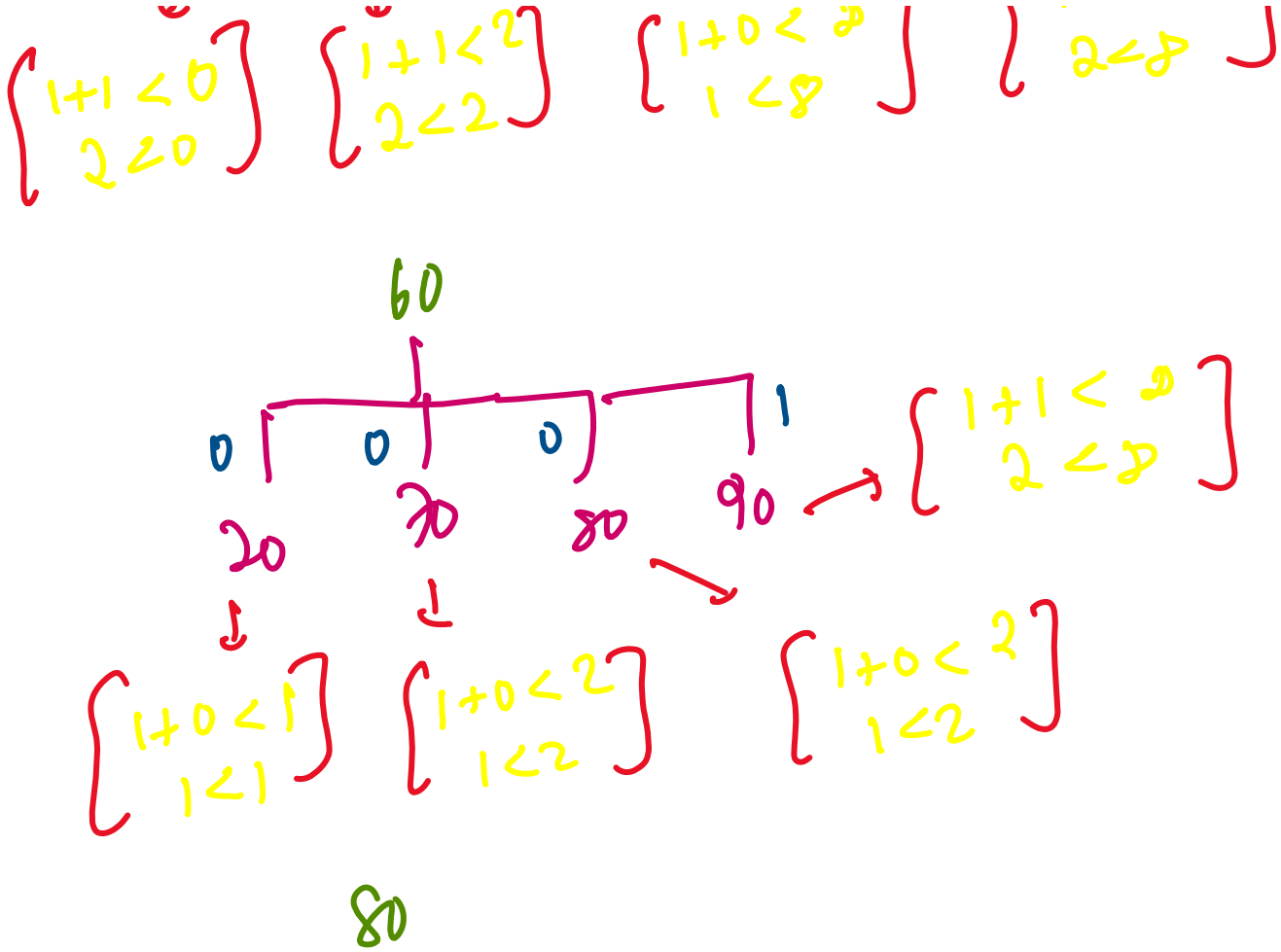
20

1 1 0 1 1

10 35 60 20 80

[1 + 1 < 0] [1 + 1 < 2] [1 + 0 < 2] [1 + 1 < 2
2 < 2]

[1 + 1 < 2
2 < 2]


60
0
0
0
1
20
30
80
90
80

13.6.5 - Choice

Friday, January 2, 2026 1:08 AM

SCENARIO	ALGORITHM
Shortest path	BFS
Exhaust all paths	DFS
Cycle detection	DFS
Grid minimum steps	BFS
Backtracking	DFS

13.7 - Shortest Path Algorithms

Wednesday, December 31, 2025 12:16 AM

What is a Shortest Path?

The shortest path problem is about finding the **minimum cost path** between nodes in a graph

- Cost can be:
  - ***Distance***
  - ***Time***
  - ***Money***
  - ***Energy***
- Graph can be:
  - ***Directed / Undirected***
  - ***Weighted***

Most Popular Algorithms:

- **Dijkstra's Algorithm**
- **Bellman–Ford Algorithm**
- **Floyd–Warshall Algorithm**

13.7.1 - Dijkstra's

Thursday, January 1, 2026 11:59 PM

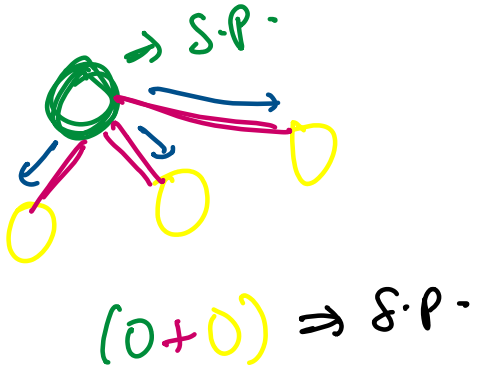
- Dijkstra's Algorithm is a shortest path algorithm used in graphs to find the minimum distance from a single source vertex to all other vertices
- It is one of the most important algorithms in DSA and graphs, especially for **weighted graphs**
- Does not work with negative weights
- Very fast for large and sparse graphs
- Helps compute the shortest path distance from the source to every other node, given:
  - ***Graph***
  - ***Source node***

Key Conditions:

- Graph can be ***directed or undirected***
- Graph must have ***non-negative edge weights (>= 0)***

Intuition: *Always expand the nearest unvisited node first*

- Start from the source
- Update distances of neighbors
- Always pick the next node with the smallest known distance
  - This is why we use a **min-heap (priority queue)**



Data Structures Used:

- **Adjacency List** → graph representation
- **Min Heap** → always get the closest node

Algorithm:

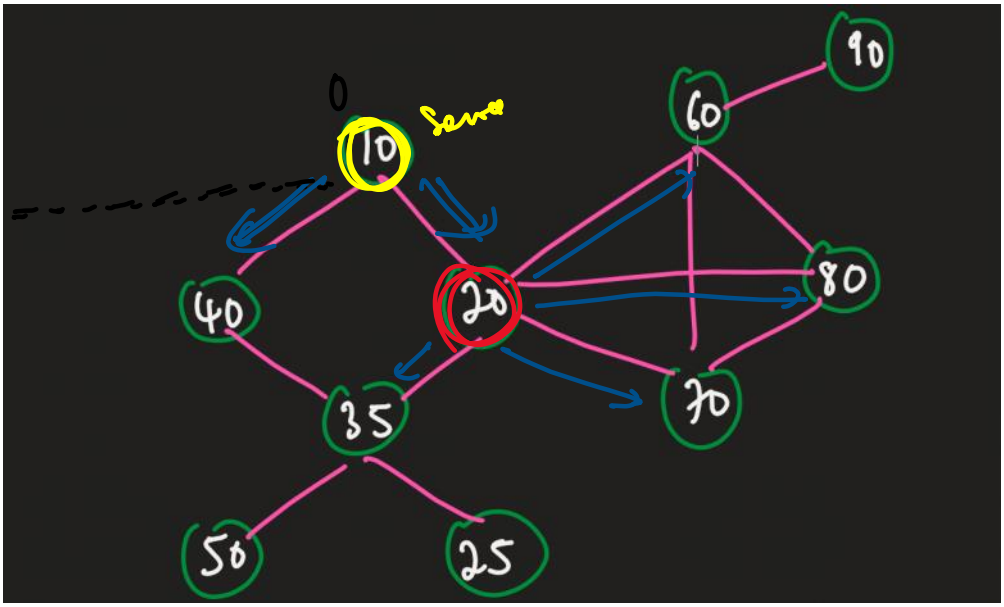
1. Set distance of all nodes = ∞
2. Set source distance = 0
3. Push (0, source) into min-heap
4. Pop smallest distance node
5. Relax its neighbors
6. Repeat until heap is empty

Initial

Repetitive

Complexity Analysis:

- **Time:** **O((V + E) log V)**
- **Space:** **O(V)**


0
10
Source
90
60
80
40
20
70
35
50
25

13.7.2 - Bellman-Ford

Thursday, January 1, 2026 11:59 PM

- Bellman–Ford Algorithm is a single-source shortest path algorithm used in graphs that can handle negative edge weights
- It is especially important because it can also detect negative weight cycles, which Dijkstra's algorithm cannot do
- Generally, its slower than Dijkstra's algorithm
- The shortest distance from the source to all other vertices, even when negative edges are present, given:
  - ***Graph***
  - ***Source node***

Key Features:

- Works with negative edge weights
- Works for directed and undirected graphs
- Can detect negative cycles

Intuition: Relax all edges repeatedly, V—1 times

- The shortest path between two nodes can have ***at most V—1 edges*** (where V = number of vertices)
- Each relaxation improves distances gradually
- If we relax all edges **V—1 times**, ***shortest paths are guaranteed***

Data Structures Used:

- **Adjacency List** → graph representation
- **Distance Array** → store shortest paths

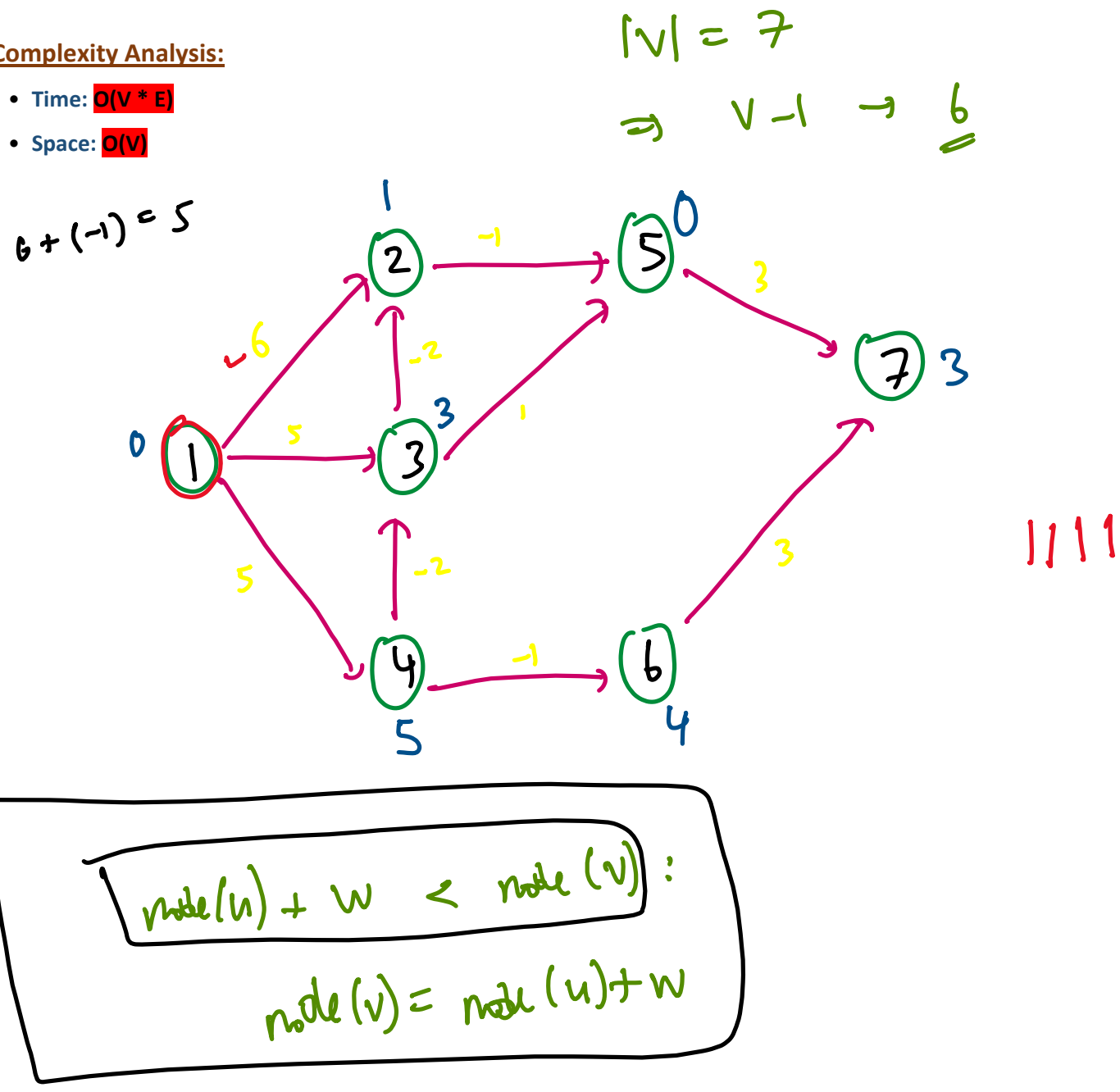
Algorithm:

- **Initialize:**
  - *Distance of source = 0*
  - *Distance of all other vertices = ∞*
- **Repeat V—1 times:**
  - *For every edge (u → v, weight w)*
- **Negative cycle check:**
  - *Do one more relaxation*

- *If any distance still reduces → **negative cycle exists***

Complexity Analysis:

- **Time:** O(V * E)
- **Space:** O(V)



13.7.3 - Floyd-Warshall

Thursday, January 1, 2026 11:59 PM

- Floyd–Warshall Algorithm is an all-pairs shortest path algorithm used in graphs to find the shortest distance between every pair of vertices
- It is especially useful for dense graphs and graphs with negative edge weights (but no negative cycles)
- Works best when the graph is **small or dense**
- Works even if negative edges exist

Key Features:

- Computes all-pairs shortest paths
- Works with negative edge weights

Intuition: Try every vertex as an intermediate point between every pair of vertices

- Try every vertex as an intermediate point between every pair of vertices

Data Structures Used:

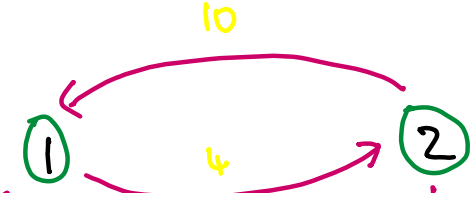
- **Adjacency List** → graph representation
- **Distance Array** → store shortest paths

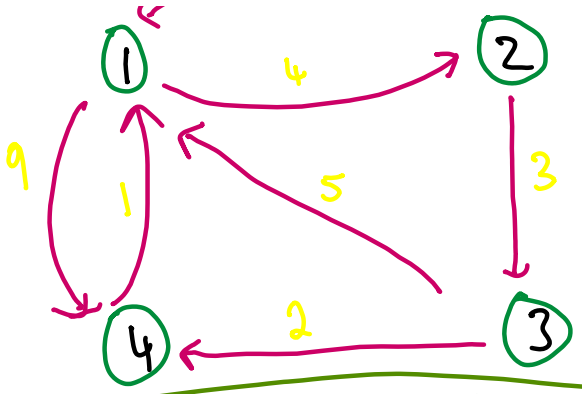
Algorithm:

- **Create a Distance Matrix:**
  - dist[i][j] = weight(i → j)
  - 0 if i == j
  - ∞ if no edge exists
- **For each vertex k, i, j:**
  - **dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])**
- **Final matrix contains shortest distances between all pairs**

Complexity Analysis:

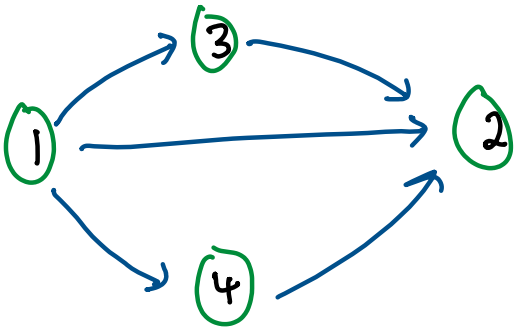
- **Time: O(V * V * V)**
- **Space: O(V * V)**





A^0 =

	1	2	3	4
1	0	4	∞	9
2	10	0	3	∞
3	5	∞	0	2
4	1	∞	∞	0



i) Consider (1) as intermediate vertex

A^1 =

	1	2	3	4
1	0	4	∞	9
2	10	0	3	19
3	5	9	0	2
4	1	5	∞	0

$\min(\infty, 10+9)$
$\min(\infty, 19)$

$\min(\infty, 5+4) \rightarrow \min(\infty, 9)$

$\min(3, 5+9) \rightarrow (3, 14)$

$\min(\infty, 1+4) \rightarrow \min(\infty, 5)$

$\min(\infty, 1+\infty)$

$A^1[2,3] = \min($
$A^0[2,3],$
$A^0[2,1] + A^0[1,3]$
$)$

$\min(2, 1+2)$

$A^k[i,j] = \min(A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j])$

$\min(3, 10+2)$
↳ $(3, 2)$

$A^2 =$

	1	2	3	4
1	0	4	7	9
2	10	0	3	19
3	5	9	0	2
4	1	5	8	0

$\min(2, 4+3) \to (2,7)$
$\min(9, 4+19) \to (9,23)$
$\min(5, 10+9) \to (5,19)$
$\min(2, 9+19) \to (2,28)$
$\min(1,5)$
$\min(2, 5+3) \to \min(2,8)$

$A^3 =$

	1	2	3	4
1	0	4	7	9
2	8	0	3	5
3	5	9	0	2
4	1	5	8	0

$\min(4, 7+9) \to \min(4,16)$
$\min(9,9)$
$\min(10, 8) \to 8$
$\min(19, 5)$ $\min(5,17)$

$$4\begin{bmatrix} 1 & 5 & 8 & 0 \end{bmatrix}$$

mín(19, 5)

mín(5, 17)

mín(1, 13)

$$A4 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 4 & 7 & 9 \\ 6 & 0 & 3 & 5 \\ 3 & 7 & 0 & 2 \\ 1 & 5 & 8 & 0 \end{bmatrix} \end{array}$$

mín(4, 14)

mín(7, 17)

mín(8, 6)

mín(3, 13)

mín(5, 3)

mín(9, 7)

13.8 - Advanced

Saturday, January 3, 2026 12:25 AM

- **Disjoint Set Union (Union–Find)**
- **Kruskal’s Minimum Spanning Tree (MST)**
- **Prim's MST**

13.8.1 - DSU

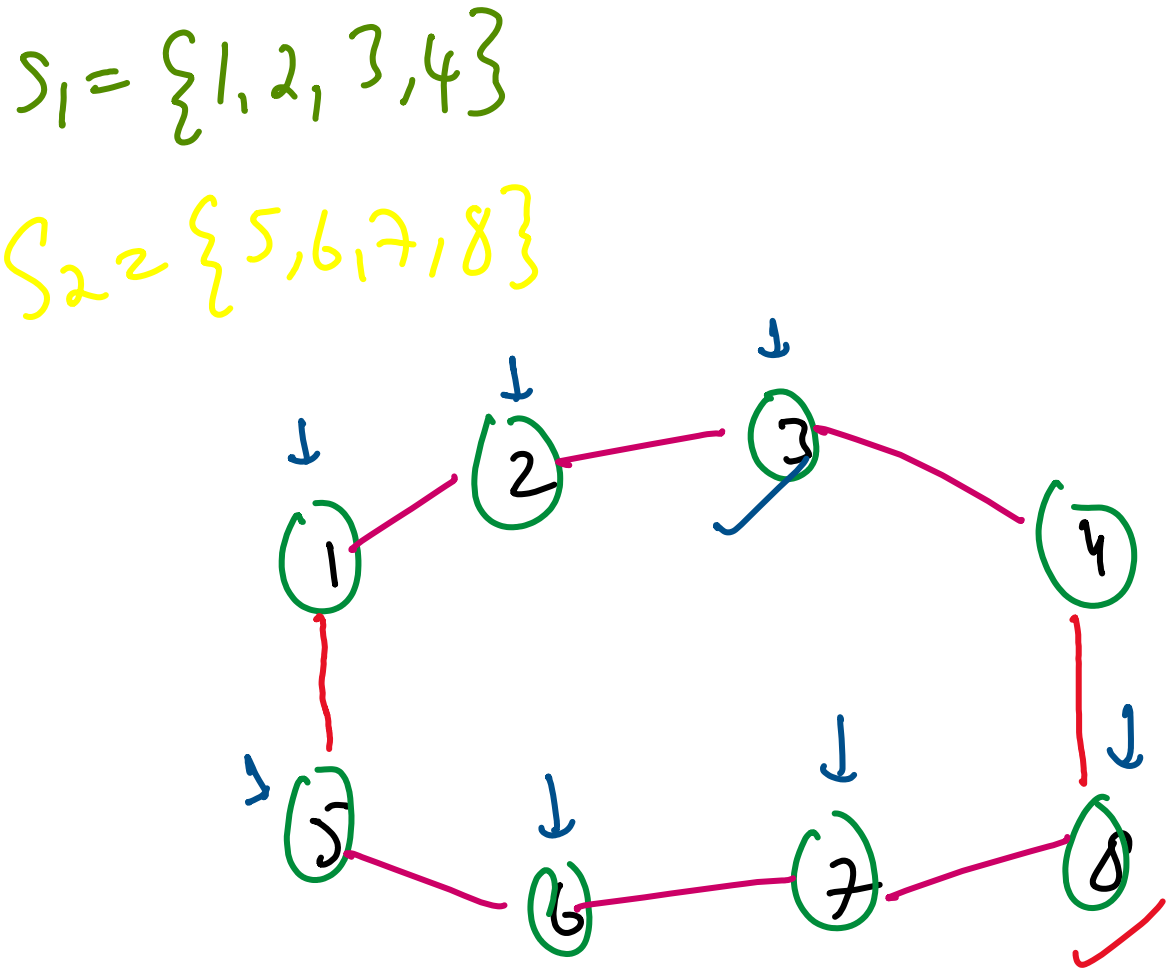
Saturday, January 3, 2026 12:26 AM

What is DSU?

- DSU is a data structure that keeps track of **connected components** dynamically
- **Used when:**
  - *Graph edges are added gradually*
  - *Fast cycle detection*
  - *Check connectivity*

Key Operations:

- **Union**
- **Find**



Find:

* The element will belong only single set
* Search set by set

$$S1 \cap S2 = \phi$$

$$S3 = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

(1, 5)

→ S3

↳ S3

∴ There's a cycle in graph

{0, 1, 2} {3, 4}

$\{0, 1, 2\}$ $\{3, 4\}$

S_1 S_2

13.8.2 - Prim's MST

Saturday, January 3, 2026 12:35 AM

What is Prim's MST:

- Prim's Algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph
- Suitable to use when:
  - ***Graph is connected***
  - ***Graph is undirected***
  - ***Graph has weights***
  - ***Best for dense graphs***

What is MST?

A spanning tree that:

- ***Connects all vertices***
- ***Has no cycles***
- ***Has minimum total edge weight***

What is a Spanning Tree:

A spanning tree is a subgraph of a connected graph that:

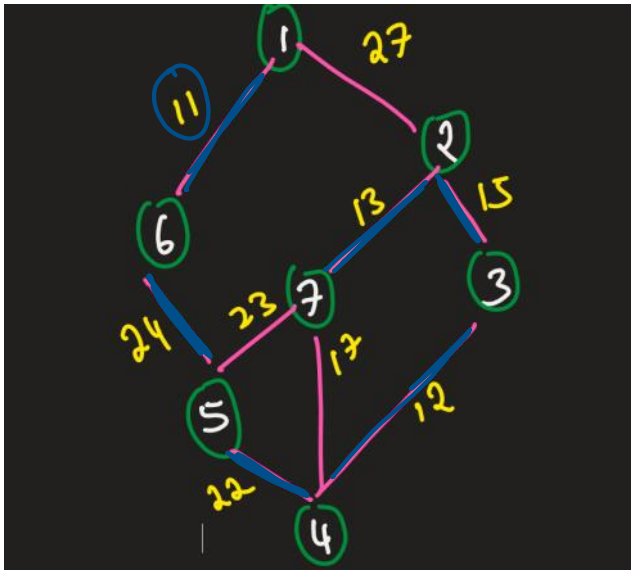
- ***Includes all vertices***
- ***Has no cycles***
- ***Uses minimum number of edges***

Intuition:

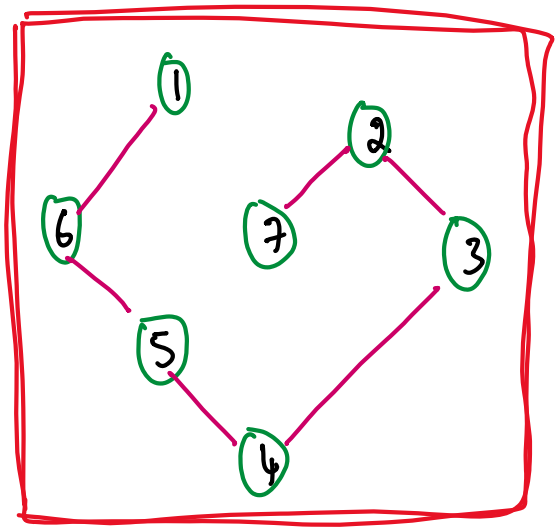
- Grow the MST one vertex at a time, always choosing the cheapest edge that connects:
  - ***a vertex inside the MST***
  - ***to a vertex outside the MST***
- **Analogy:** Expanding a network step-by-step, always picking the **lowest-cost connection available**

Algorithm:

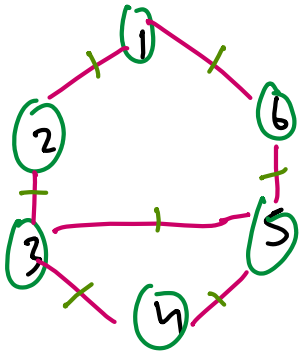
- Pick any start vertex
- Mark it as visited (part of MST)
- Push all its edges into a **min-heap (priority queue)**
- **Loop:**
  - Extract the **minimum weight edge**
  - If the destination vertex is not visited:
    - Add edge to MST
    - Mark vertex as visited
    - Push its edges into heap
    - Stop when MST has **V — 1 edges**



27, 24
22, 23,
12, 17, 15
13

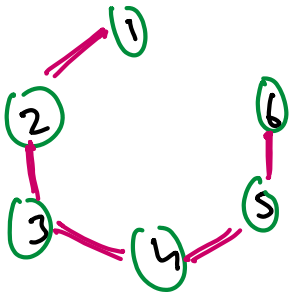


Total Cost = 97



$|V| = 6$

$|E| = 7$

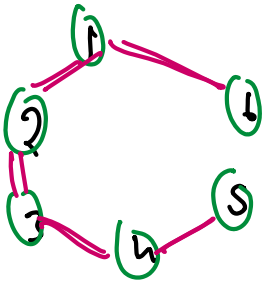


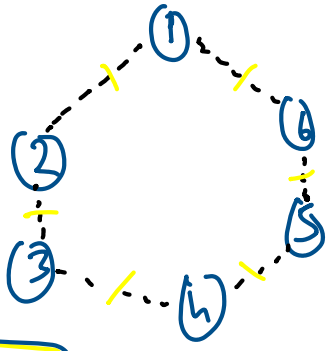
$|V'| = |V| = 6$

$|E'| \neq |E|$

$|E'| < |E|$

$|E'| = 5$





$^6C_5$

13.8.3 - Kruskal's MST

Saturday, January 3, 2026 12:31 AM

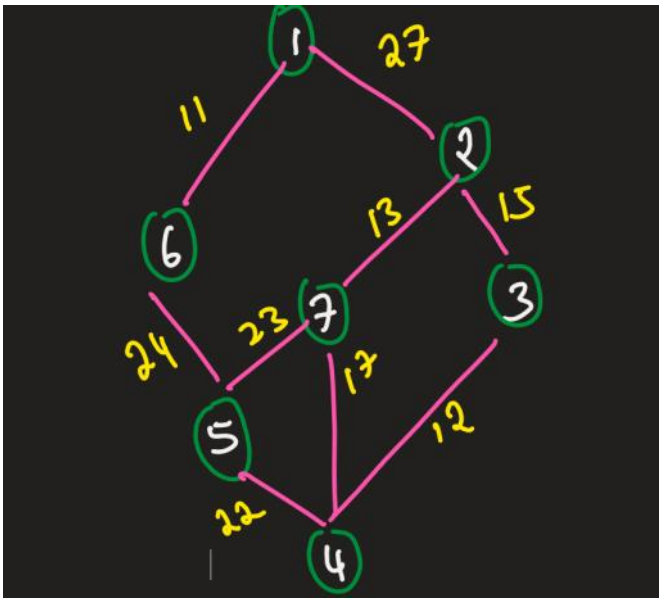
Algorithm:

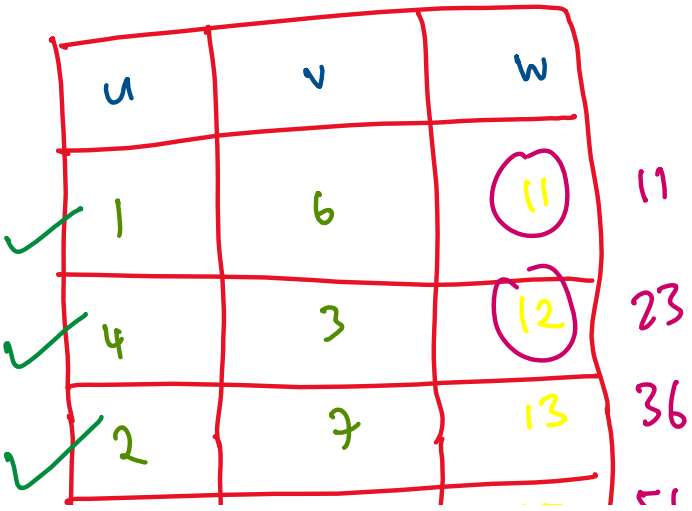
1. Sort edges by weight
2. Pick smallest edge that does not form a cycle
3. Stop after V - 1 edges

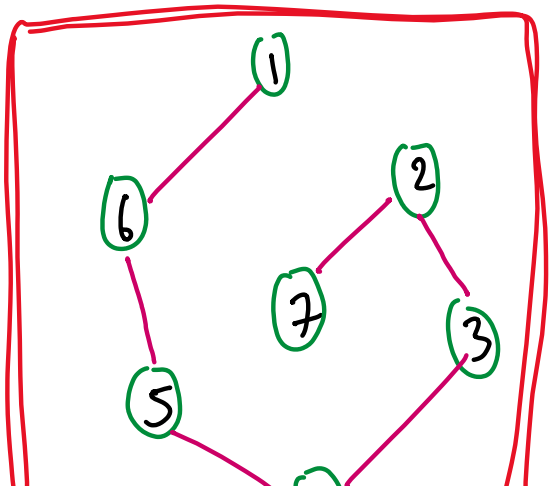
Note: ***Cycle detection is done using DSU***

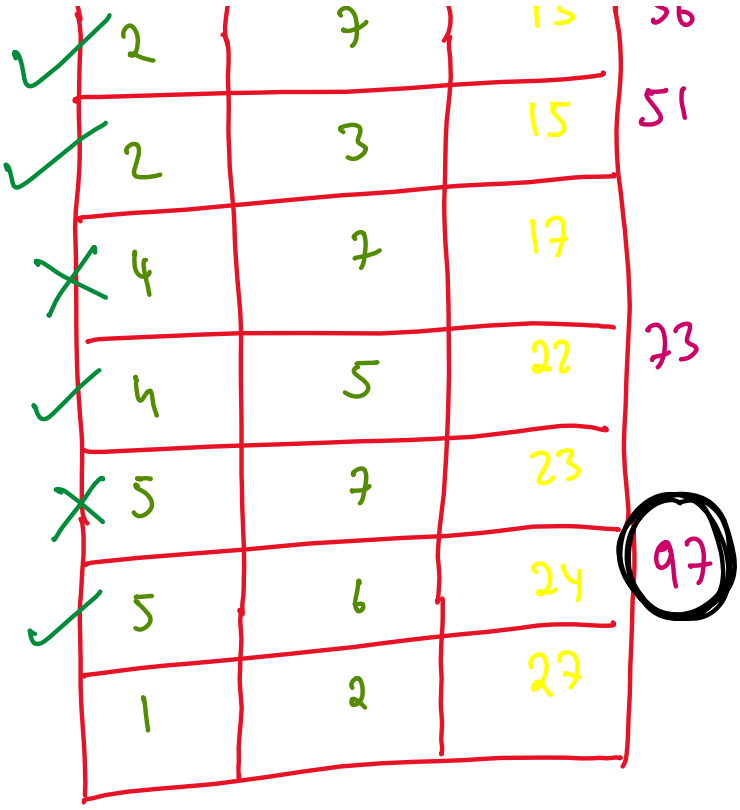
When to Use Kruskal?

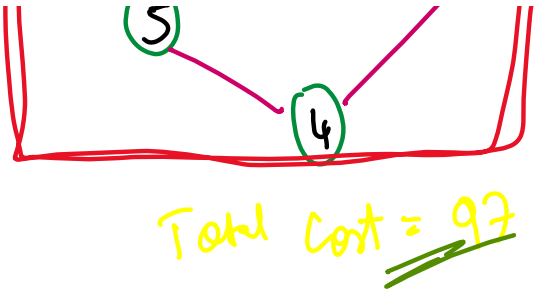
- ***Sparse graphs***
- ***Edge list available***
- ***Offline processing***









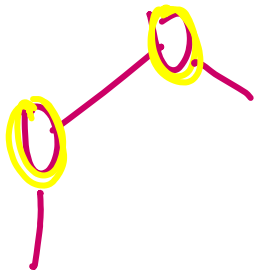


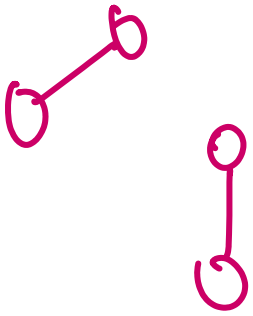
Total Cost = 97

13.8.4 - Prim vs Kruskal

Saturday, January 3, 2026 12:40 AM

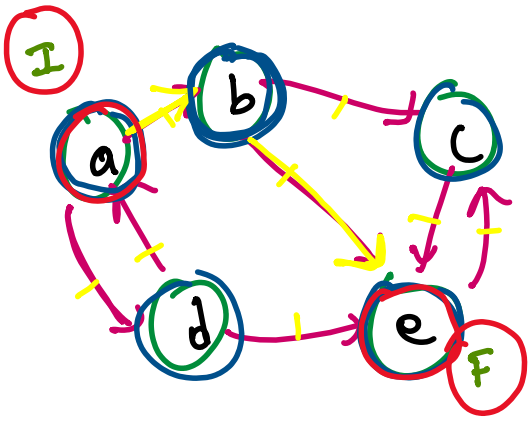
FEATURE	PRIM	KRUSKAL
Approach	Vertex-based	Edge-based
Data Structure	Min Heap	DSU (Union-Find)
Best for	Dense graphs	Sparse graphs
Graph type	Connected	Can handle disconnected (forest)





13.9 - State Space

Saturday, January 3, 2026 12:58 AM



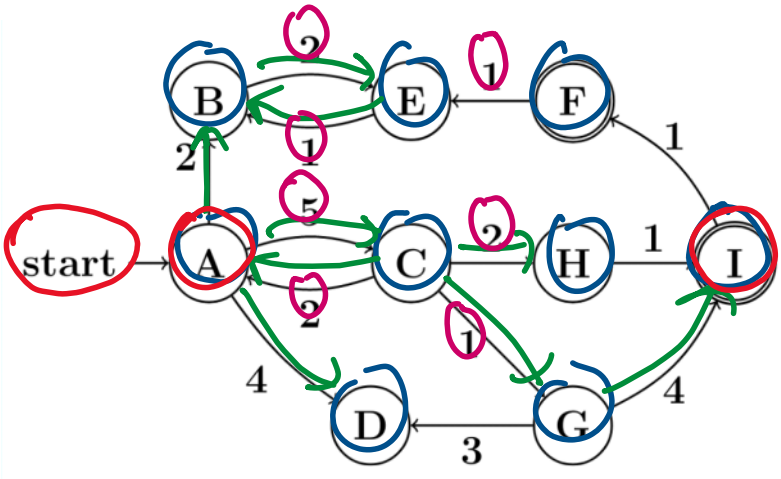
What is a State Space Graph?

A State Space Graph is a graph where:

- Each node represents a state
- Each edge represents a valid transition (action)
- We search the graph to reach a goal state from an initial state

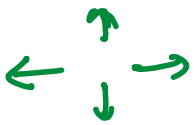
A State Space Graph is defined as:

- G = (S, A)
- S = set of all possible states
- A = set of actions that move from one state to another



Components of a State Space Graph:

1. **Initial State:**
   - The position where the problem starts
2. **State:**
   - A state captures all information needed to continue the problem
3. **Actions / Transitions:**
   - Valid moves from one state to another
   - Move Up, Down, Left, Right, etc.



4. **Goal State:**

- The state that solves the problem

5. **Constraints:**
   - Rules that decide whether a state is **valid or invalid**

How State Space Graphs Are Explored:

- **BFS:** *Shortest path / minimum steps*
- **DFS:** *Exploration*
- **Backtracking:** *Constraint satisfaction*
- **Dijkstra / A*:** *Weighted state transitions*

When to Use State Space Graph Modeling?

- The problem has choices at each step
- Need to explore all possible configurations
- Constraints decide validity
- BFS/DFS/Backtracking fits naturally

Complexity Analysis:

- **Time:** Depends on the number of states
- **Space:** Depends on visited states / recursion
- **Worst Case:** Exponential

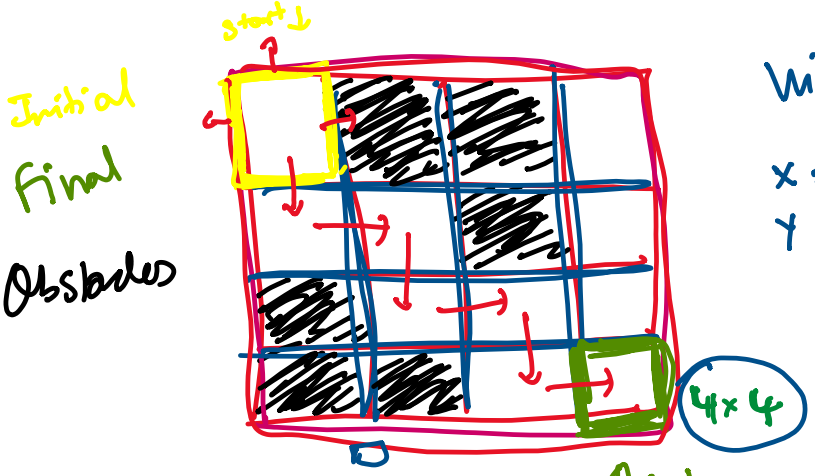
Real-World Applications:

- **AI Search Problems**
- **Game Solvers (Chess, Sudoku)**
- **Robot Navigation**
- **Path Planning**

- **Compiler optimization**

13.9.1 - Rat in a Maze

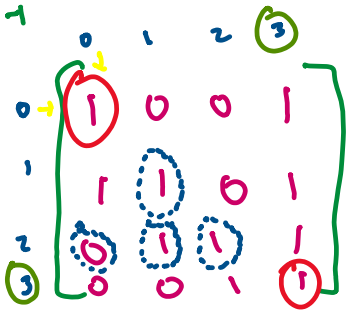
Saturday, January 3, 2026 10:06 AM



Visited = {(0,0), (1,0), (1,1),
x = ~~0 1 2~~ 3 (2,1), (2,2), (3,2),
y = ~~0 1 2~~ 3 (3,3) }

x == n-1
y == n-1

(2,1)



(0,0)
(3,3)

n → rows
n → columns

States: Cells having 1

Constraints: Position must be within (0,0) & (n-1, n-1)

Actions: up, down, right, left
(-1,0) (1,0) (0,1) (0,-1)

---

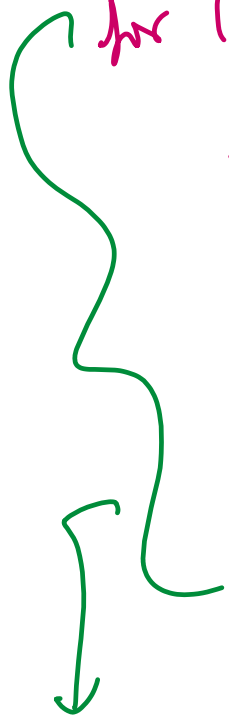
start → (0,0)

maze[0,0] = 1

(x,y)

Queue = [(0,0),

Visited = {(0,0),



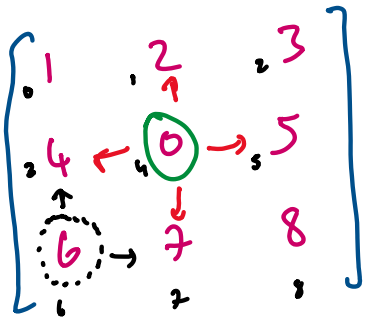
```
for (dx,dy) in directions:
    nx, ny = (x+dx), (y+dy)
    if (0 <= nx < m) & (0 <= ny < n) & maze[nx][ny] == 1:
        visited.add((nx,ny))
        enqueue((nx,ny))
```

```
while queue:
    curr = dequeue()
    if x,y == (m-1),(n-1):
        True
        loop our others
return false
```

13.9.2 - 8-Puzzle

Saturday, January 3, 2026 10:06 AM

Initial State:



Final State:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

$\begin{bmatrix} 0 \\ 4 \end{bmatrix} \rightarrow \begin{bmatrix} 4 \\ 0 \end{bmatrix}$

Swap

Arrangement of tiles (0 – 8)

Move tile '0' from initial to goal state

Represent states as tuples

Moving tile → Swapping

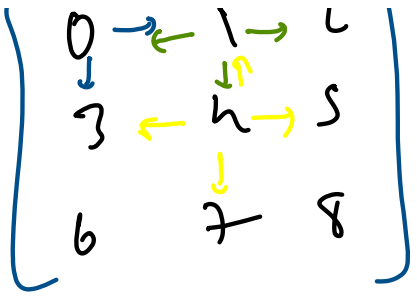
Initial → (1, 2, 3, 4, 0, 5, 6, 7, 8)

Goal → (1, 2, 3, 4, 5, 6, 7, 8, 0)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 0 & 5 \\ 6 & 7 & 8 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

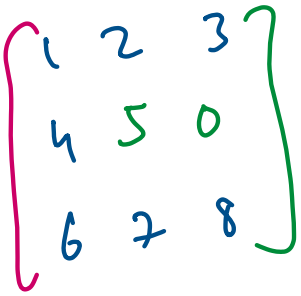
swaps = {

0: [1, 3], 1: [0, 2, 4],



0: [1,3], 1: [0,2,4],
2: [1,5],
4: [1,3,5,7]

}



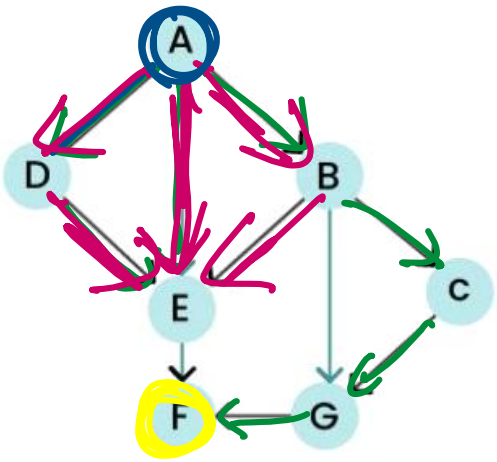
13.10 - DAG

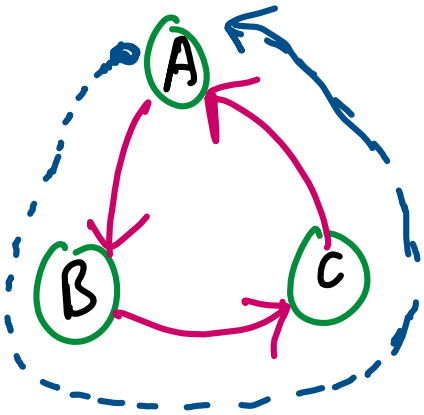
Saturday, January 3, 2026 10:20 AM

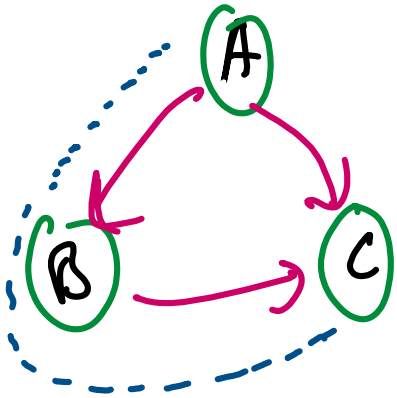
What is a DAG?

A Directed Acyclic Graph (DAG) is a graph which is:

1. **Directed:** edges have direction
2. **Acyclic:** no cycles







Key Properties of DAG:

- **At Least One Topological Ordering Exists**
- **At Least One Node with In-degree = 0**
- **At Least One Node with Out-degree = 0**
- **Longest Path Can Be Computed Efficiently**
- **Multiple Topological Orders May Exist**
- **Represents Dependency Relationships**
- **Every Subgraph of a DAG Is also a DAG**
- **Cycle Detection Is Simple**

Real-World Applications:

- **Task / Job Scheduling:**
  - Tasks depend on other tasks
  - A task can start only after its dependencies finish
  - Used in ***Project management tools***, ***Workflow Engines***
- **Build Systems (Makefiles):**
  - Some files must be compiled before others
  - Used in ***CI/CD pipelines***
- **Dependency Resolution:**
  - Used in software development
  - ***Docker image layers***
- **Data Pipelines & ETL Workflows:**
  - Data flows through **multiple processing stages**
  - ***Apache Airflow, Luigi, Prefect***
- **Version Control Systems:**
  - Commit history is a DAG
  - Each commit points to parent commit(s)
- **Neural Networks & Computation Graphs:**
  - Operations depend on outputs of other operations
  - ***Tensorflow, Pytorch***

Most Popular DAG Algorithms:

- **Topological Sorting (Kahn's Algorithm)**
- **Shortest Path in DAG**
- **Longest Path in DAG**

- **Counting Paths in DAG**

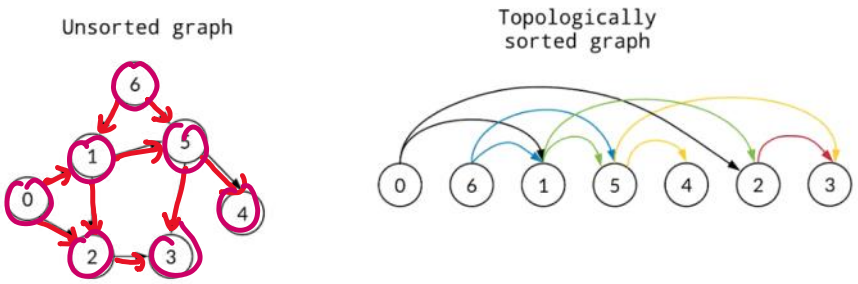
13.10.1 - Kahn's Algorithm

Saturday, January 3, 2026 10:47 AM

- Kahn's Algorithm finds a topological order of a DAG by repeatedly removing nodes with no dependencies
- If this process cannot remove all nodes, the graph contains a cycle

Topological Ordering:

- Linear arrangement of vertices in a DAG where for every directed edge from vertex **u** to **v**, **u** always appears before **v** in the list
- This shows dependency order like task prerequisites or build processes
- Allows representing workflows, ensuring that a task is listed only after its dependencies are met



Main Ideas:

1. Compute **in-degree** of every node
2. Push nodes with ***in-degree = 0*** into queue
3. Remove them and reduce neighbors' in-degree

Conclusions:

- **If at end, all nodes processed → DAG**
- **Else → cycle exists**

Algorithm:

- Find all nodes with ***in-degree = 0***
- Put them in a queue
- Process nodes level by level (BFS)
- Reduce in-degree of neighbors
- Add new ***zero in-degree*** nodes to queue

- Add new ***zero in-degree*** nodes to queue

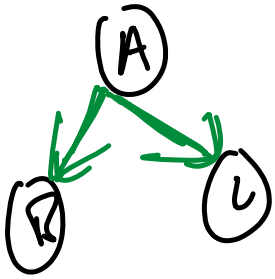
in_degree = { 0:0, 1:2→1→0, 2:2→1→0, 3:2→1→0, 4:1→0, 5:2→1→0, 6:0 }

t = [0, 6, 1, 2, 5, 3, 4]

queue: { 0, 6, 1, 2, 5, 3, 4 }

U = ∅ 6 1 2 5 3 4

DAG



A : [B, C]

13.10.2 - Shortest Path

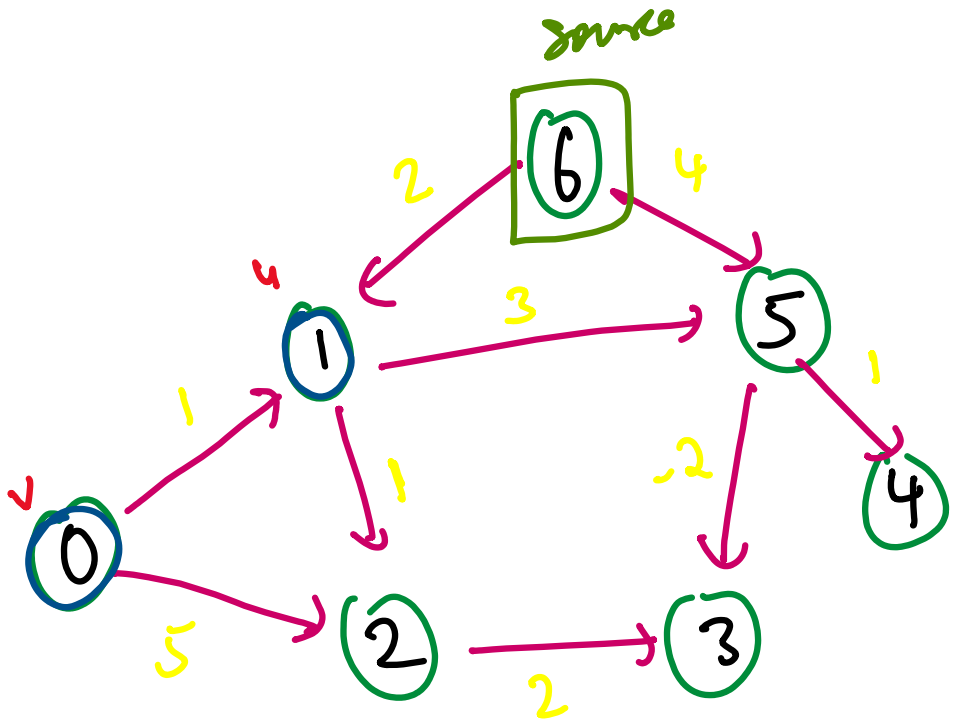
Saturday, January 3, 2026 10:48 AM

Properties:

1. **Handles Negative Edge Weights**
2. **Faster Than Other Algorithms** (Bellman, Dijkstra)

Algorithm:

1. **Topologically Sort the Graph:**
   - Ensures correct dependency order
   - Any valid topological order works
2. **Initialize Distances:**
   - Set distance of source = 0
   - Set all other nodes = ∞
3. **Relax Edges in Topological Order**



sp[1]

sp[0] → ?

sp[0] > sp[1] + 1

shortest path

sp[0] = sp[1] + 1

NOTE: Shortest path to node (0) will remain ∞ because it has no incoming node

13.10.3 - Longest Path

Saturday, January 3, 2026 10:48 AM

Properties:

- Works with **positive, zero, and negative weights**
- Based on **topological sorting**
- Very similar to shortest path in DAG, just with **max** instead of **min**

13.10.4 - Counting Paths

Saturday, January 3, 2026 10:48 AM

What Is the Counting Paths Problem?

- Given a DAG and a source node **S**, count the number of distinct paths from **S** to every other node (or to a specific target **T**)
- Paths must follow edge direction
- Nodes cannot repeat (because DAG has no cycles)

Properties:

- Only works correctly for DAG
- Uses topological sorting
- Each node's path count is finalized once
- Linear time complexity
- **Key Idea:** Number of ways to reach a node = sum of ways to reach its parents

Algorithm:

1. **Topologically Sort the DAG:**
    - Ensures all incoming paths are counted before processing a node
2. **Initialize Path Counts**
3. **Propagate Path Counts**

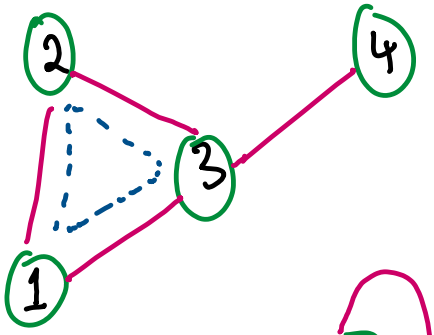
13.10.5 - Summary

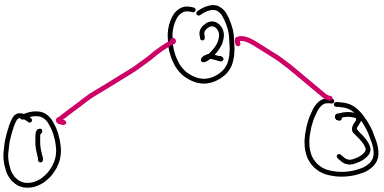
Saturday, January 3, 2026 11:46 AM

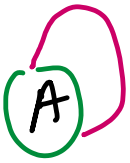
FEATURE	Kahn's Algorithm	Shortest Path in DAG	Longest Path in DAG	Counting Paths in DAG
Purpose	Find a linear order of vertices respecting dependencies	Find minimum distance/cost from source to all vertices	Find maximum distance/cost from source to all vertices	Count number of distinct paths from source to target
Input	Directed Acyclic Graph (DAG)	Weighted DAG (can be negative)	Weighted DAG (can be negative)	DAG
Output	Order of vertices	Shortest distance to each vertex	Longest distance to each vertex	Number of paths to each vertex
Key Technique	BFS + in-degree tracking (Kahn)	Topo-sort + edge relaxation	Topo-sort + edge relaxation (maximize)	DP on topologically sorted vertices
Time Complexity	O(V + E)	O(V + E)	O(V + E)	O(V + E)
Real-world Use Cases	Task scheduling, build systems, course planning	Project scheduling, routing, CPU instruction scheduling	Critical path analysis, bottleneck detection, project duration	Workflow analysis, counting decision paths, network reliability

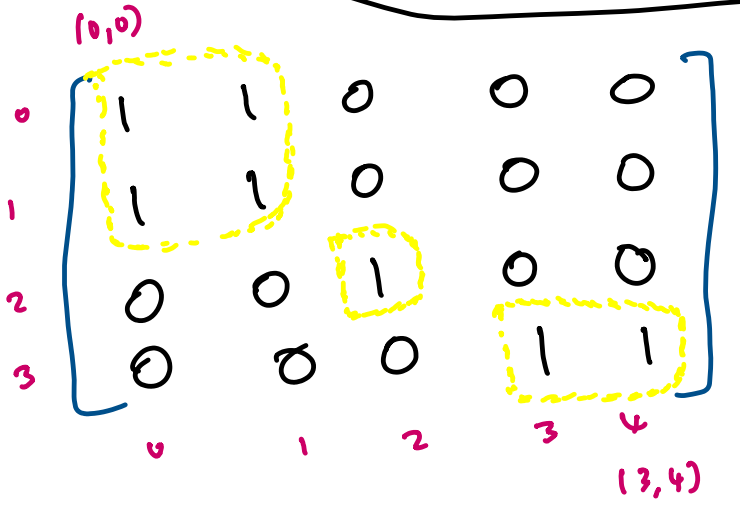
13.11 - Coding Problems

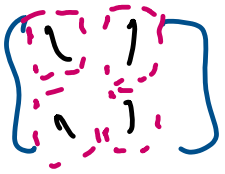
Saturday, January 3, 2026 11:58 AM









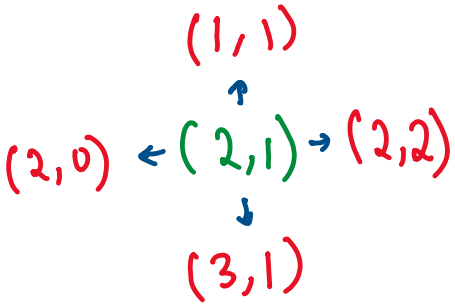


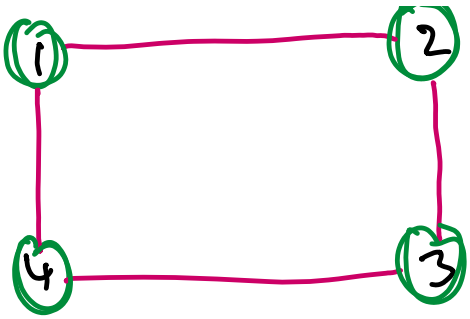
3

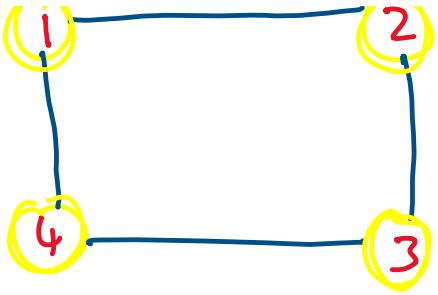
n = 4
n = 5

(0,0)
(m-1, n-1)

up → (-1, 0)
down → (1, 0)
left → (0, -1)
right → (0, 1)







graph1 = graph2 ✓

graph1 is graph2 ✗

map = {
node1 : copy1,
node2 : copy2,
node3 : copy3,
node4 : copy4
}

node1 → [node2, n4]

copy = 1
[copy(2), copy(4)]

[n1, n3]

copy = 2
[copy(1), copy(3)]

[n2, n4]

copy(3)
[copy(2), copy(4)]

[n1, n3]

copy(4)
[copy(1), copy(3)]

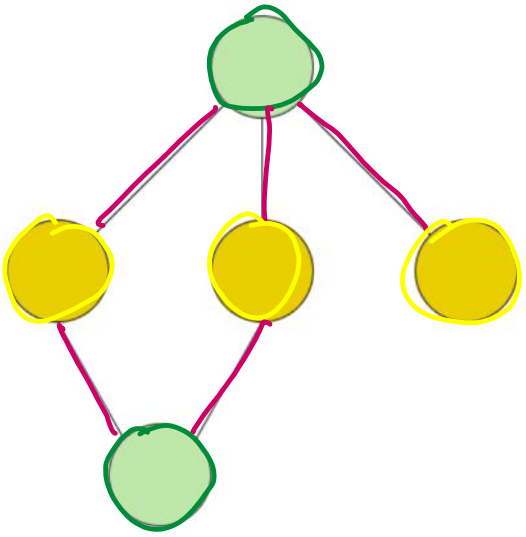
{ copy(1), copy(3) }

Bipartite Graph:

- Its vertices can be divided into two sets such that
- No two adjacent vertices are in the same set
- The graph can be colored using 2 colors
- No edge connects vertices of the same color

Idea:

- Assign a color (0 or 1) to a node
- All its neighbors must get the opposite color
- If a conflict appears → ***not bipartite***



colors → [0, 1]

color switch : 1-c

↳ 0 → 1

1 → 0

all(T, T) → T

all(T, F) → F

all(F, F) → F

1 → 0

num_courses → int

prerequisites → list of lists

4 → 0, 1, 2, 3

[1,0], [3,2], [2,1]

DAG

Kahn's Algorithm

(sr, sc)

color

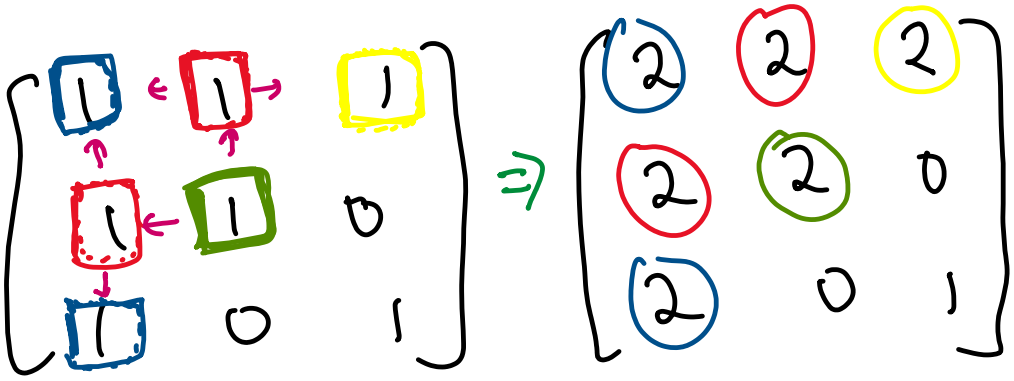
1. Set (sr, sc) to color
2. Set connected pixels to color

2. Set connected pixels to color which one of same original color

$(sr, sc) \Rightarrow (1, 1)$

connectivity $\Rightarrow$ up, down, left, right

color $\Rightarrow$ 2



cell $\rightarrow$ node

connection $\rightarrow$ edge

visited $\rightarrow$ traversal (DFS/BFS)

m, n
↳ 0, 1, 2, ..., n-1
↓
0, 1, 2, ..., m-1

$0, 1, 2, \ldots, N-1$

$(1,0), (-1,0), (0,-1), (0,1)$

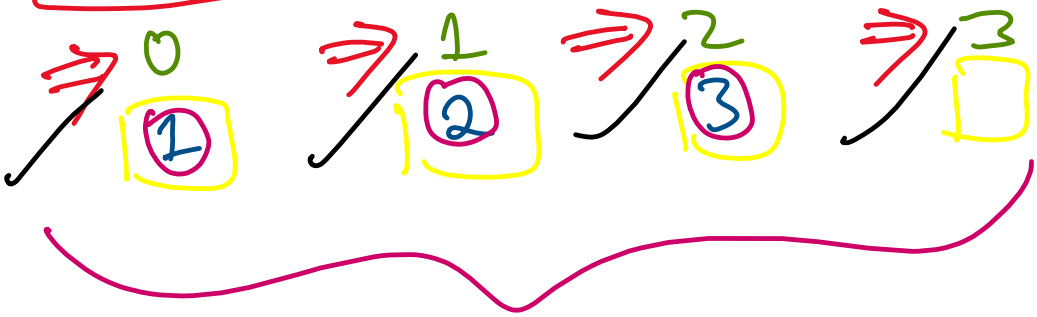
$R_1 \rightarrow [R_2]$

$R_2 \rightarrow [R_3]$

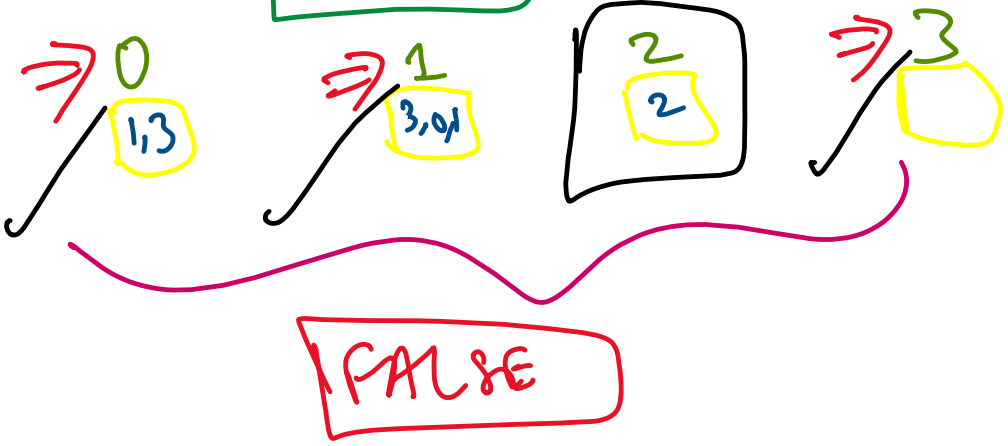
$R_3 \rightarrow [\,]$

R_1, R_2, R_3

1. visit the room
2. Explore the keys
3. If not visited, visit that room

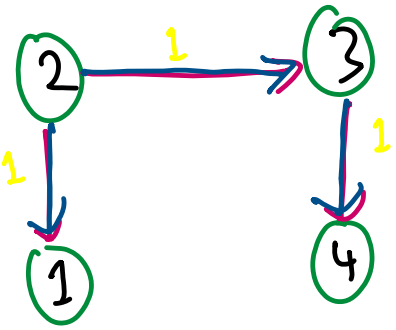


TRUE



FALSE

FALSE



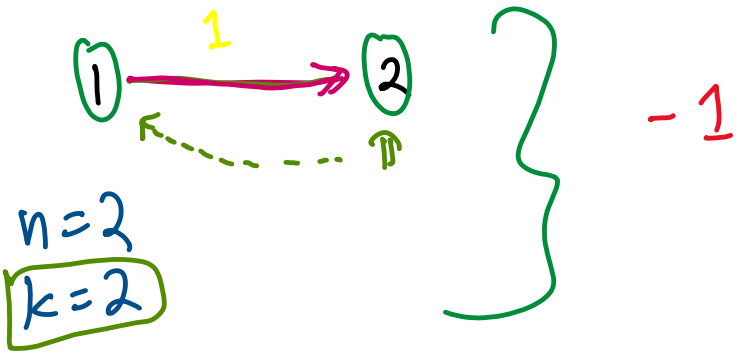
Given:

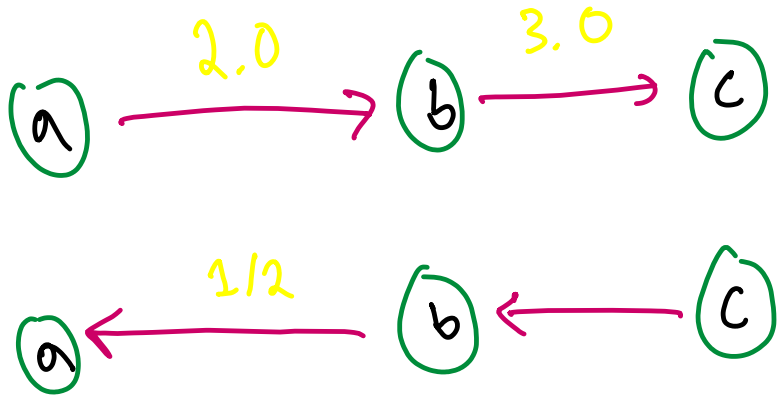
1. Network of nodes (u, v, w)
2. # nodes (n)
3. Starting node (k)

Minimum time taken by signal to reach all nodes

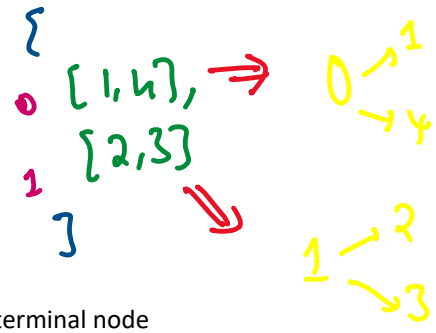
If signal can't reach all nodes return -1

```
graph = {
    1: [(2,1), ( )],
    2: [ ],
    3: [ ],
    4: [ ]
}
```





Eventual Safe States:



You are given a directed graph where:

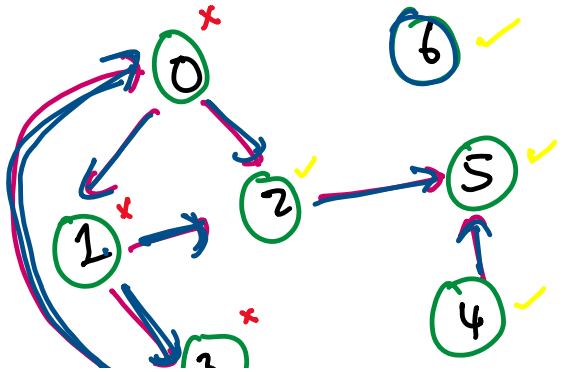
- **graph[i]** = list of nodes you can go to from node **i**

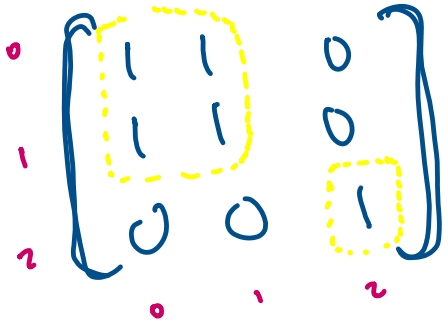
Safe Node:

- Starting from that node, every possible path eventually ends at a terminal node
- A terminal node has no outgoing edges

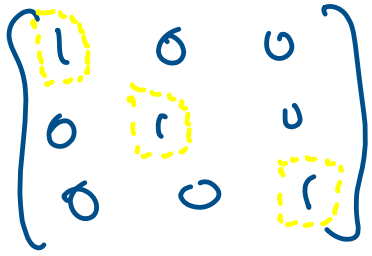
Unsafe Node:

- A node that is part of a cycle
- Can reach a cycle

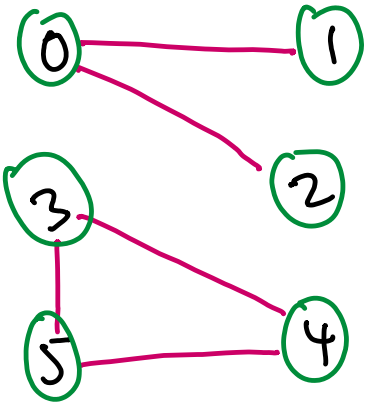




2

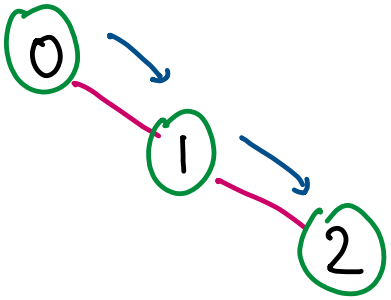


3



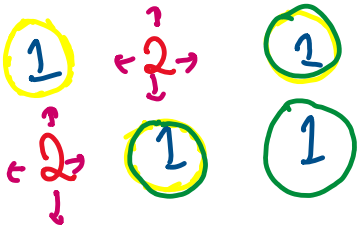
S = 0, d = 5

False



S = 0, d = 2

True



* Store the rotten oranges

* store the rotten oranges

* store the count of fresh oranges

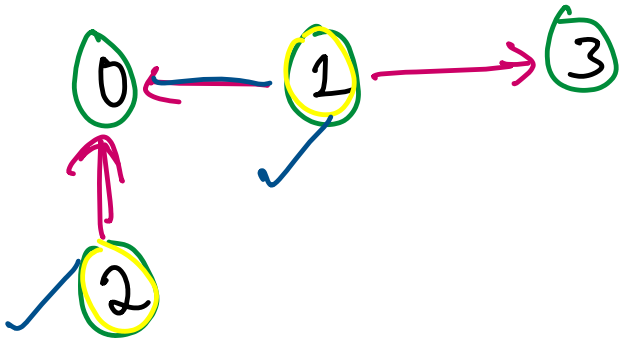
cell → node

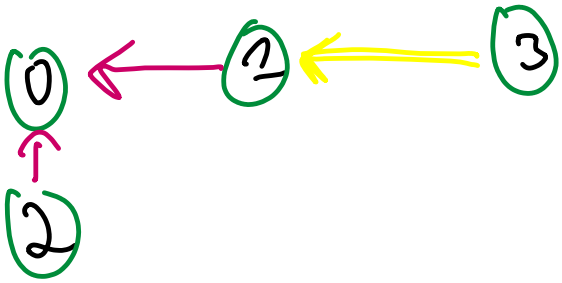
adjacency → edges

* BFS

* multi-source BFS

* queue to store rotten oranges





13.12 - DS Context

Wednesday, January 7, 2026 12:48 AM

1. Recommendation Systems:

Graph Modeling

- Nodes: Users, Items
- Edges: Click, view, purchase, rating

ML Tasks

- Link prediction
- Node embeddings
- Personalized ranking

Algorithms Used

- Random Walks
- Node2Vec
- Graph Neural Networks (GNNs)

Real-World Examples

- Netflix movie recommendations
- Amazon product suggestions
- Spotify song recommendations

2. Fraud / Anomaly Detection:

Graph Modeling

- Nodes: Accounts, cards, users
- Edges: Transactions, transfers

ML Tasks

- Node classification (fraud / non-fraud)
- Subgraph detection
- Cycle detection

Industry Examples

- Banking fraud
- Insurance fraud
- Money laundering detection

3. Social Network Analysis:

Graph Modeling

- Nodes: Users
- Edges: Follow, friend, message

ML Tasks

- Community detection
- Influence maximization
- Link prediction

Algorithms

- PageRank
- Louvain clustering
- Graph embeddings

Examples

- Facebook friend suggestions
- Twitter (X) influencer ranking
- LinkedIn connection recommendations

4. Search Engines & Ranking Systems:

Graph Modeling

- Nodes: Web pages
- Edges: Hyperlinks

ML Tasks

- Page ranking
- Authority detection

Algorithms

- PageRank
- HITS
- GNN-based ranking

Real-World Example

- Google search ranking

5. Bioinformatics & Healthcare:

Graph Modeling

- Nodes: Proteins, genes, drugs
- Edges: Interactions, reactions

ML Tasks

- Graph classification
- Link prediction
- Node classification

Use Cases

- Drug discovery
- Protein interaction prediction
- Disease pathway analysis

6. Computer Vision:

Graph Modeling

- Nodes: Objects in image
- Edges: Spatial relationships

ML Tasks

- Scene graph generation
- Object interaction detection

Examples

- Autonomous driving
- Image captioning

7. NLP:

Graph Modeling

- Nodes: Words, entities, documents
- Edges: Co-occurrence, dependency

ML Tasks

- Document classification
- Entity linking
- Semantic similarity

Examples

- Knowledge-based QA
- Document clustering

Topics

Sunday, July 20, 2025 10:08 AM

1. **Mathematical Foundations**
    - ***Discrete Mathematics***
    - ***Recurrences & Growth Functions***
    - ***Probability & Expected Value***

2. **Trees**
    - ***Binary Search Tree***
    - ***Tree DP***
    - ***N-ary Tree***

3. **Graphs**
    - ***Graph Theory***
    - ***Traversal Algorithms***
    - ***Shortest Path Algorithms***
    - ***Topological Sorting & DAG Problems***
    - ***Disjoint Set Union***

4. **Heaps**
    - ***Mathematical Foundation***
    - ***Implementation***

5. **Priority Queues (PQ)**
    - ***PQ Theory***
    - ***Graph & PQ Applications***

6. **Tries**
    - ***Foundations***
    - ***Advanced Problems***

7. **Backtracking**
    - ***Theory***
    - ***Classic Problems***
    - ***Advanced Problems & Optimizations***

8. **Dynamic Programming (DP)**
    - ***Mathematical Foundation***
    - ***1-dimensional DP***
    - ***2-dimensional DP***
    - ***String DP***
    - ***DP on Trees & Graphs***

9. **Segment Trees & Fenwick Trees**
    - ***Segment Tree Theory***
    - ***Fenwick Trees***

10. Advanced Algorithmic Patterns